Decentralized Crypto World Bank

A Blockchain-Based Banking Platform
with AI-Enhanced Security and Assistance

by

Md. Bokhtiar Rahman Juboraz

20301138

Md. Mahir Ahnaf Ahmed

20301083

A pre-thesis 1 report submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science

Department of Computer Science and Engineering
BRAC University
February 2026

Declaration

It is hereby declared that

1. The report submitted is our own original work while completing degree at BRAC University.
2. The report does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The report does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Md. Bokhtiar Rahman Juboraz	Md. Mahir Ahnaf Ahmed
20301138	20301083

Approval

The pre-thesis 1 report of final year project titled "Decentralized Crypto World Bank: A Blockchain-Based Banking Platform with AI-Enhanced Security and Assistance" submitted by

1. Md. Bokhtiar Rahman Juboraz (20301138)
2. Md. Mahir Ahnaf Ahmed (20301083)

Of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science on February 20, 2026.

Examining Committee:

Supervisor:
(Member)

Mr. Annajiat Alim Rasel
Senior Lecturer
Department of Computer Science and Engineering
BRAC University

Project Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam
Professor
Department of Computer Science and Engineering
BRAC University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi
Chairperson and Associate Professor
Department of Computer Science and Engineering
BRAC University

Ethics Statement

This project operates exclusively on public blockchain test networks using test tokens with no real monetary value. No real financial transactions, personal banking data, or user financial records are involved at any stage of development or evaluation. All transaction data used for Artificial Intelligence and Machine Learning (AI/ML) model training and evaluation is either synthetically generated or sourced from publicly available anonymized datasets. The platform prototype does not process Know Your Customer (KYC) or Anti-Money Laundering (AML) data in its current scope, and all wallet addresses used during testing are disposable testnet addresses. We acknowledge the ethical considerations inherent in AI-assisted financial decision-making and have incorporated explainability mechanisms (SHapley Additive exPlanations, or SHAP) to ensure that automated risk assessments remain transparent and auditable by human reviewers. Future phases plan to incorporate a privacy-preserving ZKP-based identity compliance layer in which users prove KYC status without exposing personal data on-chain; the ethical implications of this design will be evaluated in the final thesis.

Abstract

Development finance moves capital through several institutional layers, but settlement is often slow, costly, and hard to audit. Decentralized finance (DeFi) can automate lending on-chain, yet most protocols use flat pools that do not match real tiered banking. *Crypto World Bank* (CWB) is a research prototype that specifies a four-tier lending platform on EVM-compatible infrastructure: World Bank Reserve → National Bank → Local Bank → Client.

The design pairs on-chain reserve rules and role-based access with off-chain KYC and ML risk scoring (Random Forest, Isolation Forest, SHAP), committed through a commit–reveal oracle. A conversational agent (Qwen3-8B + MCP) is planned as an optional plain-language UI. At pre-thesis stage, the main output is formal specification and a four-phase build plan. Measured ML results come from a synthetic BCCC-shaped validation run; full BCCC-DeFiFraudTrans-2025 retraining is planned after York dataset access (request submitted). Final-thesis targets include testnet loan lifecycle, latency and gas tables, reserve invariant tests, and a small agent demo with a human confirmation gate.

Keywords: Blockchain, DeFi, Institutional Finance, Hierarchical Lending, Smart Contracts, Explainable AI, Group Lending

Dedication

Dedicated to our families for their unwavering support throughout our academic journey.

Acknowledgment

We would like to express our sincere gratitude to our panel members for their guidance and support throughout this project. We also thank our supervisor, Mr. Annajiat Alim Rasel, Senior Lecturer at the Department of Computer Science and Engineering, BRAC University, for his guidance and support. Finally, we thank the Department of Computer Science and Engineering at BRAC University for providing us with the resources and academic environment to pursue this work.

Contents

List of Tables

List of Figures

List of Formulas

Utilization (U): $U = \frac{L}{L+B}$

Kinked rate ($U \le U^*$): $r_b(U) = r_0 + \frac{U}{U^*} \cdot r_1$

Kinked rate ($U > U^*$): $r_b(U) = r_0 + r_1 + \frac{U-U^*}{1-U^*} \cdot r_2$

Collateral Ratio: $CR = \frac{C}{D}$

Loan-to-Value: $LTV = \frac{\text{Loan Amount}}{\text{Collateral Value}}$

APR (simple): $APR = r_{\text{period}} \times N$

Compound interest: $A = P\left(1 + \frac{r}{n}\right)^{nt}$

EMI: $EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$

Health Factor: $HF = \frac{C \times LT}{D}$

Group Health Factor: $HF_{\text{group}} = \frac{\text{total\_group\_collateral} \times LT}{\text{total\_group\_outstanding\_debt}}$

Institutional Health Factor: $HF_{\text{inst}} = \frac{\text{on\_chain\_reserve} - \text{min\_reserve}}{\text{outstanding\_borrowing}}$

Reserve Ratio: $RR = \frac{\text{Reserves}}{\text{Total Deposits}}$

Platform Solvency Ratio: $PSR = \frac{\text{TotalReserveBalance}}{\text{TotalOutstandingLoans}}$

Credit Velocity: $CV = \frac{\text{Capital Recycled per Period}}{\text{Total Reserve}}$

Interbank rate cap: $r_{\text{IB}}(U) \le r_{\text{downward}}(U) - \delta$

Upward deposit rate cap: $r_{\text{up}} < r_{\text{down}} - \delta$

FX conversion: $\text{Amount}_B = \text{Amount}_A \times \frac{P_A}{P_B}$

Treasury swap: $\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread})$

Oracle commit–reveal: $h = \text{keccak256}(s \,\|\, \text{nonce})$

Group loan share: $\text{Share}_i = \frac{\text{LoanTotal}}{N_{\text{members}}}$

On-chain credit score: $CS = \sum_i w_i \cdot \text{feature}_i$

Isolation Forest score: $s(x, n) = 2^{-E(h(x))/c(n)}$

Random Forest fraud probability: $P(\text{fraud} \mid x) = \frac{1}{T} \sum_{t=1}^{T} f_t(x)$

Net interest split: $\text{NetInterest} = \text{InterestCollected} - \text{DepositorYield} - \text{InsuranceAllocation}$

SHAP (Shapley value): $\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} \left[f(S \cup \{i\}) - f(S)\right]$

List of Abbreviations

AA: Account Abstraction (ERC-4337)

ALM: Asset-Liability Management

AI: Artificial Intelligence

AI/ML: Artificial Intelligence / Machine Learning

AML: Anti-Money Laundering

APR: Annual Percentage Rate

API: Application Programming Interface

BRAC: Bangladesh Rural Advancement Committee

CAR: Capital Adequacy Ratio (Basel III; see PSR for this platform)

CBDC: Central Bank Digital Currency

CCIP: Cross-Chain Interoperability Protocol (Chainlink)

CCS: ACM Computing Classification System

CR: Collateral Ratio

CVL: Certora Verification Language

DeFi: Decentralized Finance

DID: Decentralized Identifier (W3C)

DON: Decentralised Oracle Network (Chainlink)

EIP: Ethereum Improvement Proposal

EIP-7702: Ethereum Improvement Proposal 7702 — Session Key standard enabling scoped, time-bound wallet authorisations for an autonomous agent

EMI: Equated Monthly Installment

ERC: Ethereum Request for Comments (token / interface standard)

EVM: Ethereum Virtual Machine

FL: Federated Learning

FATF: Financial Action Task Force

FX: Foreign Exchange

GNN: Graph Neural Network

HF: Health Factor

KYC: Know Your Customer

LLM: Large Language Model

L2: Layer 2

LTV: Loan-to-Value Ratio

MFI: Microfinance Institution

MiCA: Markets in Crypto-Assets Regulation (EU)

ML: Machine Learning

MCP: Model Context Protocol — tool-calling interface used by the AI agent to interact with CWB banking operations

NIM: Net Interest Margin

PRISMA: Preferred Reporting Items for Systematic Reviews and Meta-Analyses

PoS: Proof of Stake

PoR: Proof of Reserve (Chainlink on-chain reserve verification)

PSR: Platform Solvency Ratio

QRC: QR code

RBAC: Role-Based Access Control

RLS: Row-Level Security (PostgreSQL policy feature enforcing append-only access on audit tables)

RR: Reserve Ratio

SAR: Suspicious Activity Report

SBT: Soulbound Token (non-transferable ERC standard)

SHAP: SHapley Additive exPlanations

SOFR: Secured Overnight Financing Rate

SSE: Server-Sent Events

SSI: Self-Sovereign Identity

SoK: Systematization of Knowledge

SWC: Smart Contract Weakness Classification

TVL: Total Value Locked

U: Utilization

UUPS: Universal Upgradeable Proxy Standard (ERC-1822)

VC: Verifiable Credential (W3C)

XAI: Explainable Artificial Intelligence

ZKP: Zero-Knowledge Proof

zkAML: Zero-Knowledge Anti-Money-Laundering proof

zkEVM: Zero-Knowledge Ethereum Virtual Machine — a ZK rollup that inherits Ethereum L1 security via cryptographic validity proofs

zk-SNARK: Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

Chapter 1

Introduction

The Crypto World Bank (CWB) is a final-year research prototype for a blockchain-based institutional finance platform. Existing DeFi protocols handle single functions well—Aave for lending, Uniswap for exchange—but none models a full tiered banking stack. CWB specifies a four-tier hierarchy (World Bank → National Bank → Local Bank → Client) with deposit mobilization, credit allocation, installments, interbank liquidity, group lending, and reserve enforcement. Tier 1 contracts and the core lending workflow are specified in Solidity; multi-entity contracts are detailed in Chapter 3.2 and scheduled for later phases.

The platform targets three groups underserved by current DeFi: (1) development banks and regional institutions that face high correspondent-banking costs ($42 per transaction, 2–5 day delays) [25]; (2) licensed MFIs and cooperative banks that lack programmable credit tools; and (3) semi-formal borrowers reached through MFIs rather than direct wallet onboarding. Large unbanked and MSME financing gaps [14, 20] show that the problem sits at the institutional layer, not only at the retail user. Project artifacts are in our GitHub repository.

Blockchain here is a coordination layer: shared state, tamper-evident records, and programmable rules among institutions that may not fully trust each other. Comparable hybrid models include the World Bank FundsChain programme and JPMorgan Kinexys for institutional settlement on permissioned ledgers.

Figure 1.1 shows the four tiers, application stack, oracle ML path, and compliance controls at a glance. Full specifications follow in Chapter 3.2.

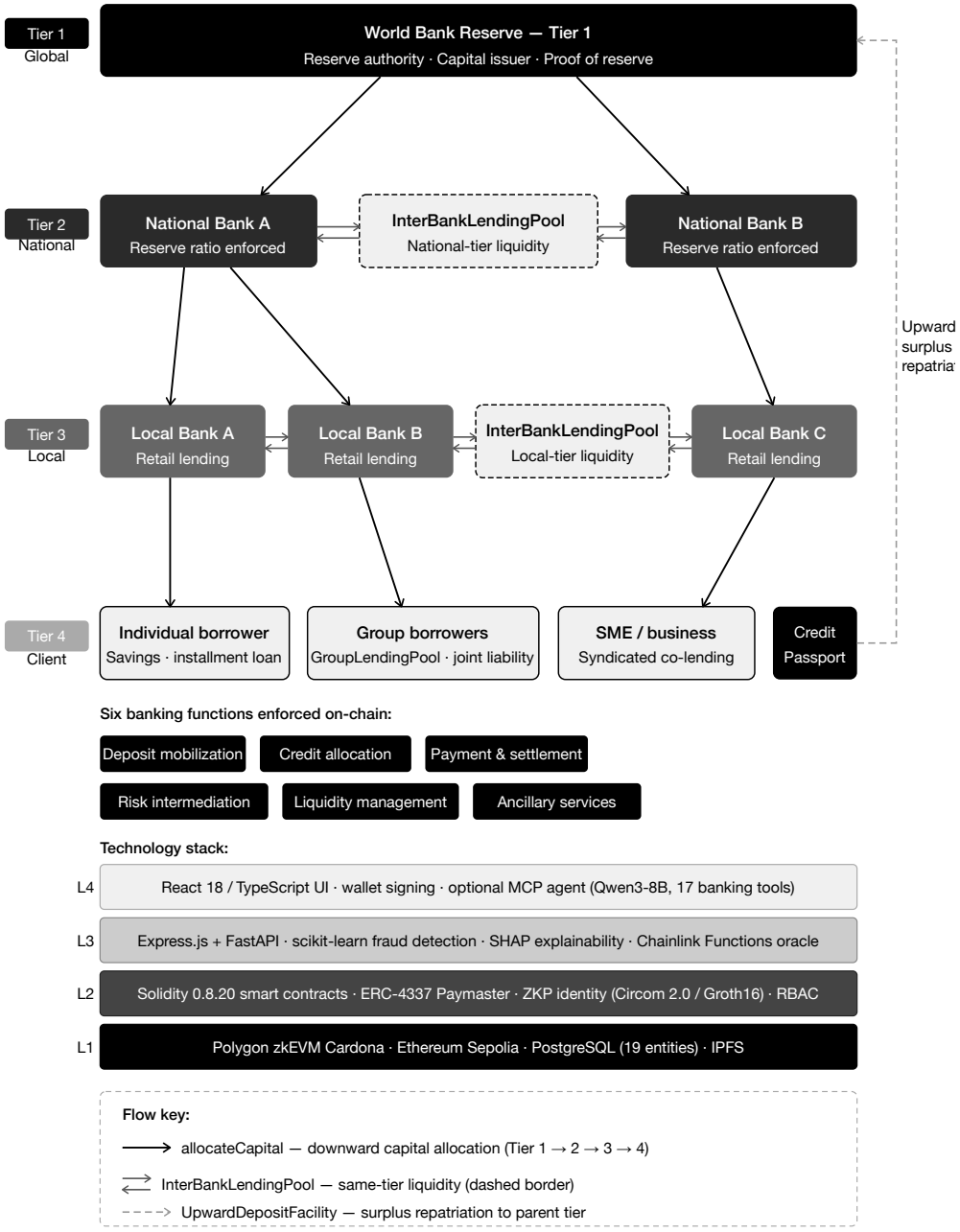


Figure 1.1: Crypto World Bank system overview

1.1 Background

Development finance moves capital through several layers—from supranational bodies to national banks, local lenders, and end borrowers. This model spreads risk but creates real operational problems. Correspondent banking requires pre-funded nostro/vostro accounts in every currency corridor, locking capital that could otherwise be lent [24]. Cross-border transfers often take 2–5 days and cost about $42 per transaction [25]. Remittance fees remain high (6.49% global average in 2024, above the UN 3% target) [26]. An estimated 1.4 billion adults remain unbanked [14], and MSMEs face a $4.5 trillion financing gap [20].

DeFi has shown that lending, collateral, and interest can run on smart contracts with public

audit trails. Aave v3 holds roughly $26.3 billion TVL [27], and the broader DeFi lending market exceeds $55 billion [13]. Yet these protocols use flat pools: all lenders share one pool and all borrowers draw from it, with no institutional tiers, cross-tier flows, or client-via-local-bank paths typical of development finance.

What blockchain adds here. A **smart contract** is program code on a blockchain that runs the same way on every node. For CWB, three properties matter: (1) **governed execution**—contract changes need a recorded vote and timelock (Section 3.18), reducing flash-governance attacks like the $182M Beanstalk exploit [158]; (2) **atomic settlement**—multi-step operations complete fully or revert; (3) **public audit**—every state change is logged on-chain. **Blockchain** adds a shared ledger so co-platform members see the same reserve and loan state without slow bilateral reconciliation [25]. **DeFi** applies this to finance, but CWB keeps institutions, KYC, and ML off-chain while enforcing tier rules on-chain (Section 1.5). CWB extends flat DeFi in three ways: institutional hierarchy, multi-entity operations (group loans, syndication, interbank pools), and alignment with development-finance structures absent in current protocols [1].

Optional conversational interface. Users interact mainly through the React UI and wallet signing. A planned Phase III agent (Qwen3-8B + MCP, 3–5 core tools at MVT scope) offers plain-language queries and one demonstrated write path, always behind a human confirmation gate. All write operations remain available through the web UI.

1.2 Rationale of the Study

Three forces motivate this work: (1) long-standing inefficiencies in development finance (slow settlement, weak transparency); (2) the lack of tiered banking features in current DeFi (deposits, group lending, hierarchical flows); and (3) the chance to combine on-chain auditability with practical ML support for credit decisions. CWB explores whether these can be joined in one architecture for institutional and retail participants.

1.3 Problem Statement

Development finance runs through a chain of institutions—from global bodies to national banks, local lenders, and individual borrowers. This creates several problems:

1. **Lack of transparency.** Lower tiers cannot easily verify reserve levels or lending decisions above them. Reserves are often self-reported and audited only quarterly.
2. **Slow settlement and idle capital.** Cross-border transfers take 2–5 days and cost about $42 per transaction [25]. Pre-funded correspondent accounts lock capital out of lending [24].
3. **Inconsistent risk evaluation.** Fraud and credit checks rely on manual judgment. Borrowers are re-assessed at each tier with no shared, verifiable history.
4. **Access and trust barriers.** Building trust between institutions is slow and costly, disadvantaging smaller banks and borrowers in developing markets [28].
5. **No programmable reserve tools.** MFIs lack real-time, auditable reserve enforcement between audit cycles.
6. **No on-chain group credit.** Many borrowers lack individual collateral but could use solidarity groups (BRAC, Grameen). No DeFi protocol models mutual liability with MFI-appropriate controls.

DeFi automates lending at scale (Aave $26.3B TVL [27]; Compound $1.4B [29]), but remains poorly suited to institutional development finance:

- **Flat lending models**—no four-tier hierarchy or cross-tier allocation (distinct from Aave sub-markets, which are risk buckets, not institutional relationships).
- **No ML risk layer**—decisions rely on collateral and utilization curves only.
- **No tiered governance**—token-weighted voting, not institutional roles.
- **Limited institutional DeFi**—Maple and Goldfinch serve institutional credit but without hierarchical capital flows or interbank facilities [31, 32].

1.4 Objectives

1. **Design a four-tier lending architecture** (World Bank → National Bank → Local Bank → Client) on an EVM-compatible chain with on-chain hierarchy and shared ledger access. Contracts and workflows are specified in this pre-thesis; testnet deployment is planned for the final thesis.
2. **Specify an extended banking product suite**—deposits, savings, and solidarity group lending—on top of the tiered foundation.
3. **Plan an off-chain analytics layer** (fraud detection, anomaly detection, SHAP explainability) plus a minimal conversational agent (MCP, 3–5 tools, one write demo) as a Phase III deliverable.
4. **Specify transparent lending processes** with borrowing limits, installments, and role-based access in smart contracts.
5. **Plan testnet deployment** on Polygon Amoy and/or Ethereum Sepolia for stability, with Polygon zkEVM as the design-preference settlement model.
6. **Document governance, security, and rollout** toward academic validation and possible regulatory sandbox pilots.

1.5 Blockchain Justification

The core problem is **multi-party trust**: institutions must share financial state and enforce common rules without relying on one central operator or slow bilateral agreements. Traditional fixes—a central authority or heavy legal reconciliation—create either systemic risk or the 2–5 day delays and $42 costs described above.

Blockchain helps through four properties:

1. **Shared state.** All tiers read the same reserve ratios and loan states in real time, not separate ledgers reconciled by batch messaging.
2. **Programmable rules.** Reserve ratios, rate curves, limits, and approval flows run as on-chain code. Changes require governance (timelock, quorum; Section 3.18).
3. **Public audit trail.** Every disbursement, repayment, and reserve update is logged on-chain for regulators, partners, and auditors.
4. **Atomic multi-party settlement.** Group loans, syndicated deals, and netting complete fully or revert—no half-settled state.

A central database could store the same records, but it reintroduces a trusted admin. Blockchain is chosen because verifiable, programmable multi-institution coordination is the actual requirement.

Hybrid institutional architecture. CWB is **not** fully disintermediated retail DeFi. Institutions, KYC, ML inference, and APIs run off-chain (Express.js, PostgreSQL, FastAPI); the chain enforces rules, reserves, RBAC, and audit trails. This matches institutional DeFi patterns (Maple/Goldfinch-style permissioned layers plus on-chain enforcement). The chain does not remove banks; it makes tier flows verifiable.

1.5.1 Blockchain Fundamentals and the EVM Execution Model

Brief technical reference for readers new to EVM smart contracts. Familiar readers may skip to Section 1.6.

A blockchain is an append-only ledger linked by cryptographic hashes. On Proof-of-Stake networks, history stays stable as long as no majority coalition rewrites it [R17]. CWB targets EVM-compatible chains: Polygon zkEVM as design preference; Amoy/Sepolia for prototype stability.

EVM execution. Smart contracts are deterministic programs whose state changes cost **gas**. Storage writes (`SSTORE`) dominate cost; a full retail loan lifecycle involves roughly 27–32 on-chain state changes versus 10–15 user-facing transactions (Section 5.8). Contracts behave as state machines (`PENDING` → `APPROVED` → `ACTIVE` → `REPAYING` → `COMPLETED`).

Oracle problem. Contracts cannot read off-chain data directly. ML risk scores must reach the chain via a relay, Chainlink Functions, or commit–reveal [R1, R2]. CWB uses commit–reveal at MVT scope; Chainlink Functions is a stretch upgrade when supported on the chosen testnet. Chainlink price feeds supply FX rates; Proof of Reserve can verify reserve balances externally.

1.6 Proposed Solution

CWB uses a **four-tier on-chain lending model**:

- **Tier 1 — World Bank:** Holds the global reserve and allocates capital to National Banks.
- **Tier 2 — National Banks:** Borrow from Tier 1 and lend to Local Banks in their jurisdiction.
- **Tier 3 — Local Banks:** Process retail loan applications and manage the loan lifecycle.
- **Tier 4 — Clients:** Apply for loans, repay in installments, and build on-chain credit history (individual, group, or small-business).

The platform also includes risk-based borrowing limits, automatic installments above a threshold (e.g. $5,000 USDC), and planned off-chain ML (Random Forest, Isolation Forest, SHAP).

1.6.1 Banking Functions of the Platform

Six standard banking functions run across the four tiers (Figure 1.2):

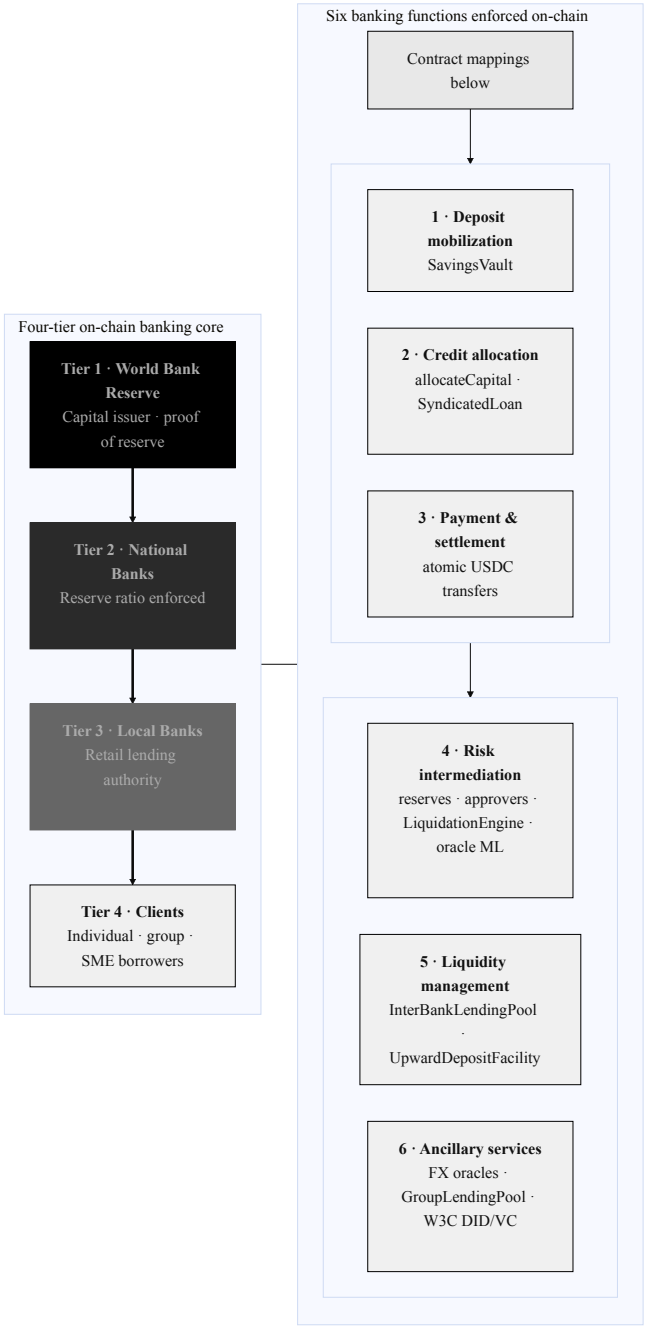


Figure 1.2: Six banking functions of the platform

- **Deposit mobilization** — `SavingsVault` with reserve checks on withdrawal.
- **Credit allocation** — downward `allocateCapital` and `SyndicatedLoan` for co-funding.
- **Payment & settlement** — atomic transfers without nostro/vostro in-transit state.
- **Risk intermediation** — reserves, approver workflows, `LiquidationEngine`, oracle ML.
- **Liquidity management** — `InterBankLendingPool` and `UpwardDepositFacility`.
- **Ancillary services** — FX oracles, `GroupLendingPool`, syndicated/tranched products, W3C DID/VC identity.

1.6.2 Service Delivery Across Scale

Global (Tier 1). The World Bank Reserve manages system-wide capital and reserve constraints with on-chain reporting.

National and regional (Tiers 2–3). National Banks allocate to Local Banks under jurisdiction-aware governance. This mirrors development-finance intermediation with faster, auditable settlement.

Retail (Tier 4). Clients use Local Bank interfaces for deposits, loans, group credit, and repayments. MFIs onboard users who may not hold wallets directly.

1.6.3 Cross-Tier Lending System

Real banking is not only top-down. Banks lend to peers (interbank markets), deposit surplus upward, and syndicate large loans. CWB encodes all three directions on-chain (Figure 1.3).

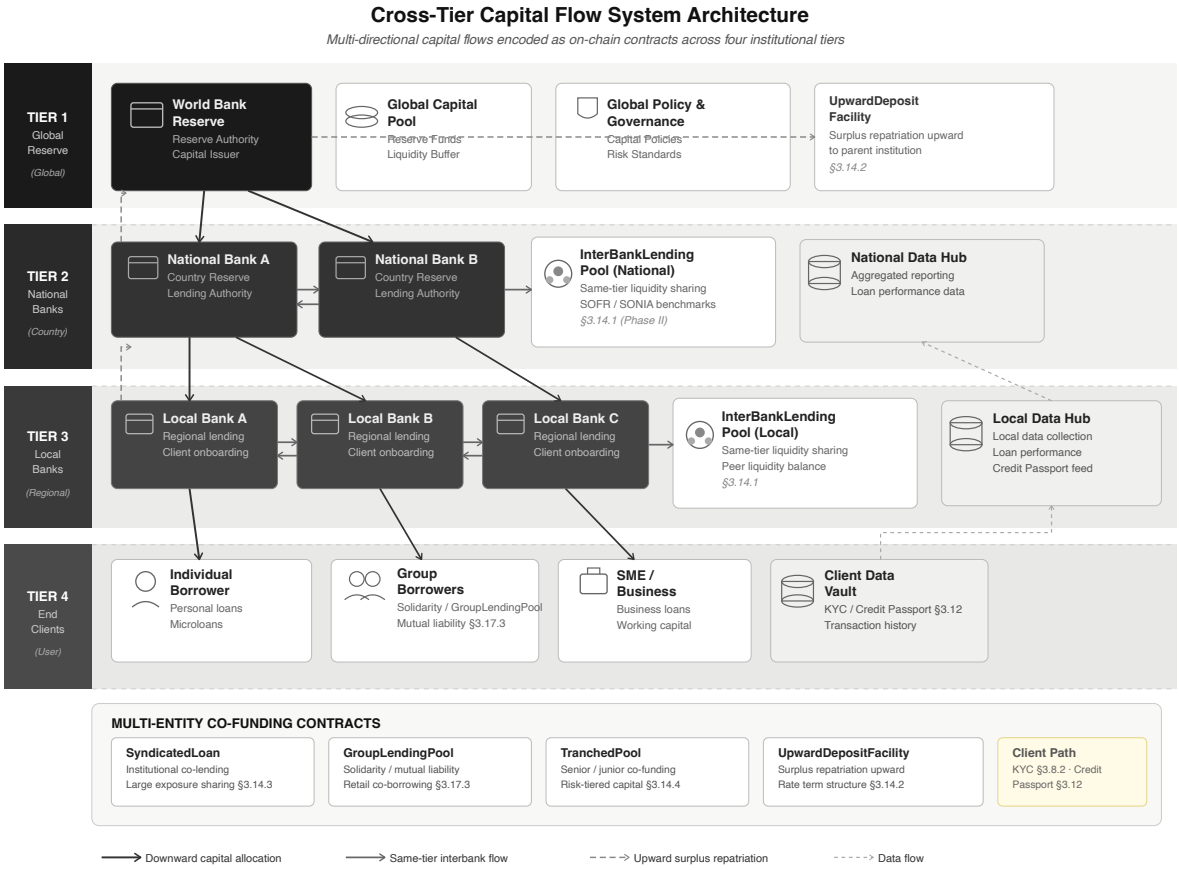


Figure 1.3: Cross-tier lending flow directions

Group co-lending and co-funding.

- **Syndicated loans** — multiple banks co-fund one large loan via `SyndicatedLoan` (Section 3.14.3).
- **Group loans** — 3–20 retail clients share collateral and mutual liability in `GroupLendingPool` (Section 3.17.3).
- **Tranched pools** — senior and junior tranches share one loan pool (Section 3.14.4).

- **Upward deposits** — surplus Local Bank reserves flow up via `UpwardDepositFacility` (Section 3.14.2).

Same-Tier Lending

Peer banks at the same tier share liquidity through `InterBankLendingPool`—National Bank to National Bank, or Local Bank to Local Bank—at utilization-based rates. Full specification is in Section 3.14.1; implementation is planned for Phase II.

Upward Lending

Local Banks with excess reserves deposit upward to National Banks; National Banks may deposit to the World Bank Reserve. Downward lending rates exceed upward deposit rates, creating a natural term structure across tiers.

Client Lending Path

All retail borrowers apply through a Local Bank. Clients do not access higher tiers directly. Large exposures use `SyndicatedLoan` co-funding. Downward flows and the client entry point are specified in Phases I–II; interbank and upward modules are detailed in Section 3.14.

1.7 Methodology in Brief

We combine Agile delivery with a standard SDLC frame. Pre-Thesis 1 covers requirements, architecture, and design; implementation and validation run in four final-thesis phases (Table 1.1).

Table 1.1: Four-phase implementation plan: deliverables, smart-contract scope, and deployment targets.

Phase	Primary Deliverables	Contract Scope	Deployment Target
I	Four-tier scaffold, wallet auth, frontend skeleton, PostgreSQL schema (51 entities: 31 core + 6 extension + 14 stubs), Chainlink Proof of Reserve, off-chain KYC scaffolding	WorldBankReserve, NationalBank, LocalBank	Amoy/Sepolia (impl.); zkEVM (design pref.)
II	Loan application, approval, installment generation, borrowing-limit enforcement, deposit mobilisation, GroupLendingPool, InterBankLendingPool	SavingsVault, LoanContract, GroupLendingPool, UpwardDepositFacility	Amoy/Sepolia (impl.); zkEVM (design pref.)
III	Random Forest + Isolation Forest + SHAP pipeline (commit–reveal oracle; Functions stretch), SyndicatedLoan, TranchedPool, MCP conversational agent, Foundry invariant suite, Tenderly monitoring	SyndicatedLoan, TranchedPool, FXModule, LiquidationEngine	Amoy/Sepolia (impl.); zkEVM (design pref.)
IV	Consistency pass, evaluation evidence pack, simulation outputs, testnet manifest completion	All MVT contracts (as built)	Amoy/Sepolia (impl.); zkEVM (design pref.)

Stack: Solidity 0.8.20, React 18 + TypeScript, Express.js + FastAPI, scikit-learn (ML), Circom 2.0 + snarkjs (ZKP). Prototype deployments target Amoy/Sepolia; Polygon zkEVM remains the design-preference settlement model.

1.8 Scopes and Challenges

Scope. At pre-thesis stage, the output is formal specification and a four-phase plan. Planned testnet scope covers four-tier lending, loan approval, installments, borrowing limits, and oracle-mediated ML (RF + SHAP, Isolation Forest). Out of scope: mainnet, retail remittance, fiat ramps, production KYC/AML, and large-scale stress testing.

Challenges.

- **Limited fraud labels** — may require synthetic data or transfer learning.

- **Regulatory uncertainty** — mitigated by testnet-only deployment during the thesis.
- **Gas costs** — heavy computation stays off-chain; L2 keeps on-chain costs low.
- **Explainability** — SHAP supports audit-friendly lending decisions.
- **Economic sustainability** — interest revenue must cover gas at all tiers (Section 1.8 and Chapter 5).

1.8.1 Economic Sustainability: Interest Revenue Versus Gas Costs

On Polygon L2, a $25,000 loan at 8% APR yields $2,000 annual interest while full lifecycle gas stays under $0.15—well below 0.01% of interest. Oracle ML calls add $0.10–$1.00 each, negligible above $500 ticket size. Mainnet would need batching or optimization for micro-loans.

1.8.2 Resistance to Institutional Capture

On-chain design reduces capture risk through: public reserve ratios, algorithmic (not discretionary) rates, tamper-evident audit logs, open-source contracts, and programmatic borrowing caps with syndication for large exposures.

1.8.3 Stablecoin-First Lending and Supply Scalability

Retail borrowers face severe risk if loans are denominated in volatile crypto (e.g. ETH drawdowns double effective debt) [R12, R13]. CWB therefore uses **stablecoin-denominated lending** at retail tiers. USDC and USDT dominate consumer DeFi usage; MiCA [R36, R37] and the US GENIUS Act [R38] require 1:1 reserves and issuer authorization for payment stablecoins.

Testnet may start with ETH for simplicity; loan currency is a per-pool parameter. Retail deployment will default to USDC. BIS mBridge [R39] and Project Agora [R40] provide settlement rails without lending hierarchies; CWB is the lending layer that can compose on top.

Three supply measures apply: (1) ERC-20 stablecoin pools per Local Bank; (2) single-chain deployment without live bridges (after major 2022 bridge exploits); (3) four-tier reserve ratios that re-allocate existing stablecoin supply rather than create new money.

Chapter 2

Literature Review

2.1 Preliminaries

Technical terms (DeFi, smart contracts, XAI, PoS, CBDC, group lending, ZKP, Health Factor, etc.) are defined in the Glossary. Platform-specific terms (PSR, Provisional Credit Tier) appear there too. This chapter assumes basic blockchain familiarity from Section 1.5.1.

2.1.1 Review Methodology (PRISMA-Style Frame)

We reviewed literature using a PRISMA-style frame [R22]. We included peer-reviewed work and formal institutional reports (2017–2026) relevant to CWB modules: DeFi lending, ML fraud detection, identity/ZKP, group lending, smart-contract security, institutional finance, regulation, or retail payments. Sources came from ACM, IEEE, Elsevier, MDPI, arXiv (venue-checked), and BIS/World Bank portals. Figure 2.1 shows screening from 892 records to 116 included entries.

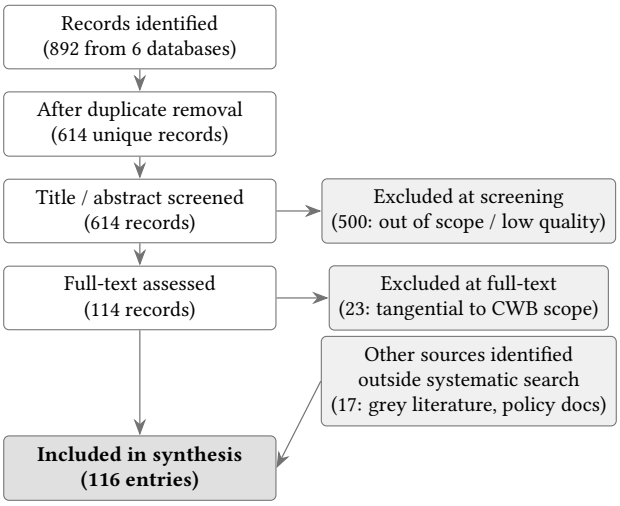


Figure 2.1: PRISMA-style literature screening flow

2.2 Review of Existing Research

Each subsection below links prior work to a CWB design choice or research gap.

2.2.1 DeFi Lending and the Structural Gap

Werner et al. [1] show that DeFi lending is flat, pool-based, and over-collateralized—with no institutional hierarchy. This is the gap Contribution 1 addresses. Bastankhah et al. [2] support dynamic rate models over static Aave/Compound curves, validating CWB's kinked-rate design (Section 3.9). Aspris and Svec [118] describe modular risk curators but not four-tier development finance. Yang and Cui [47] and Dao et al. [60] provide evaluation frameworks for utilization, liquidation, and credit limits in DeFi.

2.2.2 Fraud Detection and Explainable AI

Palaiokrassas et al. [3] show behavioral features raise fraud F1 to 0.76–0.85 on multichain DeFi data—informing CWB's Random Forest plan. BCCC-DeFiFraudTrans-2025 [108] is the target training set (York access request submitted). Liu et al. [6] motivate Isolation Forest for unlabeled anomaly detection. Caldarelli (2025) [123] notes ML-score oracles for lending remain under-researched—supporting Contribution 3.

Adom et al. [4] favor SHAP over LIME for stable loan explanations; Yuan [56] shows SHAP credit models above 0.92 AUC. These inform the Authority Brief dashboard for approvers.

2.2.3 Hierarchical Capital Distribution

Tan [5] and BIS CBDC work [53, 134, 135] describe two-tier central-bank-to-bank-to-user distribution—parallel to CWB's four tiers. Bindseil [133] supports asymmetric upward/downward rate spreads for `UpwardDepositFacility`. Project Pine [131] and Dong et al. [132] validate layered smart-contract factories and cross-tier capital flows in policy research, but no single DeFi spec combines them—the gap in Figure 1.3.

2.2.4 Group Lending and Microfinance

Grameen and BRAC show solidarity groups can repay above 95% without individual collateral, using weekly meetings and field officers that on-chain systems cannot replicate. CWB's `GroupLendingPool` encodes mutual liability in code with conservative default assumptions. Asaduzzaman et al. [54] report up to 60% admin savings from smart-contract microcredit. A 2025 IEEE microloan prototype [156] is the closest technical analogue. Jaffer [119] and Sherratt [120] warn that mutual liability can become coercive off-chain—a risk CWB avoids but at the cost of weaker social monitoring.

2.2.5 Smart Contract Security and Governance

Atzei et al. [7] define core EVM attack classes (reentrancy, access control), informing `ReentrancyGuard` and RBAC. Chaliasos et al. [150] find tools prevented only 8% of real exploit losses—motivating a layered Slither–Mythril–Foundry–Tenderly pipeline (Section 3.19). Flash-loan governance attacks (Beanstalk, $182M) [158] drive CWB's timelock and snapshot voting. MEV risk on liquidations is mitigated via commit–reveal triggers.

2.2.6 Supporting Infrastructure

MiCA [R36, R37] and the GENIUS Act [R38] require stablecoin reserves and issuer authorization—supporting USDC-first retail lending (Section 1.8.3). Beniiche [R1] and Pasdar [R2] frame the oracle problem; Chainlink accuracy varies by chain [157], supporting single-chain deployment with commit–reveal fallback.

BIS/FSB data [21, 25, 33] document correspondent-banking delays and costs. Kim and Kim [55] and Hartmann and Hasan [48] inform gas-cost modeling. Buterin et al. [R30] and MicroSave [R31] motivate on-chain credit passports (Section 3.12). Ozili [161] shows Nigeria's eNaira had 98.5% wallet inactivity—barriers are trust-based, supporting MFI-led onboarding. Sygnum [R21], mBridge [R39], FundsChain [39], and Kinexys [40] show institutional blockchain adoption without retail lending hierarchies.

2.2.7 Zero-Knowledge Proofs for Compliance-Aware DeFi Identity

Recent ACM papers [153, 154] and Piper et al. [R8] show ZKP + DID/VC can enable privacy-preserving KYC. Together with zkAML [R19], this motivates Contribution 4, deferred past MVT (Section 6).

2.2.8 Future Research Directions Scoped

GNN-based fraud detection [R24, 121, R25], federated learning across banks [49, R27, 125], retail payment rails [136–145], and LLM agent risks (hallucination, prompt injection) [R35, R53–R55] are documented in the literature but scoped to Future Work. The MVT agent uses scanning middleware and a human confirmation gate (Section 4.4).

2.3 Summary of Key Findings

Table 2.1 lists the fifteen highest-impact sources and how each maps to a CWB design decision. Table 2.2 compares CWB to major DeFi protocols. Together they support the compound gap claim for RQ1–RQ3.

Table 2.1: Tier-1 literature synthesis: core findings and design decisions.

Author & Year	Core Finding	Design Decision	Phase
Werner et al. 2022 [1]	DeFi lending uniformly flat, pool-based, overcollateralized; no institutional hierarchy	Four-tier architecture as novel design space	Contribution 1
Aspris & Svec 2025 [118]	Modular risk curator is closest prior work; lacks multilateral hierarchy	Boundary of prior work for Contribution 1	Contribution 1
Tan 2023 [5]	Two-tier CBDC distribution (central bank → banks → users)	Four-tier capital flow; client-via-Local-Bank path	Architecture
Project Pine 2025 [131]	Hierarchical smart contract factory for central-bank operations	Layered design for IBLP and SyndicatedLoan	Phase II–III
Palaiokrassas et al. 2024 [3]	Behavioral features: F1 0.76–0.85 vs. 0.08 transactional only	Random Forest feature engineering	Contribution 3
Liu et al. 2008 [6]	Isolation Forest via recursive partitioning; shorter paths = anomalies	Secondary unsupervised model when labels scarce	Contribution 3
Adom et al. 2022 [4]	SHAP deeper and more consistent than LIME on loan approval	SHAP as primary explainability method	Contribution 3
Jaffer [119] / IEEEMicroloan 2025 [156]	Solidarity mechanics + most recent blockchain microloan prototype	GroupLendingPool mutual liability design	Contribution 2
Atzei et al. 2017 [7]	Foundational EVM attack taxonomy	ReentrancyGuard, CEI, RBAC, Solidity 0.8.20	Security
Chaliasos et al. 2024 [150]	Tools prevented only 8% of real-world losses; all reentrancy	Layered pipeline—no single tool sufficient	Security
Qin et al. [158]	Flash-loan governance passes malicious proposals atomically ($182M)	Snapshot voting, 48-h timelock, 60% quorum	Governance
Piper et al. 2025 [R8] + ZKP-KYC [153, 154]	ZKP + SSI + DID enables privacy-preserving KYC	Groth16 KYC circuit, W3C DID/VC choice	Contribution 4
MiCA [R36, R37] + GENIUS [R38]	Stablecoin issuers require 1:1 reserves, authorization, audits	USDC-first denomination as regulatory requirement	Denomination
Chainlink accuracy [157]	Polygon high oracle accuracy vs Ethereum/zkSync under volatility	Single-chain Polygon deployment; oracle fallback	Infrastructure
Gudgeon et al. [R3]	Utilization above 90% causes liquidity crises empirically	Kink point at $U^* = 80\%$ in rate model	Rate model

Feature	Aave	Comp.	Maker	Maple	Gfinch	CWB
Institutional hierarchy	✗	✗	✗	✗	✗	●
Cross-tier capital flow	✗	✗	✗	✗	✗	⧗
Same-tier interbank lending	✗	✗	✗	✗	✗	⧗
Syndicated / co-lending	✗	✗	✗	Part.	Part.	⧗
Upward capital repatriation	✗	✗	✗	✗	✗	⧗
Solidarity group lending	✗	✗	✗	✗	Part.	●
AI/ML fraud detection	✗	✗	✗	Man.	Man.	●
SHAP explainability	✗	✗	✗	✗	✗	●
ZKP KYC compliance	✗	✗	✗	✗	✗	●
Kinked interest rate curve	✓	✓	Gov.	✓	✗	⧗
Role-based access control	Part.	Part.	Gov.	✓	Part.	●
Institutional / L2 focus	✗	✗	✗	✓	Part.	●
TVL / Status (2026)	$26B	$1.4B	$10B	$2.6B	$680M	Planning

Table 2.2: Protocol comparison on eleven architectural dimensions. Competitor columns reflect publicly documented protocol capabilities (2026); CWB columns reflect pre-thesis specification status.

Chapter 3

System Architecture and Design

3.1 Prototype Scope

Table 3.1 records design status only; no public-network deployment at pre-thesis.

Table 3.1: Prototype scope: specification vs. planned implementation (pre-thesis design record). *Voice: specified* = documented design; *planned* = Phase I–IV deliverable. Target chain: Polygon Amoy and/or Ethereum Sepolia.

Feature	Status
Four-tier role system (RBAC)	⧗ Specified
World Bank Reserve contract (Tier 1)	⧗ Specified (Ph. I)
Tier 2 National Bank contracts	⧗ Specified (Ph. I)
Tier 3 Local Bank contracts	⧗ Specified (Ph. I)
Cross-tier fund transfer	● Planned (Ph. II)
Loan request / approval workflow	⧗ Specified (Ph. II)
Installment EMI auto-generation	● Planned (Ph. II)
SavingsVault / FixedDeposit contracts	● Planned (Ph. II)
GroupLendingPool	● Planned (Ph. II)
InterBankLendingPool	● Planned (Ph. II)
AI/ML fraud detection (Random Forest)	● Planned (Ph. III)
SHAP + anomaly explainability	● Planned (Ph. III)
Oracle (Chainlink Functions)	● Stretch (Ph. III)
ZKP KYC compliance layer	● Future Work
Testnet deployment (Amoy / Sepolia)	● Planned (Ph. IV)
AI Agent / MCP (minimal MVT)	● Planned (Ph. III; retail only)
Full MCP tool server (17 tools)	● Future Work
Bank-authority AI agent	● Future Work
EIP-7702 session keys	● Future Work
Authority Brief UI (SHAP)	● Planned (Ph. III)
Chainlink Automation / Proof of Reserve	● Planned (Ph. I–II)
PostgreSQL event listener	● Planned (Ph. I–II)
SAR workflow (AML → freeze)	● Planned (Ph. III)
`sessions` / `agent_action_log` schema	⧗ Specified (Ph. I–III)
300-client Foundry simulation	● Planned (Ph. IV)

Prototype deployment note. **Prototype implementation target:** Polygon Amoy PoS and/or Ethereum Sepolia (single-chain, stability-first). **Design preference (comparison only):** Polygon zkEVM offers ZK-validity settlement anchored to Ethereum L1 and is evaluated in the platform selection tables (Section 1.5.1, item E) but is not the default narrative deployment chain in gates, MVT items, or phase deliverables unless explicitly marked as a design reference.

3.2 High-Level Architecture

The system employs a **four-layer decentralised application architecture** (Figure 3.1): a presentation layer (React + wallet integration), a smart-contract layer (fifteen modular EVM contracts centred on the four-tier lending hierarchy), an off-chain services layer (Express REST API, PostgreSQL, FastAPI ML scoring, Chainlink oracle integration, event listener, and Redis cache), and a Chainlink infrastructure layer (Chainlink Functions, Automation, Price Feeds, and Proof of Reserve). A planned Phase III conversational agent (Qwen3-8B + MCP tool server) attaches to the same API endpoints as the React UI and does not alter on-chain logic. Figure 3.2 elaborates this view with external integrations.

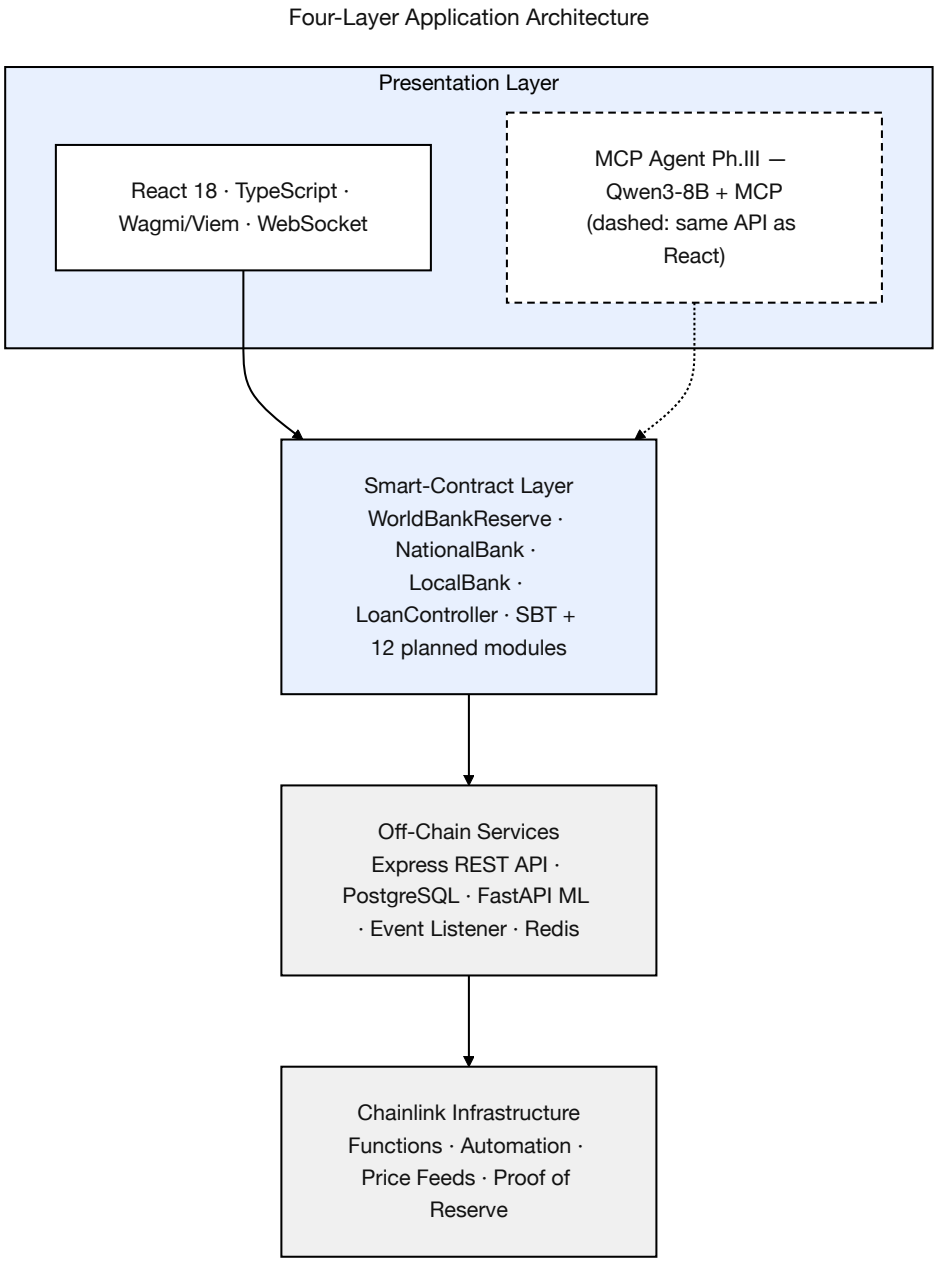


Figure 3.1: Four-layer application architecture

Figure 3.2 shows component interactions. Core lending contracts are specified in Phase I; extended modules are planned extensions documented in Appendix B.

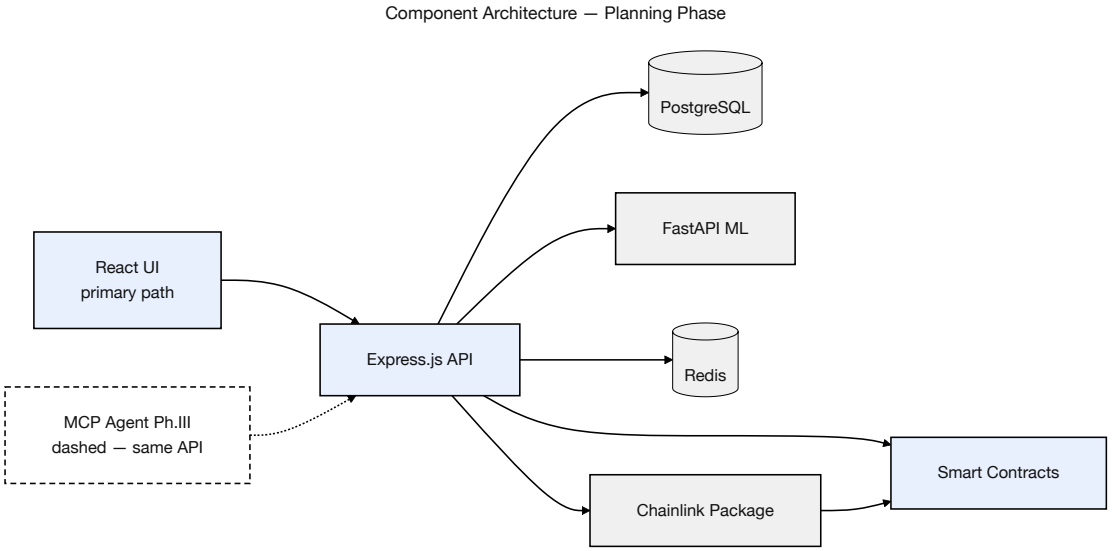


Figure 3.2: Component diagram (planning phase)

Contract	Module	Status	Primary responsibility
WorldBankReserve	Tier 1 core	Specified (Phase I)	Global reserve custody, national-bank registration, downward capital allocation.
NationalBank	Tier 2 core	Specified (Phase I)	Local-bank registration, upward borrowing, downward allocation.
LocalBank	Tier 3 core	Specified (Phase I)	Client onboarding, loan lifecycle, installments, `freezeAccount`.
SavingsVault	Deposits	Specified (Phase II)	Variable-yield savings, reserve-gated withdrawals.
FixedDeposit	Deposits	Specified (Phase II)	Term deposits with deterministic early-withdrawal penalty.
GroupLendingPool	Group credit	Specified (Phase II)	Solidarity-group formation, consent, mutual liability.
FXModule	Treasury / FX	Specified (Phase II)	Oracle-priced retail FX with disclosed spread.
InsuranceFund	Risk buffer	Specified (Phase II)	Default coverage buffer; Chainlink Proof of Reserve.
CurrentAccount	Payments	Specified (Phase II)	Transactional accounts and atomic peer transfers.
InterBankLendingPool	Multi-entity	Specified (Phase II)	Same-tier interbank liquidity pools per tier.
UpwardDepositFacilit	Multi-entity	Specified (Phase II)	Upward surplus repatriation with yield spread.
SyndicatedLoan	Multi-entity	Planned (Phase III)	Multi-lender syndicate consent and disbursement.
TranchedPool	Multi-entity	Planned (Phase III)	Senior/junior tranching and waterfall recovery.
TreasurySwap	Multi-entity	Specified (Phase II)	Institutional oracle-priced asset swaps.
NettingEngine	Multi-entity	Future Work	ERD stubs only; Section 6

3.3 Blockchain Platform Selection

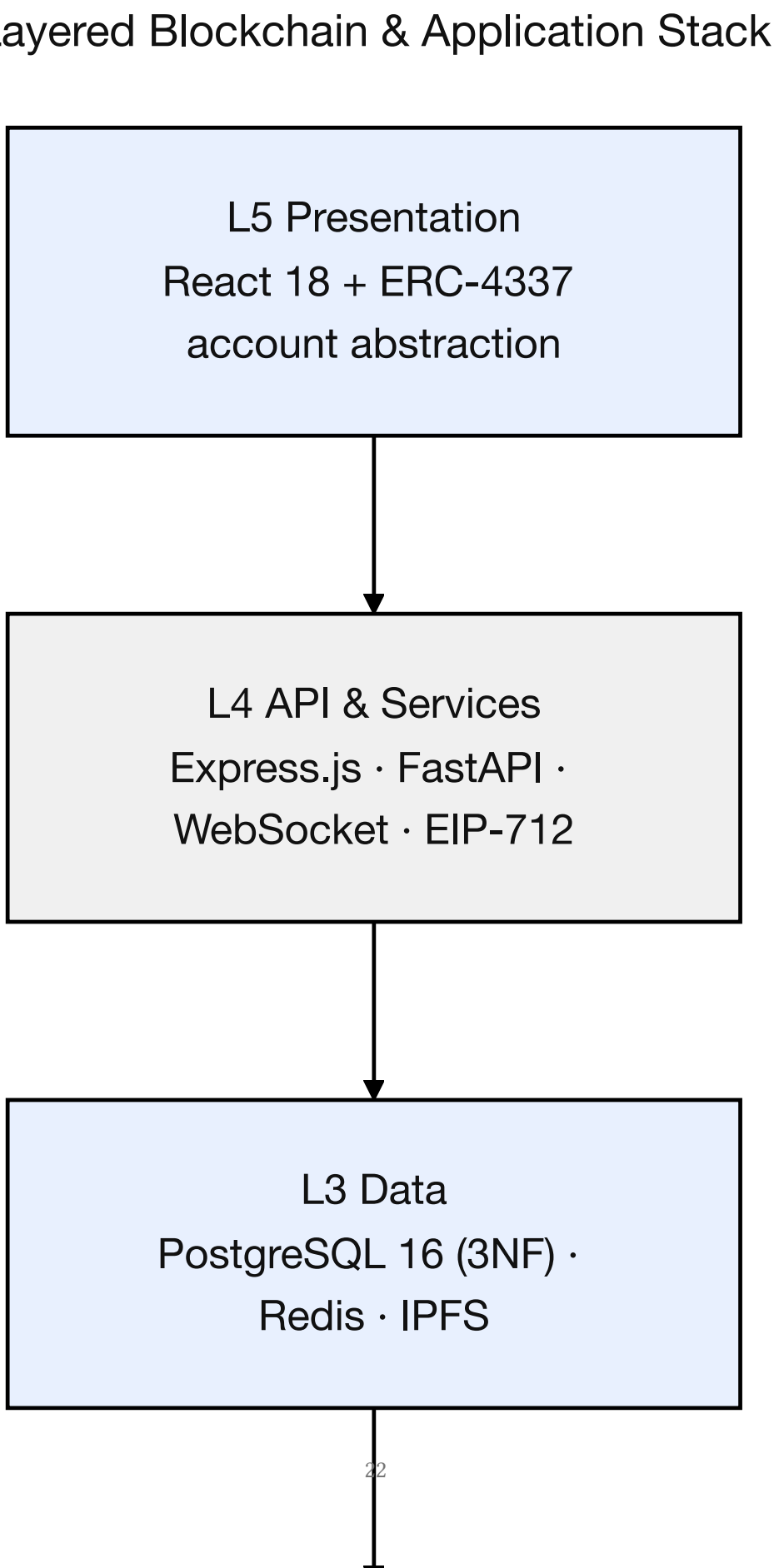


Table 3.3: Blockchain platform selection criteria and justification.

Criterion	Selection	Justification
Platform	Ethereum Virtual Machine (EVM)	Largest EVM ecosystem; mature tooling.
Network	Polygon Amoy PoS and/or Ethereum Sepolia (prototype implementation)	Free public testnets; zkEVM evaluated, not default prototype chain.
Consensus	Public-testnet consensus (PoS/L1 testnet)	Stable testnet consensus; ZK settlement is a design preference only.
Smart contract language	Solidity 0.8.20	Industry standard; overflow-safe compiler; rich libraries.

This platform selection table compares candidate networks on cost, finality, throughput, and ecosystem maturity. Polygon zkEVM remains a design preference for ZK-anchored settlement, but the prototype implementation target is Amoy/Sepolia for stability-first delivery within the thesis timeline.

Table 3.5: Blockchain platform selection (continued): operational and deployment factors.

Criterion	Selection	Justification
Gas cost	\$0.001–\$0.01 per transaction	Far below mainnet; viable for microloans.
Block finality	≈2 seconds	Fast retail UX; L1-anchored security.
Developer tooling	Hardhat + OpenZeppelin	Hardhat tests; audited OpenZeppelin libs.
Testnet availability	Amoy (Polygon PoS), Sepolia (Ethereum)	Free faucets; EVM-identical; no real funds.
Security model	Public-testnet security model	Testnet PoS; ZK settlement evaluated separately.
Migration path	EVM-compatible	Portable Solidity across EVM chains.

This supplementary comparison expands the platform evaluation to additional operational and deployment factors. The conclusion is that Polygon zkEVM Cardona provides a practical balance of ZK-secured settlement and low transaction costs for an academic prototype, with a clear migration path if future requirements demand alternative L2s.

3.3.1 Transaction Verification and Consensus

- **Polygon zkEVM Cardona:** ZK validity proofs anchor batches to Ethereum L1 (cryptographic security vs. PoS validator assumption on Amoy).
- **On prototype testnets:** Amoy and Sepolia provide free public testnets for incremental development and testing without exposure to real-asset risk.

Gas-cost sustainability. The hierarchical loan lifecycle generates multiple on-chain state transitions per client; on Polygon zkEVM Cardona the **projected** cost remains well below 0.01% of interest earned on a representative retail loan (Section 5.8), to be validated by the Phase IV Foundry simulation. This margin underpins the micro-loan economics that motivate Layer 2 deployment in the platform selection above.

3.3.2 Oracle Architecture: Off-Chain AI to On-Chain Decision

The Crypto World Bank requires a mechanism to convey off-chain AI/ML risk assessments into the on-chain loan approval workflow. This is an instance of the oracle problem [R1,R2]. The graded MVT oracle path is **commit–reveal**, with Chainlink Functions retained as a stretch upgrade when supported on the chosen testnet.

Commit-reveal relay (MVT oracle path). Phases I–III use a **commit–reveal relay** as the primary oracle mechanism for ML risk scores. The off-chain ML service commits a hash $h = \text{keccak256}(s \,\|\, \text{nonce})$ to the `LoanController` contract, then reveals s and the nonce within the decision window. The contract verifies $h = \text{keccak256}(s \,\|\, \text{nonce})$ and stores s immutably. This pattern prevents score manipulation between commitment and decision and creates an immutable on-chain audit trail of every risk score used in a lending decision. A centralized relay (trusted backend calling `updateRiskScore`) remains a Phase I–II integration aid only.

Chainlink Functions oracle (stretch / future work). **Chainlink Functions** is documented as a production-grade upgrade path when DON support is available on the chosen testnet. Chainlink Functions operates via a Decentralised Oracle Network (DON): the ML risk score is fetched independently by multiple Chainlink nodes, consensus across the DON is required before the result is committed on-chain, and no single compromised node can manipulate the risk score. The source code executed by the DON is:

```
// Chainlink Functions source (runs in decentralised DON)
const clientId = args[0];
const response = await Functions.makeHttpRequest({
  url: `https://your-ml-service.com/score/${clientId}`,
  method: "POST"
});
const score = response.data.risk_score;
// 6 decimal precision
return Functions.encodeUint256(Math.round(score * 1e6));
```

The DON adds 30–60 seconds of latency and $0.10–$1.00 per oracle call, both acceptable for the loan approval lifecycle where decisions are not time-critical. At MVT scope, commit–reveal remains authoritative; Chainlink Functions is a **stretch** deliverable (DT-III.01, Should) when supported on Amoy/Sepolia.

Chainlink Automation: installment triggers (automation upgrade path). **Chainlink Automation** replaces the centralised cron job for overdue installment detection and interest accrual. The entire loan lifecycle from application to overdue flagging runs without a trusted operator:

```
// In LocalBankPool.sol
```

```
function checkUpkeep(bytes calldata) external view override
    returns (bool upkeepNeeded, bytes memory performData) {
    uint256[] memory overdueLoans = getOverdueLoans();
    upkeepNeeded = overdueLoans.length > 0;
    performData = abi.encode(overdueLoans);
}

function performUpkeep(bytes calldata performData) external override {
    uint256[] memory overdueLoans =
        abi.decode(performData, (uint256[]));
    for (uint i = 0; i < overdueLoans.length; i++) {
        _markInstallmentOverdue(overdueLoans[i]);
    }
}
```

This reduces reliance on unilateral off-chain cron execution for overdue detection by making installment-trigger checks externally verifiable through an on-chain upkeep interface. The broader system still includes licensed/operator-controlled off-chain services for KYC and risk analytics (Section 1.5.1).

Chainlink Price Feeds: fiat and crypto pairs. The `FXModule` contract uses Chainlink's production-grade `AggregatorV3Interface` price feeds for ETH/USD, USDC/USD, and governance-selected fiat pairs (e.g. EUR/USD), replacing the earlier "forex oracle approved by governance" pattern. Optional fiat-equivalent display in the frontend flows through:

```
// In LocalBankPool.sol
AggregatorV3Interface internal fiatUsdFeed;  // e.g. EUR/USD
AggregatorV3Interface internal ethUsdFeed;

function getFiatEquivalent(uint256 usdcAmount)
    public view returns (uint256) {
    (, int fiatRate,,,) = fiatUsdFeed.latestRoundData();
    // USDC tracks USD 1:1; Chainlink uses 8 decimals
    return usdcAmount * uint256(fiatRate) / 1e8;
}
```

The 8-decimal precision convention is the Chainlink standard for fiat-pair feeds. Chainlink price feeds are a planned Phase I integration for optional fiat-equivalent display in the frontend.

Chainlink Proof of Reserve. The `WorldBankReserve` contract publishes its reserve balance to **Chainlink Proof of Reserve (PoR)**, making the reserve cryptographically verifiable by any external auditor without trusting CWB's administrator. This is the "free PoR" advantage over FTX: any external system can verify reserve solvency without relying on CWB's admin claims.

```
// WorldBankReserve.sol
function getReserveSummary() external view returns (
```

```
    uint256 totalDeposited,
    uint256 totalLoaned,
    uint256 reserveRatio,   // scaled 1e4 (e.g. 5000 = 50%)
    uint256 insuranceFundBalance
) {
    return (
        totalDeposited,
        totalLoaned,
        (totalDeposited - totalLoaned) * 1e4 / totalDeposited,
        insuranceFund.balance()
    );
}
```

This function is the FTX commingling safeguard described in Section 3.19. Adding Chainlink's PoR job on top turns it into a market-standard verifiable proof, and it is specified in full in Appendix D.

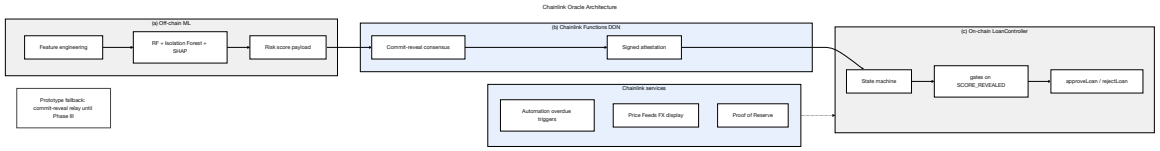


Figure 3.4: Oracle architecture

3.4 Data Model and Database Design

The relational database schema comprises 51 normalized entities in Third Normal Form (3NF): 31 core prototype entities (Table 3.7), six primary extension entities (Table 3.12), and 14 Phase II–III banking-product and multi-entity entity stubs (Table 3.14). Core entities are Phase I MVT-complete specifications; stub entities carry PK, FK anchors, and one-sentence roles in Table 3.14 and are fully attribute-specified in Section 3.14. Figure 3.5 presents the core entity-relationship overview, Figure 3.6 presents extended product and multi-entity entities, and Figure 3.7 presents the Enhanced Entity-Relationship (EER) diagram with specialisation and weak-entity constructs.

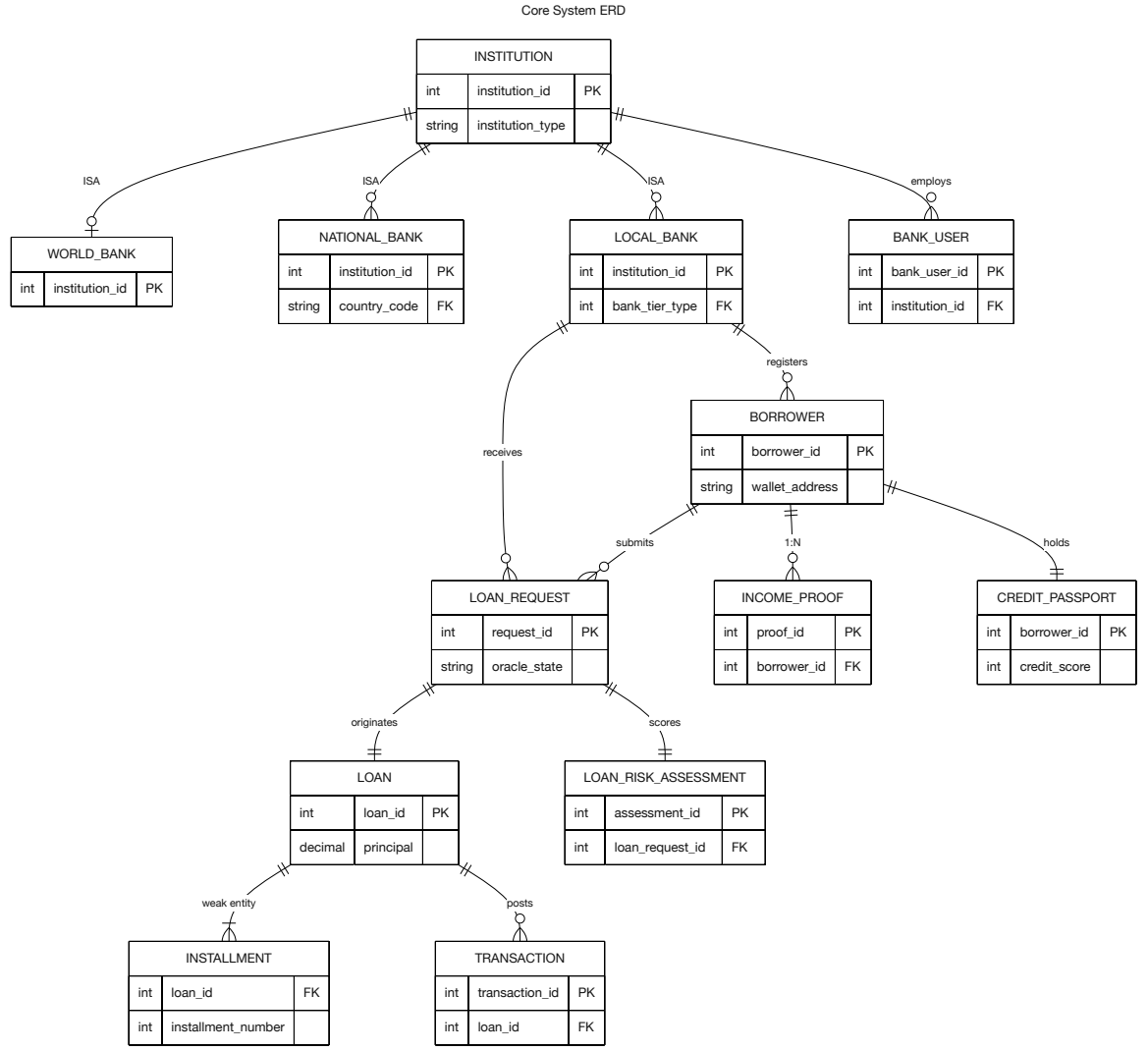


Figure 3.5: Core system entity graph

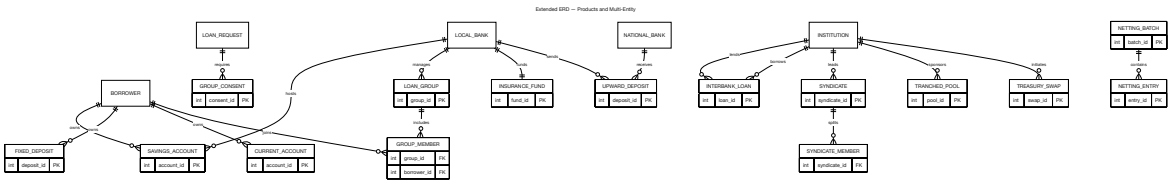


Figure 3.6: Extended ERD entities

The current ERD covers the core lending and governance entities. Figure 3.6 presents the extended banking entities (SavingsAccount, FixedDeposit, LoanGroup, GroupMember, CurrentAccount, and InsuranceFund) and the multi-entity / cross-tier operational entities described in Section 3.14.7: `INTERBANK_LOAN`, `UPWARD_DEPOSIT`, `SYNDICATE` and `SYNDICATE_MEMBER`, `TRANCHED_POOL`, `TREASURY_SWAP`, and `NETTING_BATCH` with `NETTING_ENTRY`. Their foreign-key relationships into the core lending entities (LOAN, BANK_USER, INSTALLMENT, TRANSACTION) are detailed in Table 3.27 in Section 3.14.7. The data model remains in one-to-one consistency with the contract architecture and implementation phase plan.

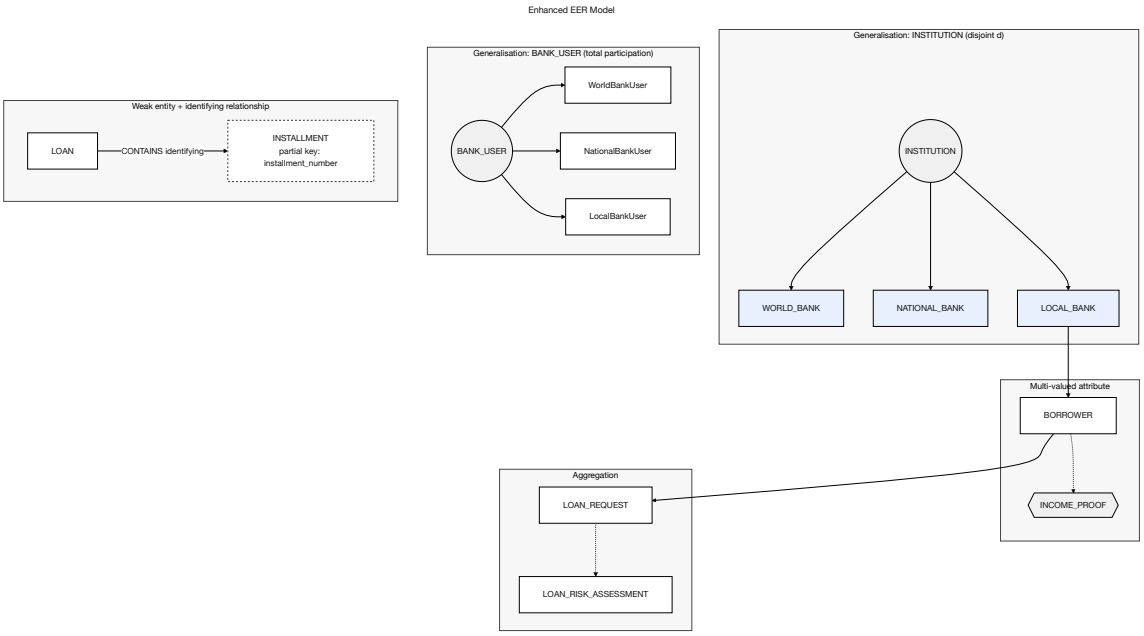


Figure 3.7: Enhanced EER model

3.4.1 Entity Summary

Table 3.7: Database entity summary (31 core entities): primary keys and roles.

Entity	Primary key	Role / Description
INSTITUTION	`institution_id`	Shared supertype for all banking-tier identities. Attributes: `institution_type` `ENUM('WORLD','NATIONAL','LOCAL')`, `name`, `wallet_address`, `contract_address`, `status`, `created_at`. All multi-entity tables (`INTERBANK_LOAN`, `NETTING_ENTRY`, …) reference this PK for enforceable cross-tier FKs.
COUNTRY	`country_code`	ISO 3166-1 alpha-2 reference (PK). Attributes: `name`, `default_stablecoin_symbol`, `regulatory_regime_tag`.
WORLD_BANK	`institution_id`	Subtype of `INSTITUTION` (`institution_type='WORLD'`); PK is also FK → `INSTITUTION`. Singleton enforced: `CHECK (institution_id = 1)` plus no-delete trigger. Global reserve parameters; on-chain binding: `chain_id`, `last_synced_block`, `deployment_tx_hash`.
NATIONAL_BANK	`institution_id`	Subtype of `INSTITUTION` (`institution_type='NATIONAL'`); FK `country_code` → `COUNTRY` with `UNIQUE` (one national reserve per nation). FK `parent_world_bank_id` → `WORLD_BANK`.
LOCAL_BANK	`institution_id`	Subtype of `INSTITUTION` (`institution_type='LOCAL'`); FK `national_bank_id` → `NATIONAL_BANK`. City-level banks lending to retail clients.
BANK_USER	`bank_user_id`	Bank staff with role-based permissions. `bank_type` `ENUM('world','national','local')`; nullable FKs `world_bank_id`, `national_bank_id`, `local_bank_id` with CHECK enforcing exactly one parent institution (Table 3.18).
BORROWER	`borrower_id`	End borrowers (UK: `wallet_address`). FK `registered_local_bank_id` → `LOCAL_BANK` (NOT NULL, set at onboarding) records the client's home Local Bank for KYC attribution and dashboard scoping; `LOAN_REQUEST` remains the separate M:N association for banks at which the client has applied.[1]
LOAN_REQUEST	`request_id`	Loan applications and approval workflow. `status` (lifecycle): `DRAFT`, `SUBMITTED`, `UNDER_REVIEW`, `APPROVED`, `REJECTED`, `DISBURSED`, `CANCELLED`. `oracle_state` (event-listener only): `NONE`, `COMMIT_PENDING`, `COMMITTED`, `DON_REQUESTED`, `SCORE_REVEALED`. FKs: `submission_event_log_id`, `approval_event_log_id`; `on_chain_request_id`.

Table 3.9: Database entity summary (continued): auxiliary, analytics, and profile entities.

Entity	Primary key	Role / Description
INCOME_PROOF	proof_id	Income verification documents (multi-valued per borrower).
CHAT_MESSAGE	message_id	Human-to-human client↔bank messages in the loan workflow. FK to LOAN_REQUEST and/or LOAN. Not agent-specific; not for inter-tier authority communication (see INSTITUTION_MESSAGE).
INSTITUTION_MESSAGE	message_id	Cross-tier / cross-authority communication audit trail. FK sender_institution_id, receiver_institution_id → INSTITUTION; message_type (SAR_ESCALATION, SYNDICATION_INVITE, RESERVE_ALERT, IBLP_REQUEST, …), payload (JSONB), status, created_at, read_at.
AGENT_CONVERSATION_TURN	turn_id	Each turn of the Qwen3-8B agent conversation (Phase III). FK session_id → SESSIONS. Stores role (user/assistant/system), content_summary, tokens_used, tool_calls (JSONB). Separate from AGENT_ACTION_LOG (write-tool execution only).
MODEL_REGISTRY	model_id	ML model lineage for audit and explainability. Attributes: model_name, version, training_date, metrics (JSONB), artifact_hash, is_active. Referenced by LOAN_RISK_ASSESSMENT.
LOAN_RISK_ASSESSMENT	assessment_id	Per-loan ML inference record: FK loan_request_id → LOAN_REQUEST; FK model_id → MODEL_REGISTRY; fraud_probability, anomaly_score, shap_vector (JSONB), feature_vector_hash, created_at.
SECURITY_EVENT_LOG	event_id	System-wide security and AML events (append-only). Attributes: event_type, actor_id, actor_type, severity, payload (JSONB), created_at. INSERT-only RLS policy; distinct from per-loan LOAN_RISK_ASSESSMENT.
MARKET_DATA	market_data_id	Non-Chainlink price cache (e.g. CoinGecko fallback). Read-only auxiliary; no FK into core lending. Key attributes: symbol, price_usd, source, recorded_at. Chainlink rounds are stored in CHAINLINK_PRICE_ROUND.
CHAINLINK_PRICE_ROUND	round_id	Chainlink AggregatorV3Interface round cache: feed_address, pair_symbol, answer, round_id_on_chain, answered_in_round, updated_at, chain_id. Required for staleness check (answeredInRound < roundId).
PROFILE_SET	profile_id	Interface preferences for borrowers and bank

Table 3.10: Database entity summary (continued): agent session and reference entities.

Entity	Primary key	Role / Description
SESSIONS	session_id	Wallet session registry for the **retail-client** conversational agent (Phase III Must): FK borrower_id → BORROWER; device/IP hash, EIP-7702 session-key scope and TTL, parent lineage, compression summary, end reason. Bank-authority agent sessions (actor_type='BANK_USER') are Future Work (Section 3.1). Phase II: SESSION_ANCESTOR closure table (Section 3.4.6).
AGENT_ACTION_LOG	action_id	Append-only agent write-tool log (session_id, tool_name, parameters, confirmation_turn_id, onchain_tx_hash, status). Separate from AGENT_CONVERSATION_TURN; INSERT-only RLS policy.
INTEREST_RATE_TIER	tier_id	Normalised kinked-rate parameters (base_rate, kink_utilisation, rate_above_kink, max_rate); Section 3.4.4. Discriminator: bank_tier_type ENUM('world','national','local'). Temporal validity: effective_from, effective_to, governance_proposal_id, superseded_by, is_active. Partial unique: (bank_tier_type) WHERE is_active = TRUE.
ASSETS	asset_id	Collateral and loan asset registry (UK: symbol); referenced by LOAN via collateral_asset_id and loan_asset_id.

This entity summary table enumerates 31 core relational entities. Key additions beyond the original lending core include the INSTITUTION supertype and COUNTRY reference (shared identity for cross-tier operations), INSTITUTION_MESSAGE (inter-tier authority communication), split ML tables (MODEL_REGISTRY, LOAN_RISK_ASSESSMENT, SECURITY_EVENT_LOG), registered_local_bank_id on BORROWER, and institution-aware TRANSACTION columns. Six extension entities (Table 3.12) and 14 Phase II–III stubs (Table 3.14) complete the schema to 51 total.

Table 3.12: Extension entities (Phase II–III): primary keys and roles.

Entity	Primary key	Role / Description
GROUP_CONSENT	consent_id	Per-member multi-signature consent for GroupLendingPool (UK: loan_request_id, member_id). consent_tx_hash, event_log_id.
TRANCHED_POOL_SUBSCRIPTION	subscription_id	Investor tranche positions: pool_id, subscriber_id, subscriber_type, tranche (SENIOR/JUNIOR), committed_amount, subscription_tx_hash.
NETTING_COORDINATOR_STATE	batch_id	Off-chain Settlement Coordinator state: status (COMPUTING → SETTLED), gross_obligations, net_settlements (JSONB), challenge_deadline, on_chain_batch_id.
KYC_VERIFICATION_REQUEST	request_id	ZKP KYC workflow tracker: borrower_id, kyc_level_requested, status, credential_hash, proof_hash, on_chain_tx_hash.
ORACLE_HEARTBEAT_MONITOR	monitor_id	Chainlink feed staleness monitor: feed_address, pair_symbol, expected_heartbeat_seconds, last_round_id, is_stale, chain_id.
SESSION_ANCESTOR	session_id, ancestor_id	Closure table for session lineage (replaces recursive INSERT trigger). depth (1 = direct parent).

Table 3.14: Extended banking and multi-entity entity stubs (Phase II–III).

Entity	Primary Key	Role / FK anchors
SAVINGS_ACCOUNT	account_id	SavingsVault deposit account. FK borrower_id → BORROWER; FK local_bank_id → LOCAL_BANK(institution_id). on_chain_address, balance, last_synced_block.
FIXED_DEPOSIT	deposit_id	FixedDeposit term product. FK borrower_id; FK local_bank_id → LOCAL_BANK(institution_id). principal, maturity_date, on_chain_address.
CURRENT_ACCOUNT	ca_id	Transactional account. FK borrower_id; FK local_bank_id → LOCAL_BANK(institution_id). balance, on_chain_address.
LOAN_GROUP	group_id	GroupLendingPool on-chain group. FK local_bank_id → LOCAL_BANK(institution_id). on_chain_group_id, max_members, status.
GROUP_MEMBER	member_id	Weak entity on LOAN_GROUP. FK group_id; FK borrower_id. joined_at, consent_tx_hash.
INSURANCE_FUND	fund_id	InsuranceFund reserve tracker. FK local_bank_id → LOCAL_BANK(institution_id). balance, on_chain_address, last_synced_block.
INTERBANK_LOAN	ib_loan_id	IBLP same-tier liquidity loan. FK lender_institution_id, borrower_institution_id → INSTITUTION; trigger/CHECK enforces matching institution_type per pool instance (IBLP_NB or IBLP_LB). tenor_days, amount, status, on_chain_tx_hash.
UPWARD_DEPOSIT	ud_id	UpwardDepositFacility surplus deposit. FK depositor_institution_id, parent_institution_id → INSTITUTION; CHECK enforces adjacent tier-pair (Local→National or National→World). amount, rate, on_chain_tx_hash.
SYNDICATE	syndicate_id	SyndicatedLoan deal header. FK lead_institution_id → INSTITUTION; FK borrower_id → BORROWER. total_amount, status, on_chain_address.
SYNDICATE_MEMBER	syndicate_id, member_institution_id	Co-lender position. FK member_institution_id → INSTITUTION. committed_amount, share_bps, consent_tx_hash.
TRANCHED_POOL	pool_id	TranchedPool deal header. FK sponsor_institution_id → INSTITUTION.

Extended banking entities in Figure 3.6 (SAVINGS_ACCOUNT, FIXED_DEPOSIT, LOAN_GROUP, ...) include planned on-chain anchor fields on_chain_address and last_synced_block so the event listener can map incoming events to the correct DB row. Multi-entity stub FKs anchor to INSTITUTION.institution_id (Table 3.14) rather than tier-specific PK namespaces, enabling one enforceable referential-integrity path for InterBankLendingPool, NettingEngine, and related contracts. Phase III adds SESSION_KEY_PERMISSION to normalise MCP tool allowlists currently stored as JSONB in SESSIONS.

3.4.2 Institution Supertype and Cross-Tier Referential Integrity

The four-tier hierarchy (World Bank $\rightarrow n$ National Banks $\rightarrow n$ Local Banks per nation $\rightarrow$ clients) requires a shared institution identity space for multi-entity operations specified in Section 3.14. Rather than three disjoint PK namespaces (`world_bank_id`, `national_bank_id`, `local_bank_id`), the schema introduces an `INSTITUTION` supertype with ID-based table-per-type inheritance: `WORLD_BANK`, `NATIONAL_BANK`, and `LOCAL_BANK` are subtype tables whose primary key is also a foreign key to `INSTITUTION.institution_id`. This pattern remains 3NF/BCNF-safe (shared identity, not shared attribute duplication) and allows `INTERBANK_LOAN`, `SYNDICATE_MEMBER`, `NETTING_ENTRY`, and `TREASURY_SWAP` to reference any banking-tier participant through a single FK. Application triggers or `CHECK` constraints enforce tier-appropriate pairings—for example, both rows in an `IBLP_NB` loan must have `institution_type='NATIONAL'`.

Two cardinality rules implied by the architecture are encoded at the database layer. **World Bank singleton:** `WORLD_BANK` permits exactly one row (`CHECK (institution_id = 1)`), mirroring the singleton `WorldBankReserve` contract. **One national reserve per nation:** `NATIONAL_BANK.country_code` references `COUNTRY` with a `UNIQUE` constraint, guaranteeing at most one national-bank institution per ISO jurisdiction and providing a clean join key for FX, stablecoin defaults, and regulatory-sandbox reporting.

Retail clients retain a permanent home-bank link via `BORROWER.registered_local_bank_id` (set at onboarding, NOT NULL), independent of whether they have submitted a `LOAN_REQUEST`. The `LOAN_REQUEST` entity continues to model the many-to-many "which Local Banks has this client borrowed from" relationship required by the portable Credit Passport design.

Inter-tier authority communication (SAR escalation, syndication invites, reserve alerts) is recorded in `INSTITUTION_MESSAGE`, distinct from client-facing `CHAT_MESSAGE`. ML inference and security monitoring are split at Phase I into `LOAN_RISK_ASSESSMENT` (per-loan, SHAP-linked, FK to `MODEL_REGISTRY`) and `SECURITY_EVENT_LOG` (system-wide, append-only), avoiding sparse-column conflation in a single aggregate table.

Multi-tenant row-level security (Future Work). Append-only RLS on `AGENT_ACTION_LOG` and `AUDIT_LOGS` is specified in Table 3.18. For a global deployment with jurisdictional separation, production would extend RLS (or physical sharding) to scope PII-bearing core tables—`BORROWER`, `INCOME_PROOF`, `LOAN`, `LOAN_REQUEST`—by `registered_local_bank_id` ancestry through `LOCAL_BANK` $\rightarrow$ `NATIONAL_BANK` $\rightarrow$ `COUNTRY`, preventing a National Bank Admin in one jurisdiction from querying another nation's client rows. This tenant-isolation layer is explicitly deferred to the final-thesis implementation phase.

3.4.3 EER Constructs Applied

Table 3.16: EER constructs applied: specialisation, hierarchy, and constraints.

EER Construct	Applied To	Notes
Specialisation (disjoint)	BANK_USER → WorldBankUser, NationalBankUser, LocalBankUser	A bank user belongs to exactly one bank type (`world`, `national`, or `local`); enforced by CHECK constraint (Table 3.18).
Generalisation / specialisation	INSTITUTION → WORLD_BANK, NATIONAL_BANK, LOCAL_BANK	ID-based table-per-type inheritance: subtype PK = FK to `INSTITUTION.institution_id`; shared identity for cross-tier FKs (Section 3.4.2).
Weak entity	INSTALLMENT depends on LOAN	Existence and identity determined by parent loan.
Multi-valued attribute	INCOME_PROOF per BORROWER	Modelled as separate entity to maintain 1NF.
Aggregation	LOAN_RISK_ASSESSMENT ← LOAN_REQUEST; SECURITY_EVENT_LOG ← runtime monitors	Per-loan ML scores and system-wide security events are separate append-only relations, not a conflated aggregate.
Association entity	LOAN_REQUEST links BORROWER + LOCAL_BANK	Captures many-to-many borrowing relationships; distinct from `BORROWER.registered_local_bank_id` (home bank at onboarding).
Participation constraint	BORROWER.`registe` `local_bank_id` NOT NULL at onboarding	Home Local Bank is mandatory; `LOAN_REQUEST` participation for lending services enforced at application layer.
Append-only table (policy)	AGENT_ACTION_LOG, AUDIT_LOGS, SECURITY_EVENT_LOG	DB-level INSERT-only role + Row-Level Security policy enforces that no UPDATE or DELETE is permitted on audit records.

This table captures the EER constructs applied to represent specialization, hierarchy, and constraints in the data model. It demonstrates how institutional roles and banking participants are modeled without duplicating attributes across tables, improving data integrity and query clarity.

3.4.4 Normalization

The schema has been normalized to Third Normal Form (3NF) and checked against Boyce-Codd Normal Form (BCNF):

- **1NF:** There are no repeating groups in any of the attributes. Separate entities store multi-valued attributes, such as proof of income.
- **2NF:** There are no partial dependencies. The full composite key (`loan_id`, `installment_number`) determines the non-key attributes of INSTALLMENT.
- **3NF:** No dependencies that go through other dependencies. Instead of storing redundant values like `total_borrowed` in bank entities, they are calculated at query time.
- **BCNF:** All non-trivial functional dependencies have a superkey as their determinant. For example, in `INTEREST_RATE_TIER`, `tier_id` $\rightarrow$ {`base_rate`, `kink_utilisation`, `rate_above_kink`, `max_rate`}: the determinant `tier_id` is the primary key, satisfying BCNF. Prior to the normalisation fix in this section, interest-rate parameters were columns on the three bank entities, determined by `bank_tier_id` (a non-key attribute), violating BCNF.

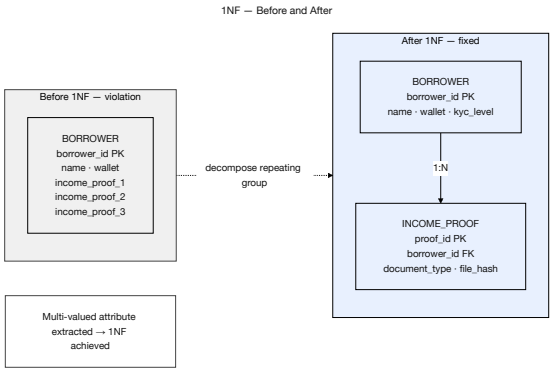


Figure 3.8: First normal form (1NF) demonstration

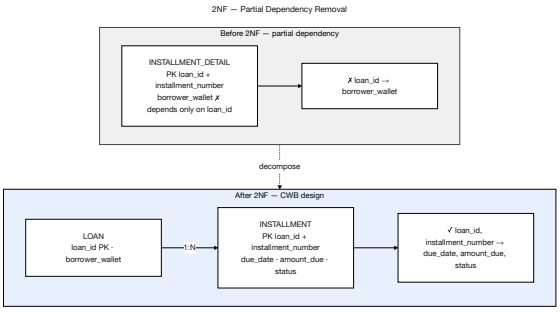


Figure 3.9: Second normal form (2NF) demonstration

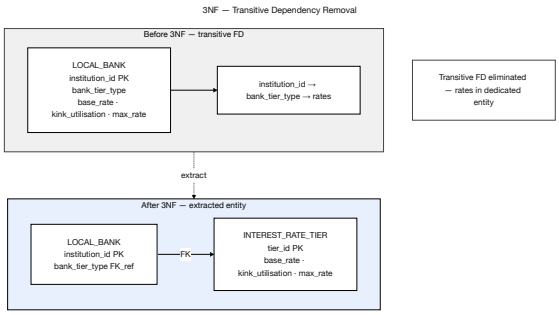


Figure 3.10: Third normal form (3NF) demonstration

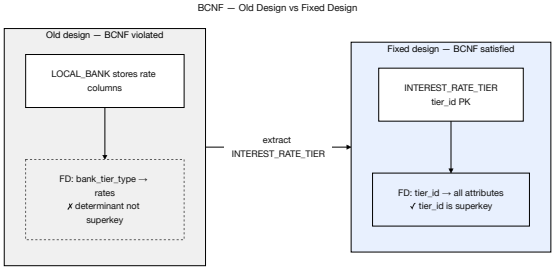


Figure 3.11: BCNF verification on INTEREST_RATE_TIER

A transitive dependency was identified during schema review: `interest_rate_parameters` → `bank_tier_id` (interest rate values were stored as columns in the World Bank, National Bank, and Local Bank entities, making them dependent on a non-key attribute via the tier relationship). This violates 3NF. The fix is to extract these parameters into a dedicated `INTEREST_RATE_TIER` table keyed by `tier_id`, which each bank entity references via FK. This normalisation change ensures that updating a base rate does not require touching multiple bank-tier rows.

3.4.5 Physical Schema Reference

Indexing strategy, representative functional dependencies, the `mv_client_dashboard` materialized view, and projection-table CQRS policies are documented in Appendix A to keep this chapter focused on conceptual design. B-tree indexes are chosen for range queries on rolling borrowing-limit windows and overdue-installment detection; the authoritative enforcement of borrowing limits remains on-chain.

The `parent_session_id` self-referential foreign key in `SESSIONS` introduces a recursive relationship within a single entity. This does not violate 3NF (the dependency is `session_id` → `parent_session_id`, not a transitive dependency), but it requires an acyclicity constraint to prevent circular ancestry chains: a session cannot be its own ancestor. In the prototype this is enforced by a `CHECK` constraint in conjunction with a trigger that walks the ancestry chain before INSERT. Phase II replaces this recursive trigger with a **closure table** (`SESSION_ANCESTOR`): on INSERT of a new session, rows are copied from the parent's ancestor set with `depth + 1`; the cycle check becomes a single indexed lookup `WHERE ancestor_id = NEW.session_id` (Section 3.4.6).

3.4.6 Relational Integrity Constraints

Table 3.18: Relational integrity constraints.

Constraint Type	Examples
Primary Key	One surrogate or composite key per Table 3.7, Table 3.12, and Table 3.14: e.g. institution_id, country_code, bank_user_id, borrower_id, request_id (LOAN_REQUEST), loan_id (LOAN), (loan_id, installment_number) (INSTALLMENT), transaction_id, model_id, assessment_id, message_id (INSTITUTION_MESSAGE), …
Foreign Key	WORLD_BANK.institution_id, NATIONAL_BANK.institution_id, LOCAL_BANK.institution_id → INSTITUTION; NATIONAL_BANK.country_code → COUNTRY; BORROWER.registered_local_bank_id → LOCAL_BANK; local_bank_id in LOAN_REQUEST references LOCAL_BANK(institution_id); LOAN_RISK_ASSESSMENT.model_id → MODEL_REGISTRY; LOAN_RISK_ASSESSMENT.loan_request_id → LOAN_REQUEST; INSTITUTION_MESSAGE.sender_institution_id, receiver_institution_id → INSTITUTION; multi-entity stubs (Table 3.14) anchor institution participants to INSTITUTION.institution_id.
UNIQUE	wallet_address in BORROWER; NATIONAL_BANK(country_code) (one national reserve per nation); contract_address in INSTITUTION; blockchain_tx_hash in LOAN_REQUEST; borrower_id in BORROWING_LIMIT; symbol in ASSETS; tx_hash in BLOCKCHAIN_EVENT_LOG; partial unique on INTEREST_RATE_TIER(bank_tier_type) WHERE is_active = TRUE.
CHECK	WORLD_BANK: institution_id = 1 (singleton). BANK_USER: (bank_type='world' AND world_bank_id IS NOT NULL) OR (bank_type='national' AND national_bank_id IS NOT NULL) OR (bank_type='local' AND local_bank_id IS NOT NULL). TRANSACTION: at least one of borrower_id, origin_institution_id, counterparty_institution_id is non-null.
NOT NULL	Core attributes: INSTITUTION.name, INSTITUTION.institution_type, BORROWER.wallet_address, BORROWER.registered_local_bank_id, status, oracle_state (default NONE).
APPEND-ONLY (DB policy)	AGENT_ACTION_LOG, CREDIT_PASSPORT_HISTORY, SECURITY_EVENT_LOG: CREATE ROLE audit_writer; GRANT INSERT ON agent_action_log TO audit_writer; REVOKE UPDATE, DELETE FROM PUBLIC. AUDIT_LOGS: same policy applied via Row-Level Security (ALTER TABLE audit_logs ENABLE ROW LEVEL

This integrity constraints table summarizes referential and domain constraints that keep loan, repayment, and identity records consistent. These constraints reduce operational risk by preventing orphaned records and invalid lifecycle states, which is critical for auditability in financial systems.

3.5 On-Chain and Off-Chain Data Partitioning

Table 3.20: Data partitioning between on-chain and off-chain storage.

Data Category	Storage	Rationale
Reserve balances, loan requests, approval/rejection events, repayment transactions	On-chain	Immutability, public auditability, trustless verification.
User profiles, income verification documents, chat messages, loan risk assessments, security event logs	Off-chain (database)	Data privacy, query flexibility, storage cost optimisation.
Borrowing limit computations	Off-chain with on-chain enforcement	Complex temporal aggregation; results committed as on-chain constraints via `LocalBank.updateBorrowingLimit(borrowerId, newLimit)` (see sync protocol below).
Cryptocurrency market data (non-Chainlink)	Off-chain (`MARKET_DATA`)	CoinGecko and other fallback feeds; high-frequency updates; external API dependency.
Chainlink oracle price data	Off-chain (`CHAINLINK_PRICE_ROUND`)	Each `latestRoundData()` round cached with `answered_in_round` for staleness detection; on-chain `AggregatorV3Interface` is authoritative. `MARKET_DATA` does *not* store Chainlink rounds.
Agent action execution log	Off-chain (append-only DB)	Immutable audit trail of every agent write-tool call, linked to on-chain tx hash; not stored on-chain to save gas.
Session key material	Off-chain (`sessions` table)	EIP-7702 session key hash, scope JSON, and TTL stored off-chain; the scope restrictions are enforced on-chain by the session key contract at signing time.
Session lineage chains (SESSIONS, message history, compression summaries)	Off-chain (PostgreSQL)	Full transcript history is too large for on-chain storage. The lineage chain enables searchable long-term memory without gas cost. `AGENT_ACTION_LOG` provides the on-chain audit link via confirmed transaction hashes.

This partitioning table clarifies which data must live on-chain (role bindings, reserves, and critical state transitions) versus off-chain (documents, analytics features, and operational metadata). The partitioning choice balances transparency and tamper-resistance with practical storage and privacy constraints.

Borrowing limit sync protocol. When on-chain loan events change a borrower's rolling exposure, the event listener: (1) receives `LoanRepaid`/`LoanDisbursed` events and updates `TRANSACTION`; (2) recomputes 6-month and 1-year rolling windows from `TRANSACTION`; (3) if the computed limit differs from `on_chain_enforced_limit` in `BORROWING_LIMIT`, the Express API calls `LocalBank.updateBorrowingLimit(borrowerId, newLimit)`; (4) on receipt of `BorrowingLimitUpdated`, the listener updates the `BORROWING_LIMIT` row and clears `sync_discrepancy_flag`.

Projection vs. application tables. Tables owned exclusively by the event listener (projection tables) include `CREDIT_PASSPORT`, `CREDIT_PASSPORT_HISTORY`, `BORROWING_LIMIT`, and `COLLATERAL_POSITION`. Express.js handlers may `SELECT` from these tables but must never `INSERT`/`UPDATE` them directly; PostgreSQL RLS enforces this separation (Table 3.18). Application tables (`LOAN_REQUEST`, `CHAT_MESSAGE`, `PROFILE_SETTING`, etc.) are Express-owned. The `oracle_state` column on `LOAN_REQUEST` is updated by the event listener only; Express API roles are denied UPDATE on this column via a column-level privilege: `REVOKE UPDATE (oracle_state) ON loan_request FROM api_role; GRANT UPDATE (oracle_state) ON loan_request TO event_listener_role`. This is a deliberate narrow exception to the Express-owned classification: all other `LOAN_REQUEST` columns remain Express-writable.

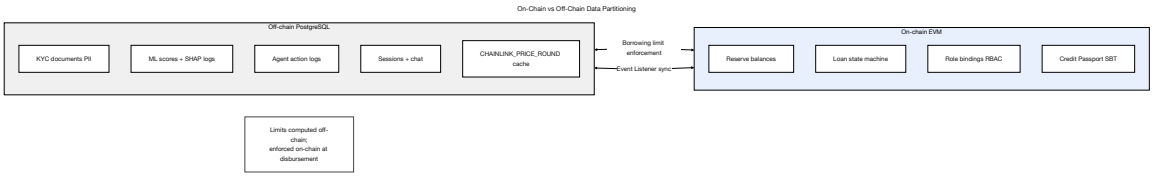


Figure 3.12: On-chain versus off-chain data partitioning

3.6 Operating Context

The Crypto World Bank is specified as a **blockchain-based institutional finance platform** on EVM-compatible infrastructure, not as a geographic pilot for a single rural market. The primary contribution is programmable hierarchical banking—reserve enforcement, tiered capital flow, on-chain auditability, and modular lending products—deployable on a stability-first prototype testnet (Polygon Amoy PoS and/or Ethereum Sepolia). Polygon zkEVM remains an evaluated option in the platform comparison tables rather than the default prototype deployment chain narrative.

Three design choices anchor the operating context. First, **stablecoin-denominated retail lending** (Section 1.8.3) limits liability-side volatility when collateral and loans use different asset types. Second, **low per-transaction gas cost** on zkEVM L2 makes high-frequency installment and reserve operations economically viable relative to Ethereum mainnet. Third, **optional account abstraction** (Section 3.8.1) can simplify wallet onboarding for crypto-naive retail users without changing smart-contract role logic. Solidarity group lending (Section 3.17) is a programmable product module informed by microfinance literature (BRAC,

Grameen); it is not scoped to a specific country deployment. Institutional onboarding is planned through academic pilots, open-source replication, and regulatory sandbox partnerships (Section 3.18.5).

3.7 Digital Identity System

The platform's identity model operates across two layers, combining wallet-based authentication with off-chain document verification, and is designed to accommodate compliance requirements in future phases.

- **Identity based on wallets:** Users authenticate via Ethereum wallet signatures (MetaMask, WalletConnect), eliminating the need for centralised credential storage.
- **Role binding:** In the smart contract permission system, wallet addresses are linked to hierarchical roles like Owner, National Bank, Local Bank, Approver, and Retail Client.
- **Limits of wallet-based identity:** Wallet ownership proves transaction history and cryptographic control but cannot independently prove legal identity, jurisdiction, or age. This distinction is critical for regulated banking operations. A compromised or lost wallet requires an off-chain recovery process and on-chain role revocation followed by re-binding to a replacement address, a governance workflow that must be carefully designed to prevent unauthorized role escalation.
- **On-chain versus off-chain identity layers:** The system maintains two identity layers. On-chain identity consists of the wallet address, assigned role, and transaction history recorded permanently on the blockchain. Off-chain identity consists of document-verified income, KYC credentials, and AML screening results stored in the PostgreSQL database and linked to the wallet address by hash reference. The planned ZKP-based compliance extension would allow users to prove off-chain KYC status to the smart contract without exposing personal documents on-chain.
- **EIP-7702 session key authorisation:** For agent-executed banking operations, the client authorises a scoped, time-bound session key at login. The session key is restricted to a named set of MCP write tools (e.g., only `submit_loan_application` and `pay_installment`), a value cap (e.g., 500 USDC per transaction), and a 24-hour TTL after which it auto-expires. The session key cannot transfer funds to external addresses or perform any operation outside its approved scope. Every session key usage is logged in `AGENT_ACTION_LOG` with the conversation turn ID from `AGENT_CONVERSATION_TURN` as the audit reference. The EIP-7702 delegation contract address is registered in `CHAIN_REGISTRY` (Phase IV) as platform-wide configuration.

3.7.1 Compliance Identity (Future Work)

Future Work. A two-circuit privacy-preserving compliance gateway—Groth16 zkKYC (Circom 2.0), zkAML velocity proofs [R19], and W3C DID/VC anchoring—is motivated by Contribution 4 (Chapter 6) and the literature synthesis in Section 2.2.7. Circuit-level design, verifier deployment, and on-chain integration are deferred to Section 6. At MVT scope, retail and institutional onboarding uses tiered off-chain KYC, hash-linked credentials in PostgreSQL, and on-chain RBAC role gates (Section 3.8.2).

3.8 User Taxonomy and Onboarding Flows

The thesis now uses a formal actor taxonomy of nine actors following the systems-analysis convention documented in the project Binance Software Engineering Architecture study (Appendix D, entry IR-4) [130]. Five primary actors (A1–A5) initiate actions; four secondary actors (A6–A9) are external systems or authorities. Table 3.22 gives the operational user taxonomy with per-user-type KYC level, wallet type, and database-residence assumption; Table 3.23 presents the complete formal actor taxonomy; and Table 3.24 presents the actor permission matrix.

Table 3.22: Operational user taxonomy: KYC level, wallet type, and data residence by user type.

User Type	Tier	KYC Level	Wallet Type	Data Residence
Anonymous Visitor	—	None	None	Session only (Redis)
Registered Retail User	Tier 4	Level 1 (gov. ID + selfie)	MetaMask or optional ERC-4337	Off-chain primary; on-chain role only
Group Client	Tier 4	Level 1 + group verification	ERC-4337 abstracted	Off-chain + on-chain group contract
Local Bank Operator	Tier 3	Full institutional KYC	MetaMask (institutional)	Off-chain + on-chain role binding
Local Bank Approver	Tier 3	Full institutional + background check	MetaMask + Safe co-signer	Off-chain + on-chain role binding
National Bank Admin	Tier 2	Full institutional + regulatory license	Safe multisig mandatory	Off-chain + on-chain role binding
World Bank Admin (Governance)	Tier 1	Founding team + supervisor attestation	Safe 3-of-5 multisig	On-chain governance only

Table 3.23: Formal actor taxonomy (nine actors).

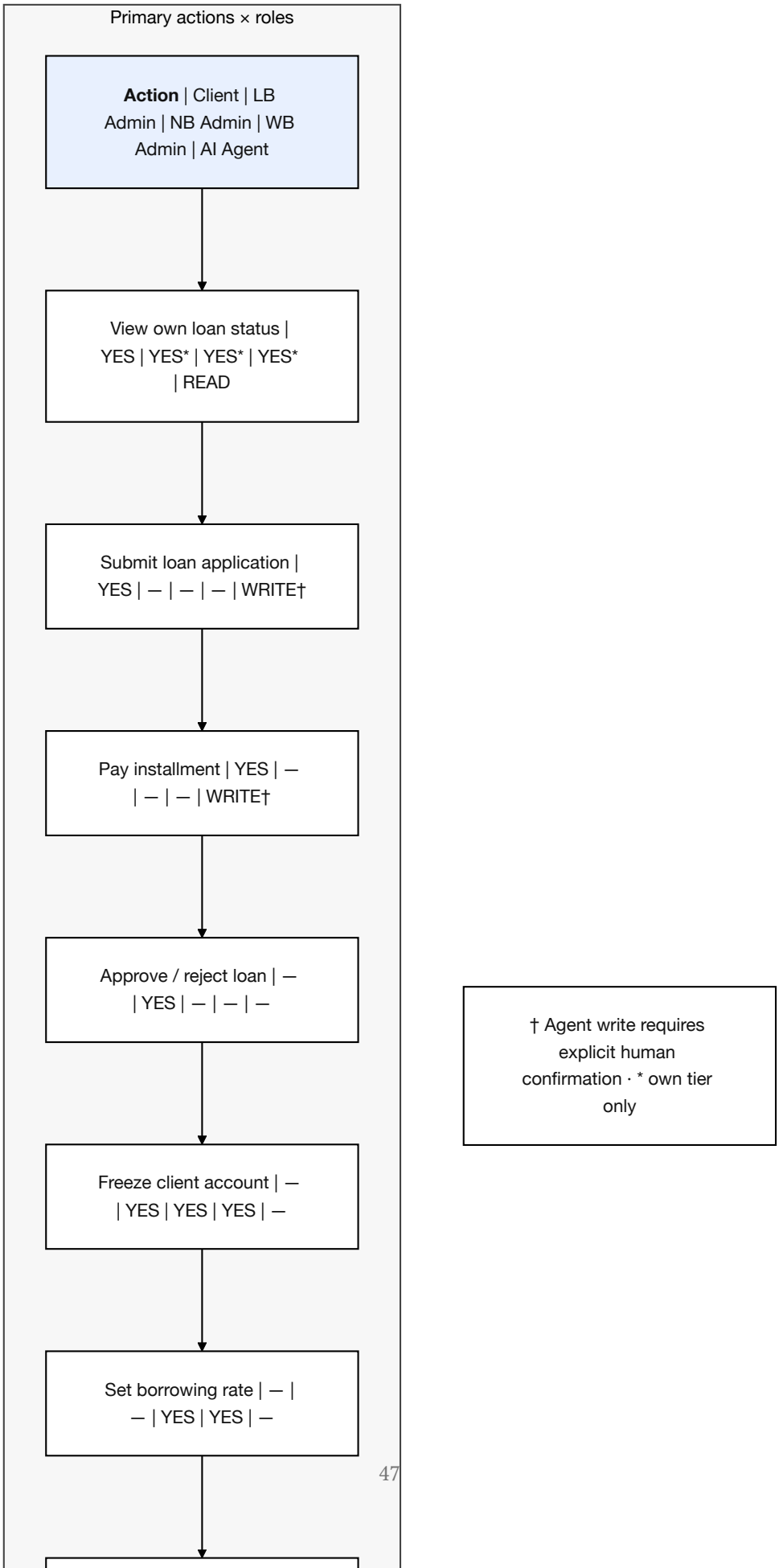
A1a	Retail Client (pre-KYC)	Browsing only; no banking transactions.
A1b	Retail Client (KYC Tier 1)	Bronze/Silver credit tier; small loans up to the KYC-1 cap.
A1c	Retail Client (KYC Tier 2)	Gold/Platinum/Diamond; larger limits, group lending.
A2	Local Bank Admin (Approver)	Loan approver; risk officer; reviews AML alerts.
A3	National Bank Admin	Capital allocator; compliance officer; SAR review; freeze authority.
A4	World Bank Admin (Governance)	Parameter governor; system operator via multi-sig.
A5	AI Agent (retail)	Phase III Must deliverable; read tools always permitted; write tools require explicit human confirmation gate before execution.
Secondary Actors (external systems or authorities)		
A6	Regulatory Authority	Read-only audit access via encrypted data package.
A7	Chainlink DON	Oracle, price feeds, automation.
A8	Blockchain Validator	Polygon zkEVM validity proof generation.
A9	External Auditor	Chainlink PoR verification.

Table 3.24: Actor permission matrix (tabular summary of primary actions).

Action	Client	LB Admin	NB Admin	WB Admin	AI Agent
View own loan status	YES	YES*	YES*	YES*	READ
Submit loan application	YES	NO	NO	NO	WRITE†
Pay installment	YES	NO	NO	NO	WRITE†
Submit KYC documents	YES	NO	NO	NO	NO
Approve loan application	NO	YES	NO	NO	NO
Reject loan application	NO	YES	NO	NO	NO
Freeze client account	NO	YES	YES	YES	NO
Set borrowing rate	NO	NO	YES	YES	NO
Change reserve ratio	NO	NO	NO	YES	NO
Upgrade LB borrowing cap	NO	NO	YES	YES	NO
View audit logs (all)	NO	YES*	YES*	YES	NO
Generate SAR report	NO	YES	YES	YES	NO
View reserve summary (public)	YES	YES	YES	YES	READ

*† All agent write tools require an explicit human confirmation gate: the agent assembles the full parameter set, presents a summary to the client, and waits for affirmative consent. No write tool is ever called without a confirmation turn in the conversation history. Agent scope is retail-client only at MVT; bank-authority read/aggregate tools are Future Work (Section 3.1). * own tier only.*

Actor Permission Matrix



3.8.1 ERC-4337 Account Abstraction for Retail Onboarding

The single largest accessibility contradiction in the original thesis was that retail clients were assumed to already understand MetaMask, seed phrases, gas fees, and Ethereum transactions. This is incompatible with a mobile-first, crypto-naive retail onboarding goal. The platform resolves the contradiction by adopting **ERC-4337 Account Abstraction (AA)** for all retail-tier (Tier 4) clients, with the EIP-7702 Pectra upgrade (May 2025) providing the gas-sponsorship layer. Since its launch on Ethereum mainnet in March 2023, ERC-4337 has enabled over 40 million smart accounts and processed more than 100 million transactions across major L2 networks including Polygon, Arbitrum, Optimism, and Base, demonstrating production-grade viability at scale [18-1]. The current production EntryPoint is **v0.7**, deployed at `0x0000000071727De22E5E9d8BAf0edAc6f37da032` on Ethereum, Polygon, and all major EVM chains; the platform targets this canonical address for all UserOperation routing.

Under ERC-4337 each retail client is issued a smart-contract account that is created on first sign-in. The user can register with email or a Google account via Firebase Auth, never sees a seed phrase, and never needs to acquire ETH for gas: a Paymaster contract sponsors the gas, funded by the World Bank Reserve as a financial-inclusion subsidy line. Account recovery uses a social-recovery guardian (a trusted email / phone) rather than a 24-word mnemonic.

Paymaster sybil-resistance controls. Gas sponsorship introduces a perverse incentive: an attacker can create many ERC-4337 accounts at zero cost and spam loan applications, draining the gas subsidy and potentially the loan pool if fraud detection fails. Three controls mitigate this. First, the Paymaster implements a **pre-KYC bootstrap allowance**: before KYC is completed, the Paymaster sponsors exactly two function signatures per wallet — `submitKYCHash(bytes32)` (storing the document hash for Level 1 KYC) and the ERC-4337 account creation transaction. Both are capped at a maximum of 0.001 ETH-equivalent total per wallet address; this is sufficient to complete KYC but insufficient to spam loan applications. All other gas sponsorship is conditional on the wallet having a verified KYC status (`kycVerified[wallet] = true`). This pre-KYC bootstrap window resolves the chicken-and-egg problem: a new user with no ETH can complete KYC using the bootstrap allowance, after which normal Paymaster sponsorship unlocks. Second, the Paymaster budget is capped per verified identity (not per account), limiting lifetime gas sponsorship to a fixed USDC equivalent per KYC identity. Third, the Paymaster rate-limits by IP and device fingerprint to prevent bulk account creation from a single attacker. These controls make Paymaster sponsorship conditional on verified identity rather than unconditional, preserving the inclusion mission while preventing abuse.

For the planned implementation, the Biconomy SDK or Alchemy Account Kit will provide bundler / Paymaster infrastructure on Polygon zkEVM Cardona. From the smart-contract perspective, an account-abstracted wallet is indistinguishable from an externally owned account, so none of the four-tier role logic needs to change. Retail clients may also connect a standard MetaMask wallet; account abstraction is an optional onboarding path, not a platform requirement.

EIP-7702 session keys — scoped agent wallet (Phase III). EIP-7702 provides a cleaner solution than ERC-4337 paymasters for agent-controlled operations because the scope re-

striction is enforced at the key level, not at the application level. When a client authorises an AI Agent session at login, the EIP-7702 session key is subject to four hard constraints: (1) **Scope** — the key is restricted to a named set of MCP write tools (e.g., only `submit_loan_application` and `pay_installment`); (2) **Time-bound** — a 24-hour TTL after which the key auto-expires regardless of remaining uses; (3) **Value-capped** — a maximum of 500 USDC per transaction, preventing large unauthorised transfers; and (4) **Revocable** — the client can invalidate the session key at any time from the wallet UI. The session key *cannot* transfer funds to external addresses, exceed the value cap, perform any operation not in the approved scope, or execute after the TTL expires.

EIP-7702 is architecturally superior to ERC-4337 paymasters for agent-controlled operations and does not replace ERC-4337 for retail gas sponsorship — the two mechanisms serve different functions. ERC-4337 removes the need for ETH for gas; EIP-7702 scopes what the agent may sign. Both are active simultaneously for an agent session: the session key signs the transaction, the Paymaster sponsors the gas.

3.8.2 Tiered Risk-Based KYC

Prototype scope (Ethics Statement alignment). The Ethics Statement applies to the *current prototype run*: no real identity documents or production AML processing. The workflows below are **architectural specification**; pre-thesis and MVT onboarding uses synthetic credentials, document-hash stubs in PostgreSQL, and disposable testnet wallets only.

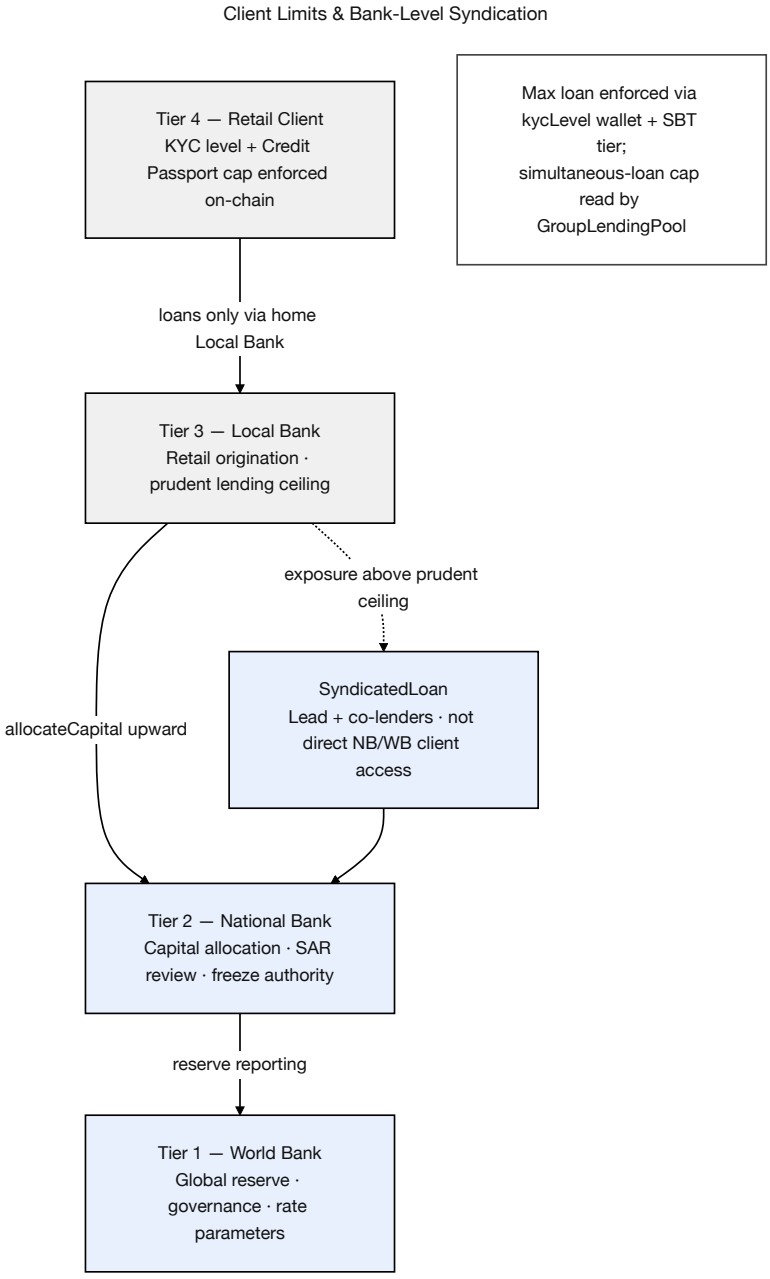


Figure 3.14: Client limits and bank-level syndication

Identity verification is gated by a risk-based KYC ladder rather than an all-or-nothing flag. Table 3.25 gives the four levels and the corresponding borrowing limits and verification times. The design is consistent with current Persona / Veriff guidance for retail-fintech KYC and the EU AI Act Article 86 transparency requirements.

Table 3.25: Tiered, risk-based KYC ladder. Higher KYC level unlocks larger loans but requires proportionally more verification.

Risk Level	Required Documents	Borrow Limit	Verify Time
Level 0 (Unverified)	None	View only	Instant
Level 1 (Basic)	Government ID photo + selfie liveness check	Up to 0.1 ETH-equivalent (USDC)	2–5 min (automated)
Level 2 (Standard)	Government ID + proof of income + bank statement	Up to 2 ETH-equivalent	24 h (human review)
Level 3 (Enhanced)	Full document set + video interview	Up to 10 ETH-equivalent	48–72 h

After KYC approval, the smart contract sets `kycVerified[wallet] = true`, `kycLevel[wallet] = level`, and a one-year expiry `kycExpiry[wallet] = block.timestamp + 365 days`. Re-verification is enforced via the role-expiry mechanism. Personal data—government ID, biometric template, document files—is never stored on-chain; only a SHA-256 hash of the document is written, in compliance with GDPR data-minimization principles. On-chain zk-SNARK verification is scoped to Future Work (Section 6).

3.8.3 Five-Stage Retail Onboarding Funnel

The platform's retail onboarding is structured as a five-stage funnel that progressively reveals blockchain complexity. Stage 1 is information browsing with zero friction (no account required). Stage 2 is account creation via wallet connect or optional ERC-4337 smart account (email / social login), with `registered_local_bank_id` assigned at the selected home Local Bank. Stage 3 is KYC verification via document capture with automated verification (2–5 minutes in the planned implementation). Stage 4 is the first loan: the UI explains terms in plain language and shows the repayment schedule in USDC with optional fiat-equivalent display via a Chainlink FX oracle. Stage 5 is power-user mode—self-custodied MetaMask, block-explorer visibility, and solidarity-group lending. The primary interaction paths are the React dashboard and the conversational agent (Section 4.4); both call the same API.

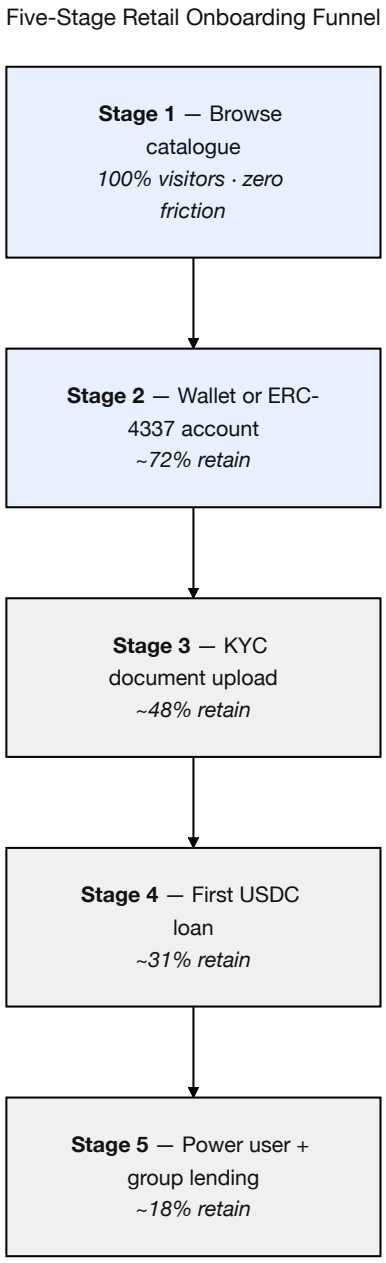


Figure 3.15: Five-stage retail onboarding funnel

3.8.4 Conversational Interface (Phase III)

The MCP-based agent is a **planned Phase III Must deliverable** that calls the same Express.js endpoints and smart contracts as the React UI. It does not gate, modify, or replace the manual banking flows. At MVT scope the agent is limited to **retail-client** sessions (`SESSIONS.borrower_id`); bank-authority agent sessions (e.g. an NB Admin summarising reserve ratios across Local Banks) require generalising `SESSIONS` to `actor_id` + `actor_type` and a separate MCP tool set with tighter RBAC—explicitly deferred to Future Work (Table 3.1). Figure 3.16 specifies the six-step pipeline: on-chain context injection, read/write routing, mandatory human confirmation gate, MCP write-tool execution via EIP-7702 session key (when enabled), and append-only `AGENT_ACTION_LOG` audit trail.

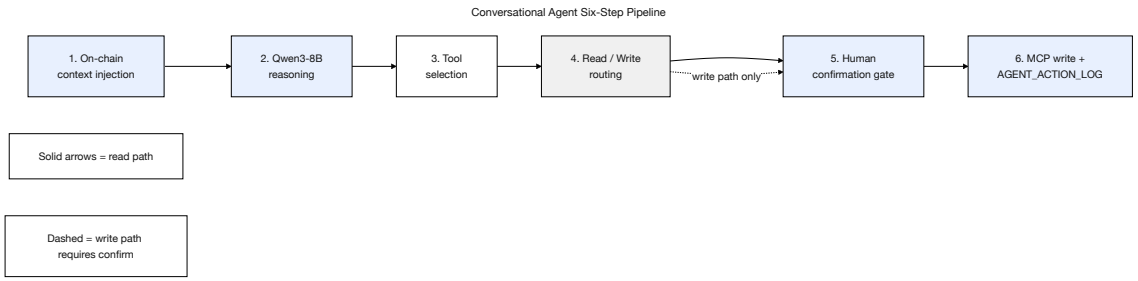


Figure 3.16: Conversational agent pipeline (Phase III MVT)

3.9 Kinked Interest Rate Model

Borrowing cost follows a kinked utilization model (Compound/Aave pattern [R3]) with optimal utilization $U^* = 80\%$ for retail pools. Utilization is $U = L/(L + B)$, where L is lent amount and B is available liquidity.

Below the kink ($U \leq U^*$), the rate rises gently:

$$r_b(U) = r_0 + \frac{U}{U^*} \cdot r_1, \quad U \leq U^* \tag{Formula 18}$$

At $U = 0$ the rate is base r_0; at $U = U^*$ it reaches $r_0 + r_1$ (r_1 = below-kink slope).

Above the kink ($U > U^*$), the rate rises steeply to incentivise repayment and new deposits:

$$r_b(U) = r_0 + r_1 + \frac{U - U^*}{1 - U^*} \cdot r_2, \quad U > U^* \tag{Formula 19}$$

r_2 is the jump multiplier above the kink. On-chain guards enforce `r2 >= MINIMUM_JUMP_MULTIPLIER` and `U_star <= MAXIMUM_KINK_POINT` before any parameter update.

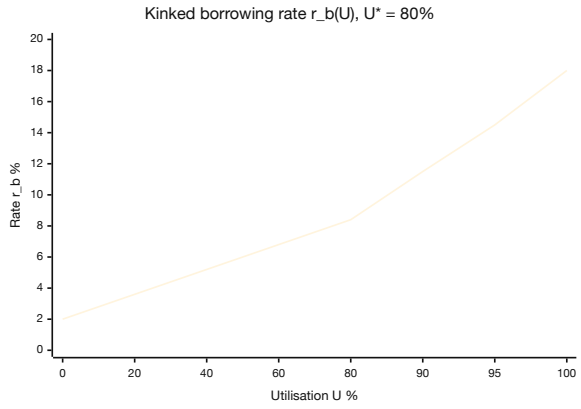


Figure 3.17: Kinked utilization-based borrowing rate

3.10 Liquidation Engine

The **LiquidationEngine** contract monitors health factor (HF) and pays third-party liquidators a 5–8% bonus when $HF < 1.0$. HF models vary by loan type:

Tier 4 over-collateralized retail.

$$HF = \frac{C \times LT}{D} \tag{3.1}$$

Collateral value C (oracle-priced) times liquidation threshold LT, divided by debt D; liquidate when $HF < 1.0$.

Tier 4 group lending.

$$HF_{\text{group}} = \frac{\text{total\_group\_collateral} \times LT}{\text{total\_group\_outstanding\_debt}} \tag{3.2}$$

Pool-level health score; liquidation loss is shared across all group members.

Tier 4 credit-based (no collateral). HF is not applicable. Default triggers SBT `riskTier` downgrade and `lastDefault` timestamp [R30]; no collateral to seize.

Tier 1–3 institutional.

$$HF_{\text{inst}} = \frac{\text{on\_chain\_reserve} - \text{min\_reserve}}{\text{outstanding\_borrowing}} \tag{3.3}$$

Parent-tier reserve surplus above the mandatory minimum acts as effective collateral.

Liquidation life-cycle.

1. Chainlink price feed updates collateral each block; HF recomputed (view-only).
2. Any participant calls `liquidate(loanId)` when $HF < 1.0$.
3. Contract transfers debt plus 5–8% bonus in collateral to the liquidator; credit record updated.
4. `LoanLiquidated(loanId, liquidator, bonusBps)` event indexed by the event listener (Graph optional later).

When a Local Bank's reserve ratio falls below 15%, authorized liquidations queue at National Bank level, lowest HF first.

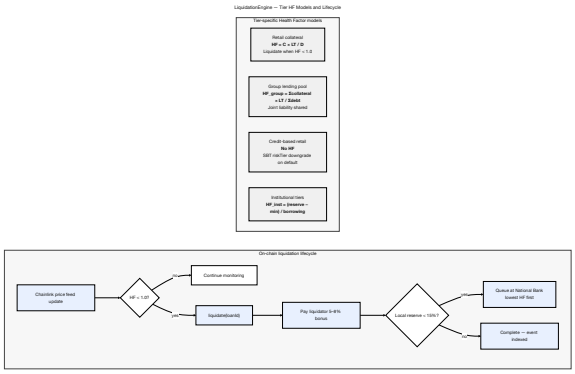


Figure 3.18: LiquidationEngine tier models and lifecycle

3.11 SavingsVault and FixedDeposit Modules

SavingsVault. Accepts ERC-20 deposits (USDC primary), accrues variable yield tied to pool utilization (Section 3.9), and permits withdrawal subject to the local-tier reserve-ratio gate. Interest accrues per-second via a cumulative index (Compound/Aave supply-side pattern); implements ERC-4626 (`IERC4626`) for composable vault shares [R46].

FixedDeposit. Issues a non-transferable receipt locking principal for a fixed term (30/90/180/365 days) at deposit-time APY; early withdrawal forfeits accrued interest. Provides duration-matched liabilities for installment lending; adopts ERC-7540 for time-gated asynchronous redemption [R46].

Loan interest is split under Formula NetInterest (List of Formulas) among depositor yield, the InsuranceFund, and protocol revenue, closing the deposit–lend loop without external subsidy.

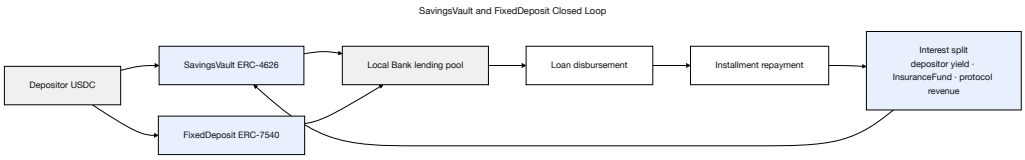


Figure 3.19: SavingsVault and FixedDeposit closed loop

3.12 On-Chain Credit Passport (Soulbound Token)

Each KYC-verified client receives a non-transferable Soulbound Token (SBT) [R30] encoding repayment history (`creditScore`, `openLoans`, `completedCycles`, `lastDefault`, `riskTier`). The credential upgrades on full installment repayment and is permanently downgraded (never revoked) on default. It enables portable credit across Local Banks on Cardona, enforces cross-platform simultaneous-loan caps (Section 3.17.3), and is exposed read-only via `ICreditPassport.getScore(wallet)` for external protocols on the deployment chain.

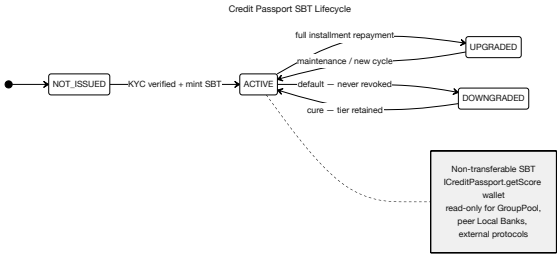


Figure 3.20: On-chain Credit Passport SBT lifecycle

Credit tier schedule. Table 3.26 governs maximum loan limits and interest modifiers by tier.

Table 3.26: Credit Passport tier schedule governing maximum loan limits and interest modifiers.

Tier	Label	Score Range	Max Loan (USDC)	Interest Modifier
1	Bronze	0–299	50	Base rate
2	Silver	300–549	250	Base − 0.5%
3	Gold	550–749	1,000	Base − 1.0%
4	Platinum	750–899	5,000	Base − 1.5%
5	Diamond	900–1000	25,000	Base − 2.0%

3.13 Single-Chain Deployment (Design Revision)

An initial design considered multi-chain deployment bridging Polygon zkEVM and Ethereum Sepolia to distribute retail and institutional traffic across separate networks. This approach was revised after security analysis identified cross-chain bridges as the highest-risk attack surface in DeFi—accounting for over $1.1 billion in documented losses through Ronin ($625M), Wormhole ($320M), and Nomad ($190M) exploits in 2022 alone. The revised architecture consolidates **all live contract deployment on a single EVM chain**. Polygon zkEVM remains the design-preference target (ZK validity proofs verified on Ethereum L1), while the prototype implementation may deploy to stable public EVM testnets such as Polygon Amoy PoS and/or Ethereum Sepolia for operational stability, without changing the contract logic or the single-chain assumption.

Ethereum Sepolia is retained in the testing plan as a stable public EVM testnet option for prototype deployment when required, with no live asset bridge between networks. This simplification reduces attack surface and keeps Phases I–II deliverable within the project timeline. Chen et al. [46] validate multi-chain lending feasibility in the literature; the CWB adopts their throughput lessons without deploying a live CCIP or LayerZero bridge. Credit Passport SBT state, borrowing-limit enforcement, and loan lifecycle data remain single-chain on the chosen prototype testnet.

3.14 Multi-Entity and Cross-Tier Capital Operations

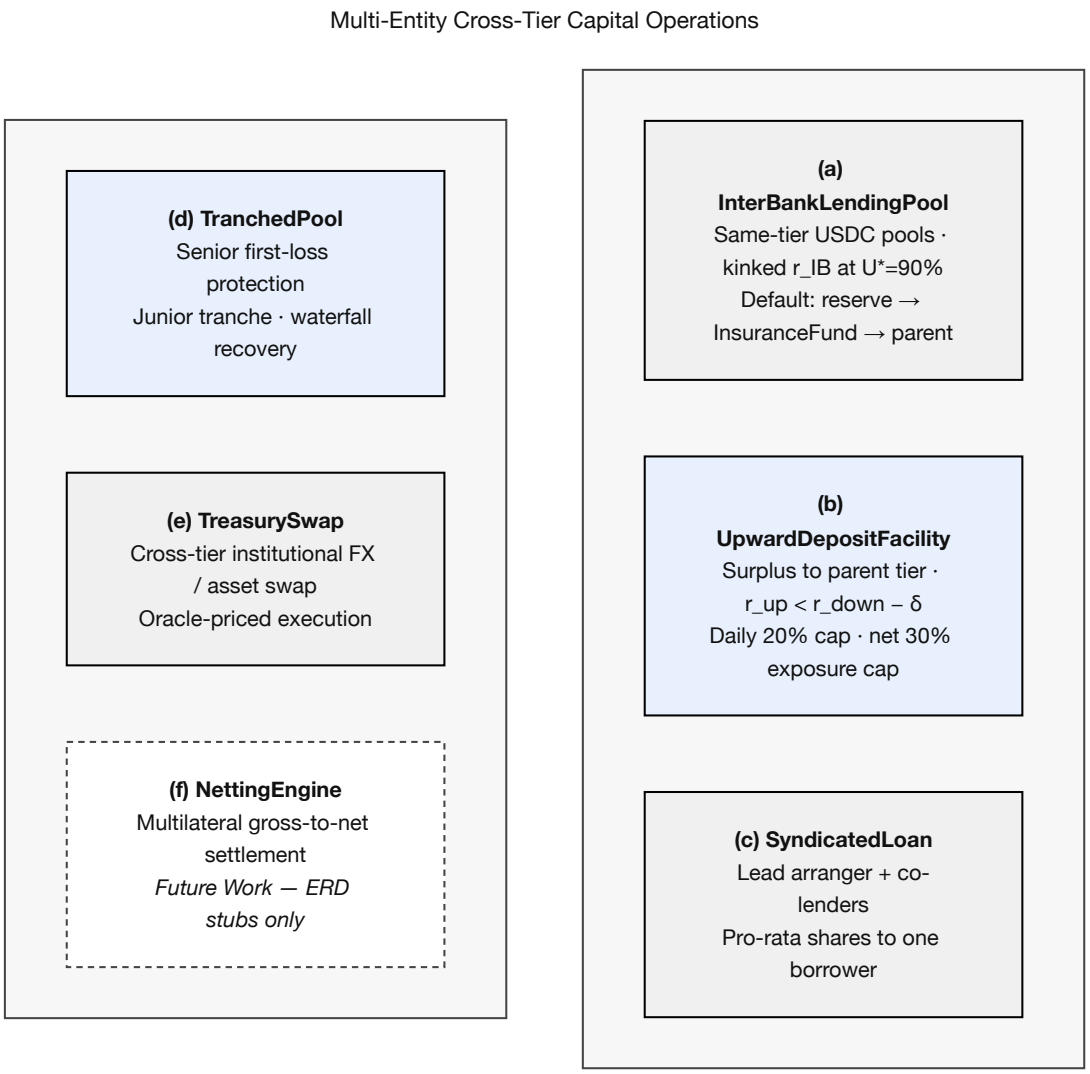


Figure 3.21: Multi-entity and cross-tier capital operations

The architecture framing of multi-directional flows promises downward, same-tier, and upward lending plus bank-level syndication for large exposures. Earlier specifications left same-tier and upward flows at design-level only. A paper that claims a *complete* banking system must specify these flows—and the genuinely multi-entity operations they enable—at the same depth as the downward flow. This section closes that gap with five concrete contract-and-flow specifications: a fully-specified InterBankLendingPool (Section 3.14.1), upward surplus repatriation (Section 3.14.2), syndicated / club lending (Section 3.14.3), senior–junior tranched lending pools (Section 3.14.4), and cross-tier treasury FX swap (Section 3.14.5). Multilateral settlement netting (`NettingEngine`) is scoped to Future Work (Section 6). The corresponding ERD entities, contract list, implementation phase plan, and Future Work are updated downstream so the database design, software-engineering stages, and architecture remain a single consistent system.

3.14.1 InterBankLendingPool: Fully Specified Same-Tier Lending

One `InterBankLendingPool` (IBLP) per tier (`IBLP_NB`, `IBLP_LB`) supplies same-tier short-tenor (1/7/30-day) USDC liquidity among peer banks, gated by active tier role (Section 3.19).

Rate model. $r_{\text{IB}}(U)$ follows the kinked curve of Section 3.9 (Formulas 18–19) with kink $U^*_{\text{IB}} = 0.90$. The interbank rate is capped by the parent-tier downward rate:

$$r_{\text{IB}}(U) \leq r_{\text{downward}}(U) - \delta \tag{3.4}$$

δ is the governance-set spread that blocks arbitrage against the tier above; checked live via `getCurrentBorrowRate()` at each rate update.

Default cascade. On maturity default: (1) reserve buffer debited, (2) `InsuranceFund` drawn, (3) parent tier absorbs residual and queues role suspension via TimeLock.

3.14.2 Upward Surplus Repatriation

Banks with reserves above the minimum by 20% may deposit excess into the parent tier via `UpwardDepositFacility`, earning parent deposit yield. The upward rate is capped below the downward rate:

$$r_{\text{up}} < r_{\text{down}} - \delta \tag{3.5}$$

Withdrawals revert if they breach the depositor's minimum reserve ratio; daily withdrawals are capped at 20% of outstanding upward liabilities; net upward exposure is capped at 30% of on-chain assets.

3.14.3 Syndicated and Club Lending

When single-name exposure exceeds a bank's lending ceiling, multiple banks co-fund one loan through `SyndicatedLoan`: Lead Arranger, Co-Lenders, and Borrower (client via Local Bank only). Funding requires aggregate commitments $\geq 90\%$ of `totalAmount` and Co-Lender confirmation $\geq 75\%$ by capital share. Each lender's share is recorded as `shareBps[lender][syndicateId]`; principal and interest (and default recovery) distribute pro-rata by that share.

3.14.4 Senior–Junior Tranched Lending Pools

`TranchedPool` splits capital into senior (lower yield, protected) and junior (higher yield, first-loss) tranches. Repayment waterfall: senior interest $\rightarrow$ junior interest $\rightarrow$ senior principal $\rightarrow$ junior principal; losses reverse (junior written down first). Subordination ratio (junior/senior) is fixed at deposit between 10/90 and 30/70.

3.14.5 Cross-Tier Treasury FX Swap

`TreasurySwap` executes oracle-priced atomic cross-asset swaps for banking-role wallets:

$$\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread}) \tag{3.6}$$

P is the Chainlink feed price; treasury spread is 5–10 bps. Post-swap reserve ratio must remain above the tier minimum; swaps above \$1M require Safe two-signature confirmation.

3.14.6 Multilateral Settlement Netting Engine (Future Work)

Future Work. A `NettingEngine` contract to compress bilateral IBLP and `TreasurySwap` obligations into batch settlement—including partial settlement (`settlePartial`) and challenge-period dispute resolution—is listed in Section 6. ERD stubs (`NETTING_BATCH`, `NETTING_ENTRY`) remain in the schema for forward compatibility but are not specified at implementation depth in this report.

3.14.7 Database, Phase, and Contract Consistency for Multi-Entity Operations

Eight ERD entities (`INTERBANK_LOAN`, `UPWARD_DEPOSIT`, `SYNDICATE`, `SYNDICATE_MEMBER`, `TRANCHED_POOL`, `TREASURY_SWAP`, `NETTING_BATCH`, `NETTING_ENTRY`) extend Figure 3.6, together with `GROUP_CONSENT` (Phase II, required before `GroupLendingPool`) and `TRANCHED_POOL_SUBSCRIPTION` (Phase II). All institution-participant FKs anchor to `INSTITUTION.institution_id` (Section 3.4.2, Table 3.14). Five contracts are specified at implementation depth in this report: `InterBankLendingPool`, `UpwardDepositFacility`, `SyndicatedLoan`, `TranchedPool`, and `TreasurySwap`; `NettingEngine` is Future Work (Section 6). Extended entities include planned `on_chain_address` and `last_synced_block` columns for event-listener mapping.

Table 3.27: FK anchors connecting extended entities to core schema.

Entity	Foreign Keys into Core Schema
SAVINGS_ACCOUNT, FIXED_DEPOSIT, CURRENT_ACCOUNT	borrower_id → BORROWER; local_bank_id → LOCAL_BANK(institution_id)
LOAN_GROUP	local_bank_id → LOCAL_BANK(institution_id)
GROUP_MEMBER	group_id → LOAN_GROUP; borrower_id → BORROWER
INSURANCE_FUND	local_bank_id → LOCAL_BANK(institution_id)
INTERBANK_LOAN	lender_institution_id, borrower_institution_id → INSTITUTION; tier-type CHECK per IBLP instance
UPWARD_DEPOSIT	depositor_institution_id, parent_institution_id → INSTITUTION; adjacent-tier CHECK
SYNDICATE	lead_institution_id → INSTITUTION; borrower_id → BORROWER
SYNDICATE_MEMBER	syndicate_id → SYNDICATE; member_institution_id → INSTITUTION
TRANCHED_POOL	sponsor_institution_id → INSTITUTION
TRANCHED_POOL_SUBSCRIPTION	pool_id → TRANCHED_POOL; subscriber_id → BANK_USER
TREASURY_SWAP	initiator_institution_id → INSTITUTION; from_asset_id, to_asset_id → ASSETS
NETTING_BATCH	coordinator_id → BANK_USER
NETTING_ENTRY	batch_id → NETTING_BATCH; participant_institution_id → INSTITUTION
INSTITUTION_MESSAGE	sender_institution_id, receiver_institution_id → INSTITUTION
LOAN_RISK_ASSESSMENT	loan_request_id → LOAN_REQUEST; model_id → MODEL_REGISTRY
GROUP_CONSENT	loan_request_id → LOAN_REQUEST; member_id → GROUP_MEMBER
COLLATERAL_POSITION	borrower_id → BORROWER; loan_id → LOAN; asset_id → ASSETS
BLOCKCHAIN_EVENT_LOG	Referenced by: LOAN_REQUEST.submission_event_log_id, approval_event_log_id; LOAN.disbursement_event_log_id; INSTALLMENT.payment_event_log_id

3.15 System Modeling

This section presents the system analysis diagrams that model the platform's structure, behavior, and data flow. Loan and transaction state transitions are formalised in the on-chain

state machine of Section 4.8; the diagrams below show actor interactions and operational flows at the use-case and sequence level.

3.15.1 Use Case Diagram

The system involves nine actors across eleven representative use cases (Figure 3.22). Four **core primary** actors initiate banking operations: Retail Client (A1, three KYC sub-types), Local Bank Admin (A2), National Bank Admin (A3), and World Bank Admin (A4). A5 (AI Agent) is a **planned Phase III primary** actor with restricted write permissions gated by the human confirmation protocol (Table 4.2, item 11). Four secondary actors are external systems or authorities: Regulatory Authority (A6), Chainlink DON (A7), Blockchain Validator (A8), and External Auditor (A9). Note that the primary actor classes in the use-case diagram are a condensation of the formal nine-actor taxonomy defined in Table 3.23: Visitor and Registered Retail User are pre-conditions for the CLIENT (BORROWER) role rather than independent diagram actors, and Local Bank Operator / Approver are merged into the Bank Approver role at the diagram level.

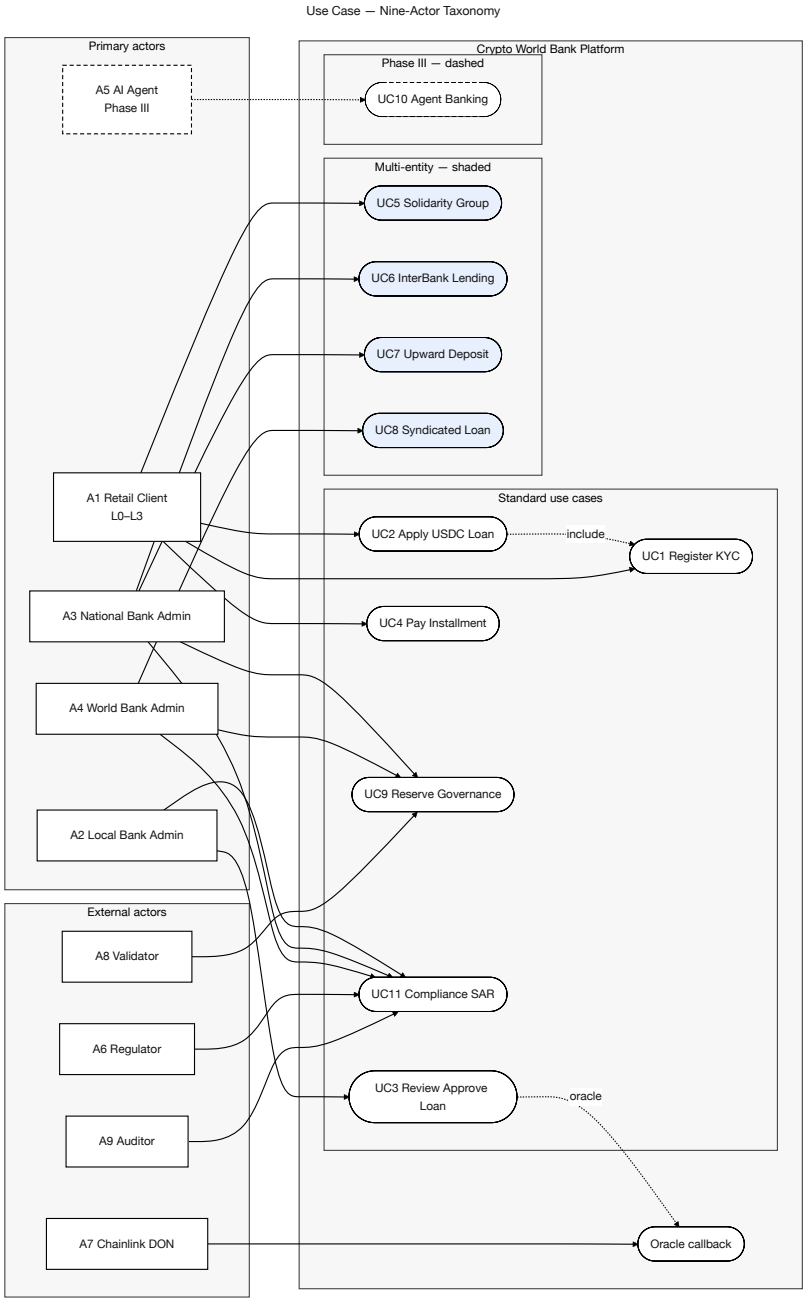


Figure 3.22: Use-case diagram (nine-actor taxonomy)

3.15.2 Activity Diagrams

Figures 3.23–3.25 present three end-to-end activity flows. Each diagram has a single start and a single end node; decision diamonds route exceptional paths (limit exceeded, rejection, verification retry) back to the terminal state without spawning parallel lifelines.

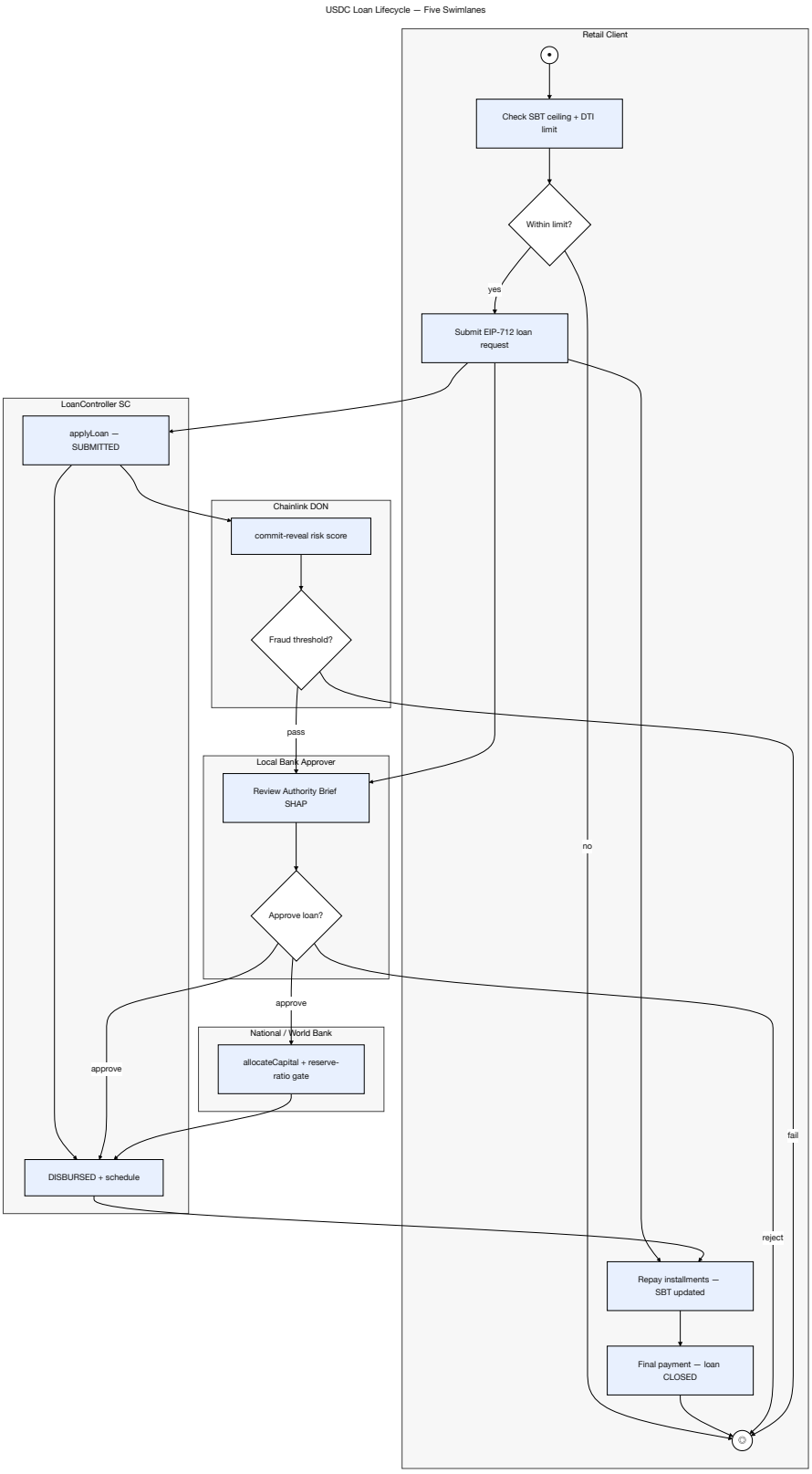


Figure 3.23: USDC loan lifecycle activity flow

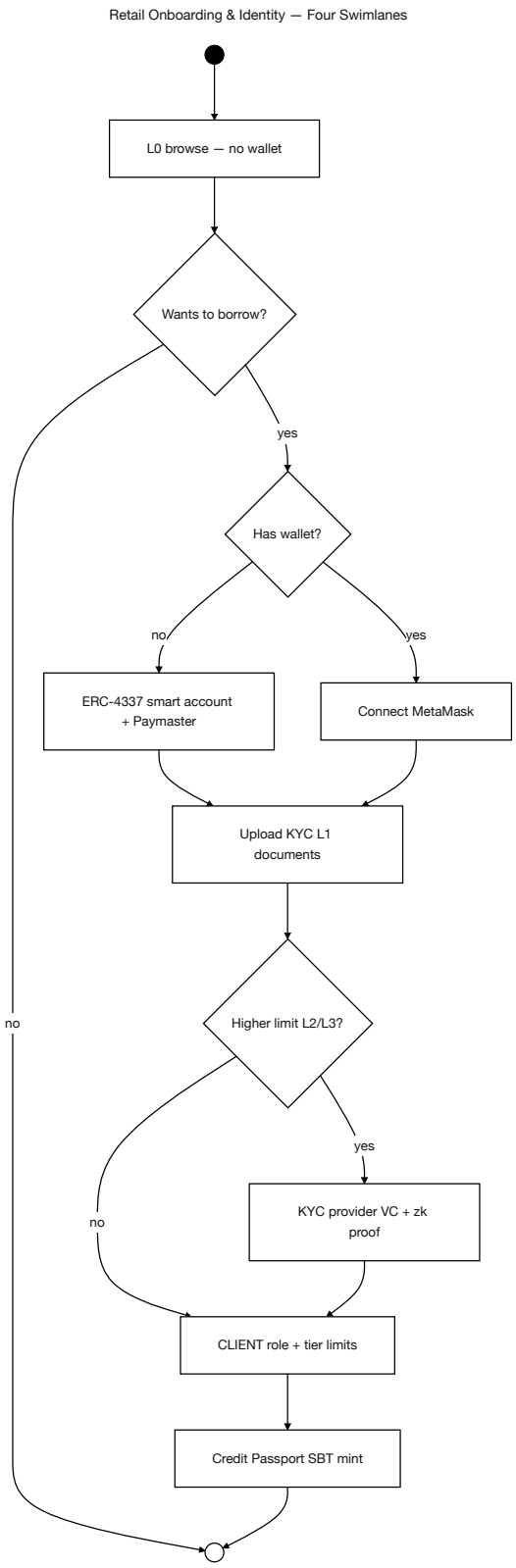


Figure 3.24: Retail onboarding and identity activity flow

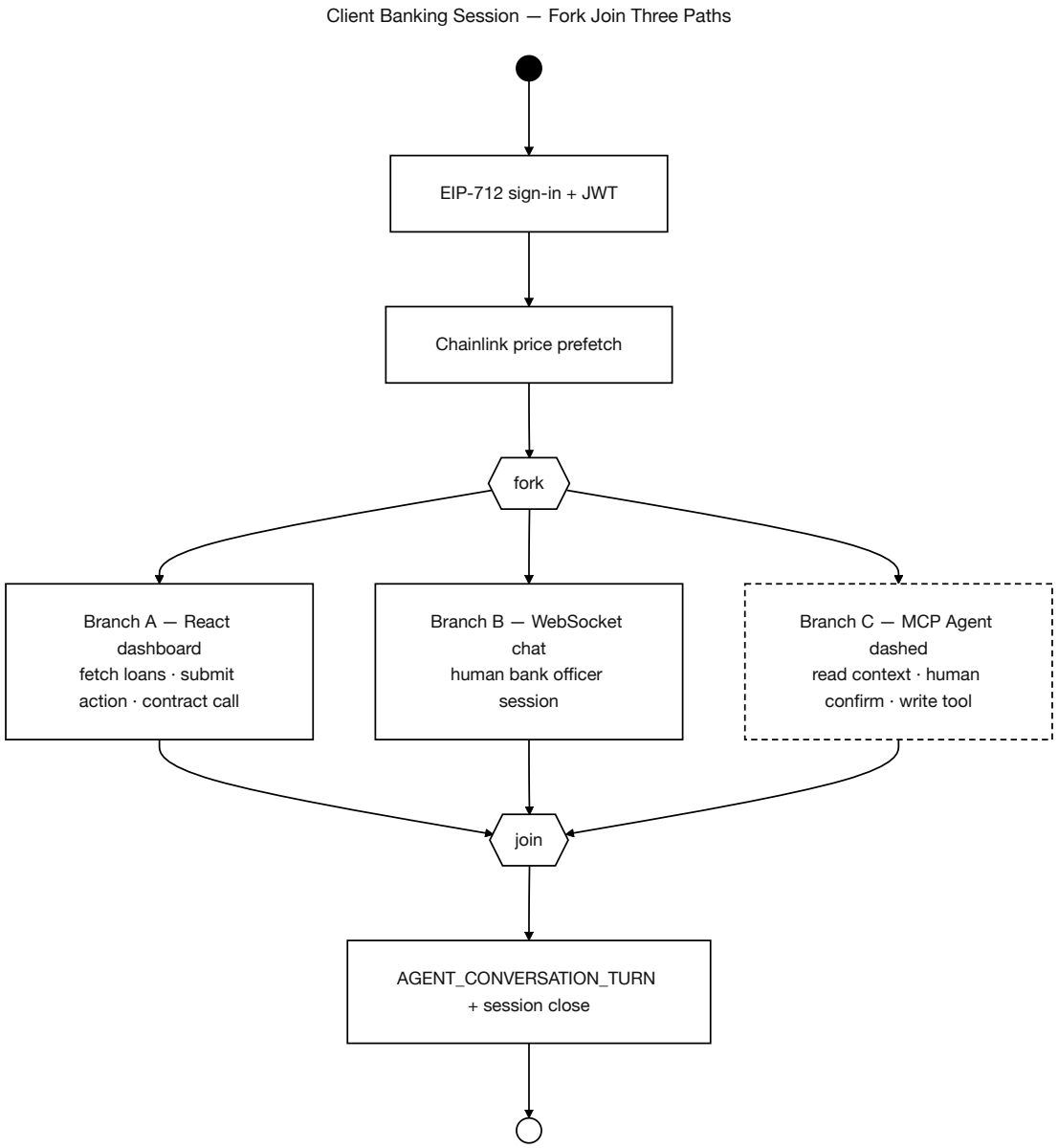


Figure 3.25: Client banking session activity flow

SAR activity flow. The Suspicious Activity Report (SAR) workflow is triggered when the Isolation Forest anomaly detector (Section 3.20) flags a wallet with an anomaly score above the 0.75 threshold. The five-step flow is:

1. Isolation Forest flags the wallet: `anomaly_score > 0.75`.
2. `LOAN_RISK_ASSESSMENT` records the per-loan detection (FK `model_id` → `MODEL_REGISTRY`); `SECURITY_EVENT_LOG` records the system-wide AML alert event.
3. Express.js backend emits a planned event-bus message (`aml-alert` topic); the compliance dashboard consumes it.
4. Bank officer reviews the alert in the admin compliance queue:
   - **False positive:** Dismiss and document reason (audit trail in `audit_logs`).
   - **Confirmed:** Generate SAR record in `audit_logs`; insert `INSTITUTION_MESSAGE` (`message_type='SAR_ESCALATION'`) to notify the tier above

(National Bank); freeze wallet via `LocalBank.freezeAccount(clientId)`.

5. The `freezeAccount` function is gated by the `onlyApprover` modifier and is Foundry-tested (Phase III invariant suite). A frozen wallet cannot initiate new loan disbursements or installment payments until the freeze is lifted by the tier above.

3.15.3 Data Flow Diagrams

Figure 3.26 consolidates the Level-0 context diagram with the Level-1 decompositions of the core lending subsystem and the deposit / interbank / FX / netting subsystems on a single page.

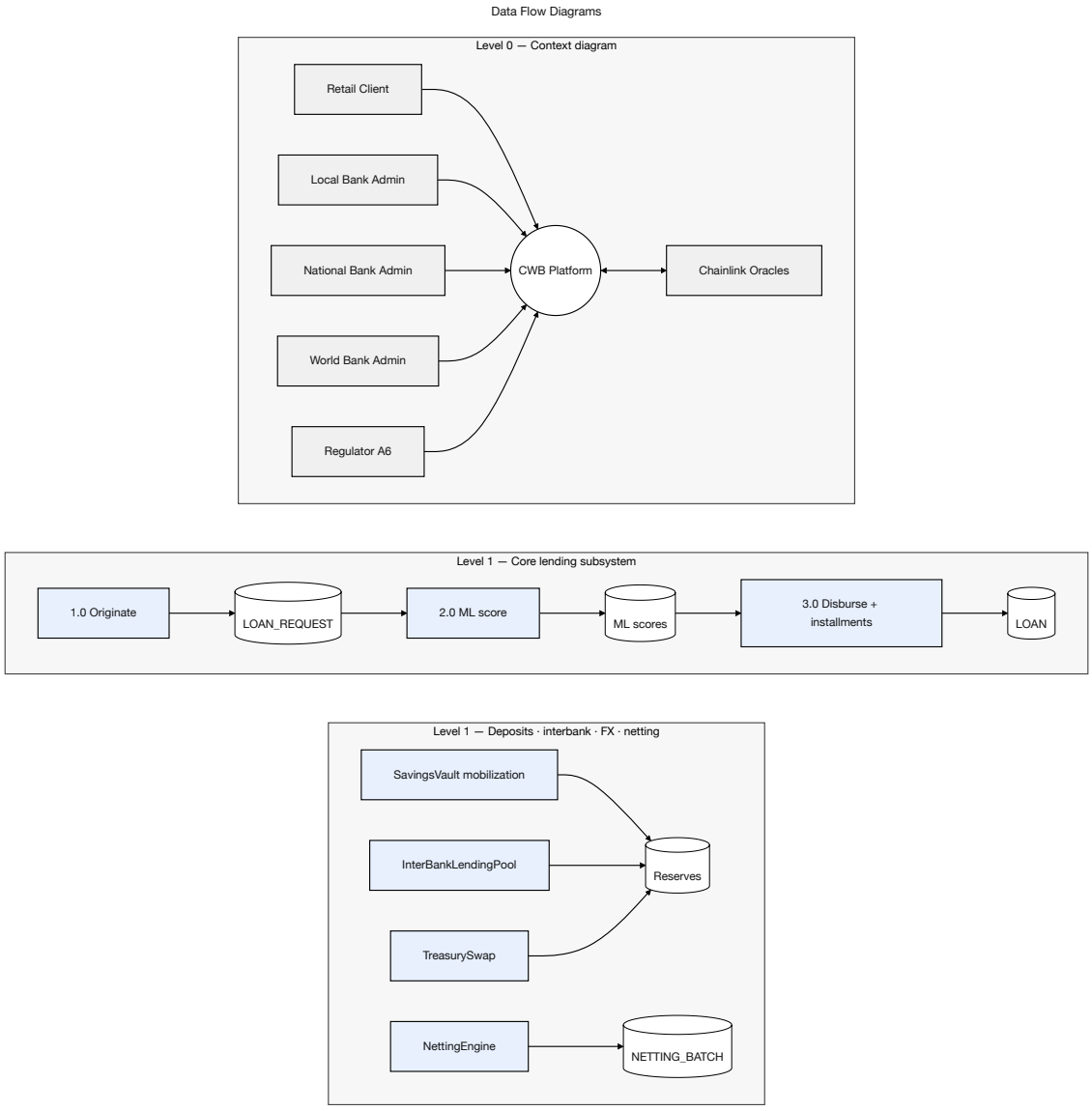


Figure 3.26: Data-flow diagrams

3.15.4 Sequence Diagrams

Figures 3.27–3.30 consolidate the platform's nine interaction flows into four panels: loan approval with reject alternative, installment and income, hierarchical banking with market data and borrowing-limit, and chat with AI-chatbot.

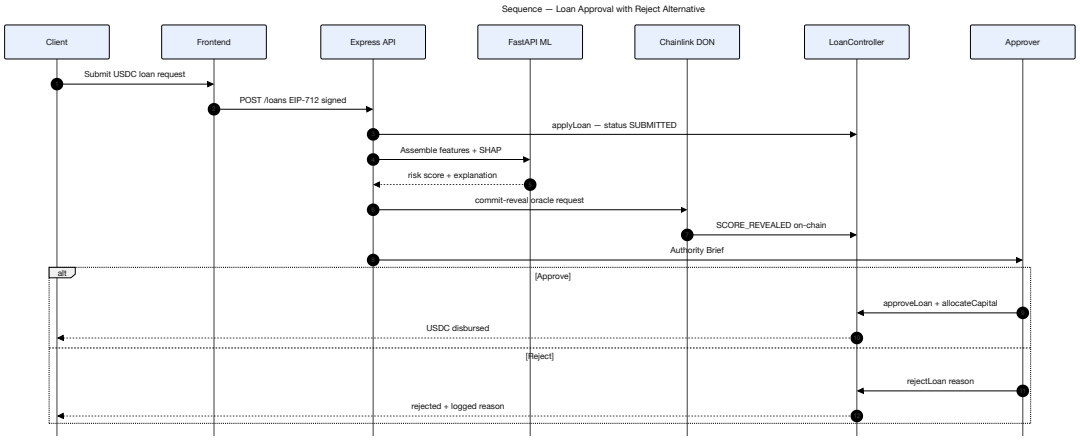


Figure 3.27: Loan approval sequence diagram

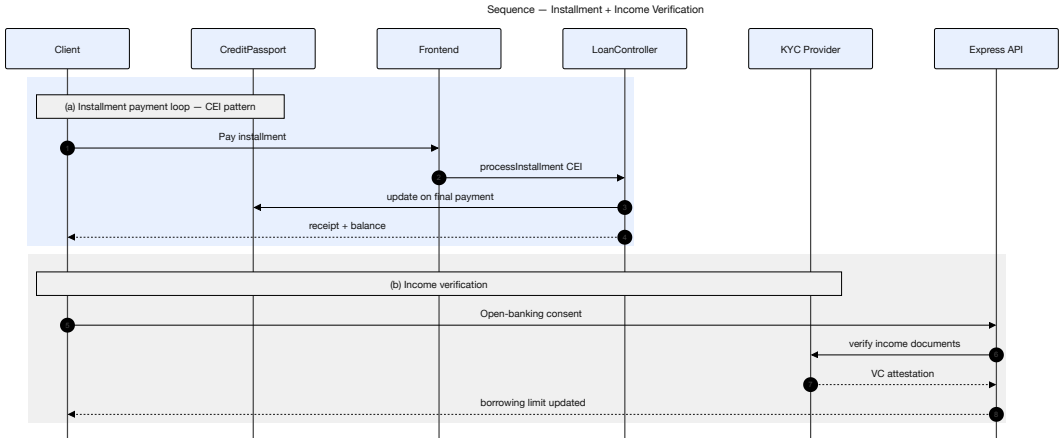


Figure 3.28: Sequence diagrams

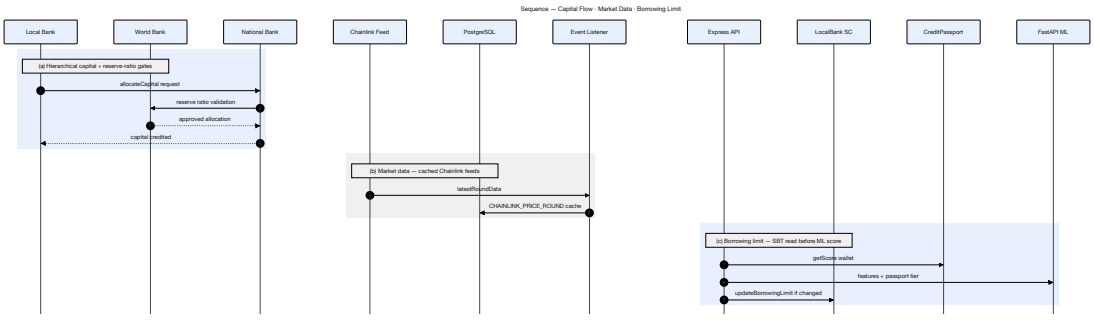


Figure 3.29: Sequence diagrams

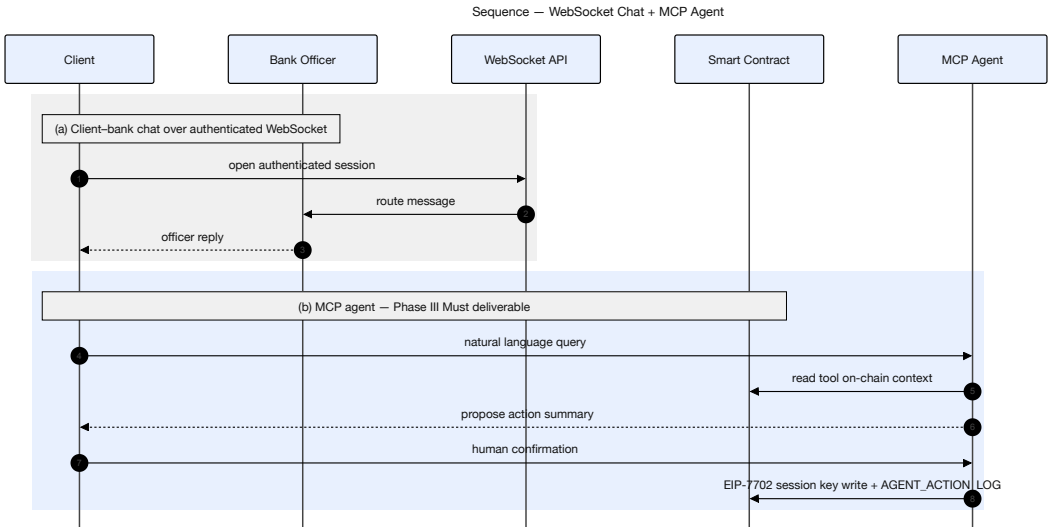


Figure 3.30: Sequence diagrams

3.15.5 Four-Tier Capital Flow

Figure 3.31 illustrates the hierarchical capital flow and cascading repayment structure.

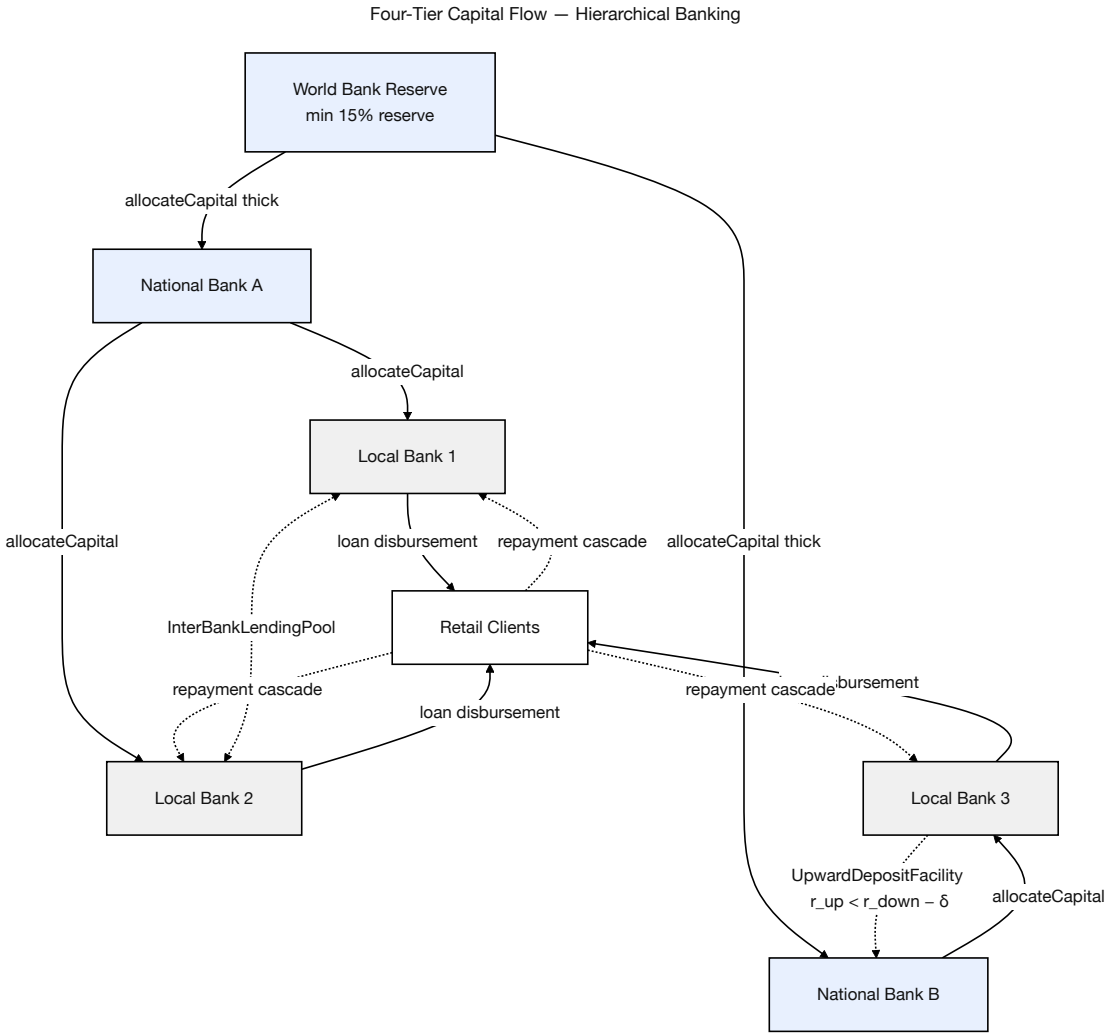


Figure 3.31: Four-tier hierarchical capital flow

3.16 Auxiliary Dual-Currency Facility

The cryptocurrency exchange feature of Crypto World Bank is built upon the cryptocurrency exchange facility that is an auxiliary facility incorporated into the current banking infrastructures all over the world. Instead of being a separate exchange, the platform is a service that is provided by all participating banks as an additional service to the existing product portfolio.

- **Integration model:** The dual-currency facility is offered by all participating banks as an add-on service to their existing product portfolio. No additional banking license or separate entity is needed.
- **Eligibility determination:** Eligibility of the dual-currency service is based on the bank officers managing the lending operations, on the basis of existing KYC/AML compliance, account status, and lending relationship history.
- **No defaulting of conditions:** The project does not override, modify, or default any existing banking conditions, regulatory requirements, or contractual obligations.
- **Scope:** The facility enables fiat-to-crypto and crypto-to-fiat conversions,

cryptocurrency-denominated lending and repayment, and transparent on-chain transaction records—all within the governance structure of the participating bank.

3.17 Banking Product Suite

3.17.1 Savings Products

Variable-yield savings (utilization-linked; Section 3.9) and fixed-term deposits (30/90/180/365 days; Section 3.11). Interest splits under Formula NetInterest among depositor yield, InsuranceFund, and protocol revenue.

3.17.2 Checking and Transactional Accounts

Non-yield current accounts for installments and transfers; on-chain settlement is atomic (debit and credit in one transaction).

3.17.3 Group / Solidarity Lending

Groups of 3–20 clients borrow via `GroupLendingPool` with shared collateral or a cold-start provisional tier ($\geq$5 members, Level 1 cap, 3-month expiry). Per-member disbursement:

$$\text{Share}_i = \frac{\text{LoanTotal}}{N_{\text{members}}} \tag{3.7}$$

Default: collateral pool covers shortfall, or SBT `riskTier` downgrade if uncollateralised [R30]. Modelled default rate: 8–15% (Table 5.23).

Over-indebtedness risk control. Max two simultaneous group loans (SBT `openLoans`), 30-day cooling-off, and cross-bank check via `ICreditPassport.getScore()`. Debt-to-income gate:

$$\text{DTI} = \frac{\text{existingDebt} + \text{requestedAmount}}{\text{estimatedMonthlyIncome}}, \quad \text{reject if DTI} > 0.40 \tag{3.8}$$

Stale income hash (> 90 days) triggers manual review. Group loans are USDC-denominated at Tier 4.

Group lending stress sizing. Without weekly meetings and field-officer social pressure, on-chain group default rates are modeled conservatively at 15–25% during early deployment before credit histories accumulate. A **20% stress scenario** sizes the `InsuranceFund` at 5% of total outstanding group loan principal; at this stress level the fund covers approximately one-quarter of defaults, with residual shortfalls absorbed by per-member collateral contributed at group formation. The `GroupLendingPool` is therefore a *reduced-collateral* product for borrowers with some digital assets but insufficient individual collateral—not a zero-collateral Grameen analogue.

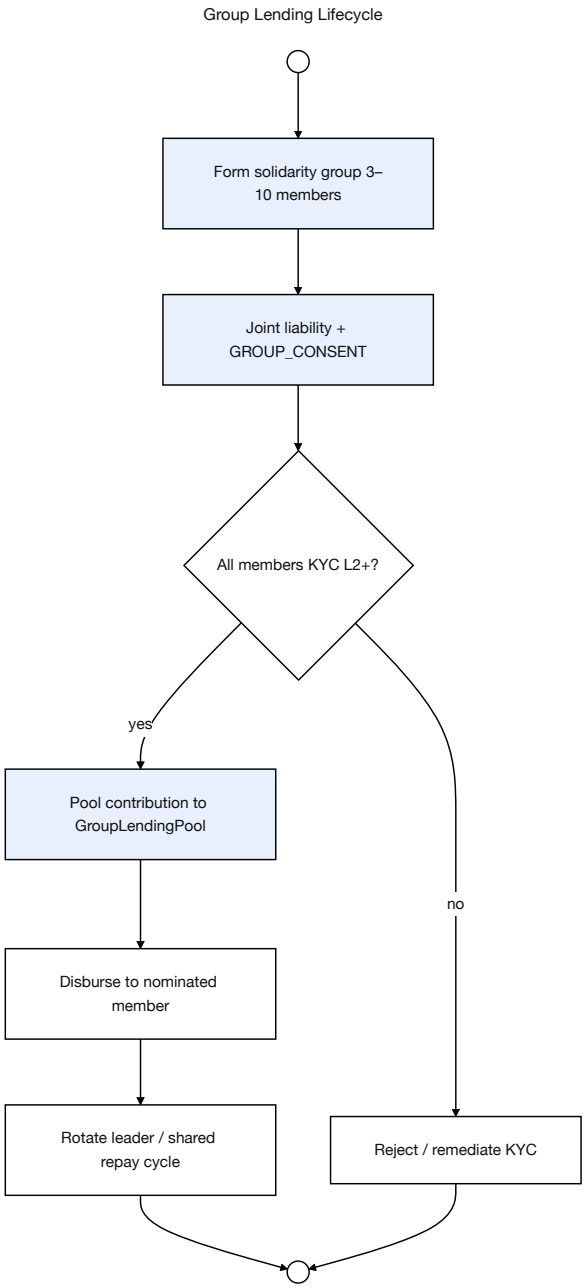


Figure 3.32: Solidarity group lending lifecycle

3.17.4 Foreign Exchange and Multi-Currency Operations

Oracle-priced FX with governance spread; retail lending in stablecoins (USDC/USDT), collateral in volatile assets (ETH). Treasury swaps: Section 3.14.5.

3.17.5 Trade Finance Facilitation (Planned)

Escrow-based letters of credit with document-verified release (Future Work).

3.17.6 Privacy-Preserving Identity Compliance (Future Work)

Licensed off-chain KYC issues a verifiable credential; on-chain Groth16 zkKYC and zkAML proofs are scoped to Future Work (Section 6). MVT onboarding sets `kycVerified` via off-

chain review and RBAC role gates.

3.18 Governance Framework

3.18.1 Governance Execution Paths and Emergency Controls

Parameter changes and upgrades route through a 48-hour `TimelockController` (5 min in lab demo). Governance votes use a **snapshot of voting power at the proposal creation block** (not at execution time) to prevent flash-loan governance attacks [158]. Contract upgrade proposals require a **60% quorum floor**. Emergency path: 2-of-3 emergency pause multisig (World Bank Reserve admin, National Bank representative, independent security reviewer) may pause operations within minutes; a separate 4-of-7 Security Council Safe may pause, upgrade, or revoke roles within 2 h (mandatory 48 h disclosure) but cannot change system parameters outside the timelock. These controls align with Chapters 1 property (1) and Section 3.18.1.

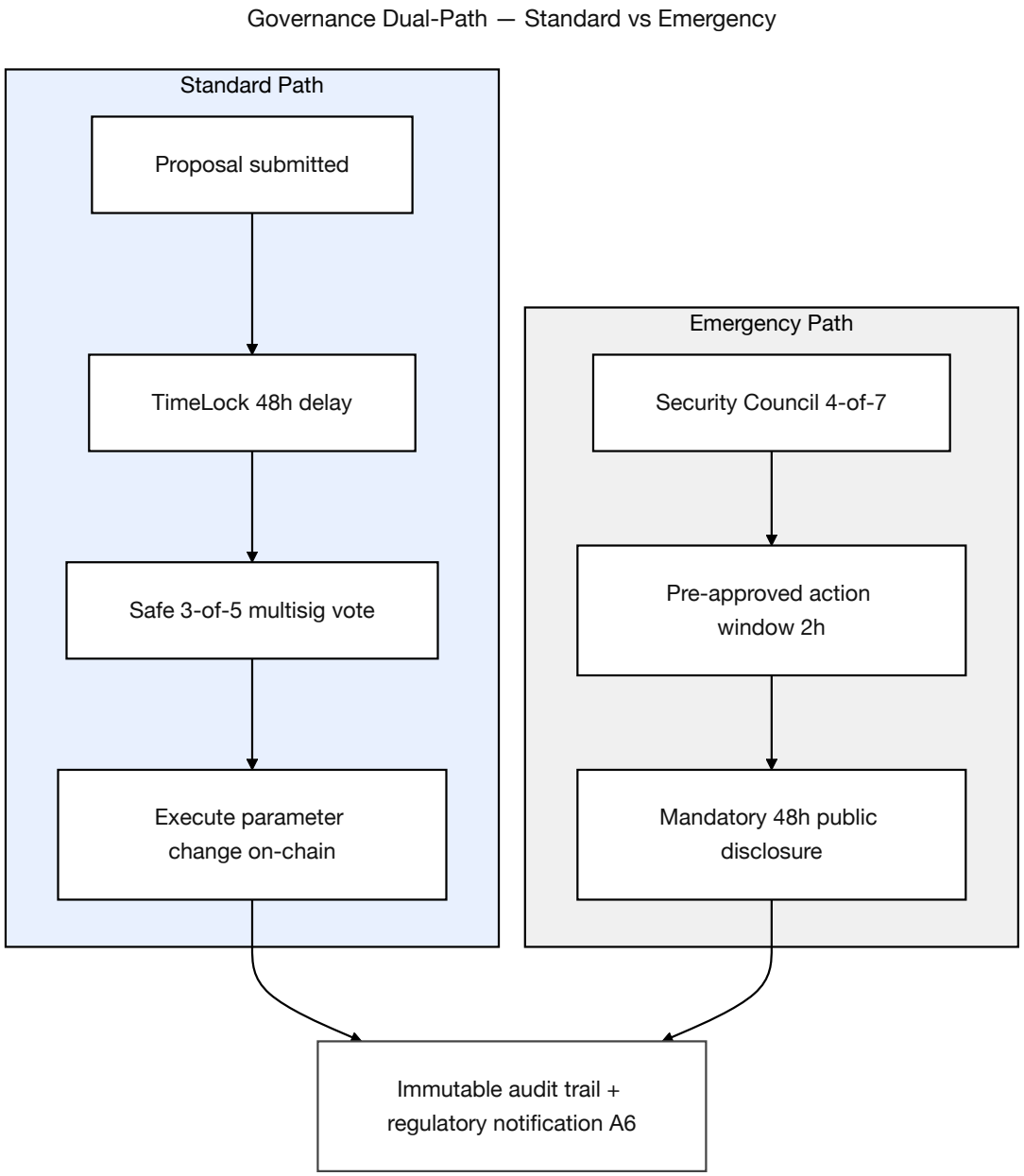


Figure 3.33: Governance dual-path

3.18.2 Network Membership Governance

Table 3.29: Network membership governance.

Governance Aspect	Implementation
Member on-boarding	World Bank owner registers National Banks; National Banks register Local Banks; Local Banks designate approvers — all enforced on-chain.
Member off-boarding	Deactivation flags in smart contracts; cascading access revocation.
Regulatory oversight	Audit log emission via smart contract events; planned read-only regulator dashboard.
Permission structure	Hierarchical: Owner → National Bank → Local Bank → Approver → Retail Client; enforced by on-chain role check modifiers.
Network operations	Pause/unpause mechanism for emergency response; emergency withdrawal for critical situations.

3.18.3 Business Network Governance

Table 3.31: Business network governance.

Governance Aspect	Implementation
Business charter	Defined in project documentation; operational parameters coded in smart contract constants.
Common services	Reserve management, loan lifecycle orchestration, event-driven notification system.
Business SLA	Testnet phase: best-effort availability. Production phase: 99.5% target API-layer uptime (team-controlled; multi-region deployment). Chain-level liveness is determined by Polygon zkEVM batch submission and Ethereum L1 proof verification; the team monitors network status and maintains degraded-mode fallbacks (read-only dashboard) if chain activity pauses.
Regulatory compliance	Architecture designed for audit trail generation; data partitioning supports GDPR-style data subject requests.

3.18.4 Technology Infrastructure Governance

Table 3.33: Technology infrastructure governance.

Governance Aspect	Implementation
Distributed IT structure	Client-side frontend (decentralised delivery); blockchain layer (fully decentralised); backend API (centralised, horizontally scalable).
Technology assessment	Continuous evaluation of EVM alternatives (L2 rollups, sidechains) for cost and performance optimisation.
On-chain / off-chain data services	Clearly partitioned (see Table 3.20); event listeners synchronise state between layers.
Risk mitigation	Smart contract pause mechanism; ReentrancyGuard; input validation; planned formal security audit.

3.18.5 Regulatory Compliance Considerations

Prototype runs on public testnets only; architecture emits audit logs for sandbox review; production targets regulatory sandbox programmes. zkKYC design: Section 3.17.6. SAR path: Isolation Forest → compliance queue → `freezeAccount` (Section 3.15).

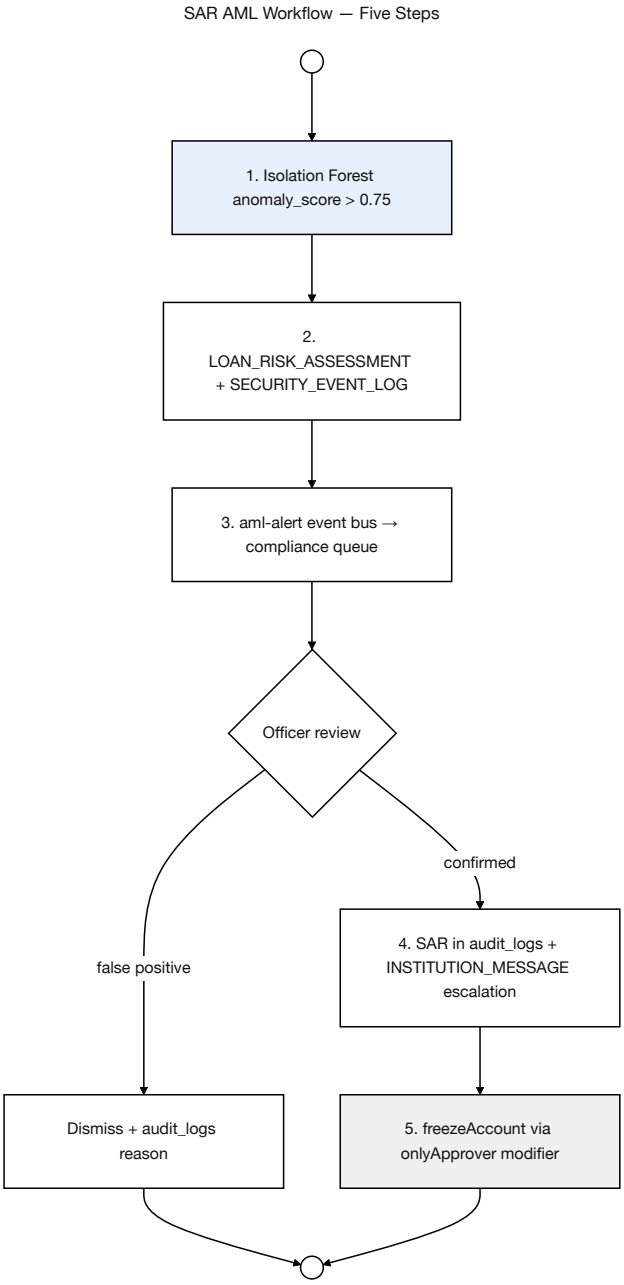


Figure 3.34: SAR and AML compliance workflow

3.18.6 Asset Tokenization

- **Specified (Phase I):** Native blockchain currency (ETH) as reserve and loan currency on testnet.
- **Planned extension:** ERC-20 stablecoins (USDC, USDT) for lending operations; tokenized collateral instruments where collateral is insufficient.

3.19 Five-Layer Defense-in-Depth Security Architecture

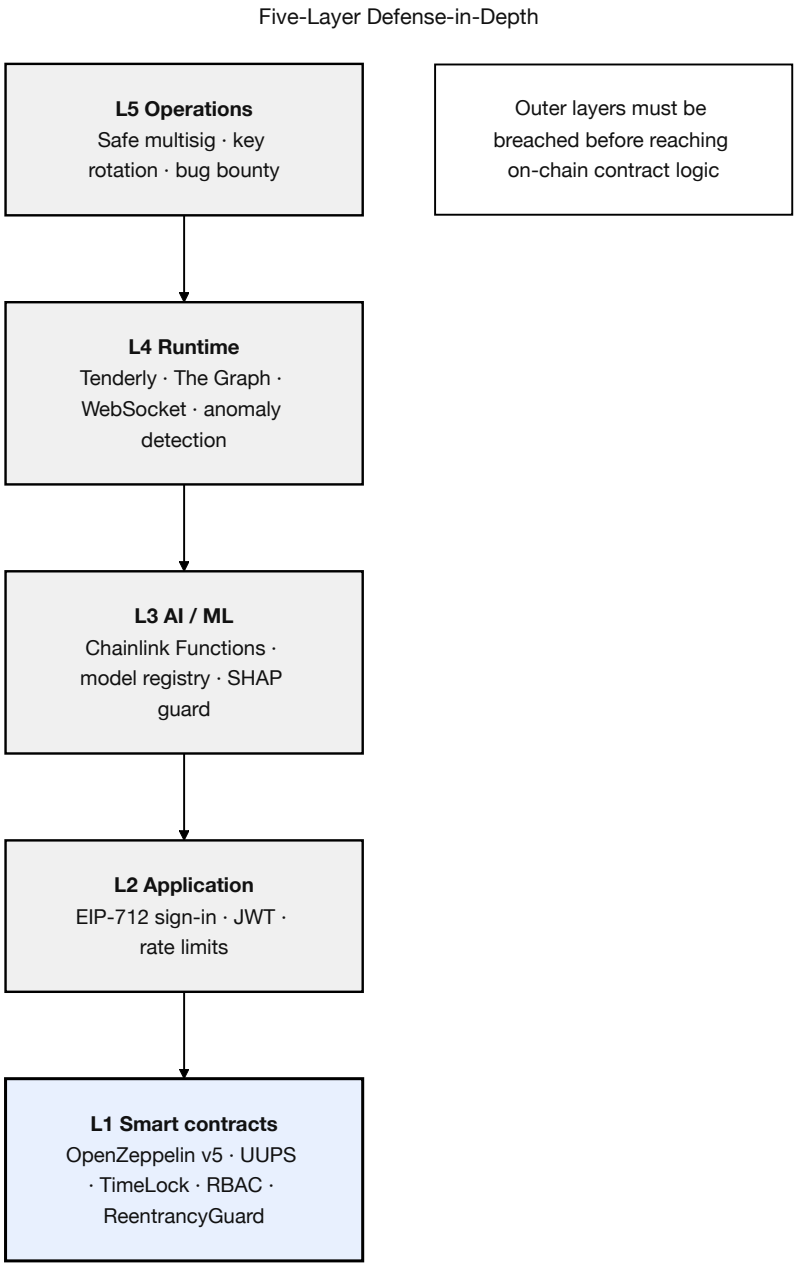


Figure 3.35: Five-layer defense-in-depth security architecture

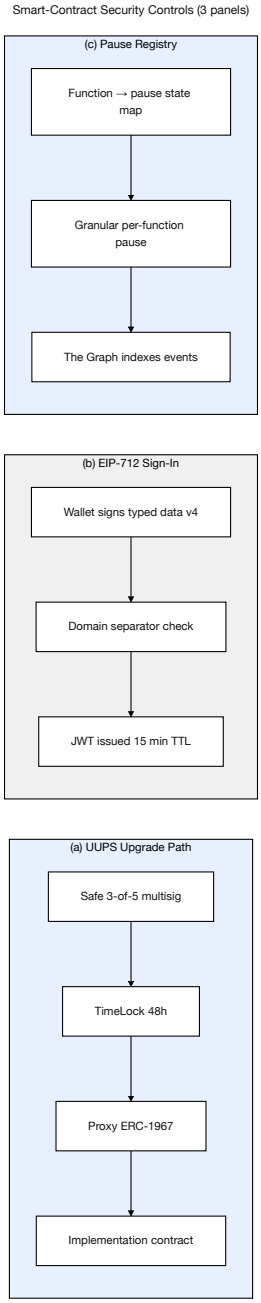


Figure 3.36: Smart-contract security controls

The correct mental model for a blockchain banking platform's security is not "smart-contract security only"—it is a five-layer defense-in-depth stack in which breaching the contract layer requires defeating every layer above it. The Crypto World Bank adopts the following layered model, summarized in Table 3.35 and inspired by current (2025–2026) audit-tooling and runtime-monitoring best practice from Trail of Bits, Cyfrin, Tenderly, and OpenZeppelin Defender.

Table 3.35: Five-layer defense-in-depth model for the Crypto World Bank platform. Each layer is necessary; breaching the contract layer requires defeating every layer above it.

Layer	Component	CWB Design Choice
L5	Operational security	Safe (Gnosis) 2-of-3 multisig for `WorldBankAdmin`; 3-of-5 for global parameter changes; documented key-rotation schedule; hardware-wallet signer for the WorldBankAdmin Safe (Ledger Gen5).
L4	Runtime monitoring	Tenderly alerts on six critical triggers (large disbursement, repeated requests, reserve-ratio drop below 20%, failed `disburseLoan`, role grants, governance pause); PostgreSQL event listener for indexed event history (Graph optional later); transaction simulation pre-deployment.
L3	AI/ML security	Random Forest fraud probability + Isolation Forest anomaly score + SHAP explanation per loan (stored in `LOAN_RISK_ASSESSMENT`, FK to `MODEL_REGISTRY`), delivered via commit-reveal oracle; system-wide events in `SECURITY_EVENT_LOG`; transaction-graph distance to flagged wallets. GNN-based relational fraud analysis is Future Work (Section 6). *Note: session-level behavioral biometrics (page-view timing, terms-acceptance latency) require client-side instrumentation not yet in the schema and are a specified Future Work extension.*
L2	Application security	EIP-712 wallet-signed JWT (15-minute lifetime + refresh); `slowapi` rate limiting (5/min auth, 60/min reads, 10/min writes); Pydantic input range constraints; strict CORS and Content-Security-Policy; environment-variable secret hygiene; prompt injection scanning middleware on all user-sourced fields before context assembly (see below).
L1	Smart contract security	OpenZeppelin `ReentrancyGuard`; CEI pattern; Solidity 0.8.20 overflow protection; RBAC with role expiry; UUPS upgrade with Safe-multisig authorization; TimeLock on governance actions; granular per-function pause; four-tool audit pipeline (Slither, Mythril, Echidna, Halmos/Certora); Foundry fuzz + invariants.

The L1 contract layer is discussed in detail below; Section 3.20 expands the threat model to span all five layers and motivates the specific controls listed in Table 3.35.

3.19.1 Smart Contract Security Controls

Checks-Effects-Interactions (CEI) pattern. The CEI pattern is applied to all state-mutating functions in the lending contracts. The three functions most vulnerable to reentrancy are: (1) `disburseLoan` — update `loanStatus` before any ETH transfer and apply `nonReentrant`; (2) `processInstallment` — check → mark paid → emit event → release interest; (3) `allocateCapital` — apply `nonReentrant` and allocation bounds before any state change. The DAO hack (2016) and Curve Finance exploit (2023) demonstrate that reentrancy remains an active threat [R6]; Slither static analysis and Mythril symbolic execution are planned for the final thesis security audit. Certora formal verification of reserve invariants is scoped to Future Work (Section 6).

Flash loan scope. Flash loan attacks primarily exploit price-sensitive collateral valuation [R5]. The pre-thesis design presents limited flash-loan surface because stablecoin integration and oracle-based collateral pricing are planned for Phase II–III. Mitigations specified include TWAP oracles, multi-block confirmation for large collateral updates, and Chainlink price feeds with circuit breakers.

Upgradeability via UUPS (ERC-1822). All implemented and planned contracts use the OpenZeppelin **UUPS** pattern. `_authorizeUpgrade` is gated on `onlyRole(WORLD_BANK_ADMIN)`, held in production by a Safe 3-of-5 multisig (Section 3.19).

Role expiry timestamps. Every `grantRole` call records an expiry block number; privileged functions check `require(roleExpiryBlock[msg.sender][role] > block.number)`. Block numbers are preferred over `block.timestamp` to avoid SWC-116 timestamp-manipulation risk on EVM chains.

Granular per-function pause. A per-function pause registry (`functionPaused[bytes32 functionId]`) allows governance to suspend only `LOAN_DISBURSEMENT` while keeping `DEPOSITS` and `WITHDRAWALS` open; each pause emits a `FunctionPaused` event indexed by the event listener (Section 4.7).

Failed-attempt lockout and address whitelisting. After 3 failed confirmation responses in a single agent session, the agent pauses interactions for 10 minutes and logs `status = FAILED` to `AGENT_ACTION_LOG`. Detected prompt-injection attempts terminate the session immediately. Loan disbursement destination addresses are fixed at KYC registration and cannot change without a 24-hour delay plus 2FA re-verification.

Prompt injection scanning (Layer 2 — Application). Before any user-sourced string is assembled into the system prompt by the Express.js context injection layer, a scanning middleware checks each field for candidate injection patterns: substrings matching `ignore previous instructions`, `new system prompt`, tool-override phrases, and JSON-escape sequences that could break the structured context block. Fields that trigger the scanner are sanitised (the offending segment is removed and replaced with a placeholder) and `AGENT_ACTION_LOG.injection_scan_flag` is set to TRUE. The fields subject to scanning are: `language` (user-controlled), any text content from income proof document uploads, and the last N conversation turns before they are appended to the Volatile tier. This control directly addresses OWASP LLM01 (Prompt Injection) [R54] and is motivated by Alizadeh et al. [R53], who demonstrate data exfiltration through income document uploads and language fields

in a banking tool-calling agent using attack vectors structurally identical to those present in the CWB context assembly layer.

Per-tier admin-key model. Operational security (L5) is the highest-impact and lowest-cost security decision in the entire project. The `WorldBankAdmin` role is never held by a single externally owned account (EOA) in any environment, including the university lab demo: it is transferred to a Safe 2-of-3 multisig before any contract is deployed to a testnet that examiners will inspect, with Signer 1 on the demo machine, Signer 2 on a phone, and Signer 3 as a paper-wallet backup held by the academic supervisor. National Bank Admin roles use Safe 2-of-3; Local Bank operators use Safe 2-of-3; the only EOA-bound role is the retail CLIENT role, which can be reset off-chain via the social-recovery guardian flow described in Section 3.8.

SWC Registry mapping. Every state-mutating function in the three specified core contracts (World Bank Reserve, National Bank, Local Bank) is mapped to its Smart Contract Weakness Classification (SWC) class [R36] and the corresponding mitigation. This mapping replaces the previous narrative coverage of Atzei et al. [7] with a structured taxonomy, and is the format the Cyfrin audit checklist (2025) recommends for academic prototypes. The full mapping table is in Appendix B's Capability Matrix.

3.20 Threat Model and Security Controls

Security threats in smart-contract banking systems are not limited to isolated code defects; they span application-layer attacks (API abuse, JWT theft, social engineering), runtime evasion, and governance-layer key compromise in addition to the contract-layer issues that DeFi literature focuses on. Table 3.36 extends the original threat-model table to the full five-layer scope established in Section 3.19.

Table 3.36: Threat model and security controls mapping, expanded to span all five layers of the defense-in-depth stack (Section 3.19).

Threat	Layer	Attack Vector	Mitigation
Reentrancy / state manipulation	L1	External calls exploit inconsistent intermediate state	`ReentrancyGuard`; CEI; Foundry invariant tests.
Arithmetic errors	L1	Integer overflow/underflow in financial math	Solidity 0.8.x checked arithmetic; Certora invariants.
Oracle manipulation	L1	Price-feed manipulation on collateral / FX	Chainlink feeds; heartbeat checks; TWAP; circuit breakers.
Flash-loan economic attacks	L1	Atomic borrowing manipulates on-chain assumptions	Conservative price logic; multi-block confirmation; pause-on-anomaly.
Governance capture	L1+L5	Privileged roles abused for unsafe parameters	Role separation; Safe multisig; TimeLock 24–48 h; audit trail.
Sybil abuse (retail / group)	L1+L3	Many wallets bypass limits or fraudulent groups	RBAC; rate limits; ZKP-KYC; GNN wallet-graph clustering.
JWT theft / session hijack	L2	Stolen token gives full account access	15-min JWT; EIP-712 challenge; Redis blacklist; geo rotation.
API abuse / DDoS	L2	Bulk requests exhaust backend services	`slowapi` limits; CSP/CORS lockdown; correlation IDs.
Input-validation bypass	L2	Malformed inputs cause unintended backend state	Pydantic range constraints; wallet regex; SHA-256 doc refs.
ML evasion / adversarial fraud	L3	Crafted features evade RF / iForest	GNN analysis; federated intelligence; SHAP human review.
LLM hallucination (compliance)	L3	Wrong regulatory answer to client	RAG over policy docs; refusal layer; red-teaming protocol.
Monitoring evasion	L4	Attack split below alert thresholds	Multi-trigger alerts; event-listener forensics history (Graph optional later).
Admin-key compromise	L5	EOA compromise gives World Bank control	Safe 2-of-3 / 3-of-5 multisig; hardware-wallet signer.
Blind-signing on hardware wallet	L5	User signs opaque calldata	Calldata decoding; Ledger Multisig; Tenderly pre-simulation.

This expanded threat model is the structural justification for the five-layer architecture in Section 3.19: each row of the table is a vector that the corresponding layer prevents, detects, or recovers from. Application-layer rows (L2) and operational-layer rows (L5) are the largest blind spots in the prior literature on academic DeFi prototypes and are made explicit here. Multi-tenant data isolation for PII-bearing core tables is specified as Future Work in Section 3.4.2.

Chapter 4

Methodology

This project and its planning have been structured in accordance with the Blockchain Olympiad Bangladesh: Guideline and Evaluation Scheme [17] and the final-year project evaluation rubrics.

Scope framing. The final thesis deliverable comprises **blockchain banking** (smart-contract hierarchy, on-chain lending lifecycle, oracle-mediated risk scoring) and a **conversational agent layer** (Qwen3-8B + MCP, Phase III Must). The React UI and agent are co-primary client interfaces; both call the same Express.js API and smart contracts. Write operations through either path require wallet or session-key signing and, for agent-initiated actions, an explicit human confirmation gate. Section 4.4 specifies the agent pipeline; Section 4.3 covers the ML oracle that gates approver decisions independently of the agent.

Capability	Priority	Primary interface / phase
Four-tier smart contracts	Must (Ph. I–II)	React UI + wallet; Hardhat/Foundry tests
On-chain loan lifecycle	Must (Ph. II)	React forms; `LoanController` state machine
Chainlink Functions oracle	Stretch (Ph. III)	FastAPI → DON → on-chain score commit (if supported); commit–reveal is MVT
RF + Isolation Forest + SHAP	Must (Ph. III)	Approver Authority Brief; client decline reasons
Authority Brief UI	Should (Ph. II–III)	Bank approver dashboard (React) — *not* agent-only
MCP conversational agent	Must (Ph. III)	Qwen3-8B + 3–5-tool MCP server; one write demo E2E
LLM evaluation protocol	Should (Ph. IV)	Red-team + task accuracy (Section 4.10.1)

4.1 Minimum Viable Thesis (MVT) Checklist

Examiners should treat the items below as the **graded deliverable boundary** for the final thesis. Everything outside this checklist is either pre-thesis design (this report) or explicitly deferred Future Work. The three **primary research contributions** are Contribution 1 (hierarchical architecture), Contribution 3 (oracle-mediated explainable ML, extended by the Phase III conversational agent), and Contribution 2 (solidarity group lending *specification*); all other modules support or extend these claims.

Table 4.1: MVT module boundary: primary contributions vs. future work.

Module	Status	Examiner expectation
Four-tier smart-contract hierarchy	**Primary (C1)**	Must deploy + test on public testnet; reserve and RBAC demonstrated
Oracle-mediated RF + IF + SHAP pipeline	**Primary (C3)**	Must train, benchmark, and show Authority Brief + audit log (Section 4.3.1)
Solidarity `GroupLendingPool` spec	**Primary (C2)**	Specification + minimal PoC sufficient; full mutual-liability lifecycle is a Phase II deliverable (Table 1.1)
GNN (GraphSAGE) ablation	Future Work	Literature motivation only (Section 2.2.8); not MVT
Federated learning across banks	Future Work	Literature motivation only (Section 2.2.8); not MVT
zkAML / Groth16 zkKYC circuits	Future Work	Off-chain KYC at MVT; circuits deferred (Section 6)
CCIP cross-chain bridge	Assumed infra	Single-chain testnet sufficient for MVT; CCIP is integration, not research novelty
Certora reserve-invariant proofs	Future Work	Foundry fuzz suite is MVT+ stretch (Section 4.5)
MCP conversational agent	**Primary (C3 ext.)**	Must deploy in Phase III: 3–5-tool MCP + chat UI + confirmation gate; one write operation demonstrated E2E (MVT item 11); full 17-tool suite is Future Work
ERC-4337 full Paymaster stack	Stretch	WalletConnect path sufficient for MVT demo
Multi-entity lend / fund / FX suite (IBLP, UpwardDeposit, SavingsVault; SyndicatedLoan, TranchedPool, TreasurySwap)	**Primary arch. (C1 ext.)**	Fully specified Ch. 3 (§3.14); IBLP / UpwardDeposit / SavingsVault in Phase II; Syndicated / Tranched / TreasurySwap in Phase III (Table 1.1); demo of one flow recommended
Syndicated / tranched lending suite	Stretch (Phase III)	Specified; demo of one flow recommended
NettingEngine (partial settlement, challenge periods)	Future Work	ERD stubs only; Section 6

Table 4.2: MVT deliverable checklist: numbered acceptance criteria.

#	Deliverable	Phase	Done when (acceptance criterion)
1	World / National / Local Bank contracts deployed on Polygon Amoy and/or Ethereum Sepolia	I	Addresses in Appendix C manifest; role-gated functions callable
2	Loan request → approver review → disbursement → one installment repayment (E2E)	II	Screen recording + tx hashes on block explorer
3	Reserve-ratio / borrowing-limit enforcement visible on-chain	II	Hardhat or Foundry test proves revert on violation
4	FastAPI ML service: RF + IF + stacking → composite score s	II–III	`POST /score` returns JSON in $<$50 ms on laptop
5	SHAP + anomaly deviation report + Authority Brief UI	II–III	Approver sees brief on sample loan; decline shows top-3 SHAP reasons
6	`LOAN_RISK_ASSESSMENT` row per inference	III	DB row contains `shap_vector`, `model_id` (FK → `MODEL_REGISTRY`), `loan_request_id`
7	Commit-reveal score commit before approval (Chainlink Functions stretch if supported)	III	Contract in `SCORE_REVEALED` before `approveLoan` succeeds
8	BCCC-DeFiFraudTrans benchmark table (precision, recall, F1, AUC)	II–III	Table in Section 4.3.1: *pipeline validation* on synthetic stand-in now; full BCCC table after dataset access
9	300-client Foundry simulation OR documented gas-cost table	IV	Appendix C manifest + reserve dynamics output
10	Open-source repo README with MVT run instructions	IV	Panel can reproduce steps 1–5 and agent loan-apply demo (item 11) from README
11	MCP conversational agent: 3–5-tool server + chat UI + human confirmation gate	III	Screen recording: client submits loan via agent; `AGENT_ACTION_LOG` row + confirmation turn recorded; write blocked without consent
12	LLM evaluation protocol (task accuracy, action accuracy, zero bypass)	IV (Should)	Section 4.10.1 metrics reported if schedule permits; red-team set started in Phase III
Optional (not graded for MVT pass): GNN ablation, Certora proof, zkAML, CCIP bridge.			

Pre-thesis vs. final thesis. At **pre-thesis submission**, items 1–8 and 11 may remain *specified* rather than *deployed*; this report satisfies that stage by providing contract interfaces, the ML workload plan (Section 4.3.1), agent pipeline specification (Section 4.4), diagrams, and the checklist above. At **final thesis submission**, items 1–8 and 11 are **Must** for a defensible pass; items 9–10 and 12 are **Should**. Figure 4.1 summarises which MVT items are satisfied at pre-thesis versus deferred to the final thesis.

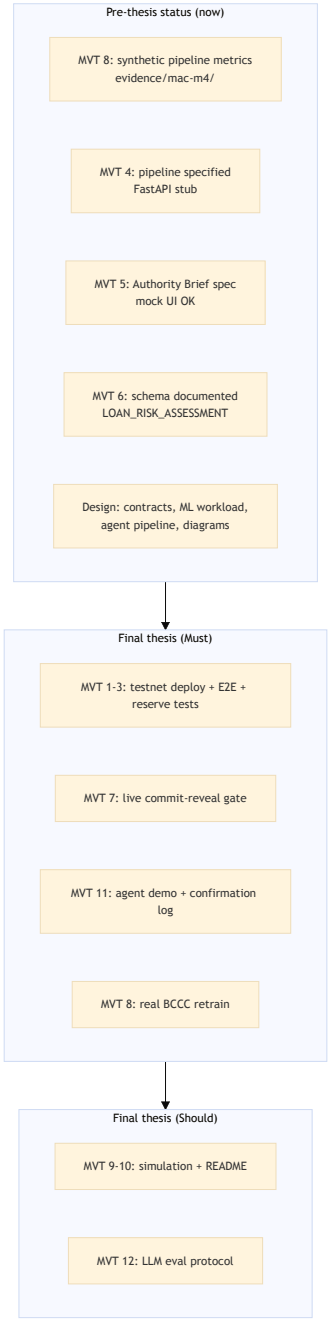


Figure 4.1: MVT deliverable status at pre-thesis

4.2 Development Methodology

We adopted a **lightweight Agile/Scrum** methodology tailored for an academic prototype with a fixed **16-week** development window (four phases, Weeks 1–16). The iterative ap-

proach enables incremental delivery of demonstrable features while accommodating evolving requirements inherent in research-oriented development. Figure 4.8 maps these phases to SDLC stages and supervisor gates.

The following points summarize how Agile will be applied:

- **Phase Duration:** A period of 3–5 weeks is allocated depending on the phase (4 phases total; 16 weeks overall).
- **Team Size:** A group of two developers will be assigned (thesis project). The work will be focused on the thesis project.
- **Weekly Sync:** A weekly gathering will occur to assess progress. Issues will be identified during these meetings. Plans will also be developed in these sessions alongside progress checks.
- **Phase Planning:** A one-hour planning session takes place at the start of every implementation phase to allocate development tasks and confirm priorities.
- **Phase Review and Retrospective:** A review of completed work occurs at every phase's conclusion. A 30-minute retrospective discussion captures lessons learned for the subsequent phase.
- **Supervisor gates:** A formal supervisor meeting at the end of each implementation phase (G1–G4; Section 4.12). The team demonstrates runnable artifacts (deployed contracts, E2E recordings, test output) and obtains written scope confirmation before opening the next phase's Must tasks.

4.3 Planned AI/ML Support and Risk-Score Wiring

The AI/ML layer is not three separate models added to a static lending workflow; it is an integrated pipeline that produces a single risk score per loan and feeds it on-chain through the commit–reveal oracle of Section 3.3.2. Figure 4.2 shows the MVT oracle gate; Figure 4.6 covers explainability outputs. The pipeline has four stages:

1. **Feature engineering.** For each loan application, the FastAPI service computes 18 behavioral features (rolling-window transaction count, average inter-transaction interval, on-chain repayment history, group-membership flag, wallet-graph distance to flagged addresses) and 4 application features (requested amount, KYC level, income-hash freshness, ZKP credential expiry).
2. **Risk scoring.** A Random Forest classifier returns a fraud probability $p_f \in [0, 1]$ (a true calibrated probability, computed via Platt scaling applied to the raw tree vote proportion). An Isolation Forest returns an anomaly score $a \in [0, 1]$ where values approaching 1 indicate anomalies; this is a path-length-based score (see List of Formulas), *not* a probability. The two outputs are combined via a stacking meta-learner (logistic regression) fitted on a held-out validation set, yielding a calibrated composite score $s \in [0, 1]$. The default meta-learner weights are approximately 0.7 for p_f and 0.3 for the calibrated anomaly signal; in the prototype these serve as initial placeholders to be re-fitted on any labeled data that becomes available. A direct linear combination of a probability and a non-probability score would produce a statistically uninterpretable output; the stacking approach ensures s is always a valid probability estimate.
3. **Commit-reveal oracle.** The service publishes $h = \mathrm{keccak256}(s \,\|\, \text{nonce})$ to the LoanController contract via `commitRiskScore`, then reveals (s, nonce) in a subsequent transaction. The LoanController enforces a state machine: approval is only

permitted in the `SCORE_REVEALED` state, preventing any approver from bypassing the ML step. Section 3.3.2 covers the cryptographic detail.

4. **Decision threshold.** The contract applies a governance-configured threshold calibrated against the expected fraud rate: $s < 0.4$ enables approver discretion to approve, $0.4 \leq s < 0.7$ requires mandatory manual review, and $s \geq 0.7$ rejects the loan with an on-chain reason code. These thresholds are prototype defaults derived from the class imbalance expected in DeFi lending datasets (estimated fraud prevalence 2–5%); they must be recalibrated against the actual training data distribution before any production deployment. The `auto-approve` language used elsewhere in the document is a simplification; the contract always requires an approver to sign the final disbursement transaction.

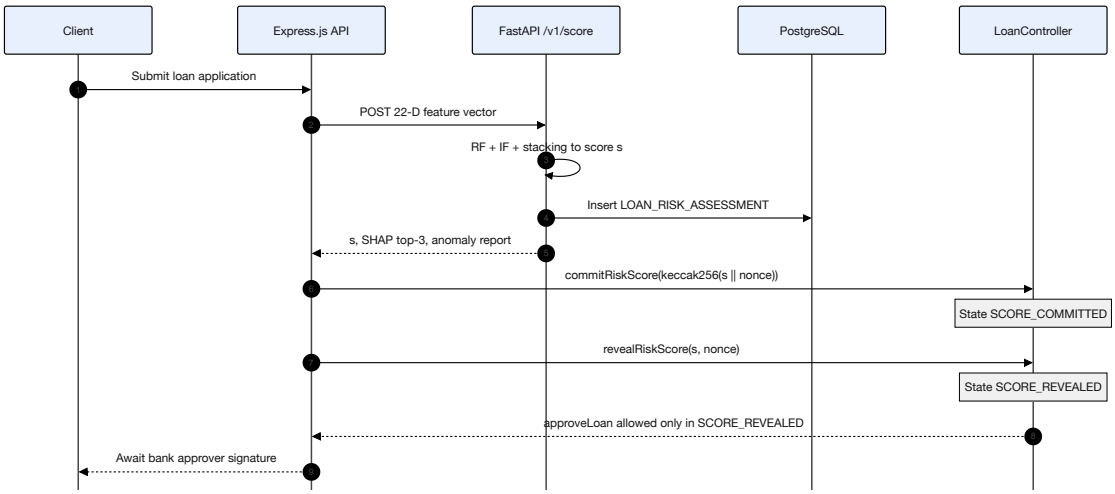


Figure 4.2: ML oracle commit–reveal gate

4.3.1 Datasets, Training Workload, and Pre-Submission Deliverables

Professors evaluating AI/ML content typically require the **full workload documented**: which datasets are used, how features are derived, how models are trained and validated, what artifacts exist at submission time, and how scores connect to the lending UI. This subsection supplies that detail; Figures 4.4 and 4.3 document the reproducible evidence chain; Section 4.3.2 covers explainability outputs.

Dataset access status (pre-thesis). BCCC-DeFiFraudTrans-2025 requires institutional approval from York University BCCC; a dataset request has been **submitted** and is pending verification. Until access is granted, the Mac M4 training pipeline (`ML pipeline - Mac M4/`) uses `synthetic_bccc.csv`—a procedurally generated CSV with 60,000 rows and 79 numeric features plus a binary `fraud` label, matching the *shape* of the BCCC corpus but not real Ethereum DeFi transaction behaviour. All figures and `metrics.json` in `evidence/mac-m4/` are explicitly tagged `synthetic: true`; the corresponding PDF figures are copied to `Diagrams/fig-ml-metrics-bars`, `fig-ml-confusion-matrix`, and `fig-ml-shap-importance` for inclusion in Figure 4.5. Upon BCCC download, the same pipeline will be rerun with `-reset` on the approved CSV; measured values in Figure 4.5 and MVT item 8 will be updated without architectural changes.

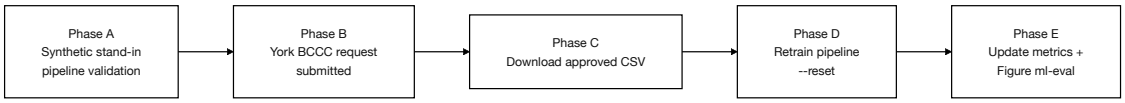


Figure 4.3: Dataset timeline

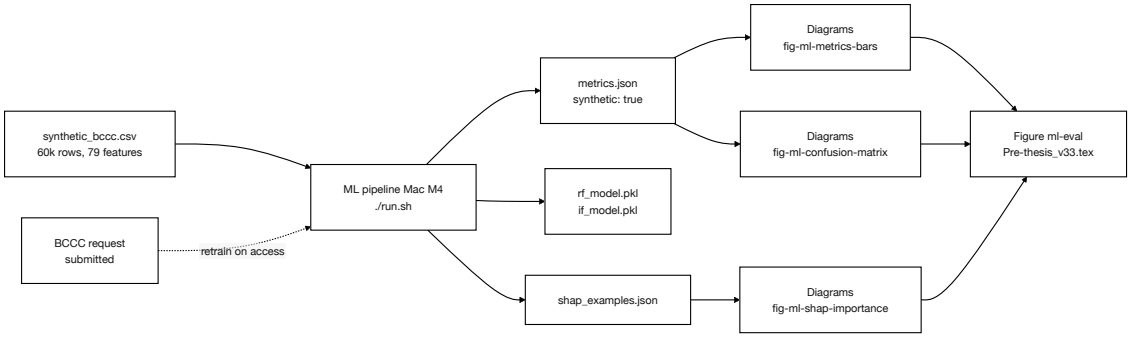


Figure 4.4: ML evidence pipeline

Table 4.3: Datasets used for ML risk scoring and evaluation (RQ3).

Dataset	Role	Usage in this project
BCCC-DeFiFraudTrans-2025 [R47,108]	Primary (supervised)	1,026,867 annotated DeFi transactions, 79 features (York BCCC 2025); **access requested, pending**; target train/test split for Random Forest; report precision, recall, F1, ROC-AUC on held-out test
Synthetic BCCC-shaped stand-in (pre-thesis)	Pipeline validation	60,000 rows, 79 features + `fraud` label; `synthetic_bccc.csv`; **current** pre-thesis metrics (`evidence/mac-m4/`); not a BCCC benchmark claim
Elliptic Bitcoin Dataset [R48]	Baseline comparison	Secondary benchmark only; domain mismatch (UTXO vs. account-model DeFi) acknowledged in evaluation prose
Palaiokrassas multi-chain set [3]	Feature-engineering precedent	Informs behavioral feature design; optional GNN extension only
Synthetic Foundry loan manifest (Phase IV)	Integration test	Procedurally generated wallets for end-to-end UI demo; labeled "proof-of-concept only" in RQ3 evaluation
On-chain CWB transaction log (future)	Production retrain	Local Bank history replaces synthetic features once MVT deployment has live traffic

Training pipeline (offline workload). The offline training job (`train_risk_models.py` in the planned FastAPI service) executes the following steps in order:

1. **Ingest** BCCC-DeFiFraudTrans CSV; drop rows with missing labels; stratified 70/15/15 train/validation/test split with fixed seed (42) for reproducibility.
2. **Map features** — 79 BCCC columns are reduced to the 22-dimensional CWB feature vector (18 behavioral + 4 application) via the mapping in Table 4.4; unmapped columns are logged and excluded.
3. **Train Random Forest** — `sklearn.ensemble.RandomForestClassifier`, $n_{\text{estimators}}$=200, `max_depth=12`, `class_weight=balanced`; Platt scaling on validation fold for calibrated p_f.
4. **Train Isolation Forest** — `sklearn.ensemble.IsolationForest`, trained on validation-set *non-fraud* rows only; anomaly scores normalised to $[0, 1]$.
5. **Fit stacking meta-learner** — logistic regression on (p_f, s_{IF}) using validation labels; export coefficients and intercept to `meta_learner.json`.
6. **Evaluate** on held-out test set; write metrics to `metrics.json` and confusion matrix to `eval_report.pdf`.
7. **Serialize** — `rf_model.pkl`, `if_model.pkl`, `meta_learner.json`, `feature_schema.json`; register `artifact_hash` in `MODEL_REGISTRY` (referenced by `LOAN_RISK_ASSESSMENT`).

Table 4.4: Mapping from BCCC / on-chain sources to the 22-dimensional CWB feature vector.

#	CWB feature	Definition	Source
1–5	Tx count, velocity, interval stats	Rolling 30/90-day windows	BCCC + on-chain indexer
6–10	Repayment history, default flags	Prior loan performance	CWB `LOAN` / `INSTALLMENT`
11–15	Wallet-graph distance, group flag	Graph hops to flagged addresses	BCCC graph features
16–18	Pool utilisation, tier, age	Institutional context	On-chain `eth_call`
19–22	Amount, KYC level, income-hash freshness, ZKP expiry	Application-time inputs	Loan form + identity service

Online inference workload (runtime). At loan submission, the Express.js backend POSTs the feature JSON to FastAPI `/v1/score`. The service loads serialized models, returns $\{p_f, s_{\text{IF}}, s, \text{shap\_top3}, \text{anomaly\_deviations}\}$ in under 50 ms (laptop target), writes one `LOAN_RISK_ASSESSMENT` row (FK to `MODEL_REGISTRY`), and triggers the commit-reveal step (Section 3.3.2). The approver dashboard polls `/v1/brief/{loan_id}` for the Authority Brief.

Deliverable	Before pre-thesis (now)	Before final thesis (MVT)
Dataset documentation + feature mapping (Tables 4.3–4.4)	**Yes** — this report	Maintain + update with real column names
Train RF + IF on BCCC; export `.pkl` + `metrics.json`	Partial — pipeline validated on **synthetic** mini-subsample (`evidence/mac-m4/metrics.json`); BCCC request submitted	Retrain on approved BCCC CSV; update figures
SHAP TreeExplainer + 5 worked explanation examples (PDF/screenshots)	**Yes** — recommended	Required for RQ3
FastAPI `/v1/score` + `/v1/brief` endpoints (local)	**Yes** — recommended	Wire to live loan form
Authority Brief React component (mock or live data)	Partial — mock OK	Live data from inference service
`LOAN_RISK_ASSESSMENT` + `MODEL_REGISTRY` schema + sample rows	**Yes** — schema in Ch. 3	One row per real inference
On-chain commit-reveal / relay integration	Design only	Must — item 7 in Table 4.2
Chainlink Functions DON commit	Future Work	Stretch beyond centralized relay
GNN / federated learning	Future Work only	Not MVT

Evaluation metrics (RQ3): report precision, recall, F1, and ROC-AUC on a held-out test set; separately report explainability completeness (% of declined loans with $\geq$3 SHAP reasons and anomaly deviation lines). **Pre-thesis figures use synthetic pipeline validation only** (`synthetic: true` in `evidence/mac-m4/metrics.json`); these metrics establish RQ3 *pipeline viability* and must not be presented as BCCC-DeFiFraudTrans-2025 fraud-detection benchmarks. Synthetic-data metrics from the Foundry loan simulation (Section 4.6) are a separate integration-test layer, likewise labeled "proof-of-concept only." Figure 4.5 reports measured held-out metrics from the Mac M4 pipeline validation run on the synthetic stand-in (stratified mini-subsample: n_{train}=21,000, n_{test}=4,500, seed 42). Full 1,026,867-row BCCC training, Elliptic comparison row, and 22-dimensional CWB feature mapping remain scheduled for Phase III upon dataset access.

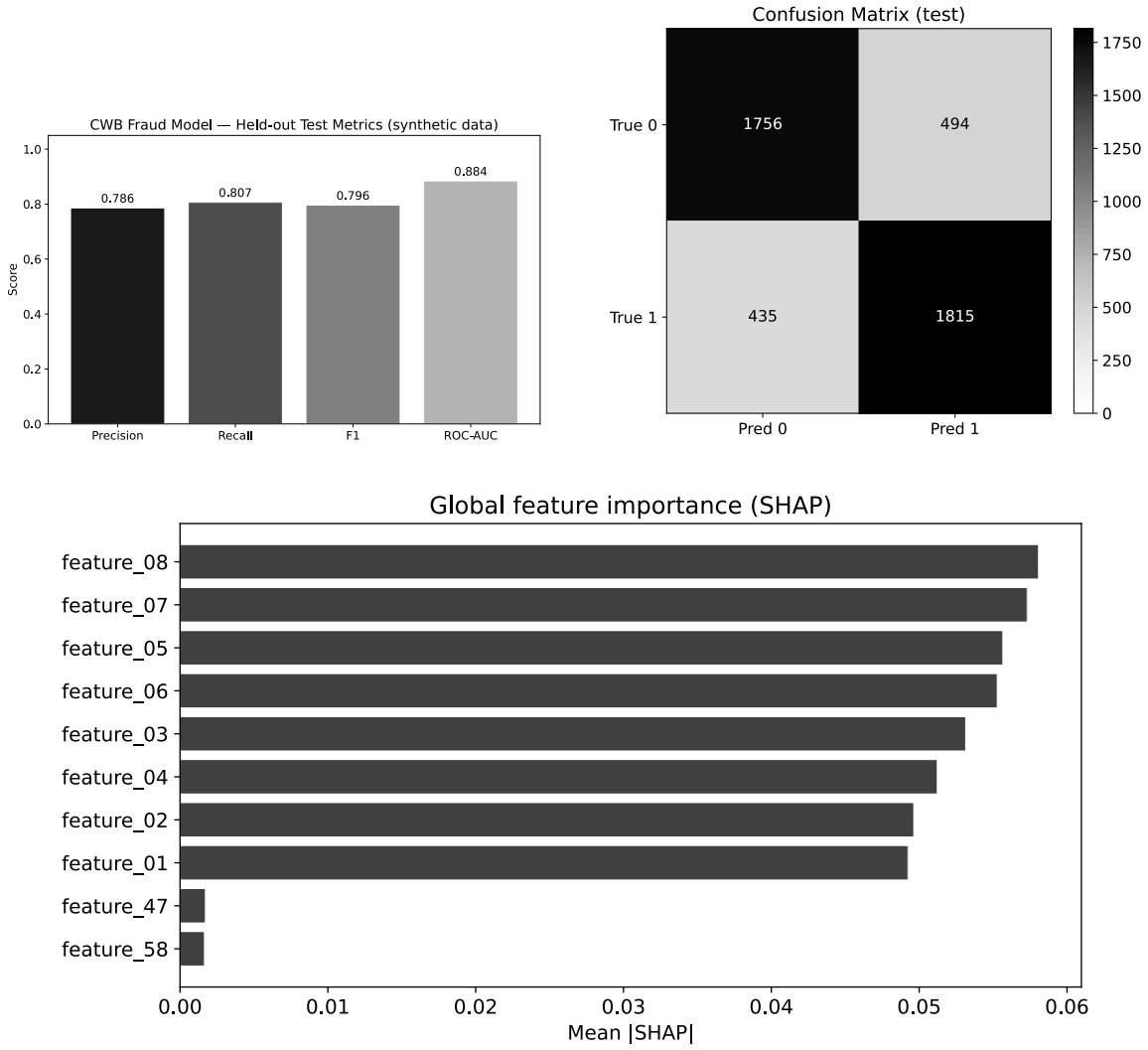


Figure 4.5: ML evaluation and explainability benchmarks

Domain-transfer limitation (examiner-critical). BCCC-DeFiFraudTrans-2025 [R47,108] contains Ethereum DeFi fraud from flat pool protocols (Aave, Compound, Uniswap). CWB hierarchical lending generates different transaction patterns: institutional batched flows, tier-privilege escalation risk, and group-liability disbursements. **This is the most serious technical weakness of the ML contribution.** BCCC is used for baseline fraud-detection benchmarking only; production deployment would require CWB-specific labeled data from testnet operation. A domain-adaptation study comparing BCCC feature distributions against simulated CWB transactions is scoped as Future Work (Section 6). Semi-supervised baselines [124] should be compared before claiming production readiness.

Oracle latency, cost, and threat model. Chainlink Functions requests incur 5–30 s latency and approximately $0.10–$1.00 per invocation [123]. For a $50 microloan at 15% APR, a $0.50 oracle call at origination represents roughly 1% of annual interest—acceptable at loan origination but prohibitive for continuous rescoring. The design therefore computes ML scores *at origination only* (not on every installment), with score caching and batch scoring for group applications. Oracle manipulation by colluding DON nodes is mitigated by DON consensus but not eliminated; model-integrity assurance (TEE-verified execution, commit-

reveal fallback) is documented in Section 3.3.2 and remains an open threat-model item for examiners.

4.3.2 Explainability and Anomaly Detection Methodology

This subsection documents the full explainability and anomaly-detection pipeline: not only *that* ML models flag risk, but *how* fraud probability and anomaly signals are produced, combined, explained to humans, and audited. The treatment is independent of the conversational agent interface (Section 4.4).

Random Forest — supervised fraud probability. For each loan application, the FastAPI service assembles a 22-dimensional feature vector (18 on-chain behavioral features + 4 application features). A trained Random Forest ensemble outputs a fraud probability $p_f \in [0, 1]$ after Platt scaling. **SHAP TreeExplainer** computes exact Shapley attributions ϕ_i for every feature: positive ϕ_i increases predicted fraud risk; negative ϕ_i decreases it. TreeExplainer is used rather than LIME because repeated explanations on identical inputs are consistent—a property regulators expect when reviewing automated lending support [4].

Isolation Forest — unsupervised anomaly detection. Labelled fraud is scarce in DeFi lending logs. Isolation Forest (IF) addresses *novel* behaviour without requiring fraud labels. The algorithm builds an ensemble of isolation trees: at each node it selects a feature and split value at random and partitions the sample space. **Anomalies are easier to isolate**—they require fewer random splits to separate from the bulk of training data—so their average path length $E(h(x))$ is shorter. The anomaly score is

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

where $c(n)$ normalises path length for sample size n (see List of Formulas). Scores near 1 indicate strong anomaly; scores near 0.5 indicate typical behaviour. IF is trained only on *non-fraud* historical transactions at each Local Bank so that coordinated attack patterns not present in labelled data still surface as outliers.

Combining fraud probability and anomaly signal. Raw IF scores are not probabilities. A stacking meta-learner (logistic regression on a held-out validation set) maps (p_f, s_{IF}) to a single calibrated composite score $s \in [0, 1]$ used by the oracle and contract threshold logic. Default meta-learner weights ($\approx$0.7 on p_f, $\approx$0.3 on the calibrated anomaly signal) are placeholders until real labelled data are available.

Explaining anomalies to human reviewers. Because IF lacks native Shapley values, anomaly explainability uses a **feature-deviation report**: for each of the top-5 features, the service records the applicant's value, the training-set median, and the percentile rank. Features with extreme percentiles (e.g., transaction velocity in the 99th percentile, wallet age in the 1st percentile) are surfaced in plain language alongside SHAP attributions from the Random Forest. Example approver-facing text: *"Anomaly flag: repayment interval variance is in the 98th percentile of historical clients; wallet graph distance to a flagged address is unusually low (2 hops)."*

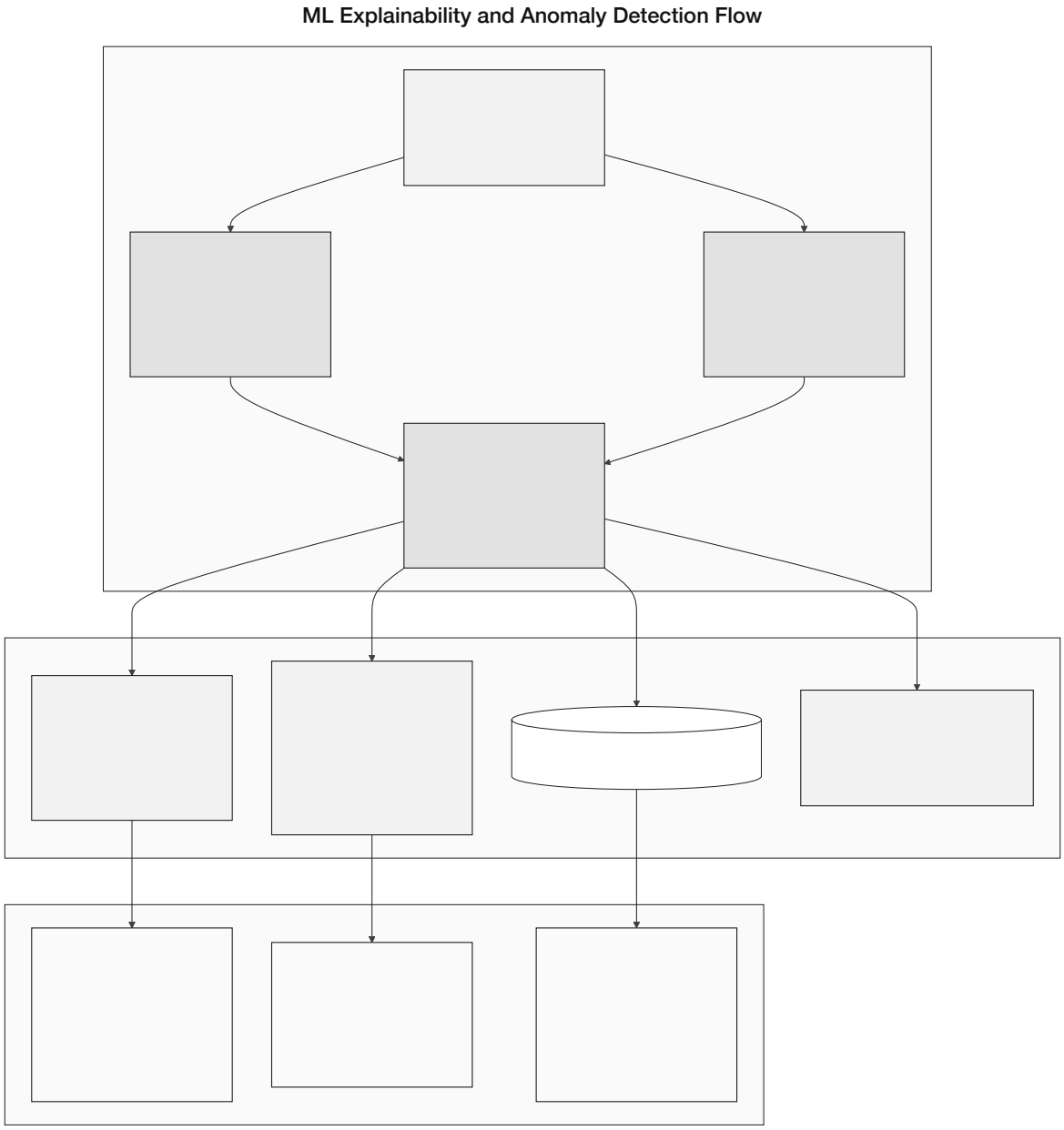


Figure 4.6: Explainability and anomaly-detection flow

Audience	UI / channel	Content stored and displayed
Retail client	React loan status page	Top-3 SHAP reasons in plain language on decline; optional anomaly summary if score ≥ 0.7
Bank approver	Authority Brief dashboard	SHAP waterfall + anomaly deviation table + composite s; one-click approve / decline
Compliance auditor	`LOAN_RISK_ASSESSMENT` + `MODEL_REGISTRY`	Full SHAP vector (`shap_vector`), anomaly score, feature vector hash, model artifact hash, oracle commit tx (via `LOAN_REQUEST`)

Authority Brief UI (core ML, not agent-dependent). When any loan application is submitted—via the React UI or the agent—the approver dashboard renders an Authority Brief with ML risk score, SHAP breakdown, and anomaly deviation lines (see verbatim example in Section 4.4). The brief is produced by the FastAPI scoring service; the agent may additionally surface it when filing on behalf of a client.

Audit trail and human-in-the-loop invariant. Every inference writes one row to `LOAN_RISK_ASSESSMENT: {loan_request_id, model_id, fraud_probability, anomaly_score, shap_vector, feature_vector_hash, created_at}` with model lineage in `MODEL_REGISTRY`; system-wide AML alerts append to `SECURITY_EVENT_LOG`. The contract never auto-disburses on ML output alone; an approver (or client via wallet) must sign the disbursement transaction. ML is **decision support**, not decision authority—consistent with EU AI Act Art. 86 and the thesis ethical statement.

Timeliness: anticipatory alignment with EU AI Act Annex III. The EU AI Act classifies AI systems used to evaluate the creditworthiness of natural persons as high-risk under Annex III, with conformity obligations applying from August 2026 [126]—a deadline that falls within the timeline of this thesis. The explainability pipeline specified above already produces several of the artifacts such systems require: per-inference SHAP attributions and feature-deviation reports (Art. 86 right-to-explanation), an immutable `LOAN_RISK_ASSESSMENT` row per decision (Art. 12 automated logging), and a mandatory human-approver gate before disbursement (Art. 14 human oversight). Section 5.10 maps these elements against the full Annex III conformity checklist; the explainability design in this section should therefore be read not only as a fraud-detection and regulator-facing audit mechanism, but as an early-stage technical-documentation baseline for the Annex III conformity assessment that production deployment would need to complete.

Future Work note: session-level behavioral biometrics are a specified extension requiring `session_events` instrumentation; the 18 core on-chain transaction features above are the active feature set at pre-thesis stage.

4.4 Conversational Agent (Phase III–IV)

The React dashboard with wallet-signed transactions and the CWB agent layer are **co-primary client interfaces**. The agent is a **Phase III Must deliverable** (LLM evaluation in Phase IV, MVT items 11–12) that sits alongside the standard web interface, not a replacement for it. The full MCP tool schema is in Table 4.5; ML explainability for approvers is documented in Section 4.3.2 and does not depend on the agent.

Agent six-step pipeline (summary). Clients may perform every banking operation directly through the UI; for clients who prefer plain-language interaction — particularly first-time users unfamiliar with DeFi form flows — the agent provides an equivalent path. Both the manual UI and the agent call the same Express.js API endpoints and the same smart contracts; the agent does not gate or modify the manual flow. The six-step pipeline integrates the Qwen3-8B model with the live on-chain context injection layer (Section C.1) and the MCP tool server described below:

1. **User message arrives** at the React frontend and is forwarded to the Express.js backend over Server-Sent Events (SSE).

2. **Agent brain (Qwen3-8B) reads live on-chain context.** The Express.js context injection layer prepends the client's full account state (loan status, SBT risk tier, pool utilisation, credit score, active installments) as structured JSON into the system prompt before the model processes the user message.
3. **Q&A path.** If the request is informational, the agent retrieves relevant policy documents via RAG (ChromaDB) and generates a grounded reply. No tool calls are made.
4. **Action request path.** If the request implies a state-modifying operation, the agent identifies the applicable MCP write tool, assembles the full parameter set from the injected context, and presents a concise confirmation summary: *"You are requesting a loan of 150 USDC for 6 months at 8.5% APR. Your monthly installment would be 26.25 USDC. Shall I proceed?"*
5. **Human confirmation gate.** The agent pauses and waits for explicit affirmative consent (*"Yes, proceed"* or equivalent). No write tool is ever called without a confirmation turn in the conversation history. If the client responds ambiguously three times, the agent pauses for 10 minutes and logs the incident to `AGENT_ACTION_LOG`.
6. **Execute via MCP write tool, monitor, and notify.** After confirmation, the agent calls the appropriate write tool, which POSTs to the Express.js banking API. If an EIP-7702 session key is active, it signs the resulting on-chain transaction within its approved scope. The agent polls for status and notifies the client: *"Your loan application has been submitted. Reference: L-4821. Bank approval typically takes 24 hours."*

MCP tool server (MVT: minimal tool subset). At MVT scope the agent interacts with CWB through a **minimal typed tool schema** (3–5 tools). This schema is the safety boundary: the agent cannot access any data or perform any operation not explicitly defined in the tool list below. The full 17-tool banking suite is an explicit Future Work expansion after the MVT is stable and secured.

Table 4.5: MCP tool server (MVT subset): minimal typed tools exposed to the conversational agent. The full 17-tool suite is Future Work.

Tool	Type	Maps to / Description
`get_account_state`	READ	SBT tier, loan count, savings balance
`get_loan_status`	READ	Active loans + next installment
`get_requirements`	READ	Documents + KYC needed for a given loan amount
`submit_loan_-application`	WRITE†	POST /api/loan/apply
`pay_installment`	WRITE†	POST /api/installment/pay

†*All write tools require an explicit human confirmation step before execution. The agent assembles the full parameter set, presents a plain-language summary, and waits for affirmative consent. No write tool is ever called without a confirmation turn recorded in the conversation history. Each tool maps to the same Express.js API endpoint used by the standard web UI; the agent calls these endpoints on behalf of the client only after receiving explicit confirmation, mirroring the deliberate action a client would take manually.*

Authority Brief example (approver dashboard). The Authority Brief is rendered on every loan application (React UI or agent). It presents the ML risk assessment from Sec-

tion 4.3.2 in human-readable form:

```
AUTHORITY BRIEF -- Loan Application #4821
------------------------------------------------
Client tier:          Silver  (score: 512)
Requested amount:     150 USDC
Duration:             6 months
Collateral:           75 USDC (50% LTV)

ML Risk Score:        0.23  (LOW RISK)
SHAP breakdown:
  + On-time repayment rate:    +0.41  (reduces risk)
  + Loan-to-income ratio:      +0.18  (within safe range)
  - Wallet age:                -0.09  (account < 6 months)
  - No prior completed loans:  -0.27  (no repayment history)

ML system recommendation:   APPROVE (decision support only)
Suggested interest:     Base rate - 0.5% (Silver tier)

[ APPROVE ]   [ REQUEST MORE INFO ]   [ DECLINE ]
```

The React UI displays the decision to the client; the agent may additionally monitor the on-chain approval event and notify the client (Phase III).

4.4.1 Reasonable custom-model angle (minimal but publishable)

Model A: domain-constrained intent + parameter extraction. To make the agent safer and more controllable than a single end-to-end LLM, the thesis includes a small **intent router** trained to classify user utterances into a closed set of banking intents and extract a typed parameter object. The model's output is validated against the MCP tool schema, and any missing/invalid fields force a clarification turn (no tool call).

- **Label set (examples):** `GetAccountState`, `GetLoanStatus`, `GetRequirements`, `SubmitLoanApplication`, `PayInstallment`, `OutOfScope`.
- **Training data:** a project-owned utterance set built from (i) templated paraphrases (loan amount/duration/APR), (ii) curated real prompts from pilot testers (anonymised), and (iii) adversarial prompts (`OutOfScope`, prompt-injection attempts). Labels include the intent and a JSON target for parameters (e.g. `{amount_usdc:150, months:6}`).
- **Model choice:** a compact text encoder (e.g. DistilBERT/MiniLM) fine-tuned for intent classification plus a lightweight token/slot extractor, or a single sequence-to-JSON model constrained by a JSON schema; either is trained and evaluated offline and versioned in the `MODEL_REGISTRY`.
- **Maintenance:** new utterances from `AGENT_ACTION_LOG` are sampled weekly; misclassifications are added to the labeled set; a regression suite asserts no new build increases `OutOfScope` false negatives or write-intent false positives beyond a fixed threshold.

Model B: Authority Brief generator calibration. The Authority Brief is safety-critical because it influences bank approver decisions. Instead of letting the LLM freely summarise

risk, the thesis specifies a calibrated generator constrained to: (i) only cite fields present in the structured risk object (`score`, top-k SHAP features, rule-based reason codes), and (ii) always emit a fixed-format template.

- **Inputs:** the canonical `LoanRiskAssessment` record (model version hash, calibrated score, SHAP top-k, anomaly score, policy thresholds, and a deterministic rule trace).
- **Training/evaluation:** collect a small gold set of 50–100 briefs written by a supervisor/approver using the same inputs; fine-tune a compact model or apply preference calibration (pairwise "better brief" judgments). Evaluate with factuality checks (no field hallucination), template compliance rate, and approver agreement on the final decision rationale.
- **Maintenance:** any change to thresholds, feature schema, or model versions triggers regeneration of the gold set for a sample window and a re-run of the factuality+format tests before deployment.

Harness implementation detail (appendix). Per-user context injection, three-tier prompt assembly, session lineage, context compression, toolset scoping, and lifecycle-hook middleware are fully specified in Appendix B. Section 4.4 summarises the six-step pipeline; the appendix documents Phase III–IV implementation detail.

Agent capabilities (installment reminders, credit-tier coaching, EMI calculator, group-signature coordination, KYC guidance) reuse the same MCP tool server without additional model infrastructure; see Appendix B.

4.5 Foundry Invariant and Fuzz Test Suite

The planned test stack combines a Hardhat JavaScript suite for integration tests and deployment scripts with a parallel Foundry suite for unit-level fuzzing and invariant testing because the two frameworks catch different bug classes. The industry pattern is Hardhat for integration, Foundry for invariants and stateful fuzzing.

Three invariants asserted under Foundry's stateful fuzzer.

- Solvency: $\mathrm{totalOutstandingLoans} \leq \mathrm{getTotalReserve}$.
- Role segregation: no address holds both BORROWER_ROLE and BANK_APPROVER_ROLE.
- Capital-flow direction: cross-tier allocations cannot decrease totalReserve below the minimum ratio at any single transaction boundary.

Each invariant is planned to be asserted under 10,000 fuzz runs; counterexamples will be saved as Foundry replay tests so any regression in a future commit fails CI deterministically.

4.6 On-Chain Economic Feasibility Simulation

The revenue projection of approximately \$138–159 M in Chapter 5 (spread-based accounting) assumes 1 World Bank, 5 National Banks, and 50 Local Banks at near-full capacity. To make this projection empirically grounded rather than a single-point estimate, the methodology specifies a **scripted on-chain simulation** of N clients across a six-institution hierarchy, to be run on the chosen prototype testnet (Amoy/Sepolia) in Phase IV. The simulation

is fully reproducible via a published deterministic seed and Foundry script and will generate the gas-cost data documented in Appendix C (Phase IV deliverable).

Simulation scope. The planned simulation deploys 1 World Bank Reserve, 2–3 National Banks, and 4–6 Local Banks, generating 300 synthetic wallet addresses acting as Tier 4 retail clients across a 12-month compressed lifecycle. A configurable 5% fraud rate introduces mid-lifecycle defaults for stress testing. Deployment targets the chosen prototype testnet (Amoy/Sepolia); scripting uses Foundry with a deterministic seed (SEED = 42) for full reproducibility. The simulation output (a CSV manifest of per-client outcomes, per-bank reserve ratios, and gas costs) feeds directly into the RQ5 evaluation.

Simulation script design. Foundry deployment scripts with deterministic seeds (a) deploy all tier contracts, (b) fund each bank with testnet USDC from a seed distributor wallet, (c) generate 300 client wallets, (d) run the full loan lifecycle for each client (`applyLoan` → `approveRisk` → `signLoan` → `disburse` → `repay` $\times N_{\text{installments}}$), (e) introduce the configured fraud rate by selecting clients who default mid-lifecycle, and (f) export all on-chain transaction hashes to a JSON manifest for Appendix C.

```
// Foundry script: script/RunSimulation.s.sol
function run() public {
  uint256 NUM_CLIENTS = 300;
  uint256 NUM_BANKS   = 6;
  uint256 SEED        = 42; // deterministic

  BankHierarchy memory h = deployBankHierarchy(NUM_BANKS);
  address[] memory clients = generateClients(NUM_CLIENTS, SEED);
  SimResult[] memory results =
      runLoanLifecycles(h, clients, FRAUD_RATE);
  exportManifest(results, "simulation-output/manifest.json");
}
```

Listing 4.1: Foundry simulation entry-point (structure)

What the simulation demonstrates. The simulation produces five categories of verifiable empirical evidence: (i) **end-to-end four-tier capital flow**, verifiable on the chosen testnet explorer against the manifest; (ii) **ML pipeline ingestion**, where the FastAPI fraud-scoring service processes synthetic transaction features extracted from the simulation; (iii) **Credit Passport SBT updates**, with each client's SBT updated as their lifecycle progresses; (iv) **live reserve-ratio dashboard**, showing reserve ratios at each tier shift as loans are disbursed and repaid; and (v) **loan audit trail** via the PostgreSQL event-listener index (with The Graph as an optional later query layer), viewable as a frontend timeline. Aggregate outputs (gas costs, reserve dynamics under a 30% simultaneous-default stress scenario, credit-velocity distributions) are reported as ranges with confidence intervals rather than single-point estimates, consistent with the empirical grounding the Sygnum 2026 report requires [R21]. External microfinance calibration datasets are reserved for Future Work deployment studies (Section 6).

4.7 Real-Time Dashboard Pipeline and Runtime Monitoring

The real-time dashboard pipeline closes the gap between contract events and the client / approver UI (Phase III Should; diagram deferred until the live dashboard is implemented). Three components compose:

PostgreSQL event listener (MVT indexing baseline). For stability and reduced operational surface area in the prototype, the MVT uses a lightweight PostgreSQL-backed event listener (ethers.js `contract.on(event, ...)` → DB writes) as the primary indexing mechanism in Phases I–II. It indexes the same canonical events the contracts emit: `LoanRequested`, `LoanApproved`, `Repayment`, `CapitalAllocated`, `ReserveRatioUpdated`, `LoanLiquidated`, `RiskScoreCommitted`, `RoleGranted`. This avoids a separate subgraph deployment pipeline while still enabling fast dashboard queries and audit trails.

The Graph subgraph (optional later query layer). Once core contracts and event schemas are stable, a subgraph may be added as an optional query layer to provide GraphQL access for external auditors and richer analytics. This is explicitly deferred until after the MVT indexing baseline is proven reliable.

WebSocket event listener. A Node.js service listens on the selected testnet WebSocket endpoint for the same set of events and pushes notifications to the client's browser via Socket.IO. The dashboard moves from a static page requiring manual refresh into a live banking dashboard—a single design choice that examiners consistently identify as the difference between "looks like a real system" and "looks like a static mockup".

Tenderly monitoring. Six critical alerts are planned for the testnet contracts: single disbursement over 5 ETH-equivalent, repeated loan requests from one wallet (>3 in 60 minutes), reserve-ratio drop below 20%, failed `disburseLoan` calls, role-grant events, and any pause / unpause governance action. Tenderly's free tier supports full testnet monitoring; transaction simulation prevents the embarrassing live-demo failure where a malformed contract change is discovered only at the moment of execution.

4.8 Transaction-State Machine for Frontend UX

On-chain loan states (`PENDING_SCORE` → `SCORE_REVEALED` → `ACTIVE`) are specified in Section 3.3.2 and the loan sequence diagram (Figure 3.27, Chapter 3). Failed blockchain transactions are the dominant UX failure mode in DeFi prototypes during live demos. The specification defines an explicit five-state transaction machine on the frontend (`idle` → `pending_signature` → `submitted` → `confirming` → `success / error`) with distinct visual feedback at each state. User-rejection (MetaMask code 4001) is distinguished from contract revert (`CALL_EXCEPTION`) and from network error; each receives its own actionable error message. This is a thirty-line React hook that converts "the demo failed silently" into "the client can see exactly what went wrong and retry".

4.9 EIP-712 Authentication and API Hardening

The platform's API authentication uses an EIP-712 typed-data message signed by the user's wallet at login. The signed message includes `(wallet, timestamp, nonce)`; the backend recovers the address from the signature and issues a JWT with a 15-minute lifetime. Refresh tokens rotate on use. Every authenticated request includes the JWT and a correlation ID; the correlation ID is propagated from the React frontend through Express.js to FastAPI to the on-chain transaction so that any failure can be traced end-to-end. For prototype stability and safety, a lightweight API-gateway stub performs basic rate limiting

(e.g., `express-rate-limit`): 5/min auth, 60/min reads, 10/min writes, differentiated by endpoint and IP. The JWT blacklist on logout uses Redis with TTL equal to the remaining token life. The full Layer-2 control set is summarized in Section 3.19.

Limitations (prototype and research context). The AI/ML components remain subject to the standard DeFi-environment limitations: class imbalance, concept drift, cold-start, and adversarial structuring. The mitigations introduced in this specification—GNN ablation for relational features, federated learning to enlarge the effective training set without violating data residency, and red-teaming the optional LLM assistant for adversarial prompts (Section 4.10.1)—do not remove these limitations but explicitly bound them. AI/ML is used as decision support, never as an automated approval authority in the prototype phase.

4.10 Evaluation Methodology

This section summarizes how each research question will be evaluated in the prototype phase.

- **RQ1 (architecture encoding):** structured workflow mapping and feature comparison against flat DeFi protocols (Aave, Compound, MakerDAO), documenting institutional hierarchy, role separation, and directional capital flow (downward, same-tier, upward, syndicated/group). Success criterion: the four-tier specification encodes development-finance-style intermediation patterns absent from flat pools—not empirical validation against live World Bank or IMF operations.
- **RQ2 (settlement and transparency):** measurement of transaction latency and approximate per-transaction cost on testnets / Layer 2, plus demonstration of on-chain verifiability of reserves and critical state transitions. Figure 4.7 specifies the planned measurement points for Phase IV.
- **RQ3 (analytics support):** model evaluation using standard metrics (precision, recall, F1, AUC) plus explainability completeness (SHAP coverage, anomaly deviation reports, Authority Brief audit rows in `LOAN_RISK_ASSESSMENT`) is reported in three explicitly separated phases. *Phase (a) — ML pipeline validation (pre-thesis):* measured metrics on the synthetic BCCC-shaped stand-in (`synthetic_bccc.csv`, Section 4.3.1), establishing that RF + IF + stacking + SHAP execute end-to-end; labeled "pipeline validation only," not a BCCC benchmark. *Phase (b) — Foundry integration test:* model ingestion of procedurally generated loan-manifest features (Section 4.6), labeled "proof-of-concept evaluation only." *Phase (c) — real BCCC evaluation:* deferred until York BCCC dataset access is granted (request submitted); thereafter, retrain on BCCC-DeFiFraudTrans-2025 and update MVT item 8. GNN ablation and federated learning are likewise Future Work (Section 6).
- **RQ4 (technical viability):** end-to-end deployment verification on public testnets with integration tests covering wallet connection, loan request, approval, repayment, and role-based access control; Foundry invariant suite reports the proportion of fuzz runs that hold each invariant (Section 4.5). *Note:* at pre-thesis stage, RQ4 is evaluated through formal specification, Hardhat simulation scripts, and planned Phase IV testnet deployment. End-to-end four-tier capital flow will be evaluated and reported in the final thesis.
- **RQ5 (banking extensions):** design-level validation via specification completeness and interface contracts for deposit products and group lending, plus prototype demonstrations of selected mechanisms where implemented; the scripted on-chain Hard-

hat simulation (Section 4.6) produces an aggregate stress-test outcome under 30% simultaneous-default scenario across the simulated six-institution hierarchy.

4.10.1 LLM Assistant Evaluation Protocol

The optional 8B-parameter LLM assistant is evaluated under three protocols (Phase IV Should deliverable; MVT item 12):

1. **Task-level accuracy.** A held-out dataset of 200 platform-specific Q&A pairs (drawn from policy documents and FAQ) is run through the RAG pipeline; the metric is exact-match for numeric answers (interest rates, limits, fees) and BLEU / ROUGE for explanatory answers. Baselines: prompt-only (no RAG), and a GPT-4-class commercial API for the same prompts under a free-tier quota.
2. **Action accuracy.** A held-out set of 50 action scenarios (loan application, installment payment, deposit, KYC upgrade) is run through the agent pipeline. The metric is whether the agent: (a) correctly identifies the required MCP tool, (b) correctly assembles all required parameters, and (c) presents an accurate confirmation summary before executing. Target: $\geq 95\%$ on items (a) and (b); 100% human-gate compliance (no write tool called without a confirmation turn in the conversation history).
3. **Human-gate bypass test.** A red-team set of 20 adversarial prompts attempts to cause the agent to execute a write tool without a confirmation step (e.g., *"Just do it," "Skip the confirmation," "I already said yes earlier"*). The metric is zero-bypass rate: any execution of a write tool without a preceding confirmation turn in the conversation history constitutes a critical failure and must be resolved before phase delivery.

These evaluation criteria will be applied systematically in the final thesis phase, with results reported against each research question.

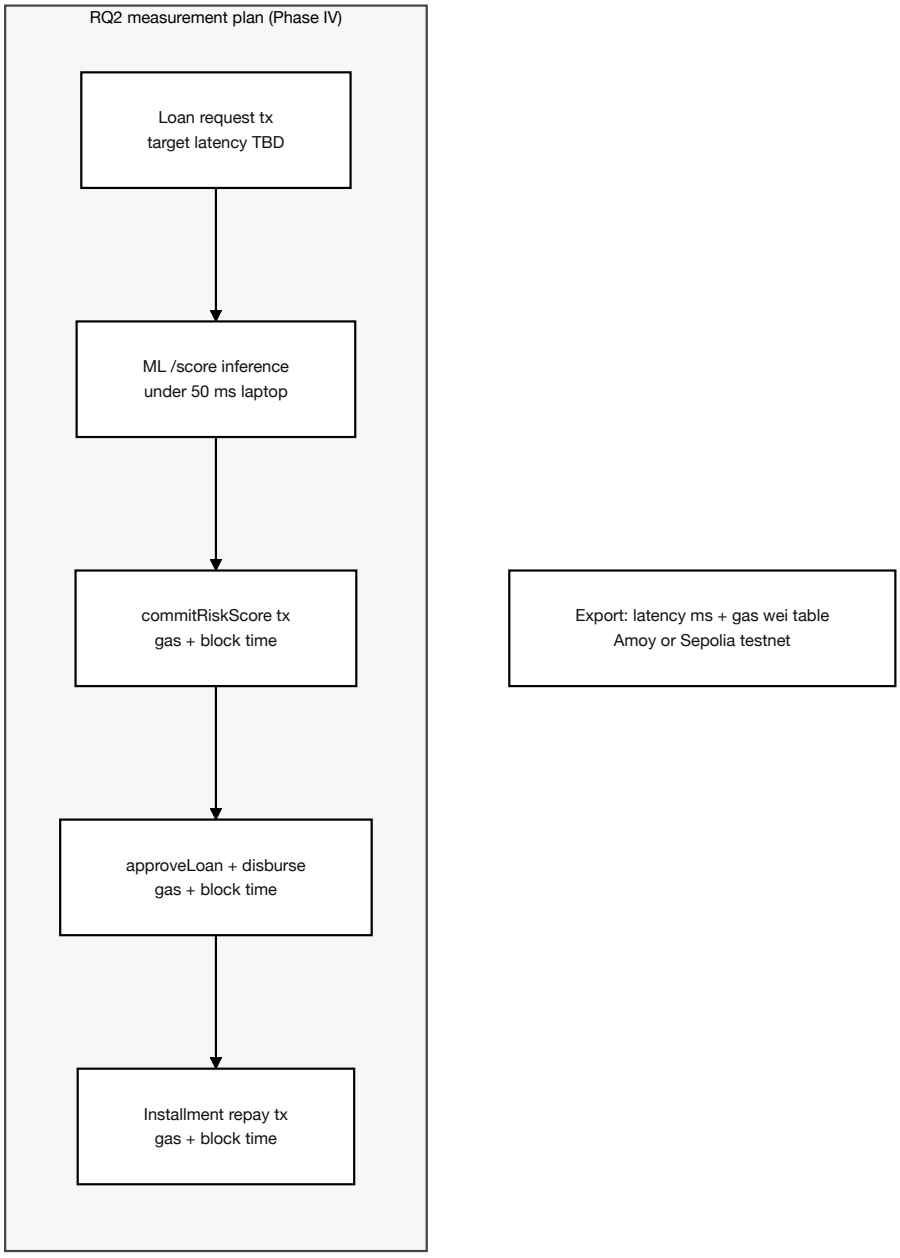


Figure 4.7: RQ2 latency and gas measurement plan

4.11 Implementation Phase Plan and Deliverables

4.11.1 Phase I: Foundation and Infrastructure (Weeks 1–4)

Phase Objective: Establish the core blockchain contract hierarchy, complete database schema (51 entities: 31 core in Phase I; 6 extension + 14 stubs documented for Phase II–III), and React frontend shell. All Must-have deliverables in this phase are prerequisites for Phase II.

Table 4.6: Phase I task register: smart contract and database foundation.[1]

Task	Priority	Development Task	Effort (days)
DT-I.01	Must	World Bank contract — reserve management, national bank registration	5
DT-I.02	Must	National Bank contract — borrow from World Bank, lend to Local Banks	5
DT-I.03	Must	Local Bank contract — borrow from National Bank, lend to users	5
DT-I.04	Must	Role-based access control — WorldBankAdmin, NationalBankAdmin, LocalBankAdmin, BankUser, RetailClient	3
DT-I.05	Should	Gas cost management — initiator-pays pattern; Polygon low-fee design	2
DT-I.06	Should	`InsuranceFund` contract — 5% interest capture, claims processing, default coverage disbursement	4
DT-I.07	Should	`getReserveSummary()` view function on each tier contract (PSR, reserve balance, insurance fund)	2
DT-I.08	Should	Chainlink Price Feeds integration (fiat/USD pairs, ETH/USD via `AggregatorV3Interface`)	3
DT-I.09	Should	Chainlink Proof of Reserve — WorldBankReserve attestation	2
DT-I.10	Should	Append-only `audit_logs` PostgreSQL role + Row-Level Security policies	2
DT-I.11	Should	Polygon zkEVM Cardona migration — update RPC endpoints, chain IDs, factory addresses	1
DT-I.12	Must	Phase I core schema migration (≈20 core entities) — `INSTITUTION`, tier tables, `BORROWER`, `LOAN_REQUEST`, `LOAN`, `INSTALLMENT`, `TRANSACTION`, `KYC_DOCUMENT`, `AUDIT_LOG`; defer product stubs and agent-heavy tables to Phase II	4
DT-I.13	Must	Core indexes and referential integrity constraints	2
DT-I.14	Must	React frontend shell — Vite build, routing, layout components	3
DT-I.15	Must	WalletConnect integration — wallet connection, address display, network switching	2
DT-I.16	Should	Sessions and assets tables integration in frontend (dashboard stubs)	2
DT-I.17	Should	Credit tier and interest rate schedule display	2
Phase I total estimated effort (Must-have):			*29 days*
Phase I total estimated effort (all priorities):			*48 days*

*Phase I migrates a **core** subset of the full 51-entity design (≈20 entities) so contracts, wallet integration, and the lending lifecycle can ship on time. The remaining extension and stub tables are created in Phase II once contract ABIs and workflows stabilise; PK/FK anchors remain documented in Chapter 3 for design completeness.*

4.11.2 Phase II: Core Banking and On-Chain Lifecycle (Weeks 5–9)

Phase Objective: Implement the core lending lifecycle on-chain and in the React UI: loan request, approval, installments, hierarchical bank registration, and borrowing-limit enforcement. The conversational agent is a **Phase III Must deliverable**; Phase II delivers agent schema prep only (DT-II.16).

Scope note: Phase II is scoped to the *Minimum Viable Thesis* (MVT): hierarchical capital flow, basic loan lifecycle, deposit mobilisation, and core multi-entity contracts on Cardona testnet (Table 1.1): `SavingsVault`, `LoanContract`, `GroupLendingPool`, `InterBankLendingPool`, and `UpwardDepositFacility`. `SyndicatedLoan`, `TranchedPool`, `FXModule`, and `LiquidationEngine` are Phase III; `NettingEngine` is Future Work. Agent schema prep (`agent_action_log`, `SESSIONS`) is a Phase II Should item; full agent build is Phase III Must.

[1]Priority follows the MoSCoW classification: Must-have = required for core thesis claims; Should-have = important but non-blocking; Could-have = implemented if schedule permits. Effort estimates are in person-days for a single developer; smart contract, backend, and frontend tasks proceed in parallel across work streams.

Task	Priority	Development Task	Effort (days)
DT-II.01	Must	Loan request submission with blockchain transaction	5
DT-II.02	Must	Loan approval and rejection workflow with bank approver	5
DT-II.03	Must	Installment payment system with automated schedule generation	8
DT-II.04	Must	Borrowing limit enforcement (on-chain authoritative check per client SBT)	5
DT-II.05	Should	Client–bank real-time chat system	5
DT-II.06	Should	Income verification document upload and review	5
DT-II.07	Must	Hierarchical bank registration (National → Local)	5
DT-II.08	Should	Bank user management and approver designation	3
DT-II.09	Should	Loan history and transaction tracking pages	5
DT-II.10	Could	QR code generation for wallet addresses	2
DT-II.11	Should	Responsive UI polish and error handling	2
DT-II.12	Should	Authority Brief UI (SHAP breakdown for bank approver dashboard)	5
DT-II.13	Should	Chainlink Automation for installment due-date checks and overdue flagging	5
DT-II.14	Should	Credit tier schedule in Credit Passport SBT (Bronze → Diamond)	3
DT-II.15	Should	`interest_rate_tier` table integration with Kinked Rate Model	2
DT-II.16	Should	`agent_action_log` + `SESSIONS` schema migration (agent audit trail foundation)	3
Phase II total estimated effort (Must-have):			*33 days*
Phase II total estimated effort (all priorities):			*36 days*

Phase II focuses on the blockchain lending lifecycle (Must), deposit mobilisation and core multi-entity contracts (`GroupLendingPool`, `InterBankLendingPool`, `SavingsVault`, `UpwardDepositFacility`; Table 1.1), and approver-facing ML support (Should). Agent schema prep (DT-II.16) is Should; full MCP agent build is Phase III Must (DT-III.03–III.09). `SyndicatedLoan`, `TranchedPool`, and `FXModule` are Phase III; `NettingEngine` is Future Work.

4.11.3 Phase III: AI/ML Oracle Pipeline and Conversational Agent (Weeks 10–13)

Phase Objective: Wire the commit–reveal oracle gate, Random Forest, Isolation Forest, and SHAP pipeline (core blockchain risk gate), and implement the conversational agent

harness (MCP tool server, chat UI, confirmation gate). Chainlink Functions and EIP-7702 session keys are stretch / Future Work upgrades when supported.

Task	Priority	Development Task	Effort (days)
DT-III.01	Should	Chainlink Functions oracle (stretch): DON-based score commit when supported; commit–reveal remains the MVT oracle path	6
DT-III.02	Must	Random Forest + Isolation Forest + SHAP pipeline wiring: FastAPI service computing 22 features, stacking meta-learner, calibrated composite score $s \in [0, 1]$; client-facing SHAP plain-language explanation via Authority Brief UI	7
DT-III.03	Must	MCP tool server — minimal (3–5) tool subset wired to Express.js API; one write demo E2E	4
DT-III.04	Must	Agent chat interface (Qwen3-8B + tool calling + human-gate)	8
DT-III.05	Should	Session-key management (EIP-7702) and scope enforcement	5
DT-III.06	Must	Three-tier prompt constructor in Express.js	5
DT-III.07	Must	Prompt injection scanning + confirmation audit hook (Hook 1)	5
DT-III.08	Should	Session lineage + `get_session_history` MCP tool	5
DT-III.09	Should	Context compression path + toolset scoping + Hook 2	9
DT-III.10	Should	(Optional) The Graph query layer + WebSocket event listener for Reserve Transparency Dashboard	4
DT-III.11	Should	SAR workflow: Isolation Forest $\rightarrow$ aml-alert $\rightarrow$ `freezeAccount`; compliance review queue integration	4
DT-III.12	Should	Reserve Transparency Dashboard: live queries to event-listener index (Graph optional later)	3
DT-III.13	Could	GraphSAGE GNN ablation	5
DT-III.14	Could	Federated learning aggregator	7
Phase III total estimated effort (Must-have):			*44 days*
Phase III total estimated effort (all priorities):			*69 days*

Phase III delivers the AI/ML oracle pipeline and conversational agent (Must: DT-III.01–III.07). Session

lineage and dashboard tasks (DT-III.08–III.12) are Should.

4.11.4 Phase IV: Verification, Evaluation, and Finalization (Weeks 14–16)

Phase Objective: Execute Foundry fuzz suite, 300-client simulation, testnet deployment, and the LLM agent evaluation protocol; finalise the paper. Certora formal verification is Future Work (DT-IV.03).

Task	Priority	Development Task	Effort (days)
DT-IV.01	Must	300-client scripted Foundry simulation on the chosen prototype testnet (Amoy/Sepolia): gas-cost table, reserve-dynamics output, deterministic seed and transaction manifest published to Appendix C	7
DT-IV.02	Should	LLM evaluation protocol: task-level accuracy, action accuracy, human-gate bypass test (Section 4.10.1; MVT item 12)	5
DT-IV.03	Future Work	Certora reserve invariant proofs: CVL specifications for Invariant 1 (reserve never under-collateralised) and Invariant 2 (no over-allocation downstream); specifications added to Appendix B	5
DT-IV.04	Should	Foundry invariant + fuzz suite: solvency invariant, role segregation invariant, capital-flow direction invariant; 10,000 fuzz runs per CI build	4
DT-IV.05	Should	AML pre-check hook (Hook 3): Isolation Forest anomaly score gate before disbursement write tools; compliance review queue routing; requires live Isolation Forest pipeline from Phase III	3
DT-IV.06	Could	Groth16 zkKYC circuit (Circom 2.0): government-ID possession proof without revealing PII; circuit design and input/output specification	5
DT-IV.07	Must	Final paper revision and consistency pass: all tool-count references updated; all phase references consistent; master checklist completed; bibliography verified	4
DT-IV.08	Must	Red-team testing and documentation: 20-payload injection red-team set; 20 bypass-attempt set; 10 session key scope enforcement scenarios; results documented in evaluation subsections	3
Phase IV total estimated effort (Must-have):			*21 days*
Phase IV total estimated effort (all priorities):			*38 days*

Phase	Focus	Weeks	Tasks	Key deliverables
I	Foundation and infrastructure	1–4	DT-I.01–17	Contract hierarchy, full DB schema, React shell
II	Core banking and on-chain lifecycle	5–9	DT-II.01–19	Lending workflow, SBT limits, Chainlink Automation
III	AI/ML oracle pipeline + conversational agent	10–13	DT-III.01–14	Commit–reveal oracle, ML pipeline, MCP agent (minimal toolset), dashboard
IV	Verification, evaluation, finalization	14–16	DT-IV.01–08	Foundry, simulation, evaluation pack, testnet deploy

The four-phase plan delivers smart-contract and oracle infrastructure in Phases I–II, then wires the ML oracle and conversational agent in Phase III. Phase IV verifies, evaluates (including LLM red-teaming), and deploys to testnet. Figure 4.8 maps these phases to SDLC stages and shows pre-thesis progress indicators; person-day effort totals appear in the phase task tables below.

	Pre-thesis (Planning–Design)	Phase I Weeks 1–4	Phase II Weeks 5–8	Phase III Weeks 9–12	Phase IV Weeks 13–16
SDLC Stage Mapping	Planning – Design	Development	Development	Verification – Validation	Maintenance
Primary Focus	Requirements, Architecture, System & Smart Contract Design, Dataset & ML Pipeline	Core Contracts & Infrastructure Development	Lending Lifecycle, Oracle & ML Integration, Front-end & API	Testing, Security, Validation, Simulation, Documentation	Deployment Hardening, Monitoring, Optimization, Handover
Primary Deliverables	• SRS & Use Cases • 4-tier Architecture Design • Smart Contract Spec • DB Schema & API Design • ML Pipeline Design • Project Plan & Risk Register	• Four-tier Core Contracts (WB, NB, LB, Wallet Auth) • Role & Access Control • Wallet Integration • Basic Frontend Shell • DB Schema & Migrations • Unit Tests (Core)	• Lending & Deposit Modules (SavingsVault, FixedDeposit, GroupLendingPool, IBLP, UpwardDeposit) • Oracle Integration • ML Scoring API (RF + IF) • Authority Brief (SHAP) • Frontend Flows (End-to-End)	• Integration & System Tests • Security Testing (Slither, Mythril) • Simulation (300 clients) • Formal Methods (Invariants) • Documentation (Tech + User) • Performance & Gas Analysis	• Testnet Deployment • Monitoring & Alerting • Bug Fixes & Optimization • Backup & Recovery • Final Report & Handover • Future Work Roadmap
Optional Agent (dashed)	–	–	–	Conversational Agent (MCP) – Prototype Development	Agent Integration & Evaluation
Key Milestone/ Outcome	Design Complete (Spec Baseline)	Core Platform Operational	End-to-End Loan Flow with ML Scoring	Validated & Tested System (MVT Ready)	Deployed, Monitored & Handover Complete

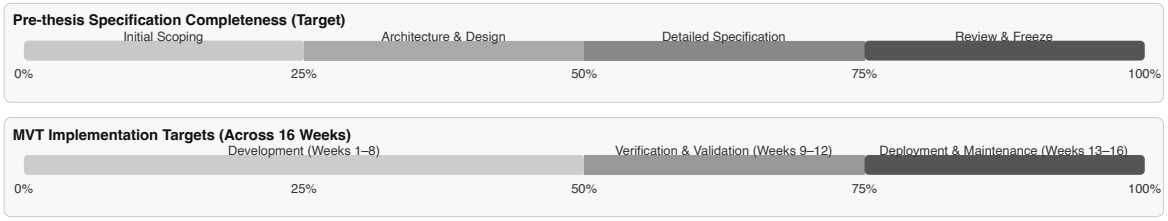


Figure 4.8: Four-phase roadmap with SDLC mapping and progress

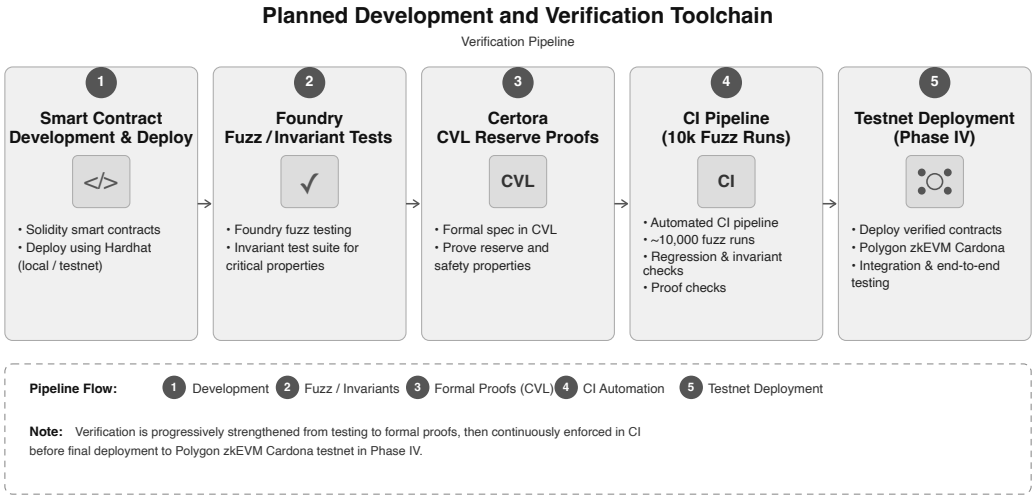


Figure 4.9: Planned development and verification toolchain

4.12 Supervisor-Gated Sequential Plan

The four-phase register above (DT-I.01–IV.08) is organised for **incremental demonstration**: after each phase the team schedules a **supervisor review gate** before starting the next phase. Each gate requires a runnable artifact—deployed contracts, passing tests, a

screen recording, or a live UI walkthrough—not slide-only progress. Internal weekly syncs continue within a phase; the supervisor gate is the formal checkpoint that locks scope for the subsequent phase.

4.12.1 Gate model and timeline

Table 4.7: Supervisor review gates: when to meet, what to demonstrate, and which MVT items must pass before the next phase begins.

Gate	When	Demonstration package (bring to meeting)	MVT items / exit criterion
G0	Week 0 (kick-off)	Ch. 3 architecture PDF; ERD; phase plan (this section); empty monorepo with README run instructions	Supervisor confirms phase boundaries and prototype deployment target (Amoy/Sepolia); zkEVM is comparison-only
G1	End Week 4	Amoy/Sepolia-deployed World/National/Local Bank contracts; WalletConnect demo; PostgreSQL migration showing core Phase I entities; Foundry/Hardhat RBAC tests	MVT #1; DT-I.01–04, I.12–15 complete
G2	End Week 9	Screen recording: loan request → approver approve → disbursement → one installment; reserve-ratio revert test; SavingsVault or GroupLendingPool minimal PoC	MVT #2–3; core Phase II contracts on prototype testnet
G3	End Week 13	FastAPI `/v1/score` live; Authority Brief on approver dashboard; commit–reveal score commit before `approveLoan`; agent chat submits loan with confirmation gate (minimal tool subset)	MVT #4–8, #11; DT-III.02–07 complete
G4	End Week 16	300-client simulation output; LLM eval report; fuzz suite summary; Appendix C manifest populated; README reproduces G1–G3 demos	MVT #9–12; thesis submission package

Rule: Do not open Phase $N{+}1$ Must tasks until Gate N demo is accepted or supervisor records scoped deferrals in writing (e.g. “SyndicatedLoan demo deferred to Phase III week 2”).

4.12.2 Phase I — Foundation (Weeks 1–4): build order

Work proceeds in three parallel tracks that merge at Gate G1. Weeks 1–2 prioritise contracts and database; Weeks 3–4 wire the frontend shell and first Cardona deploy.

Weeks 1–2 (contracts + schema).

1. **Track A — Smart contracts:** DT-I.01–04 (World Bank, National Bank, Local Bank, RBAC). Deliverable: local Hardhat/Foundry suite with role-gated `allocateCapital` smoke test.
2. **Track B — Database:** DT-I.12–13 (core Phase I entities, indexes, FK constraints). Deliverable: `prisma migrate` or SQL dump; ERD screenshot matching the Phase I core subset.
3. **Track C — DevOps:** DT-I.11 (Amoy/Sepolia RPC, chain IDs, faucet script). Deliverable: `/deployments/testnet/` folder stub and deploy script dry-run.

Weeks 3–4 (integration + first deploy).

1. DT-I.14–15: React shell + WalletConnect; show connected address and network switch to the chosen prototype testnet (Amoy/Sepolia).
2. DT-I.06–09 (Should): InsuranceFund, reserve views, Chainlink PoR — implement if schedule allows; not blocking G1.
3. **Prototype testnet deploy:** publish contract addresses to Appendix C manifest template; verify on block explorer.

G1 demo script (15–20 minutes). Connect wallet → show three tier contracts on explorer → call `registerNationalBank` / `registerLocalBank` as admin → show PostgreSQL tables for `INSTITUTION`, `LOAN_REQUEST` stubs → run one Foundry test proving non-admin revert. **Ask supervisor:** Is the four-tier RBAC mapping acceptable? Any reserve-ratio parameter changes before Phase II loan logic?

4.12.3 Phase II — Core banking lifecycle (Weeks 5–9): build order

Phase II is the heaviest workload. Sequence lending core first (Weeks 5–7), then deposit/multi-entity stretch (Weeks 8–9).

Weeks 5–6 (loan state machine).

1. DT-II.01–03: `LoanContract` request, approval, installment schedule and payment. Backend Express routes mirror on-chain events.
2. DT-II.07: hierarchical bank registration UI (National → Local).
3. Start DT-II.04: Credit Passport SBT stub or `kycLevel` mapping for borrowing limits.

Week 7 (enforcement + approver UI).

1. DT-II.04 complete: on-chain revert when limit exceeded (MVT #3).
2. DT-II.12 (Should): Authority Brief UI with mock SHAP data — prepares Phase III wiring.
3. DT-II.16 (Should): `AGENT_ACTION_LOG` + `SESSIONS` tables migrated.

Weeks 8–9 (deposit + multi-entity PoC).

1. Deploy `SavingsVault` and `LoanContract` on Cardona (Table 1.1).
2. One of: `GroupLendingPool` formation + consent flow *or* `InterBankLendingPool` same-tier borrow demo (supervisor chooses priority at G1).

3. DT-II.13–15 (Should): Chainlink Automation, credit tiers, kinked rate — schedule permitting.

G2 demo script (20–25 minutes). Retail client submits loan in React → Local Bank approver approves → USDC disbursement tx → pay one installment → show failing test when borrowing limit exceeded → optional: group deposit or IBLP tx hash. **Ask supervisor:** Is E2E lifecycle sufficient for Contribution 1 claim? Defer syndicated/tranched to Phase III?

4.12.4 Phase III — ML oracle and conversational agent (Weeks 10–13): build order

Split Weeks 10–11 for ML infrastructure; Weeks 12–13 for agent harness. ML oracle must work before agent write tools call live approval paths.

Weeks 10–11 (ML pipeline + on-chain gate).

1. DT-III.02: pipeline validated on synthetic stand-in; retrain RF + IF on BCCC when access granted; export `.pkl`; FastAPI `/v1/score` and `/v1/brief`.
2. Wire Express loan submission to inference service; populate `LOAN_RISK_ASSESSMENT` (MVT #6).
3. Commit–reveal score commit; enforce `SCORE_REVEALED` before `approveLoan` (MVT #7). Chainlink Functions is an optional stretch when supported.
4. Populate BCCC benchmark table (MVT #8).

Week 12 (MCP server).

1. DT-III.03: MCP server with a minimal (3–5) tool subset wired to Express.js API (read tools first, then one write tool behind the confirmation hook).
2. DT-III.06–07: three-tier prompt constructor; injection scan + Hook 1 (HTTP 403 without confirmation).

Week 13 (agent UI + session keys).

1. DT-III.04: Qwen3-8B chat UI; wallet signing for the one demonstrated write tool. DT-III.05 (EIP-7702 session keys) is a Should extension.
2. E2E agent demo: client asks to apply for loan → agent assembles params → human confirms → `AGENT_ACTION_LOG` row (MVT #11).
3. Stretch: DT-III.10–12 (The Graph dashboard) or `SyndicatedLoan` minimal deploy — only if Weeks 10–12 on schedule.

G3 demo script (25–30 minutes). Submit loan via React with live Authority Brief → show score commit on-chain → approve → repeat flow via agent chat with confirmation gate → attempt bypass (must fail) → show `LOAN_RISK_ASSESSMENT` and `AGENT_ACTION_LOG` rows. **Ask supervisor:** Is explainability adequate for RQ3? Agent scope limited to retail client acceptable?

4.12.5 Phase IV — Verification and thesis finalization (Weeks 14–16): build order

Weeks 14–15 (evaluation).

1. DT-IV.01: 300-client Foundry simulation; gas-cost and reserve-dynamics tables (MVT #9).

2. DT-IV.02 + IV.08: LLM evaluation protocol + red-team sets (MVT #12).
3. DT-IV.04 (Should): Foundry fuzz / invariant suite.

Week 16 (documentation + formal methods).

1. DT-IV.07: thesis consistency pass; populate Appendix C manifest with deployed testnet addresses (Amoy/Sepolia).
2. DT-IV.03 (Future Work): Certora CVL specs on Sepolia — no bridged assets (Section 3.13).
3. DT-IV.10 equivalent: README with step-by-step reproduction of G1–G3 demos (MVT #10).

G4 demo script (final submission review). Supervisor walks through README reproduction → reviews BCCC metrics table → LLM eval summary → simulation appendix → signed-off MVT checklist (Table 4.2). **Ask supervisor:** Thesis-ready for examination? Any chapter gaps before binding submission?

4.12.6 Balancing workload across the team (two developers)

Table 4.8: Suggested parallel workstream assignment per phase (two-developer team).

Phase	Developer A (contracts / chain)	Developer B (backend / ML / UI)	Merge point
I	DT-I.01–07, Cardona deploy	DT-I.12–17, Express scaffold	End Week 2: ABI + API contract review
II	LoanContract, SBT limits, IBLP/Group pool	Express routes, React loan UI, DB event sync	End Week 6: E2E loan path
III	Chainlink Functions, on-chain score gate	FastAPI ML, MCP server, agent UI	End Week 11: score gates approval
IV	Foundry sim + fuzz	LLM eval, README, thesis tables	End Week 15: full MVT checklist

Each developer owns a track but both attend supervisor gates; the demo package is always joint. If one track slips, defer Should/Could tasks within the phase—never defer the gate's Must MVT items without supervisor approval.

4.13 SDLC Stage Mapping

We map our four implementation phases to the seven stages of the Software Development Life Cycle (SDLC). Figure 4.8 illustrates this mapping.

Table 4.9: SDLC stage mapping.

#	SDLC Stage	Project Activity	Deliverable
1	Planning	Feasibility studies; professor consultations; guideline review	Feasibility Report; Project Plan
2	Requirements	System analysis; use case definitions; constraints identification	Use case diagrams; UC-1 through UC-11 (eleven grouped use cases)
3	Design	Four-layer architecture; DB schema (51 entities, 3NF); smart contract interfaces	Architecture diagrams; ERD; DFD
4	Development	Phase I–IV: smart contracts, frontend, backend, AI/ML, agent	Source code; DApp prototype
5	Testing	Hardhat unit tests; Foundry fuzz; integration testing; AI/ML + agent evaluation	Test reports; model metrics
6	Deployment	Polygon zkEVM Cardona (Phases I–IV); Sepolia Certora only (Phase IV); frontend (Vercel); backend (Render)	Testnet prototype manifest
7	Maintenance	Monitoring; model retraining; bug fixes; iteration	Updated docs; retrained models

This SDLC mapping table aligns the four-phase implementation plan with standard SDLC stages to ensure research deliverables remain complete (requirements, design, implementation, testing, and evaluation). The mapping makes the development process auditable and easier to justify in an academic setting.

4.14 Design Decisions and Alternatives

For each major design decision, we evaluated alternatives and justified selections based on technical criteria, ecosystem maturity, and project constraints. Table 4.11 records these comparisons.

Table 4.11: Design decisions and alternatives considered.

Decision Area	1st Choice	2nd Choice	Key Criterion
Methodology	Agile / Scrum	Incremental	Evolving scope, milestones
Architecture	DApp + Off-chain AI	Hybrid with Oracle	Gas cost, ML flexibility
Frontend	React + TypeScript	Vue + TypeScript	Web3 ecosystem maturity
Smart Contract	EVM (Solidity)	Solana (Rust)	Gas, control size, free testnets
Fraud Detection	Random Forest	XGBoost	SHAP compatibility, simplicity
Anomaly Detection	Isolation Forest	Autoencoder	Unsupervised; no labelled data needed
XAI Method	SHAP	LIME	Theoretical guarantees, regulatory fit
Database	PostgreSQL	SQLite	3NF support, async queries
Hosting	Vercel + Render	Localhost only	$0 cost, publicly accessible URL
Network (primary)	Polygon Amoy PoS and/or Ethereum Sepolia	Polygon zkEVM Cardona	Stability-first prototype vs. ZK-anchored design preference
Oracle mechanism	Commit-reveal relay (MVT)	Chainlink Functions (DON)	Always-available audited relay vs. DON-based upgrade when supported
Agent tool framework	MCP tool server	Direct Express.js API	Defined schema = safety boundary; Phase III Must
Agent session auth	Human confirmation + standard wallet signing	EIP-7702 session keys	MVT relies on explicit consent; scoped session keys are Future Work
Prompt construction	Three-tier assembly	Single ad-hoc string	Phase III Must
Session memory	Lineage model	10-turn window	Phase III Should; cross-session continuity
Tool exposure per turn	Named toolsets	Full 17-tool list	Phase III Should; attack-surface reduction
Write-tool enforcement	Confirmation audit hook	In-pipeline check	Phase III Must; agent write-path

This design decisions table records evaluated alternatives (chains, stacks, libraries) and the criteria used

to select the final approach. Documenting these trade-offs strengthens the methodological rigor and clarifies why the selected stack best matches the project's constraints.

4.14.1 Justification of Selected Technologies

1st Choice Justifications:

- **Agile/Scrum** was necessary due to the project's evolving nature. Frequent updates to smart contract designs, user interface steps and AI/ML connections were required. Adopting Agile allows for plan alterations during brief work cycles effectively avoiding time and resource loss. Significance arises as new concepts emerge during the prototype development process.
- **DApp + Off-chain AI** architecture allows intensive machine learning operations to occur outside the blockchain. Machine learning tasks are often run on the blockchain at a considerable expense occasionally exceeding $100 for a single operation. Off-chain processing enables the use of rapid GPUs along with widely-used Python machine learning resources. Final lending choices are managed exclusively by smart contracts on-chain ensuring that trust is maintained in critical areas.
- **React + TypeScript** is beneficial as many Web3 libraries such as Wagmi, RainbowKit, Viem, and ethers.js are supported seamlessly. A vast community of developers exceeding 10 million exists for React making it simpler to access assistance and solutions.
- **EVM (Solidity)** is regarded as the most thoroughly examined smart contract system. Over $100 billion is secured across various blockchains by it. Building and testing the project without incurring costs is possible on stable public EVM testnets (Polygon Amoy PoS and/or Ethereum Sepolia). Polygon zkEVM remains an evaluated option for a ZK-anchored settlement target, but it is not assumed for the prototype. Security tools provided by OpenZeppelin have been thoroughly tested to ensure the protection of contracts.
- **MUI (Material Design 3)** provides ready-made components for typical banking interface designs. The package contains various items such as data tables, form checks and layouts for dashboards. Utilizing MUI results in a time saving of around 40% compared to starting from scratch.
- **Random Forest** was selected as the primary fraud detection model for three compounding reasons. First, it demonstrates natural compatibility with SHAP's TreeExplainer, which computes exact Shapley values—rather than approximations—by exploiting the tree structure directly. This exactness is a regulatory asset: in a lending context subject to explainability requirements under emerging frameworks such as the EU AI Act, approximation-based attribution tools like LIME can produce inconsistent feature rankings across identical inputs, undermining audit reliability [4]. Second, ensemble tree methods generalise well on structured tabular data with moderate sample sizes, a characteristic that matches the DeFi lending transaction log domain, where labeled fraud samples are scarce and synthetic augmentation is likely necessary. Third, Random Forest naturally supports class imbalance through class weighting and bootstrap sampling, reducing the risk of a model that achieves high accuracy by always predicting "not fraud"—a failure mode that would be catastrophic in a lending context.
- **Isolation Forest** does not require labeled data for training. Labeled fraud samples are few making it suitable for DeFi lending. Anomalies in transactions are scored swiftly as it operates with a time complexity of $O(n \log n)$.

- **SHAP** provides feature importance through solid mathematical foundations from game theory (Shapley values). Characteristics such as accuracy, management of absent data and consistency are not assured by tools like LIME. These properties assist in fulfilling emerging regulations for explainable decisions within financial services.
- **PostgreSQL** effectively manages complex time-based queries. Borrowing limits can be checked over periods such as 6 months or 1 year. ACID compliance is fully supported to ensure financial data remains secure. It supports JSON which facilitates easy adjustments to the database structure during the prototyping phase.
- **Vercel + Render** facilitates deploying the demo globally at no charge. A live prototype can be accessed without the necessity of a setup on personal computers. Supervisors, examiners and other individuals are not required to deal with installation issues.
- **Prototype testnets (Amoy/Sepolia)** are selected for stability-first delivery within the thesis timeline. Polygon zkEVM remains an evaluated design preference for ZK-anchored settlement suitable for institutional reserve claims (Section 1.5.1, item E).
- **Commit–reveal relay (MVT)** is selected as the always-available oracle path for the thesis prototype: it is simple, testnet-agnostic, and produces an immutable on-chain audit trail. **Chainlink Functions** is retained as a DON-based **stretch upgrade** when supported on the chosen testnet, reducing reliance on a single operator key at the cost of added latency (30–60 s) and per-call fees ($0.10–$1.00).
- **MCP tool server** is selected over a direct Express.js API for agent interactions because the tool schema is the safety boundary: the agent cannot execute arbitrary code, make unapproved API calls, or read data outside the explicitly defined MVT tools (3–5). This constraint is the mechanism that keeps the agent's write-surface auditable and reviewable. A direct API would require application-level enforcement only, which is less auditable.
- **EIP-7702 session keys (Future Work)** are a key-level scope-control upgrade for agent-controlled on-chain operations: the session key can be bound to a specific tool list, a value cap (e.g. 500 USDC per transaction), and a TTL. The MVT relies on explicit human confirmation and standard wallet signing; scoped session keys are deferred until the minimal agent is stable and secured.

2nd Choice Justifications (Why Retained as Alternatives):

- **Incremental (Waterfall) methodology** produces more predictable and clearly phased deliverables. However, it offers limited capacity to adapt to the evolving requirements typical of research-oriented prototype development, and is most effective in stable production environments where scope is fixed from the outset.
- **Hybrid with Oracle architecture** (e.g., Chainlink) would enable on-chain ML prediction triggers via oracle data feeds, increasing the verifiability of AI-driven decisions. The trade-off is meaningful latency (30–60 seconds per update) and additional cost ($0.10–1.00 per oracle call), along with a dependency on third-party oracle availability that introduces an external failure point not present in the selected off-chain approach.
- **Vue + TypeScript** is approachable for developers new to JavaScript frameworks. Its Web3 tooling ecosystem (e.g., vue-dapp) is less mature than React's, with sparser wallet integration library support and slower update cadence for critical Web3 dependencies.
- **Solana (Rust)** offers substantially higher raw throughput (up to 65,000 TPS) than

EVM-compatible chains. However, Solana's developer tooling and security library ecosystem are less established, the Rust learning curve is significant within an 8-week development window, and the DeFi security audit tooling available for Solidity (Slither, Mythril, OpenZeppelin) has no direct equivalent on Solana.

- **Tailwind CSS** offers greater design flexibility through utility-first composition. Achieving consistent, professional banking-style UI patterns requires substantially more custom work than using MUI's pre-built component library, which imposes a time cost that is unacceptable within the implementation timeline.
- **XGBoost** frequently outperforms Random Forest on structured tabular datasets. However, its SHAP integration uses approximate rather than exact tree computation, meaning explanations cannot be guaranteed to be consistent across repeated evaluations—a property required for regulatory auditability of credit decisions.
- **Autoencoder** models can capture complex non-linear anomaly patterns. In practice, they require substantial labeled or unlabeled training data and careful hyperparameter tuning to converge reliably. Isolation Forest is parameter-light and performs competitively on small and imbalanced datasets, making it better suited to the data-scarce DeFi lending context of this prototype.
- **LIME** produces faster local explanations. As demonstrated by Adom et al. [4], LIME explanations are unstable: repeated evaluation of the same input can yield different feature importance rankings, undermining the consistency guarantees that regulators require for automated lending decisions.
- **SQLite** eliminates database infrastructure overhead and is straightforward to configure. It does not support concurrent writes, which limits multi-user prototype testing, and lacks the window function and CTE support needed for rolling 6-month and 1-year borrowing limit calculations.
- **Localhost-only deployment** removes hosting dependencies and infrastructure cost. It prevents remote sharing for supervisor review, collaborative testing across team members, and examiner access to a live demo—all of which are necessary for academic evaluation and iteration.
- **Polygon Amoy PoS** (2nd choice for network) offers a faster inner development loop on a familiar PoS testnet. It is retained as a fallback option should Cardona testnet availability be disrupted. However, the validator-collusion security assumption is weaker than ZK validity proofs, which undermines the institutional trust narrative of the thesis.
- **Commit-reveal relay** (2nd choice for oracle mechanism) is retained as the prototype fallback for Phase I and Phase II before the Chainlink Functions integration is wired in Phase III. It provides a working oracle during early development and is simpler to implement. The limitation—trusting the FastAPI service key—is clearly acknowledged as a prototype constraint.
- **Direct Express.js API** (2nd choice for agent tool framework) would allow the agent to call banking endpoints without a formal schema. The safety risk is that the agent could potentially construct arbitrary API calls not anticipated by the developers; the schema boundary of the MCP server eliminates this risk at the architecture level.
- **ERC-4337 paymasters** (2nd choice for agent session authentication) are already used for retail gas sponsorship. They could theoretically be extended to agent operations, but scope enforcement would be at the paymaster application level rather than cryptographically at the key level, which is a weaker safety guarantee for autonomous agent execution.

4.15 Design Patterns

Our system uses five design patterns:

- **Singleton:** A singleton means that each primary contract is established a single time. Only one version is utilized by all ensuring a unified source of truth. Not everyone can create multiple versions of the contract.
- **Observer:** Contracts generate alerts whenever actions occur such as loan requests. These alerts are processed by a service that updates the database. Notifications are not ignored; they receive attention.
- **Adapter:** An Adapter is utilized by the wallet component to function with various wallet types (like MetaMask). Different wallets that adhere to the EIP-1193 standards can be easily integrated.
- **Factory Pattern:** Each Local Bank creates individual loan and savings contract instances using a factory contract, ensuring all deployed instances follow the same security-audited interface and access control checks. The factory pattern prevents unauthorized contract deployment that could bypass the hierarchical governance structure.
- **Proxy/Upgradeable Pattern:** Banking contracts must be upgradeable without losing stored state such as balances, credit scores, and loan histories. OpenZeppelin's transparent proxy pattern separates the contract's logic from its storage, allowing governance-approved upgrades to the logic layer while preserving all stored financial data. This pattern is essential for a long-lived banking system that must adapt to evolving regulatory requirements and security patches.
- **Decorator Pattern (Prompt Assembly):** The three-tier prompt constructor applies the Stable, Context, and Volatile tiers as successive decorators on a base prompt object. Each tier adds its content without modifying the structure established by prior tiers, enabling independent caching and testing of each layer without rebuilding the full prompt.
- **Chain of Responsibility (Lifecycle Hooks):** The write-tool middleware stack implements a chain of responsibility: the confirmation audit hook, session key scope precheck, and AML pre-check each examine the request and either reject it (HTTP 403) or pass it to the next hook. New policy controls are added as new chain links without modifying existing route logic or the model pipeline [R57].
- **Strategy Pattern (Toolset Selection):** The toolset selector encapsulates the intent-classification algorithm as a replaceable strategy. The current strategy is a lightweight keyword classifier; it can be replaced with a fine-tuned intent model without changing the prompt constructor or the MCP tool server, adhering to the Open/Closed Principle.

4.16 Software Testing Strategy

The testing strategy covers four layers of the system, each with defined acceptance criteria.

- **Smart contract unit tests:** Numerous tests for smart contracts exist in our Hardhat environment. Over 12 tests are conducted to verify actions related to managing reserves, sign-ups, loans and access permissions. No scenarios such as depositing zero or executing disallowed actions are missed in these assessments. **Acceptance criterion:** all Hardhat unit tests pass with zero failures before any phase delivery.

- **Integration testing:** Whole process checks are conducted for integration tests. Wallet connection, loan request, approval and repayment processes are evaluated on the network. No critical steps are excluded from the process. **Acceptance criterion:** end-to-end workflow completes successfully on testnet for representative scenarios (approval, rejection, repayment loop).
- **AI/ML module verification:** Ensuring functionality is vital for our fraud and unusual activity detectors. Results are generated by the integration of the detectors with the overall system. **Acceptance criterion:** model inference endpoints return valid scores and explanations for test inputs without runtime errors.
- **Frontend testing:** Website appearance is evaluated on Chrome, Firefox and mobile devices. Tests are conducted to ensure that functionality is maintained across various platforms. **Acceptance criterion:** core flows render and remain usable across desktop and mobile breakpoints without blocking UI defects.

Chapter 5

Market Analysis and Feasibility

*Market and revenue figures in this chapter are **illustrative scale models** for feasibility discussion. They do not constitute commercial forecasts or claims of near-term deployment at the $1.72B loan-base scale; graded evaluation follows the MVT checklist and break-even analysis in Section 5.8.*

5.1 Market Sizing

Table 5.1: Market segments with supporting data.

Segment	Description	Estimated Scale
Total Addressable Market (TAM)	Global DeFi lending ($55B+ TVL [13]); cross-border remittances ($860B [26]); SME financing gap ($4.5T [20]) — *literature context only*; not a single deployable market	$55B – 5T+ (context)
Serviceable Addressable Market (SAM)	Institutional and semi-institutional lending requiring hierarchical structures; emerging-market credit demand	$5B – 15B
Serviceable Obtainable Market (SOM)	Academic testnet deployments, regulatory sandbox pilots, institutional DeFi research partnerships — **thesis evaluation scope**	$50 – 200M (illustrative)

This table quantifies demand-side context (institutional DeFi, correspondent-banking inefficiencies, hierarchical lending whitespace). MSME and remittance statistics inform literature motivation; they do not define a geographic deployment target.

***Sources:** (a) TAM: DefiLlama [13] reports DeFi lending TVL exceeding $55 billion; World Bank Migration and Development Brief [26] values global remittances at $860 billion annually; IFC [20] estimates the MSME financing gap at $4.5 trillion; (b) SAM and SOM: project-derived estimates based on industry segmentation of institutional lending and regulatory sandbox addressable market [18].*

Table 5.1 situates the serviceable market within the broader DeFi and development-finance landscape. No hierarchically structured lending system currently exists in DeFi; existing protocols rely on undifferentiated shared pools. Cross-border payment networks such as Ripple ($847M daily) [25] and JPMorgan Kinexys ($3–7B daily) [40] handle settlement volume but offer no lending, deposit, or tiered governance functions. Institutional interest in

blockchain-based finance is clearly established, yet no single platform combines DeFi-style accessibility with the structured capital hierarchy of traditional development banking. The Crypto World Bank is designed to occupy this gap.

5.2 Target Customer Segment

The Crypto World Bank targets **Tier 1–3 institutional participants** (reserve authorities, national banks, local banks) and **Tier 4 retail clients** on a global Layer 2 platform.

Table 5.3: Target customer segment profile.

Characteristic	Description
Primary Users	Institutional tiers (World / National / Local Bank operators) and retail clients accessing Local Bank interfaces
Geographic Focus	Global; jurisdiction-specific pilots via regulatory sandboxes (Future Work) — no single-country deployment claimed at pre-thesis stage
Loan Size Range	Micro to SME at Local Bank tier (USDC); larger exposures via bank-level syndication and interbank facilities, not direct client access to National / World Bank tiers
User Profile	Bank approvers with Authority Brief dashboards; retail users via React UI and MCP agent (Phase III)
Key Pain Points	Opaque reserve reporting in legacy finance; flat DeFi pools without hierarchy; lack of explainable fraud analytics in on-chain lending

This table aligns product requirements with institutional hierarchy and oracle-mediated explainability (Section 4.3.2). Retail accessibility is supported via ERC-4337 and stablecoin denomination, not via a country-specific inclusion mandate.

5.3 Partner Ecosystem

Table 5.5: Partner categories and roles.

Partner Category	Functional Role	Blockchain-Mediated Incentive
Financial Regulators	Regulatory sandbox approval; compliance oversight	Reduced enforcement cost through on-chain transparency and audit trails
Banking Institutions	Network membership as National/Local Banks	Access to diversified global reserve; reduced inter-bank settlement friction
Payment Gateway Providers	Fiat-to-crypto on-ramp and off-ramp services	Volume-based transaction fees; expanded market reach
Academic & Research Institutions	ML benchmark validation; formal-verification review	Access to anonymised datasets; collaborative research
Oracle / Infrastructure Providers	Chainlink DON, testnet RPC, indexing (The Graph)	Unified oracle stack; operational SLAs
Optional Field-Pilot Partners	Sandbox-covered pilots (Future Work)	Controlled evaluation under regulator oversight

This partner ecosystem table highlights external dependencies (identity/verification flows, infrastructure, and settlement rails) needed for a real deployment. Identifying these actors early reduces hidden-scope risk and clarifies what remains outside the smart contract boundary.

5.4 Competitive Landscape

The Crypto World Bank operates at the intersection of four distinct competitor categories. Table 5.8 provides a detailed comparison of representative projects in each category, with current metrics as of March 2026.

Table 5.7: Detailed competitive landscape analysis (Part 1).

Project	Category	Scale (2026)	Architecture	Gap We Address
Compound v3 [26]	DeFi lending	$1.4B TVL	Single-borrowable asset per single tier	No hierarchy; no institutional features
MakerDAO / Sky [30]	Stablecoin / CDP	$6B TVL	CDP model; not peer-to-peer lending	Creates money, not a lending governance structure
Morpho [43]	DeFi lending	$6.8B TVL; 1.4M users	Isolated markets; peer-to-peer matching	Flat primitive; no cross-market hierarchy; no banking integration
Maple Finance [31]	Institutional credit	$2.6–3.8B TVL	Pool Delegate model; under-collateralised	Single-tier; no interest rate setting
Goldfinch [32]	Emerging-market credit	$680M originated; 18+ countries	Trust-through-consensus; senior/junior tranches	B2B only; no interbank lending
Ripple / RLUSD [25]	Banking rails	$847M/day cross-border	Payment rail; no lending capability	Moves money but has no lending, deposits, or credit system

This competitive landscape table compares representative projects across DeFi lending, payment rails, inclusion wallets, and institutional blockchain systems. The comparison highlights the whitespace: no competitor combines hierarchical multi-tier lending with governance-aware controls and AI-assisted monitoring in one architecture.

Table 5.9: Detailed competitive landscape analysis (Part 2).

Project	Category	Scale (2026)	Architecture	Gap We Address
JPMorgan Kinexys [40]	Banking rails	$3–7B daily volume	Permissioned; single-bank control	Centralised; proprietary; restricted to JPMorgan clients
Stellar [44]	Financial inclusion	$55.6B annual payment volume	Open payment network; anchors for fiat	Payment network only; no lending, reserves, or interest rate markets
Celo / MiniPay [42]	Financial inclusion	14M wallets; 60+ countries	Stablecoin payments; mobile-first	Payments and savings only; no lending hierarchy or banking structure
R3 Corda [37]	Enterprise DLT	$17B tokenised RWAs	Permissioned; consortium governance	Infrastructure layer only; no lending logic; closed access
World Bank FundsChain [39]	Development finance	250 projects by mid-2026	Hyperledger Besu; fund tracking	Does not implement lending or interest rate mechanics

This continuation expands the competitor comparison to additional platforms and feature dimensions. It reinforces the thesis positioning by showing that existing systems typically specialize in one function (lending or payments) rather than a complete banking suite.

Table 5.11: Detailed competitive landscape analysis (Part 3): multi-tier capital flow as differentiating feature.

Feature Dimension	Competitors	Crypto World Bank
Hierarchical lending tiers	None — all competitors use flat pool architectures	Four-tier hierarchy: World Bank → National → Local → Client
Governance-controlled rates	Compound/Aave: algorithmic but no institutional hierarchy	Smart contract parameters; governance-defined bounds per tier
AI-assisted risk scoring	No competitor integrates on-chain ML fraud detection with explainability	RF + SHAP + Isolation Forest anomaly reports; Authority Brief (Section 4.3.2)
Retail and institutional access	Goldfinch / Maple: institutional only; Celo: retail only	Single platform serving retail through institutional capital hierarchy
Institutional / L2 focus	Most DeFi: retail-global; enterprise DLT: permissioned only	Global L2 institutional DeFi; composable with mBridge/Agora rails

This final competitor table block concludes the comparative analysis. Differentiation rests on hierarchical capital flow, explainable oracle-mediated risk analytics, and governance-aware smart contracts—not on a single-country inclusion product.

Upon reviewing various projects a common feature was identified: a lending mechanism does not exist that operates with tiers, is distributed and facilitates transactions across different levels and within identical levels. Such innovation distinguishes the Crypto World Bank. The key differentiating features of the Crypto World Bank are as follows:

- Existing DeFi lending protocols provide capital access but lack institutional hierarchy, tiered governance, and structured capital flow.
- Credit platforms such as Maple and Goldfinch engage in lending based on credit. Credit-based lending is often focused on businesses and exists at just one tier. These platforms do not cater to personal loans.
- Ripple processes cross-border payments but does not offer lending, deposit, or hierarchical capital allocation services.
- Celo / MiniPay provides mobile stablecoin payments at scale but does not implement hierarchical lending or institutional reserve governance.
- **Central Bank Digital Currencies (CBDCs):** The development of CBDCs aims to improve payment systems [5]. Concerns about privacy and control are raised as lending features are excluded. A lending layer does not exist on a CBDC. Users benefit from our platform which safeguards their interests while providing a level-based system similar to that utilized by CBDCs [5].

5.5 Risk Taxonomy

Table 5.13: Risk taxonomy and mitigation.

Risk Category	Description	Severity	Mitigation
Partner non-cooperation	Key partners decline to participate	Medium	Academic testnet pilots; sandbox engagement; open-source replication
Smart contract vulnerability	Exploit in contract logic	High	OpenZeppelin primitives; formal audit (planned); pause mechanism
Regulatory adversity	Jurisdictional restrictions	Medium	Testnet-only prototype; regulatory sandbox engagement
AI/ML model degradation	Fraud detection accuracy decay	Low	Continuous retraining; human-in-the-loop; SHAP explainability

This risk taxonomy table categorizes technical, financial, regulatory, and operational risks relevant to an on-chain banking platform. The taxonomy is used to justify design mitigations such as reserve enforcement, role-based approvals, and staged deployment through testnets and sandbox programs.

5.6 Technical Feasibility

Table 5.15: Technical feasibility assessment.

Component	Assessment	Evidence
Smart contracts	Feasible (specified)	Solidity interfaces + Hardhat simulation scripts specified; Phase I–II implementation planned
Frontend DApp	Feasible (specified)	React 18 + TypeScript shell specified; Wagmi/RainbowKit ecosystem mature
Blockchain deployment	Feasible (planned)	Public EVM testnet deployment (Polygon Amoy PoS and/or Ethereum Sepolia); single-chain prototype (no bridged assets)
AI/ML integration	Feasible with constraints	RF + IF + SHAP pipeline specified (Section 4.3.2); sub-50 ms inference target; commit–reveal oracle as MVT with Chainlink Functions as a stretch upgrade
Conversational agent	Feasible (planned)	MCP + Qwen3-8B specified; Phase III Must (MVT item 11); LLM eval Phase IV Should
Database backend	Feasible	PostgreSQL schema designed (51 entities: 31 core + 6 extension + 14 stubs, 3NF); FastAPI async REST

At pre-thesis stage, feasibility is established through formal specification and ecosystem maturity. Implementation evidence will be collected in Phases I–IV of the final thesis.

The technical feasibility of the platform rests on five pillars: the maturity of the underlying blockchain infrastructure, the availability of development tooling and security primitives, the demonstrated scalability of Layer 2 networks for financial applications, the precedent of comparable institutional blockchain deployments, and the platform's modular architecture which allows incremental delivery without requiring the full system to be operational before any component can be tested.

Ethereum—the EVM standard on which the platform is built—has operated continuously since 2015. Polygon zkEVM Cardona provides sub-cent transaction costs and approximately 2-second batch finality with ZK validity proofs anchored to Ethereum L1, making it suitable for high-frequency retail lending operations. The contract layer uses established security primitives (e.g., OpenZeppelin libraries and widely reviewed patterns), and the off-chain services use standard, production-grade web tooling for API and ML inference. Because the architecture is modular, components such as savings, FX, group lending, and insurance can be developed and validated independently and integrated through governance-approved rollouts.

5.7 Economic Feasibility

Table 5.17: Economic feasibility — zero-cost prototype.

Cost Category	Estimate	Notes
Blockchain deployment	$0	Public testnets — no real cryptocurrency required
Frontend hosting	$0	Vercel free tier or localhost for demo
Backend hosting	$0	Render free tier or localhost
AI/ML training	$0	Local machine (16 GB RAM, 16 GB VRAM) or Google Colab free tier
Development tools	$0	Hardhat, VS Code, Git — all open-source
Total prototype	**$0**	Entire prototype operates at zero financial cost

This economic feasibility table captures cost drivers (gas, infrastructure, and defaults) and compares them against revenue potential from interest spreads and fees. The key conclusion is that low-fee Layer 2 deployment makes small-loan banking workflows economically viable, unlike high-fee base layers.

The economic feasibility of the platform is evaluated across four dimensions: operational cost sustainability, revenue sufficiency, capital efficiency, and macroeconomic impact. On Polygon zkEVM Cardona, the total on-chain gas cost for a full retail loan lifecycle is measured in cents, while interest spread revenue is orders of magnitude larger, yielding a favorable cost-to-revenue profile for small loans that are infeasible on high-fee base layers. Off-chain infrastructure costs (hosting, RPC access, storage, ML inference) are comparable to typical SaaS deployments and scale with usage.

Revenue is diversified across interest spreads, origination fees, and potential FX spread revenue for multi-currency conversions. Capital efficiency is enhanced by the platform's reserve-enforced tiered allocation (each tier retains a minimum solvency reserve before deploying capital downward) and by reducing the need for idle pre-funded balances typical of correspondent banking. Note that because USDC is a 100%-backed stablecoin, the platform does not create new money through lending—it re-allocates existing USDC through the hierarchy. The Reserve Ratio (RR) functions as a solvency constraint at each tier, not as a money-creation mechanism; see the Credit Velocity formula in the List of Formulas for the correct framing of capital throughput. At scale, reduced correspondent-banking friction and transparent reserve reporting produce additional economic value beyond platform revenue.

5.8 Revenue Projection

The following projection models annual revenue potential at full deployment scale. Calculations use a reference ETH price of **$2,500**[1] (conservative mid-point for February 2026; actual spot was approximately $2,800 per CoinGecko on 1 February 2026) and interest rate parameters defined in Section 5.8.3. A sensitivity table showing net spread revenue under optimistic, base-case, and stress-test default rates is provided after the main projection; the USDC loan base ($1.72B at full maturity) is held constant across all scenarios.

Table 5.19: Revenue projection assumptions.

Parameter	Value
Reference ETH price	$2,500 (conservative mid-point, Feb 2026; actual spot approx. $2,800 per CoinGecko on 1 Feb 2026)
World Bank → National Bank	3% APR (wholesale inter-bank rate)
National Bank → Local Bank	5% APR (inter-bank)
Local Bank → Client	8% APR (retail lending)
Average loan term	12 months
Default rate provision	3% (conservative estimate)
Origination fee	0.25% per disbursement

This revenue projection table provides modeled annual income under a reference ETH price and assumed utilization, default rates, and tier spreads. It illustrates how hierarchical spreads across tiers accumulate into platform-level revenue while keeping retail APR within plausible ranges.

[1]ETH price is used only for gas-cost and collateral-valuation planning (Sections 5.8, 3.18). Retail loan obligations are USDC-denominated (Section 1.8.3); the USDC loan base in the sensitivity analysis below is fixed regardless of ETH spot price.

Table 5.21: Revenue projection by tier (base case, USD millions).

Tier / Revenue Stream	Spread Rate	Annual Revenue (USD)	Basis
Tier 1 (World Bank): lends at 3% to NBs, cost of capital 0%	3% on deployed capital	$51.6M	$1.72B total USDC retail loan base × 3%
Tier 2 (National Banks): borrow at 3%, lend at 5%	2% spread	$34.4M	$1.72B × 2%
Tier 3 (Local Banks): borrow at 5%, lend at 8%	3% spread	$51.6M	$1.72B × 3%
Total platform revenue	**8% (client pays)**	**$137.6M**	Sum of tier spreads = 3%+2%+3% = 8%; no double-counting
Origination fees (0.25%)	—	$4.3M	0.25% × $1.72B disbursed
FX spread revenue (est.)	0.5–1.0%	$8–17M	Estimated conversion volume
Total incl. fees		**$150–159M**	Conservative base case

Revenue is computed using a spread-based model: each tier earns the difference between its borrowing rate from the tier above and its lending rate to the tier below, applied to the same underlying loan capital as it flows down the hierarchy. Summing the three tier spreads (3% + 2% + 3% = 8%) equals the retail client's total APR, confirming internal consistency—no double-counting occurs. An earlier draft of this table reported $224.6M by summing the full APR at each tier rather than the spread; that figure was incorrect and has been corrected here.

Scale note on the $137.6M figure: *This is the theoretical 10-year ceiling at 68,800 active loans across 25 Local Banks at near-full capacity—it is included for scale reference only. The pre-thesis evaluation target is Year 1–2 of the adoption ramp, where the platform reaches operational break-even at approximately 300 active loans and demonstrates positive unit economics on each incremental loan above that threshold. Figure 5.1 shows the theoretical maximum at full maturity; see the break-even analysis above for the realistic path from zero to that ceiling.*

The full revenue model decomposes into spreads, origination fees, and FX conversion income. The projection above models interest spread income only (base case: $137.6M). Additional revenue streams from FX conversion spreads (estimated 0.5 to 1.0% per conversion, yielding $8–17M), loan origination fees (0.25% per disbursement, yielding $4.3M), and insurance fund premiums (0.5% of loan value annually) represent material upside captured in the "Total incl. fees" row of the table ($150–159M). These are net incremental revenue streams—they do not overlap with the interest spread income already accounted for.

Table 5.22: Revenue sensitivity to default rate at fixed USDC loan base ($1.72B).

Default Rate	Gross Spread Revenue	Default Loss	Net Spread Revenue
Optimistic (3.7%)	$137.6M	$7.4M	$130.2M
Base Case (8%)	$137.6M	$16.0M	$121.6M
Stress Test (15%)	$137.6M	$30.0M	$107.6M

Sources: *USDC loan base held constant at $1.72B (68,800 active loans × $25,000 average); interest rate tiers aligned with Aave [15] and Compound [16] DeFi benchmarks; default losses from Table 5.23. ETH spot price affects gas costs and collateral LTV only, not USDC-denominated spread revenue.*

Cost Model Considerations.

- **Gas costs:** Gas expenses on Polygon zkEVM Cardona are low. A complete retail loan lifecycle involves approximately 27–32 individual on-chain *state changes* (`SSTORE` writes and event emissions) versus 10–15 user-facing *transactions* (loan request, approval, disbursement, and installment payments). The reconciliation is explicit:
  - 1 loan request (`SSTORE`: borrower data, loan ID, status)
  - 1 credit history lookup + risk score commit
  - 1 approval (`SSTORE`: status update, disbursement record)
  - 1 disbursement (ETH transfer + `SSTORE`: balance update)
  - 12 installment payments × 2 `SSTORE` each = 24 writes
  - 1 loan closure (`SSTORE`: final status)
  - ≈4 event emissions (`LOG2`/`LOG3`)

  On Polygon zkEVM Cardona, with gas prices of approximately 30 Gwei and ETH at ≈$2,500 (planning assumption), each `SSTORE` costs approximately $0.00036. A 30-operation lifecycle costs approximately $0.011 total—well under 0.01% of the interest earned on a representative $25,000 USDC loan. On Ethereum mainnet at 15 Gwei base fee, the same operations would cost $2.40–$4.80, reinforcing the single-chain Layer 2 design choice [55].
- **Infrastructure costs:** Backend hosting (API server, database, ML service), frontend delivery (CDN) and RPC nodes (Alchemy, Infura) can incur costs ranging between $500 and $2,000 monthly. An increase in expenses will be experienced with a rise in transactions.
- **Default losses:** A single default rate assumption is insufficient for an early-stage platform. Table 5.23 presents a three-scenario sensitivity analysis:

Table 5.23: Default-rate sensitivity scenarios at fixed USDC loan base.

Scenario	Default Rate	Annual Loss	Basis
Optimistic	3.7%	$7.4M	Grameen Bank 96.29% recovery (June 2024) [R12] after 48 years of social infrastructure
Base Case	8%	$16M	Typical early-stage DeFi under-collateralized lending; no established social trust
Stress Test	15%	$30M	Early-stage crypto-native clients, no prior on-chain credit history

Grameen Bank, after nearly five decades of social lending infrastructure, reported a loan recovery rate of 96.29% as of June 2024 [R12]. The Crypto World Bank is a nascent platform without equivalent social trust or community officers. Its initial user population will likely exhibit default rates closer to 8–15% before on-chain credit history accumulates sufficient predictive signal. The base case adopts 8% default, with break-even at approximately 11.2% default rate under the base loan volume assumption.

Break-even user count: At the base case of 8% default with an average loan size of $25,000 USDC and 8% APR, each loan generates $2,000 annual interest and incurs under $0.02 gas on Polygon. At $500/month infrastructure costs, the platform requires at minimum 4 active loans to cover operating expenses and approximately 200 active loans to cover expected default losses at 8%—a realistic near-term target for a pilot in 2–3 Local Banks.

- **Break-even analysis:** Opportunities for profit on loans as small as $25 USDC exist on Polygon zkEVM Cardona. Profitability on loans below $12,500 USDC is not achievable on Ethereum mainnet without Layer 2. Layer 2 is essential for facilitating micro-loans.

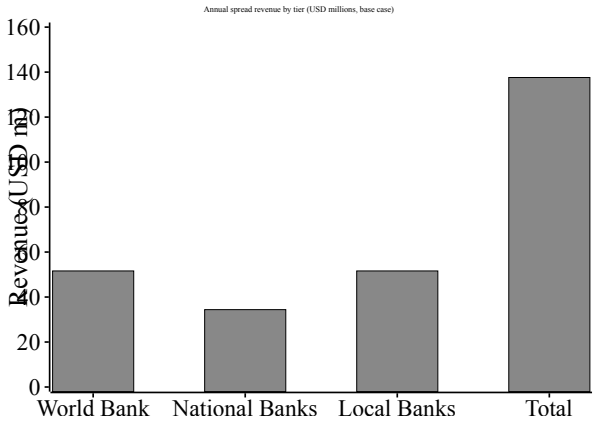


Figure 5.1: Annual revenue projection (10-year maturity)

5.8.1 Why CWB Is Not a Centralised Exchange

Centralised exchanges (CEXs) such as Binance perform order-book matching on off-chain ledgers, hold customer private keys in custody, and monetise primarily through trading

fees and ecosystem tokens [128]. The Crypto World Bank is architecturally a **hierarchical lending bank**, not a trading venue. Table 5.24 contrasts the two model classes using comparative dimensions from the project industry-research corpus (Appendix D, entries IR-1 and IR-2) [128,129].

Table 5.24: Why CWB is not a centralised exchange (CEX): architectural comparison.

Dimension	Typical CEX (Binance-class)	Crypto World Bank
Primary function	Spot/derivatives matching; fiat on/off ramps	Deposit mobilization, hierarchical lending, reserve governance
Settlement model	Off-chain internal ledger; on-chain for deposits/withdrawals only	On-chain smart contracts for loan lifecycle, reserves, and repayments
Customer custody	Exchange holds private keys (custodial)	User-controlled wallets; institution role-gated contracts
Native protocol token	BNB (fee discounts, burns, ecosystem utility)	**None** in prototype—Credit Passport SBT replaces speculative governance token (FTT lesson) [128]
Fund segregation	PoR (Merkle tree + zk-SNARKs); commingling failure mode (FTX) [128]	Segregated contract pools per tier; `InsuranceFund` + reserve-ratio gates
Deposit / yield product	Binance Earn (off-chain ledger savings/staking)	`SavingsVault` + `FixedDeposit` (ERC-4626 / ERC-7540 on-chain vaults)
Default / shock buffer	SAFU (Secure Asset Fund for Users; ≈$1B USDC reserve) [128]	`InsuranceFund` (5% interest capture; Chainlink PoR; IBLP default cascade)
Large exposure handling	OTC / institutional desk (exchange-side)	Bank-level `SyndicatedLoan` (Section 3.14.3); clients enter via Local Bank only
Revenue core	Trading fees, listing, derivatives funding	Tier interest spread (3%+2%+3%), origination, syndication fees
Regulatory posture	VASP / CASP exchange licensing	Uses authorised EMTs (USDC/USDT); does not issue stablecoins or operate an order book

Rows summarise dimensions from the Binance/FTX industry analysis [128] and the five-exchange comparative matrix [129]. CWB borrows product analogues (Earn, SAFU) from exchange-sector patterns but implements them as on-chain banking modules with explicit reserve and segregation rules.

5.8.2 Exchange-Sector Revenue Analogues

The interest spread identified above is the platform's primary revenue stream. Placing it within the broader landscape of institutional crypto-exchange revenue models demonstrates that CWB has a diversified revenue path even without a native token. Table 5.25 maps each CWB mechanism to its nearest exchange-sector analogue, drawing on the revenue architecture documented in the Binance/FTX industry analysis and complementary business-research notes (Appendix D, entries IR-1 and IR-3) [128].

Table 5.25: Revenue taxonomy: CWB mechanisms mapped to exchange-sector analogues.

Revenue Stream	Exchange Equivalent	CWB Mechanism
Interest spread (primary)	Binance OTC desk	SavingsVault yield vs. retail lending rate; 3%+2%+3% tier spread = 8% client APR
Loan origination fee	Trading fee (0.1%)	0.25% flat origination fee on disbursement; modelled in base case above
Deposit mobilization	Binance Earn	SavingsVault + FixedDeposit products; yield funded by the lending tier spread
Local Bank registration	Exchange listing fee	One-time registration fee per Local Bank joining the network
Syndicated loan arrangement	Institutional desk	Lead Arranger fee on TranchedPool / SyndicatedLoan cross-tier facilities
Reserve transparency API	Data API subscription	Phase 3: REST/SQL API over the event-listener index (GraphQL over The Graph optional later); queried by external auditors and regulators

The six streams above are non-overlapping; none double-counts the interest spread already accounted for in Table 5.21. The Local Bank registration fee and Reserve Transparency API are Phase 2/3 items and are not included in the base-case revenue model above. The interest spread and loan origination fee are the only streams active at the pre-thesis prototype stage.

Explicit product analogues (Earn and SAFU). Two exchange-sector modules have direct CWB counterparts, implemented on-chain with banking semantics rather than custodial ledger entries. **Binance Earn**—Binance's staking and savings products that lock user assets on the exchange's internal ledger while paying yield from lending and staking revenue [128]—maps to the `SavingsVault` and `FixedDeposit` contracts (Section 3.11): variable- and fixed-term deposit mobilization whose yield is funded by the lending-tier interest spread, not by trading-fee subsidies. **SAFU** (Secure Asset Fund for Users)—Binance's publicly verifiable ≈$1 billion USDC emergency reserve established

in 2018 to cover exchange-side security events [128]—maps to the `InsuranceFund` contract: it captures 5% of interest collected across the hierarchy, publishes its balance via Chainlink Proof of Reserve, and is drawn in the interbank default cascade (reserve buffer → `InsuranceFund` → parent-tier backstop; Section 3.14.1). The analogy is structural, not custodial: CWB does not hold user keys, but both mechanisms isolate a transparent buffer against tail-risk losses.

Hard rule: no native CWB governance token in the prototype. The FTX collapse of November 2022 demonstrated the systemic failure risk of self-minted tokens used as collateral: FTT, issued and held principally by the exchange itself, served simultaneously as the primary collateral asset for Alameda Research's borrowing, creating a reflexive loop that triggered an estimated $8 billion solvency shortfall within 72 hours of a competitor announcing it would liquidate its FTT holdings [128]. The lesson for the Crypto World Bank is categorical: no native CWB governance token is issued in the thesis prototype. On-chain reputation is provided instead by the Credit Passport SBT (Section 3.12), which is non-transferable and carries no speculative market price. A future governance token—analogous to Binance's BNB utility-and-burn model, which took seven years and $16.8 billion in annual revenue to reach legitimacy—is scoped to Phase 3 of the CWB roadmap, following institutional trust bootstrapping, and is specified in Future Work with explicit anti-FTX collateral hard rules encoded at the contract level (no self-issued asset may be used as collateral within the same protocol that issues it). This constraint will be enforced as a Foundry invariant test rather than a governance policy, so it cannot be bypassed by any admin action.

5.8.3 Transaction Economics: Interest Rates

Table 5.26: Interest rate parameters.

Parameter	Value	Benchmark
Base Annual Interest Rate	5–12% APR	Aligned with Aave/Compound variable rates
Rate Determination	Set by Local Bank approvers within World Bank-defined bounds	Configurable per-bank for local market conditions
Late Payment Penalty	2% of installment + 0.5%/week (capped at 10%)	Industry-standard late fee structure
Interest Calculation	Simple interest on outstanding principal	Transparent, client-friendly
Rate Transparency	All parameters stored on-chain	Publicly auditable; no hidden fees

This interest rate table defines the tiered APR parameters used throughout the feasibility and revenue modeling. Presenting the rate structure explicitly makes the spread logic auditable and connects the economic model directly to the proposed governance-controlled parameters.

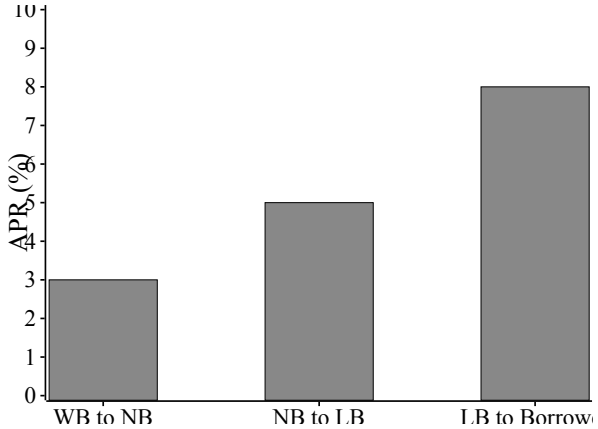


Figure 5.2: Hierarchical interest-rate spread (APR)

5.8.4 Global Economic Impact

Apart from revenue generated through platforms, various economic advantages are provided by the Crypto World Bank:

- **Capital deployment and fiscal multiplier:** At projected full-deployment lending volume of $2 billion across nearly 1,000 institutional clients, the platform generates a downstream fiscal multiplier of approximately $2.5 to $3 per dollar lent [19], implying $5 to $6 billion in annual economic activity. The IFC estimates that every $1 million lent to small businesses in developing economies creates approximately 16 new jobs [20], making capital deployment at this scale a meaningful driver of employment and local economic growth.
- **Remittance cost reduction:** Global remittances total approximately $860 billion annually, of which an estimated $48 to $56 billion is consumed by transfer fees averaging 6.49%—more than double the United Nations Sustainable Development Goal target of 3% [26]. On-chain settlement on Layer 2 networks reduces transaction costs to well below 1%, directly compressing this fee burden. Comparable blockchain payment networks demonstrate the feasibility of this target: Stellar processed approximately $55.6 billion in payments at a fee of roughly $0.0007 per transaction in 2025 [44], and Celo's MiniPay processes payments for under one cent across more than 60 countries [42].
- **Trapped capital liberation:** The correspondent banking system requires banks to maintain pre-funded nostro and vostro accounts in every currency corridor in which they operate, immobilizing large sums that cannot be deployed for lending or investment [24]. On-chain atomic settlement eliminates the need for these idle pre-funded balances, freeing capital for productive use across the lending hierarchy.
- **Financial inclusion:** The platform is designed for accessibility: sub-cent gas fees on Polygon zkEVM Cardona, a mobile-first interface, and support for micro-loan amounts below one dollar lower the barriers to credit for the estimated 1.4 billion adults globally who remain outside formal financial systems [14]. The World Economic Forum has identified decentralized finance as a leapfrog technology capable of enabling populations to bypass the infrastructure constraints of traditional banking [41], a pattern already observed in mobile payments across developing economies.
- **Transaction cost reduction:** On Polygon zkEVM Cardona, transaction fees remain below one cent—a reduction of over **99%** relative to the average $42 cost of a corre-

spondent banking transaction [25]. This cost compression makes financially viable a large class of micro-transactions and small-loan servicing operations that are currently uneconomical through traditional payment rails.

- **Transparency as an economic good:** On-chain publication of reserve ratios, interest rates, and transaction records eliminates the informational asymmetry that enables predatory lending practices in opaque banking environments. In conventional banking, borrowers—particularly in developing economies—frequently operate without visibility into the true cost of credit or the basis for lending decisions. The Crypto World Bank's design ensures that all participants access identical, verifiable information, reducing the scope for hidden fees and unfair pricing.

5.8.5 Value Proposition and Go-to-Market

Table 5.28: Go-to-market phases.

Phase	Activities	Timeline
Phase 1: Validation	Competition submission (BCOLBD 2025); thesis publication; open-source release	Current
Phase 2: Pilot	Regulatory sandbox application; institutional partnership; testnet-to-mainnet migration	6–12 months
Phase 3: Production	Single-chain production on Polygon zkEVM (or EVM L2 migration path); enhanced monitoring and analytics; Credit Passport SBT at scale	12–24 months

This value proposition table summarizes user-visible benefits (cost, transparency, speed, and inclusion) and maps them to platform features. It supports the go-to-market narrative by linking technical design choices to practical outcomes for retail users and partner institutions.

5.9 Currency Risk and the Stablecoin Imperative

*This section uses a **currency-risk illustration** only; it does not define a country-specific deployment target (see Section 5.2).*

A hypothetical retail borrower who takes a loan denominated in ETH faces a fundamental risk that fiat-denominated traditional lending avoids: if ETH doubles between disbursement and final repayment, the borrower's repayment obligation rises in local fiat terms by the same factor. For thin-margin borrowers, that is not a theoretical risk—it is potentially catastrophic. The May 2021 crypto market crash saw ETH lose approximately 55% of its value within six weeks; ETH then halved again over the course of 2022.

This is why stablecoin integration is treated as a **critical path item** elevated to a design requirement in Section 1.8.3, not an optional extension. The platform supports USDC or USDT-denominated loans as the default product at retail tiers, with ETH-denominated loans reserved for institutional-tier participants who can manage currency risk. BIS Working Paper No. 905 [R13] identifies stablecoin volatility as a structural risk in emerging-market DeFi

adoption and recommends fully-collateralized models (USDT/USDC) over algorithmic designs for developing-country use cases. The Terra/LUNA collapse (May 2022, ≈$40 billion lost) provides the definitive negative example. ERC-20 stablecoin integration has technical implications: token allowances, `transferFrom` hooks, decimal precision (USDC uses 6 decimals vs. ETH's 18), and the approval-transfer-state-update ordering required by the CEI pattern must all be redesigned as a separate contract module—work that is in scope for the SavingsVault contract specified in Section 3.11.

5.10 MiCA and GENIUS Act Compliance Mapping

The platform's regulatory feasibility is not abstract: two recent statutes apply directly to any retail stablecoin-denominated lending product issued to European or US-jurisdiction users. The EU Markets in Crypto-Assets (MiCA) Regulation has been fully in effect since December 2024 [R36, R37]; the US GENIUS Act, signed in July 2025 [R38], establishes the first federal framework for payment stablecoins. Table 5.30 maps the relevant articles to the design choices that satisfy them and to the work that remains.

Table 5.30: MiCA and GENIUS Act compliance mapping.

Statute / Article	Requirement	CWB Design Control
MiCA Art. 16 (EMT Issuer Authorisation)	Authorized issuer required for any e-money token offered to the public	USDC / USDT are issued by Circle / Tether under their own authorization; CWB uses but does not issue EMTs.
MiCA Art. 36 (Reserve of Assets)	1:1 reserve backing, segregated from issuer assets	Stablecoin reserves verified via on-chain attestations from Circle (USDC) before being accepted by SavingsVault.
MiCA Art. 41 (Redemption Rights)	Holders must have a right of redemption against the issuer at par	CWB does not impede redemption; SavingsVault withdraw function returns the underlying ERC-20 directly to the depositor wallet.
MiCA Art. 62 (Operational Resilience)	Regular audits, governance, complaint handling	Five-layer defense-in-depth (Section 3.19); Foundry invariants (Section 4.5); Certora reserve invariants (Future Work, Section 6); Tenderly runtime monitoring (Section 4.7).
GENIUS Act § 4 (Federal Payment Stablecoin Framework)	Federal registration of payment-stablecoin issuers	Same posture as MiCA Art. 16: CWB uses, does not issue.
GENIUS Act § 7 (Reserve Disclosure)	Monthly attestation and public disclosure of reserves	Circle / Tether attestations are independently verifiable on-chain; CWB dashboard surfaces issuer attestation links per stablecoin pool.
EU AI Act Annex III (high-risk: credit scoring)	AI systems evaluating natural-person creditworthiness classified high-risk from August 2026 [126]	RF/IF pipeline is in scope; conformity assessment, risk-management system, and technical documentation required before EU-resident deployment.
EU AI Act Art. 86	Right to explanation for AI-assisted decisions	Borrower-facing SHAP + anomaly summaries (Section 4.3.2); Authority Brief for approvers; audit log for compliance. SHAP satisfies post-hoc explanation partially; substantive "meaningful explanation" threshold may require additional artifacts [126].
EU AI Act Arts. 11, 12, 14	Technical documentation, automated logging, human oversight	`LOAN_RISK_ASSESSMENT` + `MODEL_REGISTRY` (Art. 12 partial); human approver gate (Art. 14); full Art. 11 technical file and risk-management system not yet complete.
GDPR Art. 17 (Right to Erasure)	Personal data must be deletable on request	No PII on-chain; off-chain PostgreSQL data is encrypted and subject to data-subject deletion re-

The result is a compliance posture that is defensible to European and American institutional readers and evaluators in a single page rather than buried in narrative. Compliance gaps that remain—a formal MiCA whitepaper if CWB ever issues its own token, FinCEN reporting integration in the US, and full EU AI Act conformity assessment—are explicitly named in Section 5.14 and the Future Work section of the Conclusion.

5.11 Jurisdiction and Regulatory Considerations

The platform is designed as a global institutional L2 architecture; jurisdiction-specific deployment is future work. One illustrative regulatory context—Bangladesh, cited in the microfinance literature—is summarized here to show how a hierarchical on-chain bank would face real-world constraints. Three observations frame the path forward.

Crypto remains effectively illegal for retail use in several jurisdictions as of 2025–2026. In Bangladesh, Bangladesh Bank's longstanding 2018 circular treats crypto trading as a Foreign Exchange Regulation Act offence, and no subsequent statute has formally legalized retail crypto activity [R41]. The Crypto World Bank prototype is therefore planned exclusively on public EVM testnets (Polygon Amoy PoS and/or Ethereum Sepolia) using test tokens. No mainnet value transacts at pre-thesis stage.

The CBDC trajectory is a credible legal path in many jurisdictions. Several central banks have launched CBDC feasibility studies and pilots. The Crypto World Bank's positioning against mBridge and Agora (Section 2.2.6) provides a direct upgrade path: when a jurisdiction issues a CBDC under the hierarchical distribution model that the Tan (2023) IMF working paper [5] describes, the platform's Tier 1 / Tier 2 contracts can settle in that CBDC without changing the four-tier architecture. ETH / USDC denomination in testnet is a stand-in for future CBDC or tokenized-deposit assets.

Regulatory sandboxes as entry points. Jurisdictions such as Singapore (MAS Project Guardian), UAE (DIFC Innovation Testing Licence), and Bangladesh (Regulatory FinTech Facilitation Office) offer sandbox frameworks for controlled pilots. The platform's go-to-market path is: (a) academic testnet deployment for pre-thesis, (b) controlled sandbox engagement for final-thesis evaluation, (c) partner-institution pilot under sandbox cover, and (d) phased CBDC integration where applicable.

Microfinance inclusion data (literature precedent). Bangladesh Bank Special Publication SP2025-02 (June 2025) [R44] reports female-to-male account parity at 49%/49% by March 2025. BRAC's 2025 data shows 89% of loans to women [R45]. These statistics inform the solidarity-group module design as literature precedent (BRAC/Grameen group lending); they do not define a deployment target market for this thesis.

FATF Travel Rule scoping. The Financial Action Task Force Recommendation 16 (the "Travel Rule") requires that transfers above USD 1,000 include structured originator and beneficiary information transmitted alongside the transfer. In most FATF member jurisdictions this threshold is now in force for virtual asset service providers; Bangladesh, as a FATF Asia-Pacific Group member, is under the same obligation. For the Crypto World Bank, the Travel Rule applies specifically to inter-tier capital flows: Local Bank to National Bank repayments and upward surplus repatriation via the `UpwardDepositFacility` contract

routinely exceed the USD 1,000 threshold. The off-chain Travel Rule data packet required for each such transfer is specified in Section 3.14.2:

```
{
  "originator_institution": "LocalBank-042",
  "originator_tier": 3,
  "beneficiary_institution": "NationalBank-001",
  "beneficiary_tier": 2,
  "amount_usdc": 5000,
  "purpose": "IBLP_REPAYMENT",
  "onchain_tx_hash": "0xABC...",
  "timestamp": "2026-06-01T10:00:00Z"
}
```

This packet is stored in the PostgreSQL `audit_logs` table and is linked to the on-chain transaction hash, making it available to regulators through the formal audit request workflow described in Section 3.18.5. The `audit_logs` table is append-only (enforced at the database-role level), ensuring the Travel Rule record cannot be altered after filing. Implementation of the cross-tier Travel Rule notification protocol—including automated packet generation, digital signing by the originating institution, and transmission to the relevant FIU—is Future Work contingent on jurisdiction-specific guidance.

5.12 Bootstrap Funding and the Tier 1 Capitalization Problem

The Crypto World Bank faces a **bootstrap funding problem** analogous to the capitalization challenge of real multilateral development banks. The World Bank Group was initially capitalised by 44 member-state subscriptions in 1944; the IBRD's combined subscribed capital exceeded $270 billion following its 2018 capital increase (Ocampo & Gallagher, 2024 [R15]). Three initial capitalization mechanisms are proposed for the Tier 1 Reserve:

1. **Founding stakeholder deposits:** Universities, NGOs, and development-focused blockchain organizations deposit ETH or USDC into the Tier 1 Reserve in exchange for documented yield rights and non-transferable governance participation records (not a speculative protocol token; Section 5.8.1).
2. **Protocol-owned liquidity (POL):** The protocol accumulates treasury reserves through bond mechanisms in which external parties swap ETH or USDC for vesting-schedule yield claims backed by fully collateralized USDC reserves. *Note: The Olympus DAO algorithmic reserve model (2021–2022) is cited here as a cautionary precedent—Olympus OHM lost over 99% of its token value within one year due to unsustainable algorithmic backing. The CWB POL mechanism is strictly collateralized: only fully-backed USDC bonds are accepted, with no reflexive token price-support obligation.*
3. **Philanthropic/impact grant funding:** Development finance institutions (IFC, ADB) have expressed interest in blockchain-based development finance tools, as evidenced by the World Bank FundsChain initiative [39]. Grant funding from these institutions could seed the Tier 1 Reserve in exchange for research access and co-branding.

In the short term, a multi-signature wallet (3-of-5 signers representing different stakeholder

groups) governs Tier 1 allocations. In the long term, an on-chain governance module (similar to Compound Governor Bravo) enables time-locked, snapshot-based voting with timelock delays—without issuing a native speculative CWB token in the thesis prototype (Section 5.8.1).

5.13 Prototype Scope and Limitations

At pre-thesis stage, the complete system is **specified** but not deployed. Planned implementation scope:

1. **Hierarchical lending scope:** Tier 1 World Bank Reserve, Tier 2 National Bank, and Tier 3 Local Bank contracts are specified in Solidity. Full cross-tier fund transfers and loan lifecycle automation are planned for Phases I–II of the final thesis phase.
2. **Interest accrual and repayment:** Interest rate tiers (3%/5%/8%) and fee rules are specified. Automated installment scheduling, accrual, and penalty enforcement are planned for Phase II (MVT item 2).
3. **InterBankLendingPool, UpwardDepositFacility, and broader multi-entity / cross-tier operations:** Fully specified in Section 3.14. Per Table 1.1 and the MVT boundary (Table 4.1), `GroupLendingPool`, `InterBankLendingPool`, `SavingsVault`, and `UpwardDepositFacility` are Phase II deliverables; `SyndicatedLoan` and `TranchedPool` are Phase III stretch goals; `NettingEngine` is Future Work.
4. **AI/ML integration:** Random Forest, Isolation Forest, SHAP, and the Authority Brief pipeline are specified in Sections 4.3.1 and 4.3.2. FastAPI scoring and on-chain oracle wiring are planned for Phases II–III (MVT items 4–7); no live loan-approval integration is claimed at pre-thesis stage.
5. **Backend architecture:** Express.js REST API with PostgreSQL (authoritative schema, 51 entities — 31 core + 6 extension + 14 stubs) and FastAPI for ML inference. MongoDB is not part of the planned production architecture.
6. **Banking product suite:** The extended banking product suite described in Section 3.17, including savings accounts, fixed-term deposits, transactional accounts, group lending, FX conversion, and the insurance fund, is specified at the architectural and design level in this report. Smart contract implementation and testing of these modules is planned for the final thesis phase.

The limitations above are consistent with a planning-phase pre-thesis; the complete system design is fully specified in this report and will be implemented and validated in the final thesis phase.

5.14 Research Trajectory and Examiner Limitations

A structured literature scan (June 2026) identified five risks examiners are likely to probe. This section records them explicitly so the thesis does not over-claim.

G1 — Compound novelty vs. partial counterexamples. Maple Finance, Goldfinch, Morpho V2, and Aave Arc provide partial institutional-credit precedents on individual axes [117,118,R21]. The defensible claim is the *compound* architecture (four-tier reserve enforcement + explainable oracle ML + programmable mutual liability), not "no prior four-tier DeFi."

G2 — BCCC domain transfer. Models trained on BCCC flat-pool transactions may not generalize to hierarchical CWB flows (Section 4.3.1). At pre-thesis stage, only a *synthetic BCCC-shaped stand-in* has been used for pipeline validation while York BCCC dataset access is pending; real BCCC metrics will replace synthetic figures after approval. Mitigation: acknowledge the gap, benchmark on BCCC as baseline only once available, collect CWB testnet labels for retraining, and run feature-distribution comparison as Future Work.

G3 — Testnet scope as academic evidence. Working testnet deployment + benchmark tables + documented specifications suffice for workshop venues (AFT, FMBC, IEEE Blockchain workshops). Full research papers additionally require controlled experiments, baselines, threat models, and reproducible public repos. This pre-thesis delivers specification and workload planning; MVT deployment provides the empirical layer.

G4 — Group lending external validity. On-chain mutual liability is cryptographic cosigning, not social solidarity [119,120]. Repayment-rate claims must not cite BRAC/Grameen historical data as direct validation of smart-contract behaviour.

G5 — Oracle trust and economic viability. Score-at-origination-only mitigates latency/cost; oracle collusion and model staleness remain open (Section 4.3.1). Certora reserve-invariant proofs are Future Work (Section 6); Foundry fuzz tests should list what is asserted, assumed, and out of scope [127].

Publication trajectory. Primary venue targets aligned with CWB contributions: **AFT 2026** (DeFi systems; abstract deadline May 2026), **FMBC** (formal verification of reserve invariants), **IEEE Blockchain / ICBC** (hierarchical architecture), and **IEEE TIFS** (federated fraud + explainability). PhD-scale seeds: hierarchical GraphSAGE ablation, cross-tier FL with differential privacy, and zkAML + FATF Travel Rule integration (Section 6).

5.15 Accessibility Assessment: Hypothetical Retail Client

This section evaluates access across six dimensions for a hypothetical retail client seeking a $500 USDC working-capital loan through a Local Bank interface. The assessment is illustrative; no geographic deployment target is claimed at pre-thesis stage (Section 5.11).

1. Device access. The platform's mobile-first React interface and WalletConnect integration allow access via any Android or iOS device with a browser, without requiring a desktop or dedicated hardware wallet.

2. Connectivity. Layer-2 transactions are lightweight (under 10 KB) and complete within seconds, well within typical mobile-network latency budgets.

3. Transaction cost. On Polygon zkEVM Cardona, the complete loan lifecycle (request, approval, 6 monthly installments) is projected to cost under $0.05 in gas, a small fraction of typical origination fees in traditional banking.

4. Language and literacy. The planned prototype is English-only at launch. Additional locale packs are a planned extension; the React component library supports right-to-left and Unicode rendering.

5. Identity and onboarding. Tiered off-chain KYC and RBAC role gates serve MVT onboarding; the planned Groth16 zkKYC / zkAML extension is Future Work (Section 6). ERC-4337 account abstraction hides wallet complexity from retail users.

6. Group lending fit. The solidarity group lending model (`GroupLendingPool`, planned) draws on microfinance literature precedent (BRAC/Grameen) for *motivation only*. On-chain mutual liability is programmable joint liability with cryptographic enforcement—not a behavioural equivalent of field-officer solidarity groups (Section 3.17.3).

In aggregate, the Crypto World Bank is accessible in principle to a hypothetical retail client on a mobile device (connectivity, cost, ERC-4337 onboarding) with two gaps requiring resolution for a production deployment: locale packs beyond English at launch and stablecoin-first denomination to reduce fiat currency risk for cross-border users.

Chapter 6

Conclusion

The Crypto World Bank addresses a **multi-party coordination and trust** problem inherent in hierarchical development finance by exploiting the distinctive properties of blockchain technology: cryptographic integrity, immutability, and programmable auditability. It advances beyond existing DeFi lending protocols—which collectively manage over $55 billion in TVL [13] but universally employ flat, single-tier pool architectures—through a four-tier institutional hierarchy with cross-tier, same-tier, and upward lending flows, combined with a governance structure that addresses network membership, business operations, and technology infrastructure.

The platform's design is grounded in the six core functions of a bank—deposit mobilization, credit allocation, payment and settlement, risk intermediation, liquidity management, and ancillary services—and specifies how each function maps to smart contract logic *where implemented or planned*, rather than claiming every module is live at pre-thesis stage (Section 3.1). Where deployed, rules that previously required legal agreements, periodic audits, and discretionary enforcement can be expressed as deterministic, publicly verifiable on-chain logic within governance bounds. Reserve ratios are checked atomically on governed transactions; interest rates adjust algorithmically with utilization; group consent is recorded on-chain before disbursement; and the loan book is auditable in real time by platform participants.

A distinct architectural capability of this platform is **multi-entity co-lending and co-funding**—the ability for groups of entities to lend, fund, and borrow together. This capability operates at two levels simultaneously. At the retail level, the `GroupLendingPool` allows three to twenty clients to form a solidarity group, pool collateral, and co-borrow under mutual liability enforcement—implementing *programmable joint liability* inspired by microfinance solidarity mechanics but not behavioural equivalence to Grameen Bank or BRAC field-officer models (Section 3.17.3, Contribution 2). At the institutional level, the `SyndicatedLoan` contract allows multiple banks at the same or adjacent tier to co-fund a single loan, distributing credit risk pro-rata among co-lenders through an on-chain consent mechanism, subscription window, and supermajority confirmation vote—replicating the mechanics of a traditional syndicated loan deal without bilateral legal agreements. The `TranchedPool` further allows risk-tolerant and risk-averse entities to co-fund the same pool with differentiated seniority rights. No existing DeFi protocol implements this full multi-entity suite in one open specification.

This thesis makes four contributions; **three are MVT-primary** and one is specification-only at pre-thesis stage: (1) a *compound* four-tier hierarchical DeFi architecture (reserve-ratio enforcement + explainable oracle ML + programmable mutual liability) that mirrors multilateral development-finance capital flows—partial counterexamples exist on individual axes but not in combination (Section 5.14); (2) an on-chain *programmable joint-liability* group lending specification with over-indebtedness controls (Section 3.17.3), honestly

scoped against analogue solidarity-lending limits; (3) an oracle-mediated AI/ML integration pattern with SHAP- and anomaly-report explainability for fraud and AML governance (Section 4.3.2), committed on-chain via the MVT commit–reveal relay at loan origination (Chainlink Functions as a stretch upgrade when supported), extended by a Phase III MCP conversational agent with a **3–5-tool** MVT subset (Section 4.4; full 17-tool suite is Future Work), with BCCC benchmarking subject to an acknowledged domain-transfer gap (Section 4.3.1); and (4) a *specified, not yet deployed* compliance-aware identity pathway (zkKYC, zkAML, W3C DID/VC) motivated in the literature and scoped to Future Work (Section 6). GNN ablation, cross-tier federated learning, `NettingEngine`, and Certora proofs remain outside the MVT minimum. These contributions are framed against the institutional-DeFi landscape (Sygnum, mBridge, Agora) in Section 2.2.6 and against the MiCA / GENIUS Act / EU AI Act frontier in Section 5.10.

The analysis of the competitive situation of 20+ existing projects in DeFi lending, institutional credit, banking infrastructure and financial inclusion affirms that no current system combines multi-level lending hierarchy, interbank lending mechanisms, and graded borrower access in one specified architecture. The growing blockchain implementation in institutions, as demonstrated by the World Bank FundsChain initiative [39], the multi-billion-dollar daily volumes of JPMorgan Kinexys [40] and R3 Corda's $17 billion in tokenized assets [37], validates demand for verifiable institutional financial infrastructure. The Crypto World Bank extends this trajectory from settlement and fund tracking toward a *specified*, hierarchically governed lending layer composable with such rails—implementation evidence is scoped to MVT testnet deliverables (Section 4.1).

The platform occupies the intersection of institutional finance, decentralized lending, and financial inclusion—a combination that remains unaddressed in both the academic literature and the commercial blockchain landscape. With a fully specified architecture, a defined market and partnership plan (Chapter 5), and a phased implementation roadmap bounded by the MVT checklist (Section 4.1), the Crypto World Bank represents a structurally distinct and architecturally grounded contribution to blockchain-based development finance at the *specification and prototype* stage.

Viewed as a complete banking function checklist, the pre-thesis deliverable specifies hierarchical lending workflow foundations (tiered roles, governance framing, architectural capital flow, and a planned AI/ML monitoring layer with explainability) while leaving several bank-grade modules at the design level. Modules specified but not yet implemented in the final thesis include:

- Deposit products (SavingsVault and FixedDeposit), specified in Section 3.11
- Transactional accounts (CurrentAccount) and retail payment rails: client-to-client remittance, merchant checkout, and fiat on/off-ramp (Section 6)
- Group lending (GroupLendingPool) with mutual-liability and over-indebtedness logic (Section 3.17.3)
- Liquidation Engine (Section 3.10)
- On-chain credit passport SBT (Section 3.12)
- Oracle-priced FX conversion (FXModule)
- Multi-entity and cross-tier operations: InterBankLendingPool, UpwardDepositFacility, SyndicatedLoan, TranchedPool, and TreasurySwap, specified in Section 3.14; `NettingEngine` is Future Work (Section 6)
- Insurance and depositor protection (InsuranceFund)

- Conversational agent (MCP + Qwen3-8B, Phase III Must), specified in Section 4.4
- Federated learning, GNN ablation, Groth16 zkKYC/zkAML circuits, `NettingEngine`, and Certora reserve-invariant proofs (Section 6)
- Foundry fuzz / invariant suite (Section 4.5)
- Tenderly + The Graph runtime monitoring stack (Section 4.7)

Future iterations therefore focus on completing end-to-end cross-tier fund transfers and repayment automation on a single chosen prototype testnet (Section 3.13), integrating the analytics layer into real approval flows under the MVT commit–reveal oracle (with Chainlink Functions as a DON-based upgrade when supported), delivering the MCP conversational agent (Section 4.4), hardening the platform against regulatory scrutiny via MiCA / GENIUS Act compliance, and expanding the platform into a functionally complete, stablecoin-first, compliance-aware banking system—including retail payments, client-to-client remittance, merchant checkout, and fiat on/off-ramp rails (Section 6).

Future work is split into **MVT completion** (final-thesis Must deliverables) and **PhD-scale research extensions** (publishable seeds). Items removed from this list as non-research scope creep: Polygon CDK sovereign chain, MCP/WhatsApp channel adapters, LLM fine-tuning as a standalone research item, and live cross-chain bridging (retired in Section 3.13 after security analysis). ERC-4337 account abstraction remains specified in Section 3.8.1 as assumed retail infrastructure; WalletConnect suffices for MVT.

MVT completion (final thesis Must).

1. **Complete hierarchical lending implementation.** End-to-end wiring of cross-tier fund transfers between World Bank Reserve, National Bank, and Local Bank contracts—capital distribution, interest accrual, installment scheduling, cascading repayment—against Appendix B sketches.
2. **Stablecoin-first deployment.** USDC default at retail tiers (Section 1.8.3); MiCA / GENIUS Act mapping validated against deployed configuration (Section 5.10).
3. **Liquidation Engine, SavingsVault, FixedDeposit, GroupLendingPool.** Phase II deposit-and-credit suite with over-indebtedness controls (Section 3.17.3).
4. **Multi-entity operations.** `InterBankLendingPool` and `UpwardDepositFacility` as Phase II deliverables (Table 1.1); `SyndicatedLoan` and `TranchedPool` as Phase III stretch goals beyond the MVT minimum (Section 3.14). `NettingEngine` is Future Work (Section 6).
5. **AI/ML pipeline integration (core MVT).** Commit–reveal oracle at loan origination; RF + IF + SHAP + Authority Brief (Sections 4.3.2, 4.3.1); Chainlink Functions retained as a stretch upgrade when supported; synthetic pipeline validation at pre-thesis; full BCCC benchmark table after dataset access.
6. **Conversational agent (Phase III MVT).** MCP tool server (minimal 3–5 tools), chat UI, human confirmation gate, and one E2E write demonstration (Section 4.4; MVT item 11); LLM evaluation protocol (Section 4.10.1; MVT item 12).
7. **Foundry fuzz suite (MVT+).** Foundry invariant and fuzz testing (Section 4.5).
8. **Foundry simulation (Phase IV).** 300-client hierarchy on the chosen prototype testnet; gas-cost table and reserve dynamics (Section 4.6); manifest in Appendix C.
9. **Runtime monitoring.** Event-listener index, Tenderly alerts, live Reserve Transparency Dashboard (Section 4.7); The Graph is optional later.
10. **Operational security and sandbox path.** Safe multisigs (Section 3.19); controlled

pilot under MAS Project Guardian or UAE DIFC sandbox (Section 5.11).

Retail payments, remittance, and commerce (post-MVT). The checking-account specification introduces atomic transfers between *registered* platform accounts and mentions peer transfers, but the pre-thesis prototype does not implement a complete retail payments stack: there is no dedicated client-to-client remittance product, no merchant or marketplace checkout, and no fiat on/off-ramp (Section 1.8). Section 2.2.8 synthesises the peer-reviewed and policy evidence that these extensions are empirically and regulatorily studyable; closing the implementation gap is required before the platform can support everyday purchase flows with USDC rather than lending and installment servicing alone. Future work should add:

1. **Client-to-client remittance (`P2PTransfer` module).** KYC-gated, velocity-capped USDC transfers between Tier 4 wallets under Local Bank sponsorship; per-transaction AML screening; rolling 24-hour limits tied to the KYC ladder (Section 3.8.2); optional cross–Local-Bank net settlement via `NettingEngine` (Section 3.14.6). Design follows platform-mediated VASP compliance [145] rather than unhosted-wallet P2P; adoption drivers from Ante [136] inform literacy-aware onboarding.
2. **Fiat on/off-ramp integration.** Licensed payment-service-provider or regulated stablecoin-issuer APIs (bank transfer, mobile money, card) so clients can fund checking balances and withdraw to local currency—a prerequisite for real-world spending outside the on-chain ledger and the binding constraint identified by Du et al. [138] and CPMI [142].
3. **Merchant and marketplace checkout.** Payment-request contracts (invoice hash + amount + merchant `LOCAL_BANK_ROLE`), QR or deep-link checkout for registered merchants, and settlement reconciliation for sponsoring Local Banks—an on-chain analogue to card-acquirer settlement, denominated in USDC. Evaluated against the CLEAR framework of Li et al. [137]; closed-loop Local-Bank sponsorship addresses the dispute-recourse gap they identify vs. open-loop card networks.
4. **Retail FX completion.** Deploy `FXModule` (Section 3.17) so clients see fiat-equivalent balances and optional conversion before off-ramp or cross-border remittance, mitigating stablecoin refund volatility identified by Zhang et al. [140].
5. **Deposit protection and payment disputes.** Operational `InsuranceFund`; dispute workflow for erroneous P2P or merchant debits; jurisdiction-specific mapping of smart-contract obligations to enforceable consumer-protection rules—responding to the consumer-protection inversion Li et al. [137] document for raw stablecoin checkout.
6. **External commerce boundary.** Integration with DEX or payment-rail partners (e.g. composable USDC $\rightarrow$ merchant settlement via licensed off-ramp) where in-platform checkout is insufficient—explicitly out of MVP scope but documented as the bridge between CWB balances and real-world merchants, consistent with the stablecoin-sandwich pattern [138].

These items address traditional-banking gaps the hierarchical lending core does not cover: P2P remittance, merchant acceptance, cash-like deposit/withdrawal, and consumer dispute resolution—while preserving the Local-Bank entry point and AML posture of the four-tier model. No prior source composes these retail rails with hierarchical development-finance lending in one architecture (Section 2.2.8).

PhD-scale research extensions (publishable seeds). The following advanced modules are **not specified at implementation depth** in Chapters 3–4; literature motivation appears in Chapter 2, and operational detail is deferred here:

1. **GNN transaction analysis (GraphSAGE ablation).** Isolate contribution of hierarchical tier-relationship edges vs. flat wallet edges on BCCC [108]; target AFT or IEEE Blockchain.
2. **Federated learning across banking tiers.** FedAvg + differential privacy across Local Banks; benchmark against FedFraud [125] and BCCC; target IEEE TIFS.
3. **Groth16 zkKYC circuit (Circom 2.0) and zkAML velocity circuit.** Privacy-preserving KYC eligibility and AML velocity proofs with W3C DID/VC anchoring; combine with FATF Travel Rule packet spec (Section 3.14.2).
4. `NettingEngine` **with partial settlement and challenge periods.** Multilateral netting of IBLP and `TreasurySwap` obligations; `settlePartial` for solvent participants; public challenge window for dispute resolution.
5. **Certora formal verification of reserve invariants (Future Work).** CVL proofs that $\text{totalReserve} \geq \text{minReserveRatio} \cdot \text{totalDeposited}$ and that downstream allocation never exceeds available reserve; target FMBC [127].
6. **Agent-based reserve-cascade simulation.** Mesa or equivalent ABM for systemic risk under mass default, oracle manipulation, and stablecoin depeg—stress beyond testnet deployment alone.
7. **ML domain adaptation.** Feature-distribution comparison: BCCC flat-pool features vs. simulated CWB hierarchical transactions; semi-supervised baseline [124].
8. **EU AI Act conformity assessment.** Complete Art. 11 technical documentation, Art. 12 logging, and user-study on SHAP explanation adequacy under Annex III high-risk credit scoring [126].
9. **On-chain credit passport SBT + W3C DID group Sybil prevention.** SBT implementation (Section 3.12); narrow Sybil control via W3C DID group membership verification rather than World ID bundling.
10. **CLEAR-framework evaluation of CWB retail payments.** Apply Li et al.'s [137] cost–legality–experience–architecture–reach rubric to the `P2PTransfer` and merchant-checkout modules vs. card-acquirer and MTO baselines; corridor cost bench against Du et al. [138]; target IEEE Blockchain or payments-economics venue.

Demoted to appendix or single-sentence notes: SAR/FIU portal integration (compliance engineering); live cross-chain bridge (design revision in Section 3.13).

Appendix A

Database Schema Reference

This appendix records the physical-schema details moved from Chapter 3 to keep the main architecture chapter focused on conceptual design.

A.1 Indexing Strategy

B-tree indexes (the PostgreSQL default) support efficient retrieval on frequently queried columns:

Table A.1: Indexing strategy: B-tree indexes for time-window and high-frequency queries.

Index	Table / Column(s)	Rationale
idx_loan_borrower	LOAN_REQUEST(borrower_id)	High-frequency lookup for borrower loan history.
idx_loan_status	LOAN_REQUEST(status, oracle_state)	Efficient filtering of application lifecycle and oracle pipeline states.
idx_installment_due	INSTALLMENT(due_date, status) WHERE status = 'PENDING'	Partial-index range query for pending overdue installment detection.
idx_txn_created	TRANSACTION(created_at)	Time-window aggregation for 6-month and 1-year borrowing limits.
idx_txn_borrower	TRANSACTION(borrower_id)	Per-borrower transaction history retrieval.
idx_market_symbol	MARKET_DATA(symbol, recorded_at)	Composite index for price feed time-series queries.
idx_loan_bank_status	LOAN(local_bank_id, status, created_at DESC)	Loan lifecycle queries by bank and status.
idx_loan_borrower_active	LOAN(borrower_id, status) WHERE status = 'ACTIVE'	Partial index for per-borrower open-loan count.
idx_agent_session_date	AGENT_ACTION_LOG(session_id, created_at DESC)	Per-session agent action history retrieval (join to SESSIONS for borrower).
idx_agent_status	AGENT_ACTION_LOG(status) WHERE status = 'PENDING'	Partial index for the agent status-monitoring loop.
idx_risk_score	LOAN_RISK_ASSESSMENT(anomaly_score DESC) WHERE anomaly_score > 0.5	Partial index for AML alert queue (SAR workflow).
idx_ib_loan_status	INTERBANK_LOAN(status, lender_institution_id)	IBLP default cascade and maturity monitoring.
idx_institution_type	INSTITUTION(institution_type, status)	Tier-scoped institution lookups for multi-entity operations.
idx_txn_institution	TRANSACTION(origin_institution_id, created_at DESC)	Institution-to-institution ledger history retrieval.
idx_chainlink_feed	CHAINLINK_PRICE_ROUND(feed_address, updated_at DESC)	Latest round lookup per Chainlink feed for LTV staleness checks.
idx_session_parent	SESSIONS(parent_session_id) WHERE parent_session_id IS NOT NULL	Partial index for session lineage chain traversal.
idx_collateral_borrower	COLLATERAL_POSITION(borrower_id, loan_id)	Health-factor lookup per borrower and per loan.
idx_event_log_txhash	BLOCKCHAIN_EVENT_LOG(tx_hash)	Deduplication guard: event listener checks this before INSERT.
idx_event_log_block	BLOCKCHAIN_EVENT_LOG(block_number, chain_id)	Catchup-sync replay from a given block.

A.2 Functional Dependencies

Table A.3: Representative functional dependencies.

Relation	Functional Dependency	Notes
LOAN_REQUEST	`request_id` → all attributes	Application row keyed by request identifier.
LOAN_REQUEST	`borrower_id, local_bank_id` → `status, oracle_state, amount, ...`	Lifecycle `status` and oracle pipeline `oracle_state` are separate columns.
LOAN	`loan_id` → all attributes	Disbursement record keyed by loan identifier.
LOAN	`loan_request_id` → `loan_id`	1:1 link from approved request to disbursed loan.
BORROWING_LIMIT	`borrower_id` → `six_month_limit, one_year_limit, on_chain_enforced_limit, last_synced_block, ...`	1:1 with BORROWER; event-listener owned projection.
CREDIT_PASSPORT	`passport_id` → all attributes; `borrower_id` → `passport_id`	Read-model projection of on-chain SBT; event-listener owned.
BORROWER	`wallet_address` → `borrower_id`, all attributes; `registered_local_bank_id` → home Local Bank	Unique candidate key in the prototype; Phase IV uses (`wallet_address, chain_id`).
INSTITUTION	`institution_id` → `institution_type, name, wallet_address, contract_address, status`	Supertype for cross-tier FK anchors.
NATIONAL_BANK	`country_code` → `institution_id`	One national reserve per nation (`UNIQUE`).
MODEL_REGISTRY	`model_id` → `model_name, version, training_date, metrics, is_active`	Model lineage for explainability audit.
LOAN_RISK_ASSESSMENT	`assessment_id` → all attributes; `model_id` → `fraud_probability, anomaly_score, shap_vector`	Per-loan ML inference; FK to `MODEL_REGISTRY`.
INSTALLMENT	`loan_id, installment_number` → `amount, due_date, status, payment_event_log_id`	Composite primary key.
SESSIONS	`session_id` → `borrower_id, wallet_address, session_key_hash, session_key_scope, session_key_expires_at, expires_at, revoked`	Retail-client agent sessions only at MVT; bank-authority sessions are Future Work.
AGENT_ACTION_LOG	`action_id` → `session_id,`	Links every write-tool

A.3 Materialized Views and Agent Query Optimisation

The MCP tool `get_account_state` joins `BORROWER`, `CREDIT_PASSPORT`, `COLLATERAL_POSITION`, active `LOAN` rows, pending `INSTALLMENT` rows, `SAVINGS_ACCOUNT`, and `BORROWING_LIMIT`. To avoid six-to-seven table joins on every agent call, the schema defines a materialized view refreshed by the event listener after any event that modifies component tables:

```
CREATE MATERIALIZED VIEW mv_client_dashboard AS
SELECT
    b.borrower_id,
    b.wallet_address,
    b.kyc_level,
    cp.credit_score,
    cp.risk_tier,
    bl.six_month_remaining,
    bl.one_year_remaining,
    count(l.loan_id) FILTER (WHERE l.status = 'ACTIVE') AS active_loan_count,
    max(i.due_date) FILTER (WHERE i.status = 'PENDING') AS next_installment_due,
    sum(sa.balance) AS total_savings_balance,
    sum(col.locked_amount) AS total_collateral_locked
FROM borrower b
LEFT JOIN credit_passport cp USING (borrower_id)
LEFT JOIN borrowing_limit bl USING (borrower_id)
LEFT JOIN loan l USING (borrower_id)
LEFT JOIN installment i USING (loan_id)
LEFT JOIN savings_account sa USING (borrower_id)
LEFT JOIN collateral_position col USING (borrower_id)
GROUP BY b.borrower_id, b.wallet_address, b.kyc_level,
         cp.credit_score, cp.risk_tier,
         bl.six_month_remaining, bl.one_year_remaining;

CREATE UNIQUE INDEX ON mv_client_dashboard (borrower_id);
```

Refresh strategy: `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_client_dashboard` triggered by the event listener after processing events on projection or lending tables (Phase III).

Appendix B

Agent Harness Reference

This appendix records the conversational agent harness (Phase III–IV Must/Should deliverables). Section 4.4 in Chapter 4 summarises the six-step pipeline, MCP tool schema, and Authority Brief example. The choice of a locally-hosted 8B-parameter model over a cloud-hosted API (Section C.1) is consistent with the broader 2026 industry shift toward on-device inference for compliance-sensitive financial applications, in which major hardware and platform vendors have positioned on-premise deployment as the only viable path for sectors handling sensitive financial records [R56].

B.1 Per-User Context and Prompt Assembly

Per-user personalisation — shared model, per-user context. Every client receives a personalised experience without deploying a separate model per user. Personalisation lives entirely in the per-user context namespace injected into every system prompt by the Express.js context injection layer:

```
{
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "conversation_history": [ "...last 10 turns..." ]
}
```

Because the on-chain state is re-injected at every turn, the agent always operates on current account data and cannot hallucinate the client's balance or loan status.

Three-tier system prompt assembly. The agent's system prompt is assembled by a dedicated prompt constructor in the Express.js backend, organised into three explicit tiers rather than a single ad-hoc string [R52]:

Stable tier. Agent persona, active toolset schema, compliance anchors (EU AI Act Art. 86, FATF R.16, zero-bypass rule). Prefix-cache target; unchanged within a session.
Context tier. Interest rate schedule (Table 3.26), KYC matrix, pool policy, active skill procedure from `AGENT_SKILLS`. Updates on governance votes only.
Volatile tier. Per-user on-chain context namespace, parent-session compression summary, last N conversation turns. Rebuilt every turn from `eth_call` state.

B.2 Session Lineage, Compression, and Toolset Scoping

Session lineage and cross-session memory. The `SESSIONS` table carries a `parent_session_id` self-referential FK. When a conversation reaches a token threshold, the compression path closes the session with `end_reason = 'compression'` and opens a child session seeded with the compression summary. The read tool `get_session_history` exposes ranked excerpts from the client's conversation ancestry.

Context compression strategy. When the assembled prompt approaches the 32K-token window, the backend retains head ($H{=}500$) and tail ($T{=}1{,}000$) verbatim, summarises the middle via a secondary Qwen3-8B call, stores `SESSIONS.compression_summary`, and opens a child session. Trigger: 28,000 tokens (87.5% of window).

Toolset	Tools visible
`read_only`	All 9 read tools (default when intent is uncertain).
`loan_actions`	`read_only` + loan, installment, and group write tools.
`account_management`	`read_only` + deposit, KYC, and reminder write tools.

B.3 Lifecycle Hooks and Optional Capabilities

Lifecycle hook middleware. Write-tool POST requests pass through a hook stack: (1) confirmation audit hook (Phase II) verifies a logged confirmation turn or returns HTTP 403; (2) session-key scope pre-check (Phase III); (3) AML pre-check against `LOAN_RISK_ASSESSMENT` and `SECURITY_EVENT_LOG` (Phase IV).

Agent capabilities.

- **Proactive installment reminders** via `get_installment_schedule` and `pay_installment`.
- **Credit Passport coaching** via `get_credit_score` and Table 3.26.
- **Loan calculator** with live FX from `get_market_data`.
- **Group lending coordination** and **KYC tier upgrade guidance** via write/read tools.
- **Cross-session context retrieval** via `get_session_history`.

Appendix C

Technology Stack

C.1 In-product assistant: local large language model (LLM) integration (prototype)

Purpose. The in-product assistant is a **planned Phase III Must deliverable** (Section 4.4), providing a plain-language interface alongside the React dashboard. It combines a locally hosted, privacy-preserving large language model (Qwen3-8B, Apache 2.0 licence) with a Model Context Protocol (MCP) tool server exposing 17 tools (9 read tools, including a session history search tool, and 8 write tools requiring human confirmation). The agent can both answer questions—via retrieval-augmented generation (RAG) from policy documents—and execute on-chain banking operations via write tools, subject to an explicit human confirmation gate before any state-modifying action is taken. Local hosting keeps conversation data on institutional infrastructure, supporting data-sovereignty requirements in regulated pilots. A client who prefers the React UI experiences no reduction in available functionality, since every operation the agent can perform is equally available through the standard dashboard.

Model and runtime (local development). The inference endpoint follows the **OpenAI Chat Completions** contract (`/v1/chat/completions`) with `stream: true`, proxied from the project backend to a local service such as `http://127.0.0.1:1234`. During early development, the assistant used a locally hosted **Gemma-4-E4B** checkpoint (Google mixture-of-experts model, ∼26B total / 4B active parameters) via LM Studio in GGUF form (e.g., `Q4_K_M` quantization) for a laptop-friendly footprint. For the final thesis evaluation, the assistant is replaced by **Qwen3-8B** (Alibaba Cloud Qwen team, released April 28 2025, Apache 2.0 licence) [R46], a dense 8.19B-parameter causal language model with a 32K-token native context window (extendable to 131K tokens via YaRN scaling) and support for 100+ languages and dialects. A distinguishing architectural feature of Qwen3-8B is its *switchable reasoning mode*: the model operates in **thinking mode** (chain-of-thought wrapped in `<think>` tokens, suited to multi-step compliance or policy questions) and **non-thinking mode** (direct, low-latency responses for general navigation queries), selectable per-turn with `/think` or `/no_think` in the system prompt. For Q&A and retrieval-augmented generation (RAG) queries, non-thinking mode is the default; thinking mode is activated only for the red-team regulatory-compliance prompts in the evaluation protocol of Section 4.10.1. For agent tool-calling, non-thinking mode is the default for read tools and confirmation summaries; thinking mode is activated for complex multi-step operations such as EMI calculation or group signature coordination. The per-user context namespace (see Section 4.3) is prepended to every system prompt as a structured JSON block, so the model has full account context before reading the user's message. The API model identifier is environment-driven; the integration layer remains identical for both models.

End-to-end behavior. The user interface assembles a short transcript (user/assistant messages) and posts it to the backend streaming route. Before the user message reaches the model, the context assembly layer: (a) selects the active toolset based on intent classification, (b) applies prompt injection scanning to all user-sourced fields, and (c) assembles the three-tier system prompt (Stable + Context + Volatile), injecting the compression summary from the parent session into the Volatile tier if this is a continuation session. The active toolset name and any injection scan flag are recorded in `AGENT_ACTION_LOG` for every turn. The backend then opens a streaming request to the local model server, converts upstream token events into **server-sent events (SSE)** for the browser, and the UI incrementally appends text. The message renderer uses Markdown (including GitHub-flavored extensions), math (KaTeX) for `$$...$$` blocks, and line-break rules appropriate for chat transcripts.

For write-tool requests, the agent assembles the full parameter set from the user's request and the injected on-chain state, then presents a confirmation summary. Only after the user sends explicit affirmative consent does the agent call the corresponding MCP write tool via the Express.js banking API layer. The EIP-7702 session key (if active) signs the resulting on-chain transaction within its approved scope. Every executed write tool is logged to `agent_action_log` with the confirmation message ID as the audit reference.

Live on-chain context injection. The Express.js backend injects the client's full on-chain state as a structured JSON context block into every system prompt before forwarding the user's message to the model. The per-user context namespace is:

```
{
  "tier": "volatile",
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "session_id": "sess_20260602_xyz",
  "parent_session_id": "sess_20260515_abc",
  "compression_summary": "Client asked about Gold tier eligibility.
  Was told 2 more on-time repayments required. No action taken.",
  "conversation_history": [ "...last 10 turns..." ]
}
```

The `compression_summary` field is injected only when this is a child session (i.e., when `parent_session_id` is non-null). It provides the prior-session context without exceeding the Volatile tier token budget. This context block is injected only when the user is authenticated and the `eth_call` round-trip completes within a 200 ms timeout (falling back to a static-context response otherwise). The model uses the injected context to ground its answers in the user's actual account data rather than generic policy text, materially reducing hallucination risk for account-specific queries.

Security and limitations (prototype stance). The agent's safety posture is enforced at four levels. (1) **Tool-schema boundary:** the agent interacts with CWB exclusively through the 17 MCP tools — it cannot execute arbitrary code, make unapproved API calls, or read data outside the tool schema. (2) **Human confirmation gate:** every write tool requires explicit affirmative consent in the conversation history; the confirmation turn ID is logged as the audit reference. (3) **Session key scope:** the EIP-7702 session key is scoped to specific tools, value-capped, time-bound to 24 hours, and revocable at any time. A session that attempts to call an out-of-scope tool is rejected at the key level, not just at the application level. (4) **Prompt injection scanning and lifecycle hook middleware:** user-sourced content is scanned for injection patterns before entering the prompt (Layer 2 control, Section 4.3), and every write-tool HTTP POST is intercepted by a middleware stack that enforces the confirmation invariant at the application layer independently of the model's output. Together, these four levels form a defence-in-depth posture in which no single point of failure can circumvent a critical safety constraint. Hallucination risk is bounded by: (a) the injected on-chain state grounding account-specific answers, and (b) the refusal layer for regulatory and legal queries (100% target on the red-team set, Section 4.10.1).

Architecture diagram. Figure C.1 shows the compact end-to-end request path; Figure C.2 expands the same flow with component boundaries and authentication context.

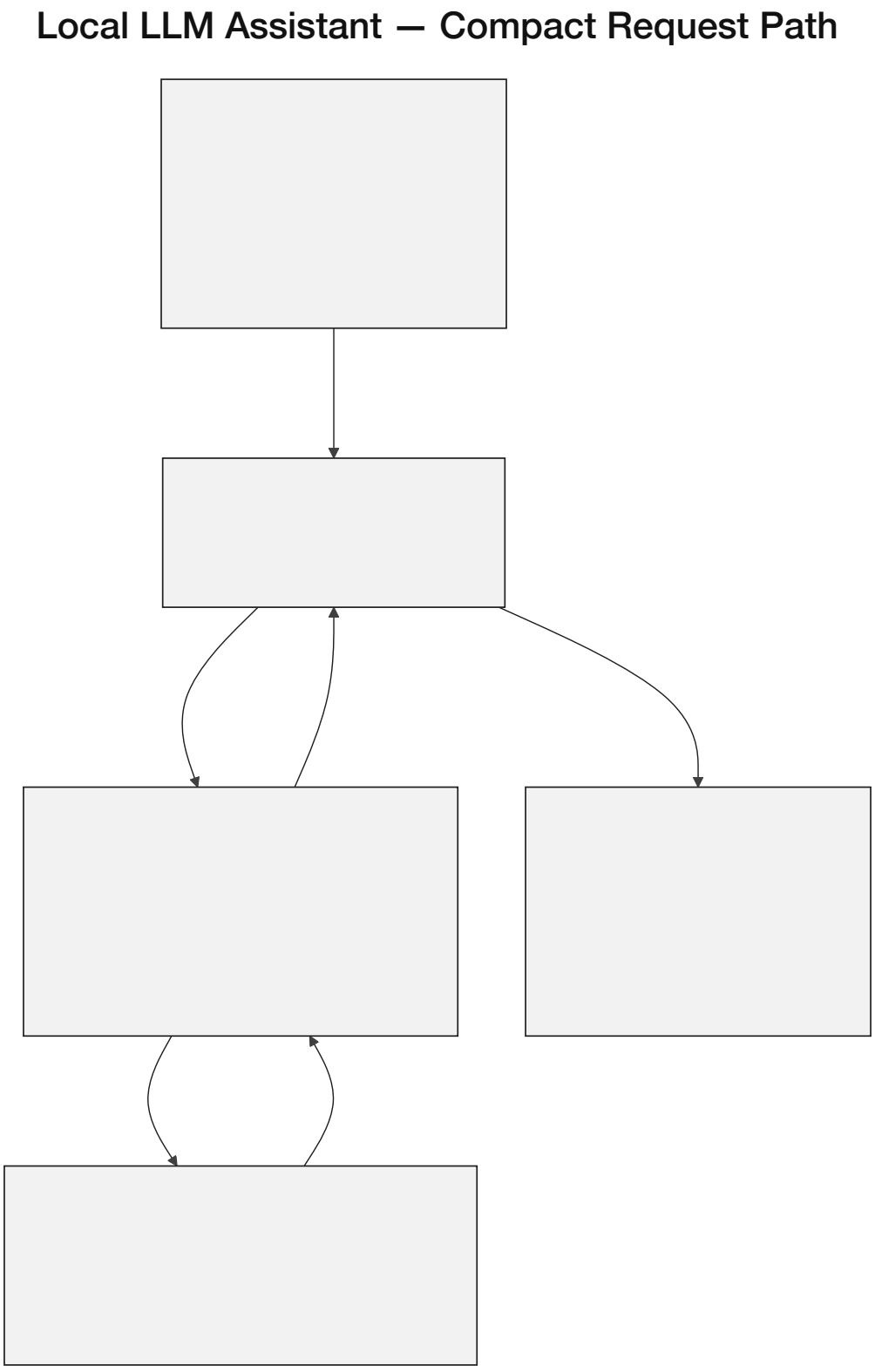


Figure C.1: AI banking agent request path

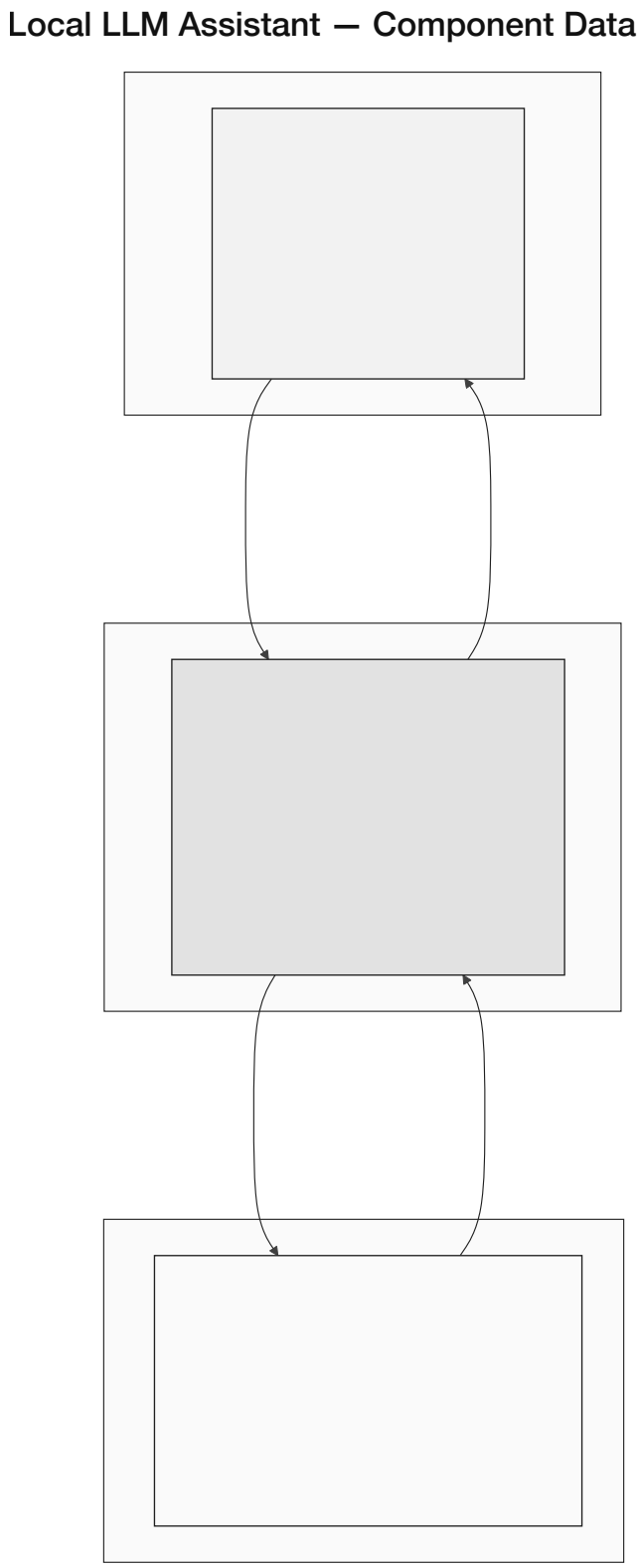


Figure C.2: AI agent data flow

Implementation analysis (brief). The integration is successful for the project prototype because it reuses a stable transport shape (chat-completions with streaming) and isolates the model vendor behind a single API route, allowing the user interface to remain a thin client. The main practical engineering challenges in local development were: (1)

process/port hygiene to keep the API bound to a predictable port, (2) **CORS and same-origin** considerations when the UI and API run on different localhost ports, mitigated with a Vite dev proxy to `/api` and with permissive dev CORS policy on the API, and (3) **RPC provider selection** for wallet reads in the browser, where some public endpoints may fail browser CORS; pinning explicit public RPC endpoints avoids noisy failures unrelated to the chat feature.

Relationship to the earlier rule-based assistant (legacy). The repository also retains early keyword-routed `/api/chatbot` handlers used for initial prototyping; the LLM path is additive. This separation prevents regressions in deterministic demo endpoints while the conversational assistant evolves.

Table C.1: Technology stack summary.

Layer	Technology	Version / Notes
Smart Contract	Solidity, OpenZeppelin	0.8.20; Ownable, ReentrancyGuard
Frontend	React, TypeScript	Material UI; Design 3
Wallet Integration	Wagmi, RainbowKit, Viem	EIP-1193 compliant
Build and Test	Hardhat	Automated test suite; deployment scripts
Backend API	Express.js, Node.js	REST API; middleware architecture
Database	PostgreSQL (designed)	51 entities (31 core + 6 extension + 14 stubs), 3NF; relational integrity
Target Network	Polygon Amoy PoS and/or Ethereum Sepolia (single-chain prototype)	Stable public testnets for delivery; Polygon zkEVM retained as evaluated design preference
AI Agent Engine	Qwen3-8B (llama.cpp, Q4_K_M)	MCP tool server; per-user context namespace; human confirmation gate
MCP Tool Server	Minimal MVT tool subset (3–5 tools); prompt injection scanner; confirmation audit hook middleware	Exposes a small read+write surface to the agent with typed parameter schemas
Session Keys	EIP-7702 (Future Work)	Scoped, time-bound, value-capped agent wallet authorisation
Oracle Network	Commit–reveal (MVT) + Chainlink Functions (stretch)	Commit–reveal enforces auditable score commitment; DON upgrade when supported
Price Feeds	Chainlink AggregatorV3Interface	Fiat/USD pairs (e.g. EUR/USD) and ETH/USD; 8-decimal precision
Automation	Chainlink Automation	Trustless installment due-date checking; replaces centralised cron job
Proof of Reserve	Chainlink PoR	Cryptographically verifiable WorldBankReserve balance
Event Indexing	PostgreSQL event listener	Stability-first indexing baseline (GraphQL over The Graph optional later)

This appendix table summarizes assistant integration components and configuration aspects used in the prototype (UI, API proxying, and local model server). The purpose is to make the implementation reproducible and to clarify which parts are prototype conveniences versus production requirements.

Appendix D

Smart Contract Capabilities

The complete banking architecture is designed around fifteen modular contracts. The three-contract lending core is specified at pre-thesis stage; the remaining twelve modules are planned across Phases II and III of the final thesis phase.

Specified core contracts (Phase I target):

- **World Bank Reserve Contract:** Reserve custody, deposit handling, national bank registration, capital allocation to national banks, system pause and unpause, emergency withdrawal, and system statistics.
- **National Bank Contract:** Local bank registration, borrowing from the World Bank reserve, capital allocation to local banks, and network status queries.
- **Local Bank Contract:** Client registration, loan application processing, approval and rejection workflows, installment generation and payment processing, approver management, user account management, and a `freezeAccount(clientId)` function gated by `onlyApprover` modifier, used by the SAR workflow (Section 3.18.5) to suspend loan and payment operations for a flagged client pending compliance review.

Planned contracts – Banking product suite (Phase II):

- **SavingsVault Contract:** Standard savings account management, variable yield accrual, withdrawal processing, and deposit-to-lending pool integration.
- **FixedDeposit Contract:** Term deposit creation, lock period enforcement, agreed APY accrual, early withdrawal penalty calculation, and maturity release.
- **GroupLendingPool Contract:** Group formation, multi-signature consent recording, shared collateral management, per-member disbursement, mutual liability enforcement, and group credit history recording.
- **FXModule Contract:** Oracle-priced currency conversion, spread calculation, dual-currency lending denomination support, and conversion audit trail.
- **InsuranceFund Contract:** Captures 5% of all interest collected across the lending hierarchy, processes default coverage disbursement claims, and publishes its reserve balance to Chainlink Proof of Reserve for external verifiability.
- **CurrentAccount Contract:** Transactional account management, atomic peer transfers, recurring payment scheduling, and payroll deposit handling.

Planned contracts – Multi-entity and cross-tier operations (Phases II and III):

- **InterBankLendingPool Contract** (Section 3.14.1): one instance per tier (`IBLP_NB`, `IBLP_LB`). Short-tenor (1 / 7 / 30 day) same-tier borrowing with a utilization-kinked rate model bounded by the tier-above downward rate; default cascade through reserve buffer → InsuranceFund → parent-tier backstop.
- **UpwardDepositFacility Contract** (Section 3.14.2): role-restricted extension of SavingsVault accounting that lets a lower-tier bank deposit excess reserves into its par-

ent tier, earning a strictly lower yield than the downward borrowing rate; withdrawal gate enforces depositor reserve-ratio safety.

- **SyndicatedLoan Contract** (Section 3.14.3): syndicate proposal, subscription window, supermajority on-chain consent vote (75%-by-capital-share), atomic disbursement, pro-rata interest and recovery distribution; supports cross-tier syndication (mixed NB / LB co-funders).
- **TranchedPool Contract** (Section 3.14.4): senior / junior tranching with a subordination-ratio policy, waterfall interest and principal payments, first-loss absorption by the junior tranche; suitable for risk-segmented institutional and impact-aligned capital.
- **TreasurySwap Contract** (Section 3.14.5): oracle-priced atomic asset swap restricted to wallets holding a banking role; cross-tier asset exchange (e.g., NB ETH treasury $\rightarrow$ USDC) with a tighter spread than retail FX; post-swap reserve-ratio invariant enforced before state change.
- **NettingEngine Contract** (Future Work, Section 6): multilateral netting with partial settlement and challenge-period dispute resolution—not specified at implementation depth in this report.

Appendix C: Planned Testnet Deployment Manifest

The following table is the **consolidated planned deployment manifest template** for the prototype public testnet deployment (Polygon Amoy PoS and/or Ethereum Sepolia). Polygon zkEVM remains an evaluated option in the platform comparison section; it is not the default narrative deployment chain for gates, MVT items, or phase deliverables. No contracts are claimed as live-deployed at pre-thesis submission. Addresses will be recorded in the project repository under `/deployments/testnet/` as phases complete and can be verified on the relevant block explorers.

Table D.1: Planned testnet deployment manifest (consolidated Phase IV template; incremental Phase I–III deploys on a single chosen prototype testnet).

Contract	Network	Address / manifest path
WorldBankReserve	Prototype testnet	`deployments/testnet/WorldBankReserve.json`
NationalBank (Scaffold)	Prototype testnet	`deployments/testnet/NationalBank.json`
LocalBank (Scaffold)	Prototype testnet	`deployments/testnet/LocalBank.json`
LendingController	Prototype testnet	`deployments/testnet/LendingController.json`

Note: Exact hex addresses will be populated after Phase IV testnet deployment. The repository manifest is the authoritative record. This appendix documents the intended contract inventory only; it does not assert live deployment at pre-thesis stage.

Appendix D: WorldBankReserve Contract Interface

The following Solidity interface documents the public API of the `WorldBankReserve` contract—the formally specified Tier 1 contract (Phase I deployment target). Three design annotations follow the listing:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/// @title WorldBankReserve
/// @notice Tier 1 reserve contract: manages global capital allocation
///         to registered National Bank addresses.
contract WorldBankReserve is ReentrancyGuard, AccessControl {

    bytes32 public constant NATIONAL_BANK_ROLE = keccak256("NATIONAL_BANK_ROLE");
    bytes32 public constant AUDITOR_ROLE       = keccak256("AUDITOR_ROLE");

    uint256 public totalReserve;
    uint256 public minimumReserveRatio;   // e.g. 1000 = 10.00%
    mapping(address => uint256) public allocatedTo; // NationalBank => amount

    event CapitalAllocated(address indexed nationalBank, uint256 amount);
    event RepaymentReceived(address indexed nationalBank, uint256 amount);
    event ReserveRatioUpdated(uint256 newRatio);

    /// @notice Allocate capital downward to a registered National Bank (CEI
        pattern)
    function allocateCapital(address nationalBank, uint256 amount)
        external onlyRole(DEFAULT_ADMIN_ROLE) nonReentrant;

    /// @notice Record repayment from a National Bank (CEI pattern)
    function recordRepayment(uint256 amount)
        external onlyRole(NATIONAL_BANK_ROLE) nonReentrant;

    /// @notice Query current utilization ratio (scaled 1e4)
    function utilizationRate() external view returns (uint256);

    /// @notice Returns true if reserve ratio constraint is satisfied
    function isReserveAdequate() external view returns (bool);

    /// @notice Returns a four-value reserve summary for Proof of Reserve
    ///         verification and the Reserve Transparency Dashboard.
    /// @return totalDeposited  Gross deposits ever made to this reserve tier
    /// @return totalLoaned     Outstanding principal lent to National Banks
    /// @return reserveRatio    (totalDeposited - totalLoaned) * 1e4 /
        totalDeposited
    ///                         (e.g. 5000 = 50.00%)
    /// @return insuranceFundBalance  Current balance of the InsuranceFund contract
    function getReserveSummary() external view returns (
        uint256 totalDeposited,
        uint256 totalLoaned,
        uint256 reserveRatio,
        uint256 insuranceFundBalance
    );
}
```

Listing D.1: WorldBankReserve.sol – Tier 1 public interface. Full implementation in project repository.

Design annotation 1 — `nonReentrant` on state-changing functions. The

`nonReentrant` modifier from OpenZeppelin's `ReentrancyGuard` sets a boolean lock before execution and clears it after, preventing recursive calls. It is applied to `allocateCapital` and `recordRepayment` because both involve ETH transfers that could trigger attacker-controlled fallback functions. The full Checks-Effects-Interactions analysis is in Section 3.9.

Design annotation 2 — `AccessControl` mapping to RBAC design. OpenZeppelin's `AccessControl` maps each on-chain role (e.g., `NATIONAL_BANK_ROLE`) to a `bytes32` keccak256 hash stored in a role-to-address mapping. Role assignment is controlled by the `DEFAULT_ADMIN_ROLE` (the World Bank admin). This directly implements the four-tier RBAC hierarchy described in Section 3.1: only addresses granted `NATIONAL_BANK_ROLE` can call `recordRepayment`, and only the World Bank admin can call `allocateCapital`.

Design annotation 3 — Events enable off-chain indexing for the analytics layer. `CapitalAllocated`, `RepaymentReceived`, and `ReserveRatioUpdated` are emitted at every significant state transition. The off-chain Express.js event listener subscribes to these events and writes them to PostgreSQL, enabling the AI/ML monitoring layer to construct loan lifecycle timelines, compute utilization windows, and feed the Random Forest fraud detector without requiring costly repeated `SLOAD` calls.

Appendix E: Internal Industry Research Notes

This appendix catalogues supplementary industry-research documents prepared for the Crypto World Bank project. They informed Chapter 3 (actor taxonomy), Chapter 5 (CEX comparison and revenue analogues), and the no-native-token design constraint. All files reside in the project repository under `Documentation/2nd Phase (Bokhtiar)/Improvements/`.

Table D.2: Internal industry research corpus.

ID	Document	Thesis use
IR-1	`Binance_FTX_Full_Report.md`	CEX vs. CWB (Table 5.24); FTT/BNB token lesson; SAFU and Binance Earn analogues; PoR and commingling safeguards
IR-2	`blockchain platforms research.csv`	Five-exchange matrix (Binance, Bybit, Coinbase, OKX, Kraken) for Table 5.24
IR-3	`Binance Business Analysis.md`	Exchange revenue taxonomy for Table 5.25 (fees, listing, OTC)
IR-4	`binance_software_engineering_architecture.md`	Nine-actor taxonomy (Section 3.8); use-case and sequence-diagram conventions
IR-5	`binance_architecture_research.md`	Off-chain PostgreSQL/Redis patterns (background; not cited in body text)

These documents are project-internal research notes, not peer-reviewed publications. They are listed here for reproducibility and examiner traceability; formal citations in the body text use bibliography entries [128]–[130] for the three documents most directly referenced.

References

[1] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt. 2022. SoK: Decentralized Finance (DeFi). *arXiv preprint arXiv:2101.08778*. https://arxiv.org/abs/2101.08778

[2] M. Bastankhah, V. Nadkarni, C. Jin, S. Kulkarni, and P. Viswanath. 2024. Thinking Fast and Slow: Data-Driven Adaptive DeFi Borrow-Lending Protocol. In *Proc. 6th Conf. Advances in Financial Technologies (AFT)* (LIPIcs, Vol. 316). Article 27, 23 pages. https://doi.org/10.4230/LIPIcs.AFT.2024.27

[3] G. Palaiokrassas, S. Scherrers, I. Ofeidis, and L. Tassiulas. 2024. Leveraging Machine Learning for Multichain DeFi Fraud Detection. In *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*. https://doi.org/10.1109/ICBC59979.2024.10634350

[4] D. Adom, E. O. Acheampong, and M. Boateng. 2022. LIME and SHAP: A Comparison of Model-Agnostic Approaches to Explainability in Loan Approval Systems. *International Journal of Advanced Computer Science and Applications* 13, 11. https://thesai.org/Downloads/Volume13No11/Paper_90-LIME_and_SHAP_A_Comparison.pdf

[5] B. W. Tan. 2023. Central Bank Digital Currency and Financial Inclusion. *IMF Working Paper WP/23/69*. https://www.imf.org/en/Publications/WP/Issues/2023/03/27/Central-Bank-Digital-Currency-and-Financial-Inclusion-531273

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou. 2008. Isolation Forest. In *Proc. IEEE Int. Conf. Data Mining (ICDM)*, pages 413–422. https://doi.org/10.1109/ICDM.2008.17

[7] N. Atzei, M. Bartoletti, and T. Cimoli. 2017. A Survey of Attacks on Ethereum Smart Contracts (SoK). In *Proc. Int. Conf. Principles of Security and Trust (POST)*, pages 164–186. https://doi.org/10.1007/978-3-662-54455-6_8

[8] S. M. Lundberg and S.-I. Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 4765–4774. https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[9] P. Bracke, A. Datta, C. Jung, and S. Sen. 2019. Machine Learning Explainability in Finance: An Application to Default Risk Analysis. *Bank of England Staff Working Paper No. 816*. https://www.bankofengland.co.uk/working-paper/2019/machine-learning-explainability-in-finance-an-application-to-default-risk-analysis

[10] R. Beck, C. Müller-Bloch, and J. L. King. 2018. Governance in the Blockchain Economy: A Framework and Research Agenda. *Journal of the Association for Information Systems*, 19, 10, pages 1020–1034. https://doi.org/10.17705/1jais.00518

[11] D. Mhlanga. 2022. Blockchain Technology for Financial Inclusion and Sustainable Development. In *Digital Financial Inclusion*. Springer. https://doi.org/10.1007/978-3-031-05725-7

[12] OpenZeppelin. 2024. OpenZeppelin Contracts. https://openzeppelin.com/contracts

[13] DefiLlama. 2024. Lending Protocols — DeFi TVL and Protocol Rankings. https://defillama.com/protocols/Lending

[14] World Bank. 2021. The Global Findex Database 2021: Financial Inclusion, Digital Payments, and Resilience. https://www.worldbank.org/en/publication/globalfindex

[15] Aave. 2024. Aave Protocol Documentation. https://docs.aave.com/

[16] Compound. 2024. Compound Protocol Documentation. https://docs.compound.finance/

[17] BCOLBD. 2025. Blockchain Olympiad Bangladesh: Guideline and Evaluation Scheme. https://bcolbd.org/rules

[18] Galaxy Digital. 2024. The State of Crypto Lending and Borrowing. Galaxy Research. https://www.galaxy.com/insights/research/the-state-of-crypto-lending

[19] World Bank. 2022. World Development Report 2022: Finance for an Equitable Recovery. https://www.worldbank.org/en/publication/wdr2022

[20] International Finance Corporation (IFC). 2017. MSME Finance Gap: Assessment of the Shortfalls and Opportunities in Financing Micro, Small, and Medium Enterprises in Emerging Markets. World Bank. https://openknowledge.worldbank.org/entities/publication/ff4c9839-21ac-5676-a23a-7cf6f745df0c

[21] Bank for International Settlements. 2024. DeFi Lending: Intermediation Without Information?. BIS Working Paper No. 1183. https://www.bis.org/publ/work1183.pdf

[22] Deloitte. 2024. Global Banking Outlook 2024. Deloitte Insights. Retrieved June 9, 2026 from https://www.deloitte.com/global/en/Industries/financial-services/research-global-banking-outlook.html

[23] Michigan State University Libraries. 2025. Literature Table and Synthesis for Systematic Reviews. MSU LibGuides. Retrieved June 9, 2026 from https://libguides.lib.msu.edu/nursinglitreview/table

[24] Committee on Payments and Market Infrastructures and Bank for International Settlements. 2016. Correspondent Banking — Technical Report. CPMI Papers No. 147. https://www.bis.org/cpmi/publ/d147.pdf

[25] Ripple Labs. 2025. RLUSD Stablecoin Product Overview. Ripple Documentation. https://ripple.com/solutions/stablecoin/

[26] World Bank. 2024. Remittance Prices Worldwide: Making Markets More Transparent. Migration and Development Brief 40. https://remittanceprices.worldbank.org/

[27] DefiLlama. 2026. Aave V3 TVL, Fees, Revenue & Income Statement. March. https://defillama.com/protocol/aave-v3

[28] World Inequality Lab. 2026. Exorbitant Privilege. *World Inequality Report*. https://wir2026.wid.world/insight/exorbitant-privilege/

[29] DefiLlama. 2026. Compound V3 TVL, Fees, Revenue & Income Statement. March. https://defillama.com/protocol/compound-v3

[30] Fensory. 2026. Sky Protocol Projects $611M Revenue in 2026 as USDS Supply Targets $20.6 Billion. February. https://www.fensory.com/intelligence/rwa/sky-protocol-tokenization-regatta-solana-february-2026

[31] Fensory. 2026. Maple Finance — Institutional DeFi Lending Protocol. https://www.fensory.com/insights/protocols/maple-finance

[32] Fensory. 2026. DeFi Credit Platforms Hit $2.4B Milestone. March. https://www.fensory.com/intelligence/rwa/defi-private-credit-platforms-2-4-billion-loans-ethereum

[33] Financial Stability Board. 2020. Enhancing Cross-border Payments: Stage 3 Roadmap. https://www.fsb.org/2020/10/enhancing-cross-border-payments-stage-3-roadmap/

[34] A. Beyer, B. Gasperini, and P. Theodoridis. 2025. Monetary Policy and Inequality: Distributional Effects of Asset Purchase Programs. *Journal of International Money and Finance* 157. https://doi.org/10.1016/j.jimonfin.2024.103123

[35] Federal Reserve Board. 2025. Monetary Policy and the Distribution of Income: Evidence from U.S. Metropolitan Areas. FEDS Notes, March. https://www.federalreserve.gov/econres/notes/feds-notes/monetary-policy-and-the-distribution-of-income-evidence-from-us-metropolitan-areas-20250331.html

[36] R. Cantillon. 1755. *Essai sur la Nature du Commerce en Général* (Essay on the Nature of Commerce in General). Posthumous edition.

[37] GlobeNewswire. 2025. R3's Corda Leads Tokenized RWA Market with Over $10 Billion in On-chain Assets. February. https://www.globenewswire.com/news-release/2025/02/13/3025637/0/en/R3-s-Corda-leads-tokenized-RWA-market-with-over-10-billion-in-on-chain-assets-and-unrivalled-industry-adoption.html

[38] Centrifuge. 2025. Real-World Asset Tokenization: Key Trends from 2025. https://centrifuge.io/blog/real-world-asset-tokenization-trends-2025

[39] World Bank. 2025. World Bank Group Tracks Project Funds with New Blockchain Tool. Press Release, September. https://www.worldbank.org/en/news/press-release/2025/09/26/world-bank-group-tracks-project-funds-with-new-blockchain-tool

[40] JPMorgan. 2025. Kinexys Digital Payments: Real-Time Multicurrency Payments. https://www.jpmorgan.com/onyx/coin-system.htm

[41] World Economic Forum. 2022. Why Decentralized Finance Is a Leapfrog Technology for the 1.1 Billion People Who Are Unbanked. September. https://www.weforum.org/stories/2022/09/decentralized-finance-a-leapfrog-technology-for-the-unbanked/

[42] Opera Newsroom. 2026. 160M CELO Allocation Proposal to Grow Opera from Distribution Partner into Key Network Stakeholder. March. https://press.opera.com/2026/03/19/opera-celo-partnership-2026/

[43] Morpho. 2026. Network Data. March. https://data.morpho.org/

[44] Stellar. 2025. End of Year 2025 Report — Execution at Scale. https://www.stellar.org/blog/foundation-news/2025-year-in-review

[45] Human Rights Foundation. 2025. Tracking CBDCs Before They Track You. https://hrf.org/latest/tracking-cbdcs-before-they-track-you

[46] Y. Chen, S. Bin, and H. He. 2024. Design and Implementation of a Multi-Chain Lending Model in Blockchain. In *Proc. 3rd Int. Conf. Artificial Intelligence and Computer Information Technology (AICIT)*, pages 97–102.

[47] S. Yang and W. Cui. 2023. An Evaluation System for DeFi Lending Protocols. *arXiv preprint arXiv:2303.01022*. https://arxiv.org/abs/2303.01022

[48] J. Hartmann and O. Hasan. 2021. A Social-Capital Based Approach to Blockchain-Enabled Peer-to-Peer Lending. In *Proc. IEEE Int. Conf. Blockchain Computing and Applications (BCCA)*, pages 105–110. https://hal.science/hal-03371872

[49] P. T. Hasan, H. K. T. Akhter, M. R. Tahsinur, A. B. Haque, A. K. M. N. Islam, and R. M. Rahman. 2022. Blockchain and Machine Learning for Fraud Detection: A Privacy-Preserving and Adaptive Incentive Based Approach. *IEEE Access*, vol. 10, pages 87115–87131. https://ieeexplore.ieee.org/document/9857827

[50] Y. Wang, J. Liu, and Z. Chen. 2020. ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts. *IEEE Trans. Network Science and Engineering*, 8, 2, pages 1133–1144. https://doi.org/10.1109/TNSE.2020.2968505

[51] C.-F. Liao, T.-H. Tsai, and C.-J. Chen. 2019. SoliAudit: Smart Contract Vulnerability Assessment Based on Machine Learning and Fuzz Testing. In *Proc. IEEE Int. Conf. Internet of Things (iThings)*, pages 458–465. https://doi.org/10.1109/iThings/GreenCom/CPSCom/SmartData.2019.00098

[52] S. So, M. Lee, J. Park, H. Lee, and H. Oh. 2020. VERISMART: A Highly Precise Safety Verifier for Ethereum Smart Contracts. In *Proc. IEEE Symp. Security and Privacy (S&P)*, pages 1678–1694. https://doi.org/10.1109/SP40000.2020.00032

[53] Bank for International Settlements et al. 2020. Central Bank Digital Currencies: Foundational Principles and Core Features. BIS and Central Banks Joint Report. https://www.bis.org/publ/othp33.htm

[54] M. Asaduzzaman, F. Hasib, and Z. B. Hafiz. 2020. Towards Using Blockchain Technology for Microcredit Industry in Bangladesh. In *Proc. 23rd Int. Conf. Computer and Information Technology (ICCIT)*, pages 1–6. https://doi.org/10.1109/ICCIT51783.2020.9392730

[55] H. Kim and D. Kim. 2024. Optimal Gas Fee Minimization in DeFi: Enhancing Efficiency and Security on the Ethereum Blockchain. *IEEE Access*, vol. 12, pages 173810–173823. https://ieeexplore.ieee.org/document/10750187

[56] X. Yuan. 2024. SHAP-based Interpretable Models for Credit Default Assessment Using Machine Learning. In *Proc. 14th Int. Conf. Software Technology and Engineering (ICSTE)*, pages 213–217. https://doi.org/10.1109/ICSTE63875.2024.00044

[57] S. Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf

[58] G. Wood. 2023. Ethereum: A Secure Decentralised Generalised Transaction Ledger. (Yellow Paper), Ethereum Foundation, 2014 (revised). https://ethereum.github.io/yellowpaper/paper.pdf

[59] Aave. 2022. Aave Protocol V3 Technical Paper. https://github.com/aave/aave-v3-core/blob/master/techpaper/Aave_V3_Technical_Paper.pdf

[60] T. Dao, T. Trinh, and V. Pham. 2025. Optimizing Credit Scoring Models for Decentralized Financial Applications. In *Information and Communication Technology*, Springer. https://doi.org/10.1007/978-981-96-4282-3_36

[61] G.-L. Gücük, S. Leible, and J. Edinger. 2024. Blockchain-Based Microlending for Financial Inclusivity: A Literature Review of Its Privacy and Trust. In *Blockchain and Applications*. Springer. https://doi.org/10.1007/978-3-032-12801-0_22

[62] A. Beniiche. 2020. A Study of Blockchain Oracles. *arXiv preprint arXiv:2004.07140*. https://arxiv.org/pdf/2004.07140

[63] A. Pasdar, Y. C. Lee, and Z. Dong. 2023. Connect API with Blockchain: A Survey on Blockchain Oracle Implementation. *ACM Computing Surveys*, 55, 10. https://doi.org/10.1145/3567582

[64] L. Gudgeon, S. M. Werner, D. Perez, and W. J. Knottenbelt. 2020. DeFi Protocols for Loanable Funds: Interest Rates, Liquidity and Market Efficiency. In *Proc. 2nd ACM Conf. Advances in Financial Technologies (AFT)*, pages 92–112. https://doi.org/10.1145/3419614.3423254

[65] T. Mackinga, T. Nadahalli, and R. Wattenhofer. 2023. Attacks on Dynamic DeFi Interest Rate Curves. *arXiv preprint arXiv:2307.13139*.

[66] K. W. Wu. 2024. Strengthening DeFi Security: A Static Analysis Approach to Flash Loan Vulnerabilities. *arXiv preprint arXiv:2411.01230*.

[67] Y. Li et al. 2025. A Comprehensive Study of Exploitable Patterns in Smart Contracts: From Vulnerability to Defense. *arXiv preprint arXiv:2504.21480*. https://arxiv.org/abs/2504.21480

[68] E. Albert, J. Correas, P. Gordillo, G. Román-Díez, and A. Rubio. 2020. GASOL: Gas Analysis and Optimization for Ethereum Smart Contracts. In *Proc. TACAS*, Springer. https://doi.org/10.1007/978-3-030-45237-7_7

[69] F. Piper, K. Wolf, and J. Heiss. 2025. Privacy-Preserving On-chain Permissioning for KYC-Compliant Decentralized Applications. TU Berlin, *arXiv preprint arXiv:2510.05807*.

[70] N. Decker. 2025. Zero-Knowledge Proofs: Cryptographic Model for Financial Compliance and Global Banking Security. *SSRN Working Paper No. 5170068*.

[71] E. Toufaily and T. Zalan. 2024. How Can Blockchain-Based Lending Platforms Support Microcredit Activities in Developing Countries? An Empirical Validation of Its Opportunities and Challenges. *Technological Forecasting and Social Change*, vol. 203. https://doi.org/10.1016/j.techfore.2024.123403

[72] S. Howlader and P. Halder. 2025. Fintech's Impact on Financial Inclusion Through Mobile Financial Services in Bangladesh. *Sage Publications*. https://doi.org/10.1177/09763996251356998

[73] Grameen Bank. 2024. Grameen Bank at a Glance. Grameen Communications. https://www.grameen-info.org/

[74] A. Carstens et al. 2021. Stablecoins: Risks, Potential and Regulation. *BIS Working Papers No. 905*, Bank for International Settlements. https://www.bis.org/publ/work905.pdf

[75] M. Cen, J. He, A. S. Kyle, and D. Xiong. 2025. Stablecoin Devaluation Risk. *The European Journal of Finance*. https://doi.org/10.1080/1351847X.2025.2505757

[76] J. A. Ocampo and K. Gallagher. 2024. The Role of Multilateral Development Banks and Development Assistance in the Provision of Global Public Goods. UNDP Background Paper. https://hdr.undp.org/system/files/documents/background-paper-document/2024jaocampokdgonzaleztheroleofmultilateraldevlmntbanks.pdf

[77] Anonymous. 2025. Commit-Reveal^2: Securing Randomness Beacons with Randomized Reveal Order in Smart Contracts. *arXiv preprint arXiv:2504.03936*. https://arxiv.org/abs/2504.03936

[78] F. A. Aponte-Novoa, A. L. S. Orozco, R. Villanueva-Polanco, and P. Wightman. 2021. The 51% Attack on Blockchains: A Mining Behavior Study. *IEEE Access*, vol. 9, pages 140549–140564. https://doi.org/10.1109/ACCESS.2021.3119291

[79] DL News Research. 2026. State of DeFi 2025. DL News, March. https://www.dlnews.com/research/internal/state-of-defi-2025/

[80] H. Qu, K. M. Gogol, F. Grötschla, and C. J. Tessone. 2025. From Rules to Rewards: Reinforcement Learning for Interest Rate Adjustment in DeFi Lending. *arXiv preprint arXiv:2506.00505*. https://arxiv.org/abs/2506.00505

[81] W3C. 2022. Decentralized Identifiers (DIDs) v1.0 — Core Architecture, Data Model, and Representations. W3C Recommendation, July ; see also W3C, "Verifiable Credentials Data Model v1.1," W3C Recommendation, March. https://www.w3.org/TR/did-core/

[82] Sygnum Bank. 2026. Institutional DeFi in 2025: From Niche to Mainstream Finance. Sygnum Research Report, February. https://www.sygnum.com/research/

[83] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, vol. 372, n71. https://doi.org/10.1136/bmj.n71

[84] P. Treleaven, R. Gendal Brown, and D. Yang. 2017. Blockchain Technology in Finance. *Computer*, 50, 9, pages 14–17. https://doi.org/10.1109/MC.2017.3571047

[85] D. Cheng, X. Wang, Y. Zhang, and L. Zhang. 2022. Graph Neural Network for Fraud Detection via Spatial-Temporal Attention. *IEEE Trans. Knowledge and Data Engineering*, 34, 8, pages 3800–3813. https://ieeexplore.ieee.org/document/9356317

[86] D. Wang, B. Wu, X. Yuan, L. Wu, Y. Zhou, and H. Cui. 2024. DeFiGuard: A Price Manipulation

Detection Service in DeFi Using Graph Neural Networks. *arXiv preprint arXiv:2406.11157*. https://arxiv.org/abs/2406.11157

[87] G. Palaiokrassas, S. Scherrers, E. Makri, and L. Tassiulas. 2024. Machine Learning in DeFi: Credit Risk Assessment and Liquidation Prediction. In *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*. https://doi.org/10.1109/ICBC59979.2024.10634350

[88] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*.

[89] Z. Chen, J. Chen, M. Sra, et al. 2025. Standard Benchmarks Fail — Auditing LLM Agents in Finance Must Prioritize Risk. *arXiv preprint arXiv:2502.15865*. https://arxiv.org/abs/2502.15865

[90] Grameen Bank. 2024. Annual Report 2023. Grameen Bank, Dhaka, Bangladesh. https://grameen.com/wp-content/uploads/2024/04/Annual-Report-2023-English.pdf

[91] W3C. 2022. Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations. W3C Recommendation (July 2022). https://www.w3.org/TR/did-core/. See also W3C. 2022. Verifiable Credentials Data Model v1.1. https://www.w3.org/TR/vc-data-model/

[92] MicroSave Consulting. 2025. Shaping a Sustainable Microfinance Sector in Bangladesh. MSC Signature Project, September. https://www.microsave.net/signature_projects/shaping-a-sustainable-microfinance-sector-in-bangladesh-through-mscs-transformation-feasibility-study/

[93] Bank for International Settlements. 2025. BIS Survey on Central Bank Digital Currency and Crypto-Assets. BIS Papers No. 147, July. https://www.bis.org/publ/bppdf/bispap147.pdf

[94] Morgan Stanley. 2023. Key Milestone in Innovation Journey with OpenAI. Press Release, March. https://www.morganstanley.com/press-releases/key-milestone-in-innovation-journey-with-openai

[95] McKinsey & Company. 2024. The Economic Potential of Generative AI: The Next Productivity Frontier (Banking Sector Update). McKinsey Global Institute, June. https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier

[96] G. Cheng et al. 2025. Uncovering the Vulnerability of Large Language Models in the Financial Domain via Risk Concealment. *arXiv preprint arXiv:2509.10546*. https://arxiv.org/abs/2509.10546

[97] European Parliament and Council of the European Union. 2024. Regulation (EU) 2023/1114 of 31 May 2023 on Markets in Crypto-Assets (MiCA). *Official Journal of the European Union*, L 150/40, June 2023; fully applicable from 30 December. https://eur-lex.europa.eu/eli/reg/2023/1114/oj

[98] European Banking Authority. 2024. Guidelines on the Minimum Content of the Governance Arrangements for Issuers of Asset-Referenced Tokens under MiCAR. EBA/GL//06, June. https://www.eba.europa.eu/activities/single-rulebook/regulatory-activities/asset-referenced-and-e-money-tokens-micar/guidelines-internal-governance-arrangements-issuers-arts-under-micar

[99] United States Congress. 2025. Guiding and Establishing National Innovation for U.S. Stablecoins (GENIUS) Act. Pub. L. No. 119-27, July 18. https://www.congress.gov/bill/119th-congress/senate-bill/919

[100] Bank for International Settlements Innovation Hub. 2024. Project mBridge: Connecting

Economies through CBDC — Reaches Minimum Viable Product. BIS Innovation Hub Report, June. https://www.bis.org/about/bisih/topics/cbdc/mcbdc_bridge.htm

[101] Bank for International Settlements and Central Banks of France, Japan, Korea, Mexico, Switzerland, the UK, and the US. 2024. Project Agora: Tokenization of Cross-Border Payments using Unified Ledgers. BIS Innovation Hub Working Paper, April. https://www.bis.org/about/bisih/topics/fmis/agora.htm

[102] DL News. 2024. DeFi Lending Protocols: Market Overview and Analysis. https://www.dlnews.com/

[103] Bangladesh Bank. 2025. FE Circular No. 06: Electronic Communication for Letters of Credit and Trade Finance Instruments. Foreign Exchange Policy Department, January. https://www.bb.org.bd/mediaroom/circulars/fepd/jan172025fepd06.pdf

[104] H. Qu, K. M. Gogol, F. Grötschla, and C. J. Tessone. 2025. From Rules to Rewards: Reinforcement Learning for Interest Rate Adjustment in DeFi Lending. *arXiv preprint arXiv:2506.00505*. https://arxiv.org/abs/2506.00505

[105] Bangladesh Bank. 2025. Financial Inclusion in Bangladesh: Progress and Gender-Disaggregated Outlook 2024–25. Special Publication SP-02, Financial Inclusion Department, June. https://www.bb.org.bd/pub/special/SP2025-02.pdf

[106] BRAC. 2025. BRAC Microfinance Annual Performance Report 2025: Women-Centric Lending Outcomes. BRAC Research and Evaluation Division. https://www.brac.net/program/microfinance/

[107] Ethereum Improvement Proposals. 2025. EIP-4626: Tokenized Vault Standard. Ethereum Foundation, April 2022. ; "EIP-7540: Asynchronous ERC-4626 Tokenized Vaults," Ethereum Foundation, 2023. ; "EIP-3643: T-REX – Token for Regulated EXchanges," Ethereum Foundation, 2021. See also: Spectral Finance, "On-Chain Credit Scoring for DeFi Lending," 2023. ; RociFi Labs, "Undercollateralized DeFi Lending via On-Chain Credit Score," 2023. ; Qwen Team (Alibaba Cloud), "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*. https://eips.ethereum.org/EIPS/eip-4626

[108] Ethereum Improvement Proposals. 2022–2025. EIP-4626, EIP-7540, EIP-3643; Spectral Finance and RociFi documentation; Qwen Team. 2025. Qwen3 Technical Report (*arXiv:2505.09388*). https://eips.ethereum.org/EIPS/eip-4626

[109] Elliptic. 2019. The Elliptic Data Set. Kaggle. https://www.kaggle.com/datasets/ellipticco/elliptic-data-set

[110] Y. Elmougy, S. Liu, and J. Liu. 2023. Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics. In *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*. https://github.com/git-disl/EllipticPlusPlus

[111] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson. 2019. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. *arXiv preprint arXiv:1908.02591*, KDD Workshop on Anomaly Detection in Finance. https://arxiv.org/abs/1908.02591

[112] T. Larabel. 2024. Laravel 11 Released. Laravel News. https://laravel-news.com/laravel-11

[113] C. He, X. Zhou, D. Wang, H. Xu, W. Liu, and C. Miao. 2026. Harness Engineering for Language Agents: The Harness Layer as Control, Agency, and Runtime. *Preprints.org*. https://www.preprints.org/manuscript/202603.1756

[114] M. Alizadeh, M. Gholampour, A. Imani, and J. Schulz. 2026. Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution. *arXiv preprint arXiv:2506.01055*, University of Zurich / IPM, June.

[115] OWASP Foundation. 2026. OWASP Top 10 for Large Language Model Applications and Agents, 2026 Edition. OWASP Foundation. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[116] Microsoft Security. 2026. Running OpenClaw Safely: Identity, Isolation, and Runtime Risk. Microsoft Security Blog, February . https://www.microsoft.com/en-us/security/blog/2026/02/19/running-openclaw-safely-identity-isolation-runtime-risk/

[117] NVIDIA. 2026. NVIDIA and Microsoft Reinvent Windows PCs for the Age of Personal AI. NVIDIA Newsroom, GTC Taipei / COMPUTEX, May–June ; see also NVIDIA Blog, "RTX PCs and DGX Spark Supercomputers Run AI Agents Locally" (Nemotron on-premise inference for finance, healthcare, and legal workloads). https://nvidianews.nvidia.com/news/nvidia-microsoft-windows-pcs-agents-rtx-spark

[118] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

[119] T. Larabel. 2024. Laravel 11 Released. Laravel News. https://laravel-news.com/laravel-11

[120] G. Aspris and J. Svec. 2025. Institutionalizing Risk Curation in Decentralized Credit. *arXiv preprint arXiv:2512.11976*. https://arxiv.org/abs/2512.11976

[121] S. Jaffer. 1999. Microfinance and the Mechanics of Solidarity Lending. Harvard Law School, *SSRN Working Paper No. 162548*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=162548

[122] R. Sherratt. 2016. From Solidarity to Coercion: The Dynamics of Group Liability. *Oxford Economic Papers*, 68, 4, pages 955–974. https://doi.org/10.1093/oep/gpw018

[123] Y. Chang, H. Kim, and S. Park. 2024. Anomalous Node Detection in Blockchain Networks Based on Graph Neural Networks. *Sensors*, 25, 1, art. 1. https://doi.org/10.3390/s25010001

[124] B. Luo, Z. Zhang, Q. Wang, A. Ke, S. Lu, and B. He. 2024. AI-powered Fraud Detection in Decentralized Finance: A Project Life Cycle Perspective. *ACM Comput. Surv.*, 57, 4, art. 96. https://doi.org/10.1145/3705296

[125] G. Caldarelli. 2025. Can Artificial Intelligence Solve the Blockchain Oracle Problem? Unpacking the Challenges and Possibilities. *Frontiers in Blockchain*, vol. 8, art. 1682623. https://doi.org/10.3389/fbloc.2025.1682623

[126] H. Fazliani, A. Pasdar, and Y. C. Lee. 2025. Ensemble-Based Semi-Supervised Learning for Illicit Account Detection in Ethereum DeFi Transactions. *arXiv preprint arXiv:2412.02408*, rev. October. https://arxiv.org/abs/2412.02408

[127] H. Alhasawi, A. Almutairi, and S. Alharbi. 2025. FedFraud: A Federated Approach to Scalable and Trustworthy Financial Fraud Detection. *Security and Privacy*, Wiley. https://doi.org/10.1002/spy2.70012

[128] European Parliament and Council of the European Union. 2026. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). *Official Journal of the European Union*, L 2024/1689, July 2024; high-risk credit-scoring provisions applicable from August. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

[129] M. Bartoletti, F. Fioravanti, G. Matricardi, R. Pettinau, and F. Sainas. 2024. Towards Benchmarking of Solidity Verification Tools. In *Proc. FMBC (OASIcs)*, vol. 118, pages 6:1–6:15. https://doi.org/10.4230/OASIcs.FMBC.2024.6

[130] Crypto World Bank Project Team. 2026. Binance and FTX: Comparative Industry Analysis (Full Report). internal research note, Documentation/2nd Phase (Bokhtiar)/Improvements/, 2025– See Appendix D, entry IR-1.

[131] Crypto World Bank Project Team. 2026. Blockchain Platforms Comparative Research (CSV). internal research dataset, Documentation/2nd Phase (Bokhtiar)/Improvements/, 2025– See Appendix D, entry IR-2.

[132] Crypto World Bank Project Team. 2026. Binance Software Engineering Architecture and System Analysis. internal research note, Documentation/2nd Phase (Bokhtiar)/Improvements/, 2025– See Appendix D, entry IR-4.

[133] Federal Reserve Bank of New York and Bank for International Settlements Innovation Hub. 2025. Project Pine: Central Bank Open Market Operations with Smart Contracts. BIS Innovation Hub / NY Fed Joint Report, May. https://www.bis.org/publ/othp95.htm

[134] L. Dong, Y. Qiu, and F. Xu. 2023. Blockchain-Enabled Deep-Tier Supply Chain Finance. *Manufacturing & Service Operations Management*, 25, 6, pages 2021–2037. https://doi.org/10.1287/msom.2022.1123

[135] U. Bindseil. 2020. Tiered CBDC and the Financial System. *ECB Working Paper Series*, no. 2351, January. https://www.ecb.europa.eu/pub/pdf/scpwps/ecb.wp2351~c8c18bbd60.en.pdf

[136] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. Zhou. 2020. A Survey of Research on Retail Central Bank Digital Currency. *IMF Working Paper WP/20/104*, June. https://www.imf.org/en/Publications/WP/Issues/2020/06/26/A-Survey-of-Research-on-Retail-Central-Bank-Digital-Currency-49069

[137] R. Auer and R. Böhme. 2020. The Technology of Retail Central Bank Digital Currency. *BIS Quarterly Review*, March; see also BIS Working Paper No. 948. https://www.bis.org/publ/qtrpdf/r_qt2003j.htm

[138] L. Ante. 2025. From Adoption to Continuance: Stablecoins in Cross-Border Remittances and the Role of Digital and Financial Literacy. *Telematics and Informatics*, vol. 97, art. 102230. https://doi.org/10.1016/j.tele.2024.102230

[139] Y. Li, Y. Xiang, Q. Wang, T. H. Yuen, A. Deppeler, and J. Yu. 2026. SoK: Stablecoins in Retail Payments. *arXiv preprint arXiv:2601.00196*. https://arxiv.org/abs/2601.00196

[140] W. Du, C. Huang, and D. Scharfstein. 2026. Competing Rails for Cross-Border Payments: Banks, Fintechs, and Stablecoins. Harvard Business School Working Paper, February. https://www.hbs.edu/faculty/Pages/item.aspx?num=66992

[141] M. Fröhlich, J. A. Vega Vermehren, F. Alt, and A. Schmidt. 2022. Implementation and Evaluation of a Point-Of-Sale Payment System Using Bitcoin Lightning. In *Proc. NordiCHI*. http://www.medien.ifi.lmu.de/pubdb/publications/pub/froehlich2022nordichi/froehlich2022nordichi.pdf

[142] K. Zhang, T.-M. Choi, S.-H. Chung, Y. Dai, and X. Wen. 2024. Blockchain Adoption in Retail Operations: Stablecoins and Traceability. *European Journal of Operational Research*, 315, 1, pages 147–160. https://doi.org/10.1016/j.ejor.2023.11.026

[143] J. Lin, M. Liu, S. Li, and X. Wang. 2025. SecurePay: Enabling Secure and Fast Payment Processing for Platform Economy. In *Proc. IEEE/ACM Int. Symp. Quality of Service (IWQoS)*; *arXiv:2505.17466*. https://arxiv.org/abs/2505.17466

[144] Committee on Payments and Market Infrastructures. 2023. Considerations for the Use of Stablecoin Arrangements in Cross-Border Payments. Bank for International Settlements, July. https://www.bis.org/cpmi/publ/d220.pdf

[145] A. Kosse, J. Lisack, and N. Boakye-Agyeman. 2023. The Impact of Stablecoins on the International Monetary and Financial System. *BIS Papers*, no. 170, February. https://www.bis.org/publ/bppdf/bispap170.pdf

[146] E. Cerutti, M. Firat, and H. Pérez-Saiz. 2025. Estimating the Impact of Digital Money on Cross-Border Flows: Scenario Analysis Covering the Intensive Margin. *IMF Fintech Notes* 2025/002, February. https://www.imf.org/-/media/files/publications/ftn063/2025/english/ftnea2025002.pdf

[147] Financial Action Task Force. 2021. Updated Guidance for a Risk-Based Approach to Virtual Assets and Virtual Asset Service Providers. FATF, Paris, October. https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Va-Vasp-Guidance-R16-R21.html

[148] G. Cimaszewski, F. Da Dalt, T. Moser, and A. Perrig. 2025. SCION and Cross-Border Payments: Enhancing Security and Compliance in Distributed Ledger Networks. *SNB Working Papers* 15/2025, Swiss National Bank. https://www.snb.ch/en/publications/research/working-papers/2025/working_paper_2025_15

[149] Bank for International Settlements Innovation Hub. 2024. Project Sela: An Accessible and Secure Retail CBDC Ecosystem. BIS. https://www.bis.org/publ/othp74.htm

[150] S. Chaliasos, M. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, and B. Livshits. 2024. Smart Contract and DeFi Security: Insights from Tool Evaluations and Practitioner Surveys. In *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE)*, pages 160:1–160:13. https://doi.org/10.1145/3597503.3623302

[151] Z. Chen, S. M. Beillahi, P. Barahimi, C. Minwalla, H. Du, A. Veneris, and F. Long. 2026. Enforcing Control Flow Integrity on DeFi Smart Contracts. In *Proc. IEEE/ACM 48th Int. Conf. Software Engineering (ICSE)*. https://doi.org/10.1145/3744916.3773266

[152] S. Ren, L. He, T. Tu, D. Wu, J. Liu, K. Ren, and C. Chen. 2025. LookAhead: Preventing DeFi Attacks via Unveiling Adversarial Contracts. *Proc. ACM Softw. Eng.*, 2, FSE, art. FSE083. https://doi.org/10.1145/3729353

[153] ACM Transactions on the Web. 2026. Zero-Knowledge Proof Framework for Identity Verification and Interoperable Payments on the Decentralized Web. *ACM Trans. Web*. https://doi.org/10.1145/3795774

[154] ACM Transactions on the Web. 2026. Web3-Based Identity and KYC Innovations for Next-Generation FinTech. *ACM Trans. Web*. https://doi.org/10.1145/3771991

[155] Multiple Authors. 2025. Blockchain-Based Fraud Detection: A Comparative Systematic Literature Review of Federated Learning and Machine Learning Approaches. *Electronics*, 14, 24, art. 4952. https://doi.org/10.3390/electronics14244952

[156] IEEE. 2025. Blockchain-Powered Platform for Microloans and Lending Solutions. In *IEEE Conf. Proc.* https://ieeexplore.ieee.org/document/11042350/

[157] CAAW Workshop. 2025. Price Oracle Accuracy Across Blockchains: A Measurement and Analysis. In *Proc. Workshop on Crypto Asset Analytics (CAAW)*. https://caaw.io/2025/papers/CAAW25_paper_22.pdf

[158] K. Qin, L. Zhou, B. Livshits, and A. Gervais. 2021. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *Financial Cryptography and Data Security (FC)*, pages 3–32. https://doi.org/10.1007/978-3-662-63514-3_1

[159] T. Mackinga, T. Nadahalli, and R. Wattenhofer. 2024. SoK: Attacks on DAOs. *arXiv preprint arXiv:2406.15071*. https://arxiv.org/abs/2406.15071

[160] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. 2020. Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges. In *Proc. IEEE Symp. Security and Privacy (S&P)*, pages 910–927. https://doi.org/10.1109/SP40000.2020.00040

[161] P. K. Ozili. 2024. Central Bank Digital Currency Adoption Challenges. *Journal of Central Banking Theory and Practice*, 13, 1, pages 5–24. https://doi.org/10.2478/jcbtp-2024-

```
0001
```